Doctoral theses at NTNU, 2024:49

Erling Rennemo Jellum

Predictable Computing for Cyber-Physical Systems

Doctoral thesis

NTNU
Norwegian University of Science and Technology
Thesis for the Degree of Philosophiae Doctor
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

NTNU
Norwegian University of Science and Technology

Erling Rennemo Jellum

Predictable Computing for Cyber-Physical Systems

Thesis for the Degree of Philosophiae Doctor

Trondheim, February 2024

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

Norwegian University of
Science and Technology

NTNU
Norwegian University of Science and Technology

Thesis for the Degree of Philosophiae Doctor

Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

© Erling Rennemo Jellum

ISBN 978-82-326-7700-9 (printed ver.)
ISBN 978-82-326-7699-6 (electronic ver.)
ISSN 1503-8181 (printed ver.)
ISSN 2703-8084 (online ver.)

Doctoral theses at NTNU, 2024:49

Printed by NTNU Grafisk senter

Abstract

Cyber-physical systems encompass a vast category of systems where computers operate in feedback with physical systems. Cyber-physical systems hold the potential for enormous societal impact, comparable to that of the internet revolution in the 2000s. However, unlike typical internet applications, where crashes and downtime are commonplace and acceptable, cyber-physical systems depend on correctness and predictability. Cyber-physical systems often interact with critical infrastructure or humans and failures are unacceptable. The correctness of a cyber-physical system depends on a predictable interaction between the computers and the physical system. There are two important aspects of this interaction. The first aspect is the value of the signals being communicated between the computer and the physical system, e.g. the setpoint of an actuator. The second aspect is the time at which this signal is being communicated. Most cyber-physical systems today are built with technology that admits non-determinism concerning both the values of the signals and the timing of the signals.

To realize the full potential of cyber-physical systems, engineers need access to high-performance computers that are easy to program to deliver reliable, predictable and correct computations. Three research areas will be important to achieve this vision. First, a better understanding of cyber-physical systems requirements for predictability is needed. Particularly in the field of multi-sensor fusion where jitter in the timestamping of sensor measurements can be thought of as noise coloring the measurement. Second, new architectures, languages and models for general-purpose CPUs that emphasize both performance and predictability are needed. Particularly, there is a need for predictable multi-core systems and distributed systems. Lastly, new models and languages are needed to efficiently design correct computations on emerging heterogeneous computer architectures such as System-on-Chip (SoC) Field Programmable Gate Arrays (FPGA).

This thesis is a compilation of peer-reviewed articles addressing topics within these three research areas. On the topic of understanding the predictability requirements of a cyber-physical system, one paper is included. It introduces the Syncline model, an intuitive performance model relating timestamping accuracy, sensor noise and the dynamics of the system.

On the topic of predictable general-purpose CPU, three papers are presented. The first paper provides support for Lingua Franca, an emerging design language for cyber-physical systems, on resource-constrained microcontrollers. The second

paper introduces InterPRET a new time-predictable multicore processor architecture The third paper extends Timed C, a C-based programming language with timing semantics, with primitives for inter-task communication over networks, enabling simplified design of distributed systems.

On the topic of predictable heterogeneous architectures, this thesis include three papers. The first paper presents fpga-tidbits, a framework for designing and evaluating accelerators for SoC FPGAs. The second paper builds on this framework to implement an accelerator for the linear programming problem known as the assignment problem. The third and last paper introduces hardware reactors, a model of computation for hardware-software codesign and an extension to Lingua Franca enabling single-source reactor-oriented hardware-software codesign.

Preface

This thesis is submitted in partial fulfillment of the requirements for the degree Philosophia Doctor (Ph.D.) at the Norwegian University of Science and Technology. It is a compilation thesis consisting of an introductory part, seven scientific articles and a concluding part.

Associate Professor Torleiv Håland Bryne was the main supervisor, with Associate Professor Milica Orlandic and Professor Tor Arne Johansen as co-supervisors. The work was funded by the university internally, but supported by the Research Council of Norway grant number 327538, Multi-Sensor Data Timing, Synchronization and Fusion for Intelligent Robots and grant number 223254, the center of excellence NTNU AMOS. The work was mainly carried out at NTNU Department of Engineering Cybernetics in Trondheim, Norway and partially at University of California, Berkeley in Berkeley, U.S. and the Technical University of Denmark in Lyngby, Denmark.

Acknowledgements

I am very grateful for the support and advice from my supervisors Torleiv Håland Bryne, Milica Orlandic and Tor Arne Johansen. You have supported me in complementary ways, from the big picture down to the nitty gritty details of hardware design and coordinate transformations. I also appreciate the freedom, as well as financial support, I got to explore the topics that interested me most.

I am also very thankful for the support and advice from Sverre Hendseth. I attribute most of what I know about real-time systems to my interactions with him. I thoroughly enjoyed the discussions and arguments we had through our paper discussion meetings with the real-time systems club which also included Henrik Austad, Øyvind Teig and Anders Petterson. Our sporadic meetings have been among the highlights of my Ph.D. journey.

I would like to thank Henrik Austad for a great collaboration and in general very good discussions. He is a great engineer and has taught me most of what I now know about TSN, PTP and Linux in general.

My visit to UC Berkeley was pivotal and I would like to thank Edward Lee for being a great host. I discovered Edward's work a year into my Ph.D. and I can honestly say that he has influenced my perspective more than anyone else. Not only on our field, computer science, but on the world in general. Marten Lohstroh has also been instrumental. I admire his work on Reactors and Lingua Franca which became pillars in my research. I also met several other great researchers at Berkeley and I would like to thank Shaokai Lin, Christian Menard, Peter Donovan, Mirco Theile and Chadlia Jerad for enriching discussions and collaborations on several papers included in the thesis.

I would also like to thank Martin Schoeberl from DTU for a very interesting collaboration and for inviting me to DTU. I have learned a lot from him on computer architecture, real-time systems, digital design and Chisel. I have also really appreciated his honest opinion in times of doubt.

I am also grateful for the friendships I have made with my fantastic colleagues at the Department of Engineering Cybernetics. They have made these last years a real pleasure.

Finally, I want to thank my amazing partner Ingrid for all her support and understanding, and my parents Grete and Edgar.

Contents

Abstract . . . . . **iii**
Preface . . . . . **v**
Acknowledgements . . . . . **vii**
Contents . . . . . **ix**

I Overview **1**
1 Introduction **3**
1.1 Motivation 3
1.2 Research Questions 4
1.3 Publications 5
1.4 Thesis Outline 7
2 Background **9**
2.1 Systems and Models 9
2.2 Cyber-Physical Systems 11
2.3 Layers of Abstraction in Computing 14
2.4 Models of Computation 15
2.5 Lingua Franca 19
2.6 Timed C 21
2.7 Computer Architectures 22
2.7.1 PRET machines 23
2.7.2 System-on-chip FPGAs 24
3 Research Summary **27**
3.1 Overview 27
3.2 RQ1: What is the Importance of Predictable Timestamping in CPS? 28
3.3 RQ2: How Can We Enable Predictable Computing on General-Purpose CPUs? 29
3.4 RQ3: How Can We Enable Predictable Computing on SoC FPGAs? . 32

II Papers **35**
A The Syncline Model - Analyzing the Impact of Time Synchronization in Sensor Fusion **37**
A.1 Introduction 38
A.2 Background 39

A.2.1 Sensor synchronization primitives . . . 39
A.2.2 Timestamping methods . . . 40
A.3 Preliminaries . . . 40
A.3.1 Mathematical Notations . . . 40
A.3.2 Attitude representations and relationships . . . 40
A.4 Related Work . . . 41
A.5 Syncline Model . . . 42
A.5.1 State-space formulation . . . 42
A.5.2 Sensor measurement model . . . 42
A.5.3 System dynamics . . . 43
A.5.4 Estimation error in sensor fusion . . . 43
A.6 Examples . . . 46
A.6.1 Example 1: Georeferencing with GNSS, INS and LiDAR . . . 46
A.6.2 Example 2: Underwater survey with USBL, MBE, INS and GNSS . . . 53
A.7 Conclusions . . . 58
Bibliography . . . 59
B Beyond the Threaded Programming Model on Real-Time Operating Systems . . . 61
B.1 Introduction . . . 62
B.2 Embedded Programming in Lingua Franca . . . 63
B.2.1 Embedded design patterns . . . 64
B.3 Related Work . . . 67
B.4 Design and Implementation . . . 67
B.5 Evaluation . . . 68
B.5.1 Memory footprint . . . 69
B.5.2 Timing precision . . . 70
B.5.3 Performance . . . 72
B.6 Conclusion . . . 75
Bibliography . . . 77
C Composable distributed real-time systems with deterministic network channels . . . 81
C.1 Introduction . . . 82
C.2 Contributions . . . 83
C.3 Background and Motivation . . . 84
C.3.1 Real-time Systems . . . 84
C.3.2 Distributed Real-Time Systems . . . 85
C.3.3 Composability and Composable DRTS . . . 86
C.3.4 Real-Time Networks and TSN . . . 86
C.3.5 Precision Time Protocol . . . 88
C.3.6 Linux, PREEMPT_RT and rt-tests . . . 89
C.3.7 Timed C . . . 89
C.3.8 Related work . . . 91
C.4 Deterministic Network Channels and Federated Timed C . . . 93

C.4.1 Formalization . . . 93
C.4.2 Reference Implementation Overview . . . 98
C.4.3 Detailed Architecture . . . 100
C.4.4 Automatic Stream Reservation . . . 104
C.4.5 Description of Protocol Format . . . 104
C.4.6 Integration with Timed C . . . 104
C.5 Experimental Setup and Evaluation . . . 105
C.5.1 Test scenarios . . . 105
C.5.2 Test tools . . . 106
C.5.3 3rd party tools . . . 108
C.5.4 Running the Test Scenarios . . . 110
C.6 Results . . . 111
C.6.1 PTP Accuracy . . . 111
C.6.2 Kernel Wakeup Accuracy . . . 112
C.6.3 Package Transmission Delay . . . 113
C.6.4 E2E Delay with Network Noise . . . 113
C.6.5 Network Triggered Wakeup Accuracy . . . 115
C.6.6 Final Remarks . . . 116
C.7 Conclusion . . . 116
C.8 Future work . . . 117
C.9 Code examples . . . 117
C.10 Linux RT Setup . . . 120
Bibliography . . . 121
D InterPRET: A Time-predictable Multicore Processor . . . 127
D.1 Introduction . . . 128
D.2 Related Work . . . 129
D.3 Background . . . 130
D.3.1 FlexPRET . . . 130
D.3.2 The Network-on-Chip . . . 131
D.4 InterPRET . . . 132
D.4.1 Predictable Timing . . . 133
D.4.2 Software-defined Devices . . . 134
D.4.3 Hardware Devices . . . 134
D.4.4 Bootloader . . . 135
D.5 Evaluation . . . 135
D.5.1 Resource Utilization . . . 135
D.5.2 Latency, Jitter, and Bandwidth . . . 136
D.5.3 Case Study 1: Multicore PingPong . . . 137
D.5.4 Case Study 2: Software-defined UART . . . 138
D.6 Conclusion . . . 138
Bibliography . . . 141
E fpga-tidbits: Rapid Prototyping of FPGA Accelerators in Chisel . . . 145
E.1 Introduction . . . 146
E.2 Background . . . 147

E.2.1 Hardware-Software Codesign on SoC FPGAs 147
E.2.2 Chisel 149
E.3 Related Work 150
E.3.1 OpenCL 150
E.3.2 SYCL 150
E.3.3 FPGA shells 150
E.3.4 Chipyard 150
E.3.5 Vitis Unified Software Platform 151
E.4 fpga-tidbits 151
E.4.1 Generic Accelerator 154
E.4.2 Platform Wrappers 154
E.4.3 On-chip Memories 155
E.4.4 Off-chip Memory Access 155
E.4.5 Streams 156
E.4.6 High-level Synthesis 156
E.4.7 Hardware-Software Cosimulation 156
E.5 Evaluation 157
E.5.1 Case Study 1: BISMO 157
E.5.2 Case Study 2: Auction Accelerator 158
E.5.3 Case Study 3: Reactor-Chisel 159
E.6 Future Work 160
E.7 Acknowledgements 161
E.8 Conclusion 161
Bibliography 163
F Solving Sparse Assignment Problems on FPGAs 167
F.1 Introduction 168
F.2 Background 169
F.2.1 The assignment problem 169
F.2.2 Methods for the assignment problem 170
F.2.3 The assignment problem in MOT 173
F.3 Related Work 175
F.4 Sparse and Speculative (SaS) Auction 176
F.4.1 Sparse problems 177
F.4.2 Dependency speculation 179
F.4.3 Hardware Architecture 180
F.4.4 Comparison to SoA Auction 184
F.4.5 Performance model for speculation 185
F.5 Evaluation 187
F.5.1 Performance 189
F.5.2 Performance model evaluation 194
F.5.3 Area 197
F.6 Conclusions 198
Bibliography 199

G Codesign of Reactor-oriented Hardware and Software for Cyber-Physical Systems . . . 203
G.1 Introduction . . . 204
G.2 Background . . . 206
G.2.1 Models of computation . . . 206
G.2.2 Synchronous data flow . . . 207
G.2.3 Lingua Franca and the reactor model of computation . . . 207
G.3 Related Work . . . 209
G.3.1 OpenCL . . . 209
G.3.2 SYCL . . . 210
G.3.3 Compaan/Laura . . . 210
G.3.4 ReconOS . . . 210
G.3.5 ReconROS . . . 211
G.3.6 ForSyDe . . . 211
G.3.7 Vitis Unified Software Platform . . . 211
G.3.8 AADL . . . 212
G.3.9 SystemC . . . 212
G.3.10 Bluespec Codesign Language . . . 212
G.3.11 LEAP . . . 213
G.4 Reactor-oriented hardware-software codesign . . . 213
G.4.1 Blinky . . . 213
G.4.2 Vector addition . . . 214
G.4.3 Multi-sensor fusion . . . 216
G.5 Hardware reactors model of computation . . . 217
G.6 reactor-chisel . . . 218
G.6.1 Tokens . . . 220
G.6.2 Token interfaces . . . 221
G.6.3 Reaction . . . 221
G.6.4 Ports . . . 223
G.6.5 Connections . . . 223
G.6.6 State variables . . . 224
G.6.7 Reactor . . . 224
G.6.8 Triggers . . . 225
G.6.9 Hardware-software codesign . . . 227
G.6.10 Compilation . . . 229
G.7 Evaluation . . . 230
G.7.1 Image processing example . . . 230
G.7.2 Resource utilization . . . 232
G.7.3 Latency . . . 234
G.7.4 Throughput . . . 235
G.8 Conclusion . . . 236
Bibliography . . . 237

III Concluding remarks **243**
1 Conclusion **245**
2 Future Work **249**
Bibliography **251**

Part I

Overview

Chapter 1

Introduction

1.1 Motivation

Cyber-physical systems (CPS) consist of networked computers interacting with a physical system through sensors and actuators. They range from tiny medical devices embedded in humans to autonomous vehicles, air traffic control systems, power grids, and many more. CPS have the potential to revolutionize several sectors of society, much like the internet revolution [1]. However, to realize their full potential, CPS depend on both high-performant and predictable computing. By performance, we refer to the average processing time needed by the computing device to execute some benchmarks, e.g. SPEC [2]. High performance is needed to realize compute-intensive state estimation and control algorithms as the number of sensors and the amount of sensor data will continue to grow. By predictability, we mean that the *behavior* of the, possibly distributed, computation is knowable and preferably deterministic. With behavior, we are referring to both the values being computed, i.e. data-predictability, and the time at which these values are produced, i.e. time-predictability. While performance, as defined here, is mainly a property of the underlying computer hardware, predictability is connected to both the hardware architecture and the abstractions used to program the hardware. So far, we have seen incredible improvements in computer performance during the last 50 years. We have not seen the same improvements in the ability to design predictable computing. On the contrary, the advances in performance have often come at the cost of reduced predictability.

Chip designers are today still improving computer performance, mainly by pursuing (1) multicore architectures and (2) heterogeneous architecture. Both introduce interesting predictability challenges which are tackled in this thesis. Multicores are challenging because they make the design of both time-predictable and data-predictable computations hard. The challenges in time-predictability are mostly due to the hardware architectures of the most common multicores. Each core exhibits non-deterministic timing due to optimizations such as branch prediction, out-of-order execution and cache hierarchies. Moreover, when composing multiple cores sharing the same memory hierarchy the result is drastically worse.

Experiments by Lockheed Martin concluded that moving a single-threaded application from a single-core to a multi-core increased the worst-case execution time by a factor of six [3]. Data-predictability is challenging on multicores because the commonly used languages and frameworks for orchestrating computations spanning multiple cores are generally also non-deterministic.

Heterogenous systems tend to offer more time-predictability because the domain-specific accelerators often do not include general-purpose optimizations such as speculation. However, the programming of such systems remains challenging. In this thesis, we focus on system-on-chip (SoC) field-programmable gate arrays (FPGAs). Here the computation spans hardware accelerators implemented on the FPGA and sequential programs running on the CPUs. The state-of-the-art languages and frameworks for programming SoC FPGAs suffer from a lack of data-predictability. The programmer is left with the job of pruning away the non-determinism allowed by the programming models and framework. Prominent examples are ReconOS, ReconROS and Xilinx Runtime (XRT). They simplify the codesign of hardware on the FPGA and software on the CPUs by extending various concurrency models from software to hardware. By settling for non-deterministic concurrency models the design of the runtime is made easier but the design of predictable computing is made harder. This thesis addresses the problem of data-predictability of computations spanning hardware and software on SoC FPGAs.

CPS also increasingly include computations distributed over networks. This trend is made challenging by traditional networking technologies that provide no way to guarantee the timing of the delivery of messages. Recently a new set of IEEE standards known as Time Sensitive Networking (TSN) has challenged this by introducing guaranteed bounds on end-to-end latencies through networks. In this thesis, we address the challenge of how to program such networked systems. Many popular frameworks, like ROS2 and MQTT, are based on publish-subscribe which decouples sender and receiver and is inherently non-deterministic. With TSN, the sender and receiver will have tightly synchronized clocks which opens up new opportunities for building more predictable distributed computations.

1.2 Research Questions

The ever-growing need for computational power in CPS can only be satisfied by moving to parallel and heterogeneous computer architectures. This future trend motivates the main research objective of this thesis:

Enabling predictable computing on emerging computer architectures required by the next-generation cyber-physical systems.

This objective is tackled by research into the following research questions:

RQ 1: What is the importance of predictable timestamping in CPS?
RQ 2: How can we enable predictable computing on general-purpose CPUs?

RQ 3: How can we enable predictable computing on SoC FPGAs?

In RQ1, the question of why predictability is an important property in CPS is explored. The focus is on multi-sensor fusion systems where measurements from multiple sensors are fused to a single estimate of the state of a physical system. In particular, the role of predictability in the *timestamping* of the sensor measurements is explored.

In RQ2 we explore various ways to provide predictable computing on general-purpose CPUs. We propose IntePRET, a time-predictable multi-core computer architecture capable of deliver parallel computations without sacrificing predictability. We also propose using Lingua Franca, an emerging language for CPS, for resource-constrained computers such as microcontrollers. Finally, we propose an extension to Timed C enabling programming distributed systems with native support for TSN.

Finally, in RQ3, we look at predictable computing on SoC FPGAs, a new class of heterogeneous computer architectures. We present fpga-tidbits, a powerful framework for designing and verifying accelerators for the FPGA fabric of SoC FPGAs. We leverage this framework to design an accelerator for solving the assignment problem, an optimization problem found at the core of several CPS. Lastly, we propose a new model of computation and design language for the codesign of hardware and software for SoC FPGAs. We define hardware reactors as a subset of the recent reactor model of computation and extend Lingua Franca with support for designing computations spanning CPUs and FPGA.

1.3 Publications

Given below is the list of publications written as part of this thesis. There is a total of seven publications. The publications are enumerated in thematical order where Paper A address Research Question 1, Paper B, Paper C and Paper D address Research Question 2 and Paper E, Paper F and Paper G address Research Question 3.

Conference publications

A **Erling Rennemo Jellum**, Torleiv H. Bryne, Tor Arne Johansen, Milica Orlandic, 'The Syncline Model - Analyzing the Impact of Time Synchronization in Sensor Fusion,' in *2022 IEEE Conference on Control Technology and Applications (CCTA)*, 2022, pp. 1446–1453. DOI: 10.1109/CCTA49430.2022.9966179

Role of the Authors: I had the initial idea and did software development and evaluation. Bryne contributed to the formalization and mathematical notation. Orlandic and Johansen contributed with improvements to the revisions of the paper.

B **Erling Rennemo Jellum**, Shaokai Lin, Peter Donovan, Efsane Soyer, Fuzail Shakir, Torleiv Bryne, Milica Orlandic, Marten Lohstroh, Edward A. Lee, 'Beyond the Threaded Programming Model on Real-Time Operating Systems,' in *Fourth Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2023)*, F. Terraneo and D. Cattaneo, Eds., ser. Open Access Series in Informatics (OASIcs), vol. 108, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 3:1–3:13, ISBN: 978-3-95977-268-6. DOI: 10.4230/OASIcs.NG-RES.2023.3. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2023/17734
Role of the Authors: I had the initial idea and carried out the majority of the software development and evaluation. Lin contributed with porting the Savina benchmarks, Soyer and Fuzail contributed with the Thread-Metric benchmarks. Donovan contributed with writing and general development. Lohstroh and Lee supervised the project and Orlandic and Bryne contributed to the writing.

D **Erling Rennemo Jellum**, Shaokai Lin, Peter Donovan, Chadlia Jerad, Edward Wang, Marten Lohstroh, Edward A. Lee, Martin Schoeberl, 'InterPRET: A Time-predictable Multicore Processor,' in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, ser. CPS-IoT Week '23, San Antonio, TX, USA: Association for Computing Machinery, 2023, pp. 331–336, ISBN: 9798400700491. DOI: 10.1145/3576914.3587497. [Online]. Available: https://doi.org/10.1145/3576914.3587497
Role of the Authors: Schoeberl had the initial idea and I carried out the majority of the software and hardware development as well as evaluation. Lin wrote the chapter on Predictable Timing, Jerad did the Ping Pong case study, Wang contributed to the original FlexPRET design. Donovan performed the network-on-chip benchmarking. Lohstroh and Lee supervised and contributed to the writing.

E **Erling Rennemo Jellum**, Yaman Umuruglu, Milica Orlandic, Martin Schoeberl, 'fpga-tidbits: Rapid Prototyping of FPGA Accelerators in Chisel,' in *To appear in: 2023 26th Euromicro Conference on Digital System Design (DSD)*
Role of the Authors: Umuruglu had the initial idea and did the initial implementation. I picked up the project years later and ported it to chisel3. I performed the case studies on the Auction Accelerator and reactor-chisel, while Umuruglu performed the case study on BISMO. Schoeberl was the main supervisor and Orlandic contributed to the writing.

Journal publications

F **Erling Jellum**, Milica Orlandić, Edmund Brekke, Tor Johansen, Torleiv Bryne, 'Solving Sparse Assignment Problems on FPGAs,' *ACM Trans. Archit. Code Optim.*, vol. 19, no. 4, Dec. 2022, ISSN: 1544-3566. DOI: 10.1145/3546072. [Online]. Available: https://doi.org/10.1145/3546072

Role of the Authors:
Brekke had the initial observation that the auction algorithm was a bottleneck. I did most of the modeling, implementation and evaluation. Bryne and Johansen contributed to the writing. Orlandic was the main supervisor and contributed to the design, implementation, evaluation as well as writing.

C Henrik Austad, **Erling Rennemo Jellum**, Sverre Hendseth, Geir Mathisen, Torleiv Håland Bryne, Kristoffer Nyborg Gregertsen, Sigurd Mørkved Albrektsen, Bjarne Emil Helvik, 'Composable distributed real-time systems with deterministic network channels,' *Journal of Systems Architecture*, vol. 137, p. 102 853, 2023, ISSN: 1383-7621. DOI: https://doi.org/10.1016/j.sysarc.2023.102853. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762123000322
Role of the Authors: Austad and I had the initial idea together. Austad carried out the majority of the software implementation and evaluation. I contributed to the formalization of the proposed netchannels. Hendseth, Mathisen, Bryne, Gregertsen, Albrektsen and Helvik contributed with supervision and to the writing.

Publications submitted for review

G **Erling Rennemo Jellum**, Martin Schoeberl, Edward Lee, Milica Orlandic, 'Codesign of Reactor-oriented Hardware and Software for Cyber-Physical Systems,' *Submitted to: ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 2023
Role of the Authors: I had the initial idea and performed the implementation and evaluation. Lee and Schoeberl contributed with supervision and expertise on reactors and Lingua Franca. Orlandic was the main supervisor and contributed with writing and the general direction.

1.4 Thesis Outline

This thesis is organized into three parts. The first part provides the theoretical background in Chapter 2 and a summary of the research in Chapter 3. The second part consists of the seven research articles published as part of the thesis. The third, and last, part provides conclusions and potential future work.

Chapter 2

Background

2.1 Systems and Models

The distinction between physical *systems* and the *models* of these systems is a fundamental dichotomy in science and engineering. The ancient Chinese text Tao Te Ching opens with the following: "The Tao that can be told, is not the eternal Tao" [11], hinting at the difference between what can be expressed with symbols and words, and reality. In his theory of forms, Plato argues that ideas and mathematics exist in a different realm from the physical world [12]. Kant makes a distinction between the thing, which is our perception, i.e. a model, of some physical system, and the thing-in-itself, which is the true nature of the physical system [13]. Lee observes that in engineering, models and systems are often conflated [14] and engineers often refer to a set of equations as *the system*. Properties of the equations are believed to be properties of the system they model. However, any definitive statement about the stability, determinism, timeliness and reliability of a system, is actually a definitive statement about a model of the system. Lee called this the "Kopetz-principle" after the Austrian computer scientist Hermann Kopetz [15]. Jantsch defines a model as a *simplification* of another entity, either a physical thing (system) or another model [16]. The purpose of a model is to represent another entity without unnecessary information.

In his book "Plato and the nerd" [14], Lee observes that there is a fundamental dichotomy of models as well. He introduced the distinction between scientific models and engineering models, both of which abstract away from physical systems. In scientific modeling, we start with a physical system and try to construct a model that captures some properties of the physical system. Through experiments, the model is evaluated in terms of how well it can predict the behavior of the physical system. An engineering model, on the other hand, is a blueprint for a physical system that does not yet exist. Engineering models are used to design and reason about physical systems before they exist. Through experiment, the constructed physical system is evaluated against the model in terms of how well its behavior matches the behavior designated by the model. We can view computer programs, as engineering models that abstractly specify the behavior of an

aggregation of electrons, semiconductors and copper wires.

Determinism and predictability are two central concepts in this thesis. To better understand them we will define a simple model of computation inspired by the Tagged Signal model from Lee and Sangiovanni-Vincentelli [17]. We define the set of timestamps as the positive reals, i.e. $\mathbb{T} = \mathbb{R}^+$ and the set of possible data values as $\mathbb{V}$. An event $e \in \mathbb{E}$ is a value $v \in \mathbb{V}$ and a timestamp. I.e., $\mathbb{E} = (\mathbb{T} \times \mathbb{V})$. Further, $\mathbb{E}^{**}$ is the set of all finite and infinite sequences of events. We define the set of all possible computations as $\mathbb{K} : \mathbb{E}^{**} \rightarrow 2^{\mathbb{E}^{**}}$. Note that the codomain of this function is the *powerset* of all finite and infinite event sequences. This enables us to model non-deterministic computations where one input sequence leads to one out of many possible output sequences. Finally, a computation k is an element of $\mathbb{K}$. Informally, a computation is a "process" that interacts with its environment through events that occur at well-defined points in time. The inclusion of time in the definition is unusual but necessary as the timing of the interactions between the computer and the physical system is essential for the correctness of the resulting CPS.

A computation k is deterministic if it defines exactly one output sequence for each input sequence, i.e. $k : \mathbb{E}^{**} \rightarrow \mathbb{E}^{**}$. Note that this is only a subset of all possible computations. If we restrict ourselves to looking at only the timestamps of the output events, we can define a "time-deterministic" computation k_t as one that defines exactly one sequence of output timestamps for each sequence of input events, $k_t : \mathbb{E}^{**} \rightarrow \mathbb{T}^{**}$. Likewise, if we restrict ourselves to looking at the values associated with the output events we can define a "data-deterministic" computation k_d as one that defines exactly one sequence of output values for each input event sequence. $k_d : \mathbb{E}^{**} \rightarrow \mathbb{V}^{**}$.

While determinism is a binary property and fairly straightforward to define precisely, predictability is a more elusive property. Informally, we can say that predictability is connected to two aspects of a computation k. (1) The variability in the set of possible output event sequences, given an input event sequence. (2) The analyzability of k and the cost of computing various properties of the variability. Most literature focuses on the time-predictability of computations and in particular the worst-case execution time (WCET), best-case execution time (BCET) and average-case execution time (ACET). We note that our definition based on events is more general and can also express these properties.

Grund et. al. define the time-predictability τ of a computation as the variability of its execution time, i.e. $\tau = \frac{WCET-BCET}{WCET}$ [18]. This is an example of a definition based on the variability in the set of possible output event sequences, i.e. the first aspect described above. Thiele and Wilhelm focus on analyzability and define time-predictability as the difference between the estimated WCET of k and the actual WCET [19]. This implies that it might not be straightforward to analyze k and that we have to estimate the timing associated with the output event sequence. This is connected to the second aspect described above. Kirner and Putschner define time-predictability as a weighted sum of variability, as defined

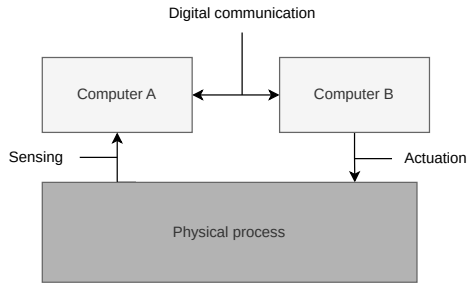


Figure 2.1: A simple model of a cyber-physical system

by Grund et. al., analyzability, as defined by Wilhelm and Thiele, and the computational complexity of performing the analysis [20].

In their recent work on the CAL theorem, Lee et. al. shows how the notion of time-predictability and data-predictably are closely linked. They use the term *consistency* to refer to the degree to which different tasks agree on the value of shared variables, and *availability* to refer to the latency experienced when interacting with the task. They show that in the face of non-negligible communication and execution times, there will have to be a trade-off between the consistency and the availability of the tasks. Consistency corresponds to our data-predictability and availability corresponds to the time-predictability. The CAL theorem shows that there will always be a compromise between time-predictability and data-predictability.

2.2 Cyber-Physical Systems

Fig. 2.1 shows a simple model of a CPS. Embedded computers monitor a physical system through sensors and communicate and coordinate to control the physical system through actuators [15]. Computation affects the physical systems, and vice versa. A vast class of systems can be thought of as CPS including biomedical and healthcare systems, avionics, automotive systems, smart power grids, HVAC, water management and robotics [15, 21].

CPS consisting of multiple sensors will typically fuse the measurements in a state estimation algorithm. This is commonly referred to as sensor fusion. Sensor synchronization refers to the process of assigning coherent timestamps to measurements from the different sensors. Sensor synchronization is an often overlooked problem but can introduce large errors in the accuracy of the sensor fusion algorithm [22].

Multi-object tracking (MOT) is a particular sensor fusion problem that we address in this thesis. In MOT, the goal is to estimate the position and velocity of multiple objects based on sensor measurements, often from multiple sources. Usually, Bayesian filtering is used to predict “a priori” state estimates for the objects

and then merge them with sensor measurements into "a posteriori" state estimates [23]. MOT is an enabling technology for autonomous vehicles [24] but is computationally very expensive [25]. This thesis includes work on accelerating the bottleneck of MOT algorithms on heterogeneous platforms.

The design of CPS is made challenging by the fundamental difference in which our most common models of the physical systems, and our most common models of computation and communication, relate to *time*. The physical systems are often modeled as ordinary differential equations where the behavior is a function of time. Computations, on the other hand, are modeled in languages like C as a sequence of discrete instructions. Time is not a part of the model underlying C, only the order of instruction. When we compose a time-dependent model of a physical system with an untimed model of computation, the result is a non-deterministic model of a CPS [26].

The Origin of Timing Requirements

To explore the origin of timing requirements to the computation. Consider the following simple control system modeled with a linear state space model,

$$\begin{aligned} \dot{x}_1(t) &= x_2(t) \\ \dot{x}_2(t) &= -\frac{b+K}{J}x_2(t) + u(t). \end{aligned} \tag{2.1}$$

This is a model of a DC motor where x_1 is the angular position of the motor shaft, $x_2(t)$ is the angular velocity of the motor shaft, b and K are motor-specific constants, J is the moment of inertia of the rotor and $u(t)$ is the input voltage applied to the motor's armature.

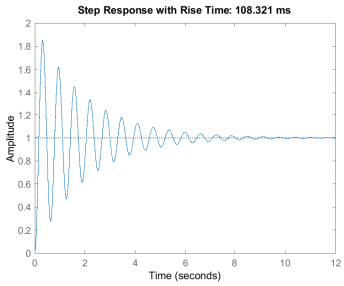
Now, a simple proportional-integral-derivative (PID) controller can be designed to control the motor towards a reference angular velocity $r(t)$:

$$u(t) = -K_p e(t) - K_i \int_0^t e(\tau)d\tau - K_d \dot{e}(t), \tag{2.2}$$

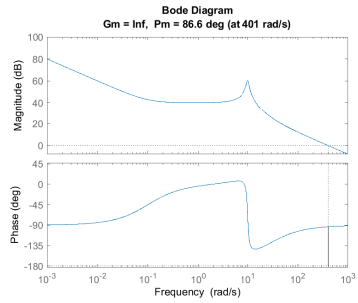
where $e(t) = r(t) - x_2(t)$, is the error between the reference and the actual angular velocity. K_p is the proportional constant, K_i is the integral constant and K_d is the derivative constant. This is a model of a CPS. The physical part is described by Eq. 2.1 and the cyber part is described by Eq. 2.2. In Eq. 2.2 we see how, at every time instant, the cyber part (i.e. the computation) consists of: (1) Sensing the current angular velocity $x_2(t)$; (2) computing the error $e(t)$; (3) computing the rate of change of the error $\dot{e(t)}$ and the accumulated error $\int_0^t e(\tau)d\tau$; and finally (4), computing the control input to apply to the actuator $u(t)$. In our model, this computation is happening continuously and instantaneously. In an actual implementation, we would have a finite, and possibly time-varying, sampling rate and a non-zero delay between sensing x_2 and delivering the input voltage u to the motor.

There are four important time-related questions to answer with respect to this computation:

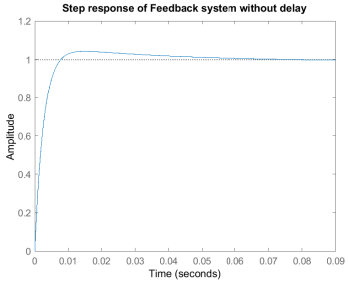
1. At what sampling rate must we run the computation?
2. What is the maximum acceptable delay from sensing to actuation?
3. How much variance in the sampling rate (e.g. input jitter) is acceptable?
4. How much variance in the actuation instant (e.g. output jitter) is acceptable?



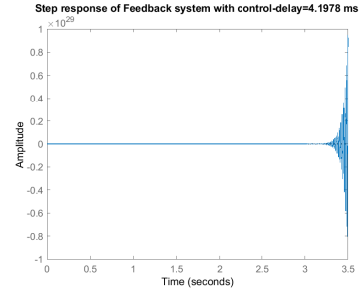
(a) The step response of the DC motor



(b) Bode plot of open-loop system of DC motor and PD controller



(c) Step response of DC motor with PD controller with zero delay



(d) Step response of DC motor with PD controller with 4.2 milliseconds delay

Figure 2.2: Analysis of the system described with Eq. 2.1-2.2. $K = 0.1$, $J = 0.001$, $b = 0.001$,$K_p = 100$, $K_d = 4$ and $K_i = 10$.

These questions can mostly be answered with classic control theory by deriving stability margins for the control system. (1) A rule of thumb is that the sampling rate should be less than one-tenth of the rise time of the step response of the open-loop system [27]. Fig. 2.2a shows the step response of the system, a sampling period of 10 milliseconds will do in this case. (2) The maximum accepted delay from sensing to actuation is called the phase margin of the system and

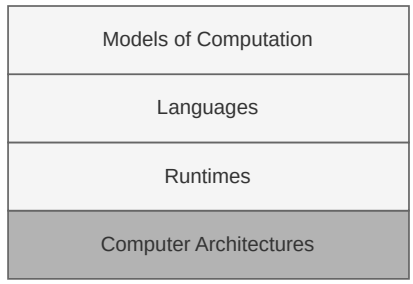


Figure 2.3: Layers of abstraction in computing relevant for this thesis

it can be estimated using a Bode plot. Fig. 2.2b shows the Bode plot which reveals a phase margin of 86.6 degrees with a cross-over frequency of 401 $\frac{rad}{s}$, yielding a delay margin of 3.7 milliseconds. Fig. 2.2c-2.2d shows the step response for the closed-loop system with 0 and 4.2 milliseconds delay from sensing to actuation, respectively. Likewise margins for (3) input jitter and (4) output jitter can be derived analytically [28–30].

An often overlooked problem, and a topic of this thesis, is the effect of input jitter in sensor fusion systems, which is connected to the sensor synchronization problem discussed in the previous section. If the controller uses samples from different sensors to estimate the current state of the system. Then input jitter will translate directly to measurement errors. The magnitude of this error depends on the rate of change of the system when the measurement is being made.

2.3 Layers of Abstraction in Computing

In this section, we will briefly discuss the different layers of computing abstractions that are touched by this thesis, shown in Fig 2.3. The proceeding sections go into more detail on a selection of these layers.

The higher the layer is in the stack, the less concrete and more abstract it is. At the bottom, we find “Computer Architectures”. This layer is concerned with the architecture of the computing hardware itself. It is of a different color because it is concerned with the actual physical topology of the computing device, i.e. the hardware, rather than the contents of memories and configuration registers, i.e. the software, firmware or bitstream. We will look closer at different computer architectures in Section 2.7.

The next layer is “Runtimes”, which refers to low-level software or bitstream configuration running on the computing devices. A runtime allows programs, often written in higher-level languages, to interact with a more abstract or generic model of the computing device. Examples are the Java Virtual Machine (JVM) and the Lingua Franca runtime which will be introduced later. According to this definition, an operating system like Linux, or a real-time operating system (RTOS)

like Zephyr or FreeRTOS, can be thought of as a runtime.

At the “Languages” level we find the programming languages. Famous examples are Python depending on a Python runtime like CPython, Java which depends on the Java Virtual Machine and Lingua Franca which depends on a Lingua Frana runtime implementation in the relevant target language [31]. We also include compiled languages like C, C++, Chisel and VHDL here, even though they do not depend on a runtime.

Finally, we have the “Model of Computation” level. A particular language will have an underlying model of computation. We represent it as more abstract than languages because it is further removed from the actual physical computing devices. In the next section, we will delve into this concept.

2.4 Models of Computation

In this section, we will take a closer look at the concept of a model of computation (MoC) from Fig. 2.3. We will look at different MoCs and their applicability to model high-performance and predictable computing required by modern CPS.

The science of computing was arguably started in Cambridge in 1936 when Alan Turing published his paper "On Computable Numbers with an Application to the Entscheidgedungsproblem" [32]. He introduced an imaginary computing device, i.e. an MoC, that reads and writes symbols onto squares on an unbounded tape. The machine contains a very simple control unit that consists of a finite set of configurations. Each configuration describes how to respond to reading different symbols of the current square. The response to reading a symbol can only include (1) Overwriting the symbol; (2) moving the tape left or right by one position and (3) changing the current configuration. Turing’s model of computation forms the bedrock of computing to this day. But it lacks a notion of *concurrency*. The Turing Machine can only do “one thing at the time”, this is contrasted with CPS where many things are happening at the same time. There might be multiple sensors, human interfaces, actuators, and emergency stop buttons. Moreover, there is no notion of *time* in Turing’s model. The operations performed by the machine are ordered, but whether they execute in milliseconds or hours does not make any difference to the computation they perform. This is, again, at odds with CPS where the timing of the interaction between computers and the physical system is a correctness criterion. In the simple MoC we defined in Section 2.1 time was included for this very purpose. Finally, Turing also realized that whilst being deterministic, his machines were unpredictable in a fundamental sense. For instance, Turing later proved that the question of whether an arbitrary Turing Machine ever reaches its final, halting state, is not computable. This has serious implications for the trade-off between time-predictability and data-predictability identified in the CAL theorem. It means that there exists no algorithm that can compute the execution time bound of all programs. This means that we cannot always provide guarantees for the time-predictability and data-predictability of a computation.

According to Jantsch, a MoC is a model of a computing device. He further observes that these models are often dynamical, time-invariant, non-linear and discrete in states and inputs [16]. A programming language will have one or several underlying MoCs. As such, a program can be viewed as a model and the MoC as the modeling framework.

Lee's distinction between engineering models and scientific models is also applicable to MoCs. However, MoCs just provide the modeling framework, it's their *use* which can be divided into engineering and scientific. Formal verification of algorithms is an example of the scientific usage of MoCs. An algorithm is modeled in a language with a well-defined underlying model of computation and then model-checking is performed. An example is the FDR4 refinement-checker [33] that accepts algorithm specifications written in the language CPSM which is based on Hoare's Communicating Sequential Processes (CSP). Another example is Timed Rebeca [34] which is a modeling and specification language based on the Actor model of computation [35].

In this thesis, we are mainly interested in the engineering usage of MoCs. Here, the *intended* behavior of the computing device is modeled in a programming language with a semantic provided by a MoC. For instance, the semantics of VHDL is based on the discrete-event MoC. As such, a VHDL module is a model of a physical computing device performing a particular discrete-event computation. It is the task of an EDA toolchain to turn such a model into a configuration of FPGA resources, or a gate-level representation of a standard cell library, that can reliably deliver the behavior specified by the VHDL module. In the same sense, a C program is also a model of a computing device performing a sequential computation [36]. The job of a compiler is to translate the program into a sequence of machine instructions prescribing the very same behavior.

Researchers have proposed numerous MoCs and languages for programming CPS. In the following, we will take a look at a few of the most impactful schools of thought.

The "CSP school" is centered around British universities that advocate using process calculi like Communicating Sequential Processes (CSP) [37]. In CSP a computation is modeled as a composition of sequential processes which do not share data and only communicate through message-passing with rendezvous. This is an elegant model for concurrency, but it does not address timing. Moreover, a network of CSP processes is in general non-deterministic, in that the same sequence of input messages might lead to multiple different sequences of output messages. Notable languages based on these models are Occam [38], Ada [39], Go [40], xC [41] and Handel-C [42]. In Handel-C, programs are compiled into synchronous digital circuits usually targeted at FPGAs.

The "Synchronous school" is predominantly French and advocates the use of the Synchronous-Reactive MoC [43]. This is inspired by synchronous circuit design

where signals are clocked through the circuits by a periodic global clock signal. This filters out "time noise" which is introduced by unpredictable delays through circuit components [44]. The circuits are modeled as performing computation and communication instantaneously and the inputs to a component are synchronous with its output. This simplification has enabled the design of extremely reliable circuits of incredible complexity. The so-called synchronous languages let the computation be divided into a discrete sequence of *reactions*. At each reaction, input signals propagate through a set of computations in a well-defined order. This yields a deterministic concurrent execution as long as the new inputs do not arrive until the previous ones are completely handled. Examples of synchronous languages are languages include Esterel [45], Lustre [46], Signal [47], ForeC [48] and SCCharts [49].

There is also the "Dataflow school" springing out of the work of Dennis [50], Kahn [51] and Lee [52]. Kahn Process Networks (KPN) [53] resemble CSP, except that the communication between the processes is asynchronous and buffered in first-in-first-out (FIFO) queues. Synchronous Dataflow (SDF) [52] introduces the notion of instantaneous reactions. Instead of a set of concurrently executing processes communicating asynchronously, in SDF, the processes are triggered in discrete reactions when a sufficient number of tokens are present at its inputs. This can simplify the scheduling and analysis of the process networks. Dataflow languages include CAL [54], Compaan/Laura [55] and LabVIEW [56]

The "Asynchronous school" can be traced back to Hewitt [57] and Agha [35] and their work on the Actor models of computation. The Actor MoC is a general model for concurrent computation. A computation is expressed as a set of actors which communicate asynchronously through message-passing. In response to receiving a message, an actor can modify its private state, create new actors and send more messages. Languages such as Erlang [58] and Scheme [59] as well as frameworks such as Akka [60] and Qp [61] are based on the Actor model.

The task models for real-time systems also deserve mention. Although not strictly engineering models, they are commonly used to model concurrent real-time software executing under priority-based scheduling by an operating system. This is, arguably, still the most common way of developing real-time software for CPS. The unit of computation in real-time task models is the *job* [3]. A job is characterized by an arrival time, a worst-case execution time (WCET) and a deadline. A job is a sequential process, and concurrency can only be expressed by introducing multiple jobs. Jobs are created by a finite set of independent *tasks* which each generate a possibly infinite sequence of jobs. One of the most known task models is due to Liu and Layland [62]. Here each task τ_i is characterized by a minimum period T_i and a WCET C_i. The deadline D_i is implicitly equal to the period of the task. The Three-Parameter Sporadic Tasks Model [3] extends Liu and Layland's model by allowing a relative deadline that is greater or smaller than the period.

There are several limiting assumptions made by these models. First, the tasks are independent, there is no way of modeling that two tasks are synchronized, or that one task causes another. Second, there is no way to model communication or synchronization between jobs of different tasks, they are assumed to execute independently. Lastly, there is no way of modeling parallelism within a single job. These assumptions are quite limiting in the context of CPS as different parts of the computation certainly are dependent, moreover, to satisfy the increasing need for computing power, multiprocessor architectures are increasingly used in CPS and the need for intra-task parallelism is apparent.

The sporadic DAG model [63] is a more recent task model addressing the need to expose intra-task parallelism. Here each job created by a task is a directed acyclic graph (DAG) $G = (V, E)$ where each vertex $v \in V$ is a sequential "sub-job" and each edge $e \in E$ represents a precedence constraint. This enables executing a single job across multiple cores.

A given task-set is *feasible* upon a platform if there exists a schedule where all timing requirements are met for all collections of jobs that can be legally generated by the task-set. A task-set and a scheduling algorithm are said to be schedulable if the scheduling algorithm generates a schedule that meets all the timing requirements for all legal collections of jobs. There is a rich literature on scheduling algorithms which is beyond the scope of this thesis. Suffice it to say, that the most common strategy is preemptive, priority-based scheduling where the scheduling algorithm decides what priorities to assign to each job.

The logical execution time (LET) model from Henzinger and Kirsch [64, 65] is a related approach that is gaining popularity in the automotive sector and was recently included in the AUTOSAR standard [66]. While synchronous languages assume zero execution time and the real-time task models assume a bounded execution time, in LET, an exact logical execution time is assumed. This enables a deterministic composition of tasks that share data. In the real-time task models, it is assumed that all tasks are independent and do not share resources; this is seldom the case. In practice, tasks that share resources must use locks to prevent data races, and this affects the WCET estimates and can lead to priority inversion. Executions modeled in LET can be viewed as engineering models that accurately specify how the underlying computing device should behave, also in the time domain. It is left to the compiler and run-time to synthesize a program that can deliver this behavior, or, reject specifications that are not realizable.

Recently, a new class of MoCs and languages have emerged that build upon ideas from the discrete-event systems, synchronous languages, dataflow, real-time task models and LET [67]. These models embrace the concept of logical time and are built around scheduled events managed by an event queue [68]. Events may represent triggerings of sequential computations, or a variable update. An execution is now a sequence of time-steps each associated with a set of events that occur on the time-step. Each time-step is handled like a synchronous program, which yields a deterministic composition of simultaneous events. These models are referred to

as sparse synchronous models. They are sparse in both the temporal and the spatial dimensions. Temporal sparsity refers to the fact that the time-steps might be sparse and bursty, rather than regular and periodic. Spatial sparsity refers to that, typically, only parts of the computations are activated at any timestep.

There are two main languages based on the sparse synchronous model. Sslang, from Hui and Edwards [69, 70], is an imperative-functional language with strong and static typing. Lingua Franca [71], from Lohstroh and Lee, is a declarative polyglot coordination language. Lingua Franca and its underlying Reactor MoC, is a central theme in this thesis.

2.5 Lingua Franca

In this section, we will take a closer look at a particular language from the "Languages"-level in Fig. 2.3. Lingua Franca is a declarative polyglot coordination language based on the reactor model formalized in Marten Lohstrohs Ph.D. thesis [72]. It is referred to as a *coordination* language, rather than a programming language because it addresses the concerns of communication and coordination, not the computation (as defined by Turing) itself. Lingua Franca is a language for specifying a network of concurrent components, called reactors. The computation performed within each reactor is expressed in a target language. It is *polyglot* as it supports multiple target languages. Finally, it is *declarative*, as opposed to imperative, in that a program describes the topology of a network of reactors rather than an imperative sequence of operations.

To informally introduce the reactor model we will use the simple example Lingua Franca program and its auto-synthesized diagram in Fig G.1. The program consists of a network of five reactors related by connections and containment. The main reactor, called `SenseToAct` is the top-level reactor and contains all the other reactors, it is defined on lines 29-36. It contains two instances of the `Sensor` reactor, which is defined on lines 2-9, one instance of the `Fusion` reactor which is defined on lines 10-20, and one instance of the `Actuator` reactor defined on lines 21-28. Reactors communicate through typed ports, in this example, all ports have type `int`. The fundamental unit of computation is the *reaction*. They are represented as dark grey chevrons and are event handlers. They are triggered whenever there are events present at any of their sources. Events consist of a value and a tag (i.e. a timestamp). Reaction invocations are, as in the synchronous languages, logically instantaneous, meaning that events produced by a reaction will have the same tag as the event that triggered the reaction. Events originate from triggers, which include fixed period *timers* (shown as clocks), input ports (shown as small triangles on the borders of the reactors) and *actions* (shown as white triangles). Logical actions are denoted with an "L" and allow reactions to schedule future events. Physical actions, denoted with a "P", allow asynchronous contexts, like an interrupt service routine, to schedule events into the system. The actual computation taking place in a reaction is specified in the reaction body, which is expressed in a target language between two delimiter brackets `{= ... =}`.

```
target C;
reactor Sensor {
  output out: int
  timer t(0, 100 msec)
  state cnt:int=0
  reaction(t) -> out {=
    lf_set(out, self->cnt++);
  =}
}
reactor Fusion {
  input[2] in: int
  output out: int
  logical action a:int
  reaction(in) -> a {=
    lf_schedule_int(a, MSEC(10), in[0]->value + in[1]->value);
  =}
  reaction(a) -> out {=
    lf_set(out, a->value);
  =}
}
reactor Actuator {
  input in:int
  reaction(in) {=
    lf_print("Driving actuator with %u", in->value);
   =} deadline(10 msec) {=
    lf_print("Handling deadline violation!");
   =}
}
main reactor {
  sensor1 = new Sensor()
  sensor2 = new Sensor()
  fusion = new Fusion()
  actuator = new Actuator()
  sensor1.out, sensor2.out -> fusion.in
  fusion.out -> actuator.in
}
```

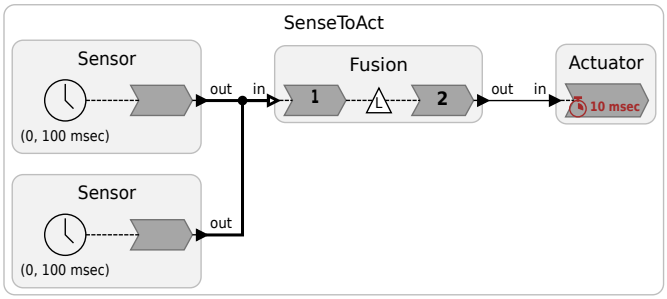


Figure 2.4: An example Lingua Franca program using the C target

Reactions are only triggered when all observable events with the same tag have been produced. Reactions inside the same reactor can share state variables and are executed in a predefined order should they be triggered at the same tag. The reactor model is deterministic under the assumption that the reaction bodies are deterministic. The execution of a Lingua Franca program is deterministic with respect to a *logical* timeline defined by the tags of the events. The program will also exist on a *physical* timeline. The logical and physical timeline interacts at four points. (1) At startup, physical and logical time is synchronized. (2) Logical time can never exceed physical time. (3) Physical actions are scheduled relative to the current physical time; this is unlike logical actions, which are scheduled relative to the current logical time. (4) A *deadline* can be associated with a reaction. A deadline specifies the maximum difference between logical and physical time, i.e., the *lag*, allowed at the invocation of the reaction. A deadline-miss handler must also be specified by the user.

Lingua Franca and the reactor MoC will be central concepts in the rest of this thesis.

2.6 Timed C

Timed C is a real-time extension to the C programming language, proposed by Broman and Natarajan [73] in 2018. Like Lingua Franca, it builds on the notion of logical time. However, it uses POSIX threads and priority-based scheduling rather than an event queue. This makes it a lightweight addition to C because it does not need any runtime infrastructure. Rather, it runs on top of POSIX and FreeRTOS. The downside is that it inherits the non-determinism which is endemic to priority-based thread scheduling.

In Timed C, the programmer can specify the intended timing of their program through *timing points*. Timing points are either soft, firm or hard which represents the severity of violating them. Logical time only progresses at the timing points and computation between timing points is logically instantaneous. When running on a particular computer, the program will also exist on a physical timeline. Timing violations occur when the program arrives at a timing point and the physical time exceeds the logical time associated with the timing point.

Concurrency is exposed through a *task* construct which maps directly to the underlying threading library. Tasks communicate through *fifochannels* and *lvchannels*. The former is a first-in-first-out queue with buffered non-blocking writes and blocking reads, the latter is just a shared variable without synchronization. Tasks can be assigned different scheduling policies and priorities which are forwarded to the underlying thread library.

A topic of this thesis is the extension of Timed C to distributed systems.

2.7 Computer Architectures

In this section, we will look closer at the "Computer Architecture" layer from Fig. 2.3. Advances in computer architecture during the last 40 years have had a big impact on the question of predictability for CPS. Traditionally, software for CPS ran on simple microcontrollers where the execution time of each machine instruction was known and constant. This enabled very accurate estimates of execution times for a given instruction trace. From around 1985 to 2003 there was a remarkable exponential improvement in the performance of microprocessors averaging at approximately 52 % per year [74]. This was driven by two main factors. First, continued advances in the manufacturing of transistors reduced their physical size. Gordon Moore famously observed that the number of transistors per integrated circuit kept doubling every year and projected that it would continue to do so, at least for another decade. This became known as Moore's law [75] (he later revised it to doubling every two years). Shrinking the length and width of the transistors has several effects. First, it reduces the capacitance of the transistor and thus the voltage required to operate it. Moreover, the circuit delay is reduced as the electrons must travel shorter distances. This means that the circuit can be clocked at a higher frequency. The overall effect is that doubling the transistor density can be done while increasing the operating frequency by around 40 % without increasing the power consumption. This relationship is known as Dennard scaling [76].

The second factor driving the performance improvement was advances in computer architecture. In particular, cache hierarchies, deep pipelines and speculative execution greatly improved the instructions-per-cycle (IPC) which is the gold standard for computer performance [2]. These improvements sacrificed predictability in execution time for increased average-case performance. All this was done whilst keeping the interface to the CPU, i.e. the instruction set architecture (ISA), unchanged. This was possible since the ISA does not put any restrictions on execution times, only on observable order of execution. In the context of CPS, this development is problematic. Although enabling more computationally intensive algorithms, the increasing non-determinism in the execution time makes worst-case execution time analysis and scheduling of real-time software increasingly difficult.

Dennard scaling eventually came to an end as the static leakage current of the transistors does not scale with decreasing voltage and size. This leakage current ended up being the major contributor to power consumption. This has put an end to the exponential improvement in the single-threaded performance of CPUs. There have been two types of responses in the industry to this, multicore systems and heterogeneous systems. AMD, followed by Intel, moved to multicore designs in the early 2000s. There are several challenges with using multicore systems for CPS which we address in this thesis. (1) There is the potential for considerable interference between the different cores, particularly if they share memory. (2) The

task of expressing parallelism can be challenging. The standard approach is to use classical sequential languages, like C or C++, together with a threading library. This has serious drawbacks [77]. Finally, (3) the process of scheduling the parallelized work on the different cores is difficult. The results from classical real-time scheduling assume single-core architectures.

Heterogeneous systems refer to a broad set of architectures that combine CPUs and domain-specific accelerators like Graphical Processor Units (GPUs), Neural Processor Units (NPUs) and Field Programmable Gate Arrays (FPGAs). There is not a clear distinction between multicore systems and heterogeneous systems as most multicores include some domain-specific architecture and most heterogeneous systems include multiple CPU cores. Heterogeneous systems enable performance improvement by specializing in specific workloads. Oftentimes, the domain-specific accelerators provide very time-predictable execution which is a promising development for CPS [78].

We will, in the following two sections, look closer at two types of computer architectures that are promising for CPS and central to this thesis.

2.7.1 PRET machines

The decreasing time-predictability of modern CPUs triggered research into alternative computer architectures for CPS. Notable examples are the embedded real-time Java processor JOP [79], the VLIW architecture Patmos [80] and its multicore version T-CREST [81], all due to Schoeberl. At UC Berkeley the so-called Precision Timed (PRET) machines were proposed at around the same time [82]. The PRET machines were organized around fine-grained multithreading where a set of hardware threads are interleaved into the pipeline. This keeps the IPC per thread sufficiently low such that speculation is not needed while giving a high overall IPC when considering all threads. Instead of a cache hierarchy, single-cycle access scratchpad memories are used. Overall, this creates computers with constant and predictable execution time for any instruction trace. There are three generations of PRET machines. The orignal PRET machine [83], PTARM [84] and finally FlexPRET [85]. FlexPRET is a five-stage in-order processor implementing the base 32-bit RISC-V ISA (RV32I). FlexPRET has no branch predictor and uses fine-grained multithreading with a barrel register file to avoid pipeline bubbles. Compared to earlier-generation PRET machines, it has a more flexible thread scheduler which enables round-robin scheduling of threads configured as *hard-real time threads* (HRTTs) and opportunistic scheduling of threads configured as *soft real-time thread* (SRTTs). One of the key features of the FlexPRET is the ability to handle interrupts without introducing any timing jitter in the HRTTs. This can enable the integration of networking stacks with real-time embedded software without hurting the time-predictability.

The FlexPRET introduces three timing instructions for controlling the timing of each thread: (1) `delay_until` halts execution until some absolute point in the

future; (2) `wait_until` halts until an interrupt, with the possibility of a timeout; and (3) `interrupt_expire` schedules an interrupt to occur at an absolute point in time.

The company XMOS develops commercial multi-core PRET machines. Their XS1, XS2 and XS3 is based on fine-grained multithreading and scratchpad memories, like the Berkeley PRET machines.

In order to handle the workloads of modern CPS, PRET machines will also be forced to move to multicore architecture or heterogeneous architectures. Little research exists in this space. Notable exceptions are T-CREST [81], MultiPRET [86] and MERASA [87]. One of the topics of this thesis is the development of multi-core PRET machines.

2.7.2 System-on-chip FPGAs

Field-Programmable Gate Arrays (FPGAs) are computing devices containing arrays of lookup tables, fixed-function units, signal routing, clock networks and memories. FPGAs can implement any digital circuit by configuring the lookup tables and the signal routing. The configuration of an FPGA can easily be changed, even dynamically at runtime. FPGAs are commonly used to simulate digital circuits like CPU pipelines [88].

System-on-chip (SoC) FPGAs refer to devices that integrate CPUs and FPGAs on the same integrated circuit. These belong to the class of heterogeneous architectures introduced in Section 2.7. SoC FPGAs offer a unique combination of versatility (due to general-purpose CPUs), flexible I/O (due to the large number of I/O pins connected to the reconfigurable FPGA) and performance (due to the opportunity to accelerate complex workloads on the FPGA). Accelerators are custom circuits, often optimized for a particular workload, which can be synthesized to the FPGA. SoC FPGAs typically offer direct memory access from the FPGA, meaning that the accelerators can read and write data directly from the main memory. It is also common to have the FPGA be connected to the peripheral bus of the CPUs. This way, accelerators can be implemented as slave devices connected to this bus, offering control and status registers for the CPU to read and write from.

The process of designing computations spanning both the CPUs and the reconfigurable FPGA is called hardware-software codesign and remains a challenge. This is largely due to the fundamental difference in the compute substrate of a CPU and an FPGA and, therefore, in the MoC commonly used to design computations. The CPU is typically programmed based on a sequential MoC in a language like C or C++. The FPGA is commonly programmed based on the synchronous MoC in languages like VHDL, Verilog and more recently, Chisel. Several proposals exist for specifying hardware and software computations at the same level of abstraction. High-level synthesis (HLS) is perhaps the most common, where both hardware and software are specified as sequential imperative programs in languages like C, C++ using frameworks like OpenCL, SyCL and Vitis. The advantage of using C-like languages for HLS is that it lowers the barrier to entry

for software developers familiar with the sequential MoC. However, the C-like languages have no concurrency or timing primitives, both of which are critical aspects of digital circuits [89]. In Compaan/Laura both hardware and software are specified as Kahn Process Networks in MATLAB. In ReconOS the POSIX threaded programming model is extended to VHDL enabling inter-thread communication between software threads and hardware threads. In ReconROS the software and hardware are implemented on top of the Robot Operating System (ROS2) which is based on publish-subscribe. The Bluespec Codesign Language (BCL) supports expressing computations based on the guarded atomic actions model. Programs written in BCL can be compiled into both hardware and software. ForSyDe [90] offers a different approach. Instead of finding a single MoC that can be realized both on an FPGA and on a CPU, ForSyDe allows combining processes of multiple MoCs into a single heterogeneous process network. ForSyDe can synthesize processes using the synchronous MoC into VHDL descriptions that can target an FPGA.

An important part of this thesis is the development of a new abstraction-level for doing hardware-software codesign targeting SoC FPGAs.

Chapter 3

Research Summary

3.1 Overview

The central theme of this thesis is computing for CPS. In Fig. 3.1 the simple CPS model from Fig. 2.1 is populated with the papers published as part of the thesis. The papers are colored according to the Research Question that they address. The blue paper is on Research Question 1, the green papers on Research Question 2 and the yellow papers on Research Question 3.

Clearly, most of the work is related to the computers and the computations they are performing. There are two exceptions. Paper A addresses the connection from the physical system to the computers. This connection represents the sensors of the CPS. Paper A includes an analysis of the effect of sensor timestamping jitter on the overall performance of the CPS. Paper C addresses the connection between the computers, i.e. the networking aspect of a CPS, and in particular the use of deterministic real-time networks in CPS.

Fig. 3.2 presents a different perspective. Here we show how the different papers and research questions are related to the abstraction layers of the computing stack first introduced in Fig. 2.3. The papers in green on the left side are related to CPU-based computations and the yellow papers on the right side are related

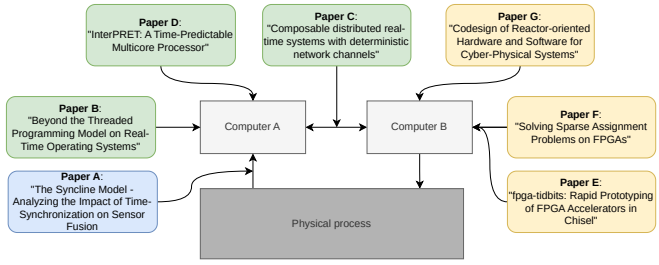


Figure 3.1: How the different papers are connected to the components of a cyber-physical system introduced in Fig 2.1.

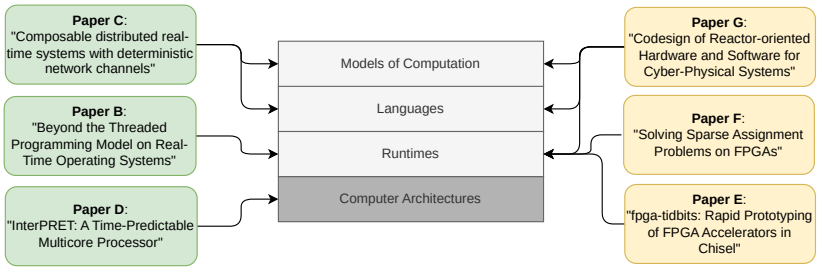


Figure 3.2: How the different papers are connected to the different layers in the computing stack introduced in Fig. 2.3.

to SoC FPGA-based computations. We are addressing topics on several levels of abstraction for both compute substrates.

In the following, we will introduce the seven articles of this thesis organized with respect to the research questions introduced in Section 1.2.

3.2 RQ1: What is the Importance of Predictable Timestamping in CPS?

With RQ1, we seek to investigate the need for predictable computing in CPS. As discussed in Section 2.2, the behavior of a CPS depends on the timing of the interaction between the cyber part and the physical part. We showed that the stability of a feedback-controlled linear system is directly connected to the sampling frequency and sensor-to-actuator delays. Also, the effects of input and output jitter are well-studied. However, in emerging multi-sensor fusion systems, input jitter takes on a new meaning. These systems are often built with sensors that perform measurements asynchronously. Non-deterministic timing, i.e. jitter, in the control software, leads to assigning incorrect timestamps to measurements. This inaccuracy can degrade the performance of the system.

We build upon previous work by Albrektsen on hardware-based time synchronization of sensors [91]. It is our goal to quantify the benefits of using Albrektsen's Syncboard. The research question is also related to the notion of predictability in computing discussed in Section 2.1. Due to modern CPUs getting less and less predictable, tasks like accurate timestamping are increasingly difficult to perform in software.

Contributions

The main contributions to RQ1 are summarized below.

- The Syncline Model, a simple and intuitive mathematical model for exploring the relationship between timestamping accuracy and CPS performance.

- A survey of sensor synchronization protocols and timestamping methods.

In Paper A "The Syncline Model - Analyzing the Impact of Time Synchronization in Sensor Fusion", we present an intuitive performance model for the effects of time synchronization on sensor fusion algorithms. It is based on the key insight that the performance, i.e. accuracy, of a sensor fusion algorithm will be limited by one out of two factors. (1) The quality of the individual sensor measurements or (2) the quality of the time synchronization between different sensors. We review the most common protocols for synchronization between computers and sensors like 1PPS [92], NTP [93], PTP [94], TOV [91] and triggering. We also review a set of commonly used sensors and robot platforms.

We draw inspiration from the Roofline model [95] and visualize the interaction between synchronization errors and sensor quality for increasing timestamping inaccuracies for different robot platforms. A combination of sensors and robot platform yields a "syncline" that relates sensor fusion estimation accuracy and timestamping accuracy. If the timestamping accuracy is too low, you are in the "sync-bound" region where the performance is mainly limited by your timestamping accuracy. We define the "critical timestamping accuracy" as the timestamping accuracy where the errors introduced by it are equal to the errors introduced by inaccuracies of the sensors themselves. For robot platforms with slow dynamics or low-quality sensors, software timestamping might be sufficient. However, for robots with fast dynamics or high-quality sensors, hardware timestamping techniques are often required. We derive synclines for many different robot platforms and sensors and compare the model against a simulation.

Paper A provides tools for *understanding* a system rather than designing it. It quantifies the effect of poor time-predictability in the computations responsible for acquiring and timestamping sensor measurements. It also points to the observation that time-predictability and data-predictability are closely related. It is shown that being wrong about the *time* of an event is comparable to being wrong about the *value* associated with the event. How wrong depends on the dynamics of the system.

3.3 RQ2: How Can We Enable Predictable Computing on General-Purpose CPUs?

While RQ1 was concerned with modeling CPS to derive timing requirements, so-called scientific modeling according to Lee [14], RQ2 is concerned with engineering models and tools for programming high-performance and predictable software systems. The work on RQ2 resulted in three papers. Returning to Fig. 3.2, Paper B addresses the "Runtimes" level by proposing reactors as an alternative to RTOSes. Paper C is at the "Languages" and "Models-of-Computation" levels, it introduces an extension to the Timed C language target distributed real-time systems. Lastly, Paper D is at the "Computer Architectures" level. It introduces a time-predictable

multicore CPU architecture for CPS.

Contributions

The main contributions to RQ2 are summarized below.

- Federated Timed C, an extension to the Timed C programming language for distributed systems.
- Extending the Time Flow Graph formalism with primitives for modeling distributed systems.
- Support for Zephyr RTOS as a platform for Lingua Franca programs.
- Evaluation of the timing precision of the Lingua Franca runtime.
- InterPRET, a time-predictable multicore architecture.
- Software-defined devices as a new direction for PRET machines.

In Paper B "Beyond the Threaded Programming Model on Real-Time Operating Systems", we contribute to the open-source Lingua Franca project. Lingua Franca is a recent and promising language for designing CPS. It has a semantic notion of time which, unlike most other software languages, enables directly expressing the timing required for stability as discussed in Section 2.2. In this paper, we address the limited support and evaluation of Lingua Franca on resource-constrained platforms like microcontrollers. We add Lingua Franca support for several hundred embedded boards through building on the recent Zephyr RTOS[1]. Unlike other RTOS, e.g FreeRTOS[2], Zephyr provides platform-independent device abstractions to the microcontroller peripherals. This enabled us to use high-resolution timer peripherals to provide timing to the LF runtime. Experiments with FreeRTOS yielded timing precision in the millisecond range, while Zephyr gave us sub-microsecond precision in I/O operations. The interrupt latency was measured to around 50 microseconds with jitter in the range of 100 nanoseconds.

To evaluate the general performance of the LF runtime we ran the ThreadMetric and the Savina benchmarks against a baseline implementation using Zephyr's thread API. We show that the LF runtime has competitive performance and in some cases even outperforms the thread implementations. Finally, we evaluated the LF runtime in terms of code and data size and found it to be competitive with alternative frameworks, such as MicroPython[3], SynchronVM [96] and Medusa [97].

We conclude that timed semantics and deterministic concurrency are achievable at low-cost and with competitive performance, also for resource-constrained platforms.

In Paper D "InterPRET: A Time-predictable Multicore Processor", we present a multicore CPU architecture with predictable timing. It combines a configurable number of FlexPRET [98] CPU cores with a S4NOC [99] network-on-chip (NoC).

[1] https://zephyrproject.org/
[2] https://www.freertos.org/
[3] https://micropython.org/

This yields an end-to-end time-predictable multicore architecture for CPS. As discussed in Section 2.7, there are two trends in computer architecture contributing to poor time-predictability in modern computers. The first is the use of speculation and out-of-order execution to improve the IPC, and the second is to scale horizontally to multiple cores sharing the same memory hierarchy. InterPRET is able to deliver high IPC without sacrificing time-predictability through hardware multithreading, it also scales horizontally without losing predictability because inter-core communication is explicit through a NoC rather than implicit through a cache coherency protocol.

We also develop C-based firmware for the InterPRET. We provide simple libraries for doing inter-core communication over the NoC. A bootloader enables re-programming the on-chip instruction memories with new programs.

We also propose a new direction for PRET machines with what we call "software-defined devices". These are software-based functions on a SoC which typically is done by dedicated hardware, e.g. digital communication interfaces like UART, SPI and USB. The time-determinism of PRET machines enables such functions to be reliably implemented in software. We provide a software-defined UART and shows how we can use it to implement a "printf-service" running on one of the cores allowing hardware threads from different cores to print out debug information.

The InterPRET is evaluated by synthesizing it to an FPGA. Here we emulated the design at 50 MHz and ran realistic benchmarks. Resource utilization was also studied. We concluded that multicore PRET machines are an interesting topic and deserve more study and work. In particular, we note that the question of which MoC should be used when programming them, remains open.

In Paper C "Composable distributed real-time systems with deterministic network channels" we present Federated Timed C (FTC) as an extension to Timed C [100]. FTC provides native support for deterministic network channels using the Time Sensitive Networking (TSN) Quality-of-Service mechanisms. Timed C was proposed by Natarajan and Broman as a way of adding temporal semantics to the C programming language through a small set of language primitives. A *timing point* is a way of letting the programmer encode the timing requirements of the applications as well as the severity (hard/firm/soft) of failing to meet the requirement. Like Lingua Franca, LET and the Synchronous languages, Timed C introduces the notion of a logical timeline into the program. Timed C also adds tasks and channels for inter-task communication.

We propose a new Timed C primitive called *netchannels* which provides inter-task communication across networks. We extend the Time Flow Graph formalism [101] providing a formal semantics for *netchannels*. This enables mechanized reasoning about the composition of multiple FTC tasks communicating through different channels.

FTC builds on Linux, off-the-shelf x86 hardware and OpenAVB from the AvNU Alliances[4]. FTC is evaluated extensively in a test setup where two FTC hosts are

[4]https://github.com/Avnu/OpenAvnu

communicating through a two-hop network subject to interfering traffic as well as interfering load on the CPUs and memory subsystem at each host. We use a Linux kernel patched with PREEMPT_RT[5] and applied several real-time optimizations to improve time-predictability. We shield a set of two cores on each host and move all movable tasks and interrupts to the remaining cores. HyperThreading and changing "cstates" is disabled. Our application is scheduled with SCHED_RR with a priority of 80 which out-ranks any standard kernel thread. LinuxPTP[6] is also scheduled with a high priority with SCHED_RR to minimize clock synchronization errors. The network traffic was managed with qdisk and mqprio giving priority to the PTP messages used for clock synchronization. The data sent over netchannels are sent using default AVB parameters and the Credit Based Shaper [102].

Our results are promising. The clock synchronization error is kept below 30 nanoseconds, even with interfering traffic. The wakeup accuracy at each host is on average 1.1 microseconds with a worst-case of 16 microseconds. We then evaluate the accuracy with which two FTC tasks, on two different hosts can coordinate actions. We show that the end-to-end delay over two hops is around 60 microseconds on average with a worst-case of 184 microseconds. The two tasks can do a coordinated wakeup within 16 microseconds accuracy, consistent with the wakeup accuracy at each host. This demonstrates the feasibility of using Linux and commercially available off-the-shelf hardware for building highly accurate distributed real-time systems.

3.4 RQ3: How Can We Enable Predictable Computing on SoC FPGAs?

SoC FPGAs are promising for CPS as they offer high performance, versatility and flexible I/O. However, as discussed in Section 2.7.2, the fundamental difference in the computing substrate offered by the CPUs and the FPGA makes them challenging to program. The research performed on RQ3 is mainly related to the development of new abstractions for programming FPGA accelerators. Returning to Fig. 3.2, Paper E deals with the "Runtimes"-level, it introduces a framework that enables rapid prototyping of FPGA accelerators and generation of software drivers for interacting with the accelerators. Paper F is also shown at the "Runtimes"-level. It presents a particular FPGA accelerator implemented using the framework from Paper E. Lastly, Paper G deals with the "Runtimes", the "Languages" and the "Models of Computation" levels. It is the most comprehensive paper included in this thesis. It proposes a new, predictability-first approach to programming SoC FPGA.

[5] https://wiki.linuxfoundation.org/realtime/start
[6] https://github.com/richardcochran/linuxptp

Contributions

The main contributions to RQ3 are summarized below.

- `fpga-tidbits`, an FPGA accelerator framework for Chisel with automatic software driver synthesis and co-simulation.
- An FPGA accelerator for the Acution algorithm based on speculative execution.
- The hardware reactor MoC.
- `Reactor-chisel`, a Chisel runtime implementing the semantics of the hardware reactor MoC
- A “Chisel target” for the Lingua Franca Compiler.
- A “Codesign target” for the Lingua Franca Compiler.

In Paper E “fpga-tidbits: Rapid Prototyping of FPGA Accelerators in Chisel”, we present a framework for prototyping FPGA accelerators in the recent hardware description language Chisel [103]. `fpga-tidbits` abstracts away low-level platform-dependent details from the hardware designer enabling platform-independent design of FPGA accelerators. This improves the current situation where FPGA accelerator designs typically are tied to a specific FPGA vendor. `fpga-tidbits` also provides a library of reusable components, like direct memory access modules, caches and stream components like forks and filters. `fpga-tidbits` autogenerates software drivers for the accelerators and includes a co-simulation environment based on Verilator. This enables the development of hardware and software without needing an FPGA board.

We evaluate the framework through three case studies. While this work does not directly address predictability or performance it enables Paper F and Paper G which are both using `fpga-tidbits` extensively.

In Paper F "Solving Sparse Assignment Problems on FPGAs", we present an FPGA accelerator for solving a linear programming problem which is the bottleneck of multiple object tracking. Multiple object tracking is commonly employed in CPS and can become the bottleneck of the entire system. In this paper, we are focused on high performance rather than predictability. We adopt the idea of speculation, which has been instrumental in modern computer architecture, and apply it to FPGA accelerators. We implemented Bersetka’s Auction algorithm [104] where the assignment of objects to agents is decided through bidding, as in an auction. Each agent compares his reward for the objects, with the current price of the object and makes a bid which updates the prices. The state-of-the-art FPGA implementation of this algorithm was pipelined, but suffered from a lot of stalling as one had to wait for the previous agent to finish their bid and update the prices before the next agent could compare their reward to the up-to-date prices. We observed that most of the time, this was not necessary as two consecutive agents seldom bid for the same object. We implemented a static speculation scheme where an agent always speculates that there is no dependency between their bid and the

uncommitted bids of agents ahead of them in the pipeline. A misspeculation occurs when there was, in fact, a dependency, in that case, the calculation is redone. This resembles static branch predictors found in smaller CPU pipelines.

As such, this work trades time-predictability for performance. This is contrasted by the rest of the work in this thesis which puts determinism and predictability first.

Paper G "Codesign of Reactor-oriented Hardware and Software for Cyber-Physical Systems" concludes the work included in this thesis. It is the core piece of the thesis and builds on the results from several of the other papers. In this paper, we present a new abstraction level for doing hardware-software codesign. We define *hardware reactors* as a subset of Lohstroh's reactor MoC as the semantic foundation of this abstraction. We present `reactor-chisel`, a hardware runtime delivering hardware reactor semantics on the FPGA. `Reactor-chisel` implements hardware reactors as a Homogenous Synchronous Dataflow Graph [52]. The programmer provides a set of event-triggered Chisel modules, their inter-dependency and timing. `Reactor-chisel` enables the modules in the right order and handles the buffering of data between the modules. `Reactor-chisel` is based on `fpga-tidbits` and is thus portable across several FPGA boards and also supports hardware-software co-simulation based on open-source tools.

We use Lingua Franca to program reactor-oriented hardware by extending it with a "Chisel-target". This allows the programmer to provide the Chisel module implementation within reactions and declare their inter-dependency and timing with Lingua Franca syntax. This provides a new and interesting abstraction level for doing FPGA accelerator design.

The main contribution of the paper is not enabling isolated FPGA accelerator design in Lingua Franca, but codesign of both FPGA accelerators and software. For this purpose, we introduce a "Codesign-target" to Lingua Franca. It supports single-source implementation of C++ reactors executing on the CPUs and Chisel reactors executing on the FPGA. A compiler transformation will turn it into two separate projects, one for the software reactors to be run on the CPUs and one for the hardware reactors to be synthesized to the FPGA. By using the hardware reactor model, we enable deterministic coordination of accelerators running on the FPGA and software running on the CPU. The coordination is based on passing messages back and forth between the CPU and the FPGA with timing information. This is based on how distributed Lingua Franca programs are coordinated across networks. This work lifts hardware computations executing on the FPGA from being mere server devices for software computations, to equal entities. Coordination and synchronization between FPGA and CPUs are bidirectional, which means that overloaded reactors in the FPGA can stall reactors on the CPU, and vice versa.

The runtime and methodology are evaluated in terms of FPGA resource utilization, timing precision and throughput on a set of benchmarks. We show high performance at an acceptable cost.

Part II

Papers

Paper A

The Syncline Model - Analyzing the Impact of Time Synchronization in Sensor Fusion

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Torleiv Håland Bryne
NTNU
Trondheim, Norway

Tor Arne Johansen
NTNU
Trondheim, Norway

Milica Orlandić
NTNU
Trondheim, Norway

Published in
2022 IEEE Conference on Control Technology and Applications (CCTA)

Abstract

The accuracy of sensor fusion algorithms are limited by either the intrinsic sensor noise, or by the quality of time synchronization of the sensors. While the intrinsic sensor noise only depends on the respective sensors, the error induced by quality of, or lack of, synchronization depends on the dynamics of the vehicles and robotic system and the magnitude of time synchronization errors. To meet their sensor fusion requirements, system designers must consider both which sensor to use and also how to synchronize them. This paper presents the Syncline model, a simple visual model of how time synchronization affects the accuracy of sensor fusion for different mobile robot platform. The model can serve as a simple tool to determine which synchronization mechanisms should be used.

A.1 Introduction

Sensor fusion algorithms are often developed with, and evaluated on, data sets with perfectly synchronized sensor measurements, e.g. the EuRoC data set [1]. In reality, separate sensors are never completely synchronized. The degree to which they are synchronized depends on the which synchronization primitives are used and how they are used. There exists few commercial products for flexible synchronizing various sensor sources. System designers are typically left to implement their own synchronization solutions [2]. Not only is this error-prone, but they lack a framework for understanding the precision requirements for such a solution. For extreme precision, one could implement a custom timestamping circuit on an FPGA yielding deterministic timestamping and parsing of sensor data. In the other end of the scale, there is software timestamping of sensor samples "on arrival" running under some non-real time operating system. The first approach can achieve synchronization precision in the order of nanoseconds, while the second solution can suffer synchronization precision in the 100s of milliseconds range [3].

A passive algorithm for estimating synchronization error is proposed in [4] and algorithms for incorporating precision- and jitter-estimates in the sensor fusion algorithms in [5].

The accuracy of the sensor fusion algorithm is affected by both the intrinsic sensor noise and the accuracy of the timestamping of the sensor measurements, i.e. the synchronization precision. The degree of which the synchronization precision affects the overall accuracy of the sensor fusion algorithm is dependent on the dynamics of the system. For a system with slow dynamics, e.g. an unmanned surface Vehicle (USV), reducing synchronization precision from nanoseconds to microseconds does affect the overall accuracy significantly. I.e. the intrinsic sensor noise is the limiting factor. However, for a system with fast dynamics, e.g. a unmanned aerial vehicles (UAVs), using more accurate sensors does not improve the sensor fusion accuracy significantly if the synchronization precision is only in the order of hundreds of milliseconds. I.e. the synchronization precision is the limiting factor.

In this work we introduce the Syncline model which quantifies the effects of synchronization precision and instrinsic sensor noise and provides a clear and insightful visual performance model for system designers integrating sensors on mobile robots. The model is applied on two sensor fusion applications using realistic sensor and robot platform models:

1. Direct georeferencing from either UAV, USV, autonomous underwater vehicle (AUV), or surface vessels, and
2. Subsea survey with a surface vessel, and an AUV.

Analytical models are derived for both examples and compared to the predictions of the Syncline model.

This paper is organized as follows. In Section II we give a brief background into sensor synchronization. Section II introduced related work. Section IV introduces the mathematical notation and coordinate frames used. Section V derives the general Syncline model mathematically before it is applied on two concrete examples in Section VI. The work is concluded in Section VII.

A.2 Background

A.2.1 Sensor synchronization primitives

Commercial off-the-shelf sensors typically include analog-to-digital conversion and have a digital communication interface to the host. The synchronization primitives exposed by these sensors typically fall in one of the following categories:

1. **1PPS** [6] These sensors typically include an embedded microcontroller which samples and timestamps sensor measurements based on its own internal clock. A 1PPS is a 1-wire input signal which expects a flank at the beginning of each second. The embedded microcontroller can then timestamp measurements relative the beginning of a whole second. Some sensors can receive and parse certain global navigation satellite systems (GNSS) timing packets from a GNSS receiver by the use of a UART receiver. This allows the sensor to timestamp measurements on the Coordinated Universal Time (UTC) timeline.
2. **Network synchronization**. High-end sensors will typically include microcontrollers or microprocessors. They can run the TCP/IP stack and support IEE1588 Precision Time Protocol (PTP) [7] or Network Time Protocol (NTP) [8] which synchronizes the sensors' clock relative a grandmaster clock on the network.
3. **Time-of-validity (TOV) synchronization**. Some sensors do not timestamp their measurements, but rather expose timing information to the host. A TOV signal is a flank on a single wire outputted by the sensor at the precise moment of sampling. The host must detect and timestamp the TOV signal and associate it with the proceeding measurement.
4. **Triggered synchronization**. Other sensor, like cameras and some inertial

measurement units (IMU), can be triggered to provide a measurement sample. This is normally done through a single wire connected to the sensor. The user can generate a flank on this pin and associate the time of the flank with the proceeding measurement.

A.2.2 Timestamping methods

There are multiple ways for a computational device to use these synchronization primitives.

1. **Hardware timestamping** To achieve deterministic timestamping accuracy the only solution is to perform it in hardware. This applies to 1PPS, TOV and Triggered synchronization. This means using dedicated Timer peripherals to timestamp and generate events. The accuracy of hardware timestamping is typically limited by the frequency of its clock source and that clock's drift.
3. **GPIO-based timestamping** In GPIO-based timestamping interrupts are used to detect and timestamp input events (TOV). To generate output events (1PPS and Triggers) one can either configure a timer interrupt or just use the sleep function if one uses a multi-threaded platform. GPIO-based timestamping is only deterministic under strong restriction. Unless the timestamping interrupt routine has the highest priority and interrupts never are disabled, like during system-calls in Linux, GPIO-based timestamping will not give deterministic timestamping.
3. **Timestamp-on-arrival** Another option is to just timestamp the sensor measurements as they arrive to the compute platform. This might be the only choice if either your sensor has no synchronization primitives, or your platform has no GPIOs. Sometimes UART to USB adapters are used to easily integrate sensors using UART/RS232/RS422 with a compute platform over USB. The ubiquitous FTDI UART-USB chips are known to introduce up to 16ms buffering delay [9].

A.3 Preliminaries

A.3.1 Mathematical Notations

The Euclidean vector norm is denoted $\|\cdot\|_2$, and the $n \times n$ identity matrix is denoted $\boldsymbol{I}_n$. The transpose of a vector or a matrix is denoted $(\cdot)^\intercal$. Coordinate frames are expressed as $\{\cdot\}$, while $\boldsymbol{z}_{bc}^{a} \in \mathbb{R}^3$ denotes a vector $\boldsymbol{z}$ from frame $\{b\}$ to $\{c\}$, resolved in $\{a\}$. $\boldsymbol{S}(\cdot) \in SS(3)$ denotes a skew symmetric matrix such that $\boldsymbol{S}(\cdot) = -\boldsymbol{S}^\intercal(\cdot)$ and $\boldsymbol{S}(\boldsymbol{z}_1)\boldsymbol{z}_2 = \boldsymbol{z}_1 \times \boldsymbol{z}_2$ and $\boldsymbol{z}_1 \cdot \boldsymbol{z}_2$ is a dot product for the two vectors $\boldsymbol{z}_1, \boldsymbol{z}_2 \in \mathbb{R}^3$.

A.3.2 Attitude representations and relationships

Table A.1 introduces the coordinate frames used in this paper. The rotation matrix, $\boldsymbol{R}_{ab} \in SO(3)$, represents the rotation between $\{a\}$ and $\{b\}$ frames such that $\boldsymbol{z}_{\star\star}^{a} =$

Table A.1: Coordinate frames

Symbol	Name
e	Earth Centered Earth Fixed (ECEF)
n	North East Down (NED)
b_r	Mobile robot reference frame
b_o	Object of interest
b_l	LiDAR
b_g	Global navigation satellite system (GNSS) receiver
b_{sv}	Surface Vessel reference frame
b_{uv}	Autonomous underwater vehicle (AUV) reference frame
b_{urx}	Hydro acoustic ultra short baseline(USBL) receiver
b_{utp}	USBL transponder
b_{mbe}	Multibeam echosounder (MBE)

$\boldsymbol{R}_{ab}\boldsymbol{z}^b_{\star\star}$.

Additionally, the Euler angles roll, pitch and yaw are defined as ϕ, θ, ψ and relate to rotation matrix from body to NED using:

$$\boldsymbol{R}_{nb}(\boldsymbol{\Theta}) = \begin{bmatrix} c\theta c\psi & -c\phi s\psi + s\phi s\theta c\psi & s\phi s\psi + c\phi s\theta c\psi \\ c\theta s\psi & c\phi c\psi + s\phi s\theta s\psi & -s\phi c\psi + c\phi s\theta s\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix} \tag{A.1}$$

where $c\star$ denotes cos($\star$) and $s\star$ denotes sin($\star$) and

$$\boldsymbol{\Theta}_{nb} = \begin{bmatrix} \phi & \theta & \psi \end{bmatrix}^\top. \tag{A.2}$$

For an estimated or measured rotation matrix $\hat{\boldsymbol{R}}_{nb}$, the true rotation matrix $\boldsymbol{R}_{nb}$ relates to the attitude error $\boldsymbol{\epsilon}_a$

$$\boldsymbol{R}_{nb} \approx \hat{\boldsymbol{R}}_{nb}\left(\boldsymbol{I}_3 + \boldsymbol{S}(\boldsymbol{\epsilon}_a)\right) \tag{A.3}$$

if $\boldsymbol{\epsilon}_a$ is small. Hence,

$$\begin{aligned} \boldsymbol{R}_{nb}\left(\boldsymbol{I}_3 + \boldsymbol{S}(\boldsymbol{\epsilon}_a)\right)^{-1} &\approx \hat{\boldsymbol{R}}_{nb} \\ \Rightarrow \hat{\boldsymbol{R}}_{nb} &\approx \boldsymbol{R}_{nb}\left(\boldsymbol{I}_3 - \boldsymbol{S}(\boldsymbol{\epsilon}_a)\right) \end{aligned} \tag{A.4}$$

due to $\left(\boldsymbol{I}_3 + \boldsymbol{S}(\boldsymbol{\epsilon}_a)\right)^{-1} \approx \left(\boldsymbol{I}_3 - \boldsymbol{S}(\boldsymbol{\epsilon}_a)\right)$ for small $\boldsymbol{\epsilon}_a$.

A.4 Related Work

State estimation subject to non-deterministic delays and clock synchronization errors have been extensively studied, see [10].

In [11] the authors introduce the Roofline model, a simple visual performance model for computational platforms. Their key insight is that the performance of

a certain application running on a computational platform is either limited by its memory bandwidth or its raw compute capabilities. The Syncline model builds on this idea. Instead of being limited by either memory bandwidth or computing power, mobile robots are limited by either sensor accuracy or synchronization quality.

The F1 Roofline model [12] is an adaptation of the Roofline model for multi rotor UAVs. They model the max flights speed of a multi rotor as either limited by sensor accuracy, processing speed or robot weight. They show that for some applications and robots using a slower and lighter compute platform could increase the max flight speed.

The Syncline model is inspired by this line of work.

A.5 Syncline Model

The Syncline model considers three kinds of sensor modalities:

1. Sensors measuring the 3D localization of the robot, e.g. GNSS,
2. Sensors measuring the attitude or orientation of the robot, e.g. INS, and
3. Sensors measuring range and bearing to other objects, e.g. LiDAR, camera, Radar, Sonar etc.

Some of the modalities will actually have internal sensor fusion which we will not consider.

A.5.1 State-space formulation

Consider a dynamic system represented by the following nonlinear state-space equation,

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}(t), \boldsymbol{u}(t)), \tag{A.5}$$

where $\boldsymbol{x}$ is the state vector and $\boldsymbol{u}$ is a vector with input signals. This could represent any type of system which needs accurately timestamped measurements in order to perform sensor fusion.

The focus of this paper is on mobile robots, in that case $\boldsymbol{x} = (\boldsymbol{p}^e_{eb}, \boldsymbol{\Theta}_{nb}, \dots)^\intercal$. Here $\boldsymbol{p}^e_{eb}$ is the position of the robots *body* with respect to Earth's center relative defined by the *ECEF* coordinate frame. $\boldsymbol{\Theta}_{nb}$ is the attitude of the robot which can be given via Euler angles. The rest of the state variables depend on the robotic system and its environment.

A.5.2 Sensor measurement model

The sensor measurements are modelled as the system output as follows

$$\boldsymbol{y}(t_k) = \boldsymbol{h}(\boldsymbol{x}(t_k + \mu_{\text{sync}})) + \boldsymbol{\epsilon}_{\text{sensor}}, \tag{A.6}$$

here $\boldsymbol{y}(t_k)$ is a sensor measurement timestamped with the time tag t_k. The measurement is assumed to be a function of the state vector $\boldsymbol{h}(\boldsymbol{x}(t_k))$ with additional

sensor noise denoted ϵ_{sensor}. The independent random variable μ_{sync} represents the *synchronization error*. The synchronization error leads to inaccurate timestamping as a sensor measurement valid at time $t_k + \mu_{\text{sync}}$ is associated with timestamp t_k. The variables ϵ_{sensor} and μ_{sync} are assumed to be uncorrelated variables.

A.5.3 System dynamics

The potential consequences of synchronization errors between sensor sources is related to the *system dynamics* of the robot. The actual error introduced depends on how much the measured states have changed in the time between the measured and actual timestamp.

A.5.4 Estimation error in sensor fusion

The goal of the sensor fusion algorithm is to estimate some states, internal or external, based on the sensor samples $\boldsymbol{y}(t_k)$ and control inputs $\boldsymbol{u}(t_k)$:

$$\hat{\boldsymbol{x}}(t_k) = \boldsymbol{g}(\hat{\boldsymbol{x}}(t_{k-1}); \boldsymbol{y}(t_k), \boldsymbol{u}(t_k)), \tag{A.7}$$

This is also known as non-linear state estimation. The scope of this paper is limited to sensor fusion algorithms that estimate the position of objects, e.g. direct georeferencing, depth estimation, inertial navigation systems, etc.

In general, the error of the estimation of a system state variable $\boldsymbol{z}$ is defined as follows:

$$\boldsymbol{\epsilon} := \boldsymbol{z} - \hat{\boldsymbol{z}}. \tag{A.8}$$

By assuming that the sensor fusion $\boldsymbol{g}$ is a linear combination of the measurements $\boldsymbol{y}$ can this estimation error can be represented by three components:

$$\boldsymbol{\epsilon} = \boldsymbol{\delta}_{\text{sync}}(\mu_{\text{sync}}) + \boldsymbol{\delta}_{\text{sensor}} + \boldsymbol{\delta}_{\text{systematic}}, \tag{A.9}$$

where $\boldsymbol{\delta}_{\text{sync}}$ is the estimation error caused by the synchronization error, here defined as the *sync-induced error*. The variable $\boldsymbol{\delta}_{\text{sensor}}$ is the estimation error caused by sensor noise, not to be confused with the sensor noise itself $\boldsymbol{\epsilon}_{sensor}$. The estimation error is the sensor noise passed through the sensor fusion algorithm, defined as the *sensor-induced error*. The last source of error is $\boldsymbol{\delta}_{\text{systematic}}$, which represent the systematic errors due to calibration, mounting, misalignment and so on. For the rest of this paper, such systematic errors are disregarded.

The Syncline model simplifies the sync-induced error to be a linear combination of the speed and the angular velocity of the robot scaled by the distance to the measured objects

$$\boldsymbol{\delta}_{\text{sync}}(\mu_{\text{sync}}) \approx \|\boldsymbol{v}^e_{eb}\|_2 \mu_{\text{sync}} + d \cdot \|\boldsymbol{\omega}^b_{nb}\|_2 \mu_{\text{sync}}, \tag{A.10}$$

where $\|\boldsymbol{v}^e_{eb}\|_2$ is linear velocity of the robot, d is the distance from the robot to the object of interest and $\|\boldsymbol{\omega}^b_{nb}\|_2$ is the angular velocity of the robot. The left term of

(A.10) is an error caused by the linear distance between the robots actual position when a measurement was taken and its position at the time of the timestamp. The right term is related to the difference in orientation of the robot between the time of the measurement sample and the time of the timestamp. This difference in orientation, i.e. angle, is multiplied by the distance to the object of interest. For small angles this is an approximation of the georeferencing error induced by the orientation error.

It is clear that the sync-induced error varies during the operation of the mobile robot, e.g. if there is no relative movement between the robot and the object of interest, then there is also no sync-induced error. An important characteristics of the sync-induced error is its *expected worst-case* magnitude during a robot operation. We define the expected worst-case sync-induced error as the error observed with the *mean* of the synchronization error μ_{sync} and the expected maximum linear and angular velocity. We also refer to this as the *upper bound* and indicate it with an asterisk as follows:

$$\delta^*_{\text{sync}}(\tau) := v_{\max}\tau + d \cdot \omega_{\max}\tau, \tag{A.11}$$

Where τ is the worst-case synchronization error and, $v_{\max}$ and $\omega_{\max}$ are norms of the expected maximum linear and angular velocities for the robot platform, respectively.

We define δ^*_{sensor} as the expected worst-case of the sensor-induced error as follows

$$\delta^*_{\text{sensor}} := \sigma_p + \sigma_r + (\sigma_\Theta + \sigma_u) \cdot d, \tag{A.12}$$

where σ_x is the standard deviation of the estimated variable x p, r, Θ and u represent localization, ranging, attitude and bearing sensors, respectively. In this context "expected" refers to that we use the standard deviation as the magnitude of the sensor error. "Worst-case" refers to the fact that we are summing the magnitudes. This implies that the error vectors are positive scalar multiples of each other. Clearly the worst-case scenario is when all the error vectors are in the same direction.

Finally, we define the Syncline as

$$\delta_{\text{syncline}}(\tau) := \delta^*_{\text{sync}}(\tau) + \delta^*_{\text{sensor}}. \tag{A.13}$$

The Syncline is an approximation of the worst case estimation accuracy as a function of the synchronization error. Fig. A.1 shows a plot of (A.13), i.e. a Syncline. Here, δ^*_{sync} is calculated with the $v_{\max}$, $\omega_{\max}$ and r estimated for a car, see Tab. A.2. Also, δ^*_{sensor} is set to the arbitrary value of 0.1 m. This signifies that the sensors employed by the robot is capable of estimating positions with worst case error of 0.1 m. The vertical axis shows the **Estimation accuracy** which is the inverse of the error. This is why the accuracy asymptotically reaches 10^1 m^{-1} which is the inverse of 0.1 m. The horizontal axis shows the **Synchronization Accuracy** which is the inverse the synchronization error $1/\tau$, i.e. if the synchronization error is

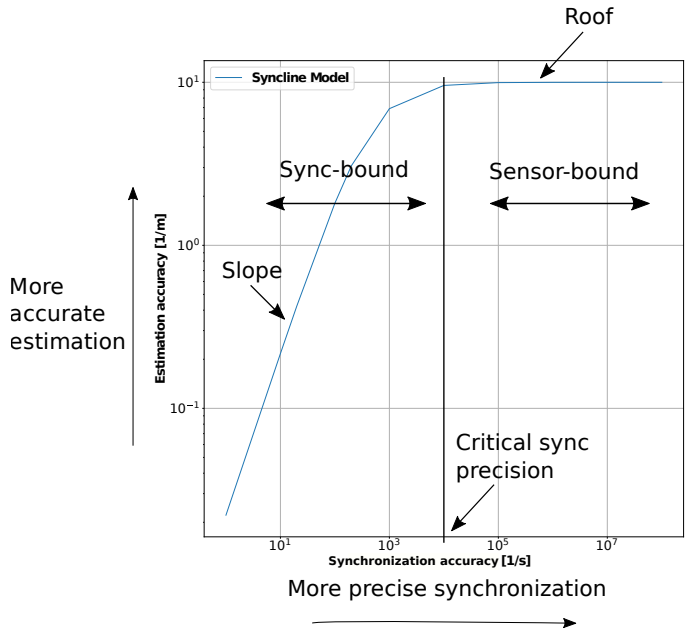


Figure A.1: A Syncline plot

1μs, then the synchronization accuracy $1/\tau$ is 10^6s^{-1}. We also define τ_{crit} as the critical synchronization error which satisfies

$$\delta^*_{\text{sync}}(\tau_{\text{crit}}) = \delta^*_{\text{sensor}}. \tag{A.14}$$

Fig. A.1 indicates two regions along the horizontal axis. The **Sync-bound** region is the region of high synchronization error where $\delta^*_{\text{sync}} > \delta^*_{\text{sensor}}$ which corresponds to $\tau > \tau_{\text{crit}}$. It is called sync-bound because the performance is mainly limited by the synchronization accuracy. The **Sensor-bound** region is characterized by $\delta^*_{\text{sync}} < \delta^*_{\text{sensor}}$ and $\tau < \tau_{\text{crit}}$. In this region, there is little use in improving the synchronization mechanism as intrinsic sensor noise is the limiting factor. The border between the two regions indicates the critical synchronization error τ_{crit}. This is where $\delta^*_{\text{sync}} = \delta^*_{\text{sensor}}$ and $\tau = \tau_{\text{crit}}$. In general, the system designer should achieve synchronization accuracy greater than the critical synchronization accuracy. This ensures that one gets maximum value from the sensors.

Fig. A.1 also indicates the **slope** and the **roof** of the curve. The slope shows how accuracy is increasing as we increase the synchronization accuracy. The roof is the maximum obtainable accuracy for the complete sensor fusion system.

Table A.2: Robot platform parameters

Robot dynamics				
	$v_{\max}$ $[\frac{\text{m}}{\text{s}}]$	$\omega_{\max}$ $[\frac{\text{deg}}{\text{s}}]$	d [m]	b [m]
UAV Fixed Wing	21	77.9	100	1
UAV Multi Rotor	5	311.8	5	0.5
USV	5	17	30	5
AUV	30	8.7	5	5
Car	30	17.3	50	3
Large SV	2.5	4.5	1000	50
Small SV	4	17	1000	10

A.6 Examples

In this section, the Syncline model is evaluated on two examples of sensor fusion in mobile robots. First, a state-space model of the sensor fusion algorithm is derived, it will serve as the "ground truth" and be the basis for the simulations of the system. Then the Syncline model is derived and evaluated against the simulator. Evaluating the model on real, experimental data, is left as future work.

To develop the Syncline model, we consider a set of robot platforms and their dynamics. Table A.2 shows the mobile robots and parameters used in these examples. The variable d is the range and signifies the typical distance between the robot and the objects to be localized. The parameter b stands for baseline which indicates the size of the robot and the maximum lever arms between the different sensors. The Syncline model is also dependent on the intrinsic sensor noise. We consider a set of commonly used sensors introduced in Table A.3 where $\sigma(x)$ is the standard deviation of the measured state x. I.e. the intrinsic sensor noise.

A.6.1 Example 1: Georeferencing with GNSS, INS and LiDAR

In the first example we will consider is a georeferencing application where a mobile robot uses three sensors to estimate the global position of an external object. A GNSS receiver is used to measure its own global position, INS for attitude measurements and a LiDAR to measure range and bearing to the external object of interest.

Analytical model

In this example, we use a state space with 9 states comprising of the robots position and attitude as well as the position of the external object that we wish to georeference:

$$\boldsymbol{x} = [\boldsymbol{p}^e_{eb}, \boldsymbol{\Theta}_{nb}, \boldsymbol{p}^e_{eo}]^T. \tag{A.15}$$

There are three sensors, each timestamps the measurements on its own clock i.e. t_{GNSS}, t_{INS} and t_{LiDAR}. The synchronization error μ_{sync} is defined as relative

the LiDAR timestamp. Thus, the GNSS synchronization error is defined as $\mu_{\text{sync}}^{\text{gnss}} = t_{\text{GNSS}} - t_{\text{LiDAR}}$.

The GNSS measurement is modeled as

$$\boldsymbol{y}_{\text{GNSS}}(t) = \hat{\boldsymbol{p}}^{e}_{eb_g} \tag{A.16}$$

$$\hat{\boldsymbol{p}}^{e}_{eb_g} = \boldsymbol{p}^{e}_{eb_g} + \boldsymbol{v}^{e}_{eb_g}\mu_{\text{sync}}^{\text{gnss}} + \boldsymbol{\epsilon}_{p}^{\text{gnss}}. \tag{A.17}$$

The first term is the true position of the GNSS sensor. The second term is the sync-induced error which is how much the GNSS sensor moves in the timestamping error period. The final term is the intrinsic sensor noise. The sync-induced error is a combination of the velocity of the GNSS receiver relative the ECEF frame and the synchronization error. The velocity can be expressed as a combination of the linear and angular velocity of the robot as follows:

$$\begin{aligned}\boldsymbol{v}^{e}_{eb_g} &= \frac{d}{dt}\boldsymbol{p}^{e}_{eb_g} = \frac{d}{dt}\left(\boldsymbol{p}^{e}_{eb_r} + \boldsymbol{p}^{e}_{b_r b_g}\right) \\ &= \boldsymbol{v}^{e}_{eb_r} + \dot{\boldsymbol{R}}_{eb_r}\boldsymbol{p}^{b_r}_{b_r b_g}.\end{aligned} \tag{A.18}$$

The rotation from the ECEF frame to the robot frame is composed of the two rotations $\boldsymbol{R}_{eb_r} = \boldsymbol{R}_{en}\boldsymbol{R}_{nb_r}$. We assume that $\boldsymbol{R}_{en}$ is constant during the time synchronization error μ_{sync}^{gnss}. We can then express the time derivative of that rotation as: $\dot{\boldsymbol{R}}_{eb_r} = \boldsymbol{R}_{eb_r}\boldsymbol{S}(\boldsymbol{\omega}^{b_r}_{nb_r})$. Inserted into (A.18) yields

$$\boldsymbol{v}^{e}_{eb_g} = \boldsymbol{R}_{eb_r}\left(\boldsymbol{v}^{b_r}_{eb_r} + \boldsymbol{S}(\boldsymbol{\omega}^{b_r}_{eb_r})\boldsymbol{p}^{b_r}_{b_r b_g}\right). \tag{A.19}$$

further inserting into (A.17) gives us

$$\hat{\boldsymbol{p}}^{e}_{eb_g} = \boldsymbol{p}^{e}_{eb_g} + \boldsymbol{R}_{eb_r}\left(\boldsymbol{v}^{b_r}_{eb_r} + \boldsymbol{S}(\boldsymbol{\omega}^{b_r}_{nb_r})\boldsymbol{p}^{b_r}_{b_r b_g}\right)\mu_{\text{sync}}^{\text{ins}} + \boldsymbol{\epsilon}_{p}^{\text{gnss}} \tag{A.20}$$

The INS measures the attitude of the robot as follows

$$\boldsymbol{y}_{\text{INS}}(t) = \hat{\boldsymbol{R}}_{nb_r} \tag{A.21}$$

$$\hat{\boldsymbol{R}}_{nb_r} = \boldsymbol{R}_{nb_r} + \boldsymbol{R}_{nb_r}\boldsymbol{S}(\boldsymbol{\omega}^{b_r}_{nb_r})\mu_{\text{sync}}^{\text{ins}} + \boldsymbol{\epsilon}_{\Theta}^{\text{ins}}. \tag{A.22}$$

The first term is the true attitude. The second term is the sync-induced error, it is the time derivative of the attitude multiplied with the timestamp offset. This assumes that the angular velocity is constant during this time period. The last term is the intrinsic sensor noise.

$$\boldsymbol{y}_{\text{L}}(t) = \hat{\boldsymbol{p}}^{b_r}_{b_l b_o} \tag{A.23}$$

$$\hat{\boldsymbol{p}}^{b_r}_{b_l b_o} = \boldsymbol{R}_{b_r b_l}(\boldsymbol{p}^{b_r}_{b_l b_o} + \boldsymbol{\epsilon}_{p}^{\text{lidar}}) \tag{A.24}$$

I.e the LiDAR outputs a position vector from itself b_l to the object of interest b_o. In reality the LiDAR outputs a 3D point cloud with azimuth, elevation and range

for each point. This signal must pass through a signal processing algorithm which performs object detection. We assume the output of this object detection is a range r and a bearing vector $\boldsymbol{n}_{b_l b_o}^{b_l}$ defined as

$$\boldsymbol{n}_{b_l b_o}^{b_l} := \begin{bmatrix} \cos(\Psi)\cos(\alpha) & \sin(\Psi)\cos(\alpha) & -\sin(\alpha) \end{bmatrix}^\intercal \tag{A.25}$$

where Ψ is azimuth and α is elevation such that the total vector from $\{b_l\}$ to $\{b_o\}$ can be stated

$$\boldsymbol{p}_{b_l b_o}^{b_l} = r \cdot \boldsymbol{n}_{b_l b_o}^{b_l} = \boldsymbol{R}_{b_l b_o}(\alpha, \Psi) \cdot \boldsymbol{d}_{b_l b_o} \tag{A.26}$$

and where

$$\boldsymbol{R}_{b_l b_o}(\alpha, \Psi) = \begin{bmatrix} \cos(\Psi)\cos(\alpha) & -\sin(\Psi) & \sin(\alpha)\cos(\Psi) \\ \sin(\Psi)\cos(\alpha) & \cos(\Psi) & \sin(\alpha)\sin(\Psi) \\ -\sin(\alpha) & 0 & \cos(\alpha) \end{bmatrix} \tag{A.27}$$

and

$$\boldsymbol{d}_{b_l b_o} = \begin{bmatrix} r & 0 & 0 \end{bmatrix}^\intercal. \tag{A.28}$$

Notice that the LiDAR measurement has no sync-induced error. This is because we have defined synchronization error as relative to the LiDAR timestamp.

The sensor fusion is defined by the function $\boldsymbol{g}$ introduced in (A.7). It combines INS, GNSS and LiDAR measurements to estimate the global ECEF position of the object, which we write as

$$\boldsymbol{g}(\boldsymbol{x}(t_{k-1}); \boldsymbol{y}(t_k)) = \hat{\boldsymbol{p}}_{eo}^{e}(t_k). \tag{A.29}$$

This is a series of rotations and translations:

$$\hat{\boldsymbol{p}}_{eo}^{e}(t_k) = \hat{\boldsymbol{p}}_{eb_g}^{e} + \boldsymbol{R}_{en}\hat{\boldsymbol{R}}_{nb_r}(\boldsymbol{p}_{b_g b_l}^{b_r} + \hat{\boldsymbol{p}}_{b_l b_o}^{b_r}). \tag{A.30}$$

This can again be expressed in terms of the true quantities, sync-induced errors and sensor noise by substituting in (A.20), (A.22) and (A.24).

Syncline model

For the sync-induced error, we use the $v_{\max}$ and $\omega_{\max}$ from Table A.2. For the sensor error, we use the σ-values from Table A.3.

Fig. A.2 compares the Synclines for different mobile robot platforms when using uBlox F9P RTK, MRU5 and VUX1-UAV. There are several interesting things to note. Firstly, the Synclines have different roofs. The roof is the accuracy achieved when you are only limited by the intrinsic sensor accuracy. The reason why the robots have different roofs is that they are assumed to be georeferencing objects at different distances. Secondly, the length of the slope of the Synclines varies. This is related to the sensitivity to synchronization errors. The shorter the roof (i.e. a lower τ_{crit}), the more sensitive a system is to synchronization errors. This depends on the robot dynamics and the intrinsic sensor noise.

Table A.4 reports the τ_{crit} for the different sensors individually. The synchronization error requirements for the sensor fusion is dictated by the sensor with the

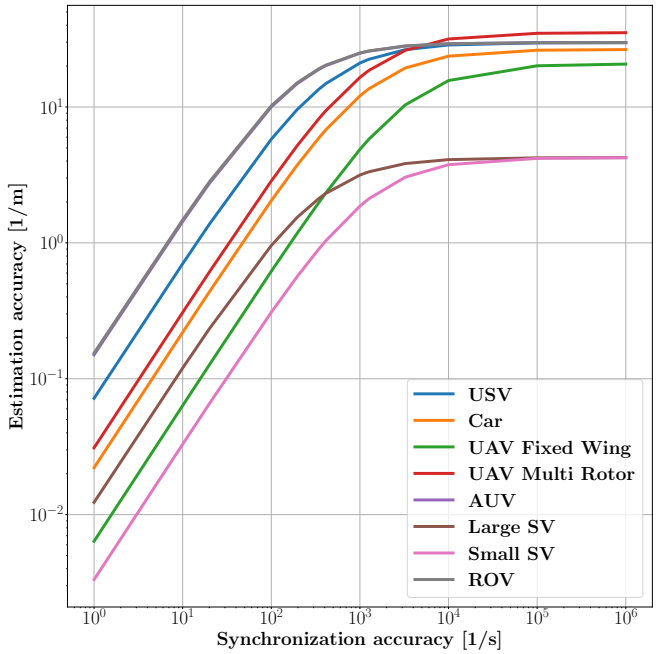


Figure A.2: Syncline for direct georeferencing with uBlox F9P RTK, MRU5 and VUX1-UAV for various robot platforms

Table A.3: Sensor noise

	GNSS			
	uBlox F9P PVT	uBlox F9P RTK	Trimble R12 DGNSS	Trimble R12 RTK
$\sigma(p)$	1.5 m	1 cm	0.25 m	8 mm
	INS			
	SBG Ellipse	SBG Apogee	Kongsberg MRU5	
$\sigma(\phi)$	0.1°	0.008°	0.002°	
$\sigma(\theta)$	0.1°	0.008°	0.002°	
$\sigma(\psi)$	0.2°	0.03°	0.002°	
	LiDAR			
	Velodyne Alpha Prime	Velodyne HDL32E	RIEGL VUX1-UAV	Faro Focus Plus
$\sigma(r)$	4 cm	2 cm	1 cm	1 mm
$\sigma(\Psi)$	0.1°	0.08°	0.006°	0.005°
$\sigma(\alpha)$	0.2°	0.08°	0.006°	0.005°
	USBL and MBE			
	Kongsberg HIPAP502	Sensodyne USBL7000	Kongsberg M3Sonar	Sonic 2026 MBE
$\sigma(r)$	2 cm	1.5 cm	1 cm	1 mm
$\sigma(\Psi)$	0.06°	0.04°	0.9°	0.45°
$\sigma(\alpha)$	0.06°	0.04°	0.5°	0.45°

lowest τ_{crit}. In Fig. A.4, a plot with a few different combinations of the sensor payload for a UAV platform is presented. Intuitively, changing the sensor payload raises or lowers the roofs of the Synclines, which depends on the intrinsic sensor noise only. It also changes the length of the roofs which is related to τ_{crit}. The shorter the roof, the more sensitive the system is to synchronization error.

To verify the accuracy of the Syncline model, we have made a simulator based on the analytical formulation in (A.30) is also implemented. We simulate a fixed wing UAV which maneuvers according to Table A.2 trying to georeference a fixed object using senors with parameters in Table A.3. For each synchronization error, the worst case georeferencing error is found and compared with the Syncline in Fig. A.3. Clearly, the Syncline model is adept for modeling direct georeferencing.

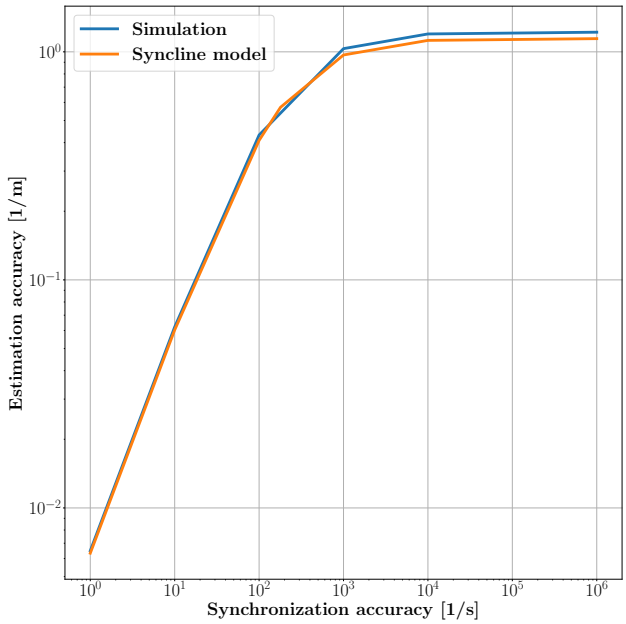


Figure A.3: Syncline model for UAV Fixed Wing compared with simulations

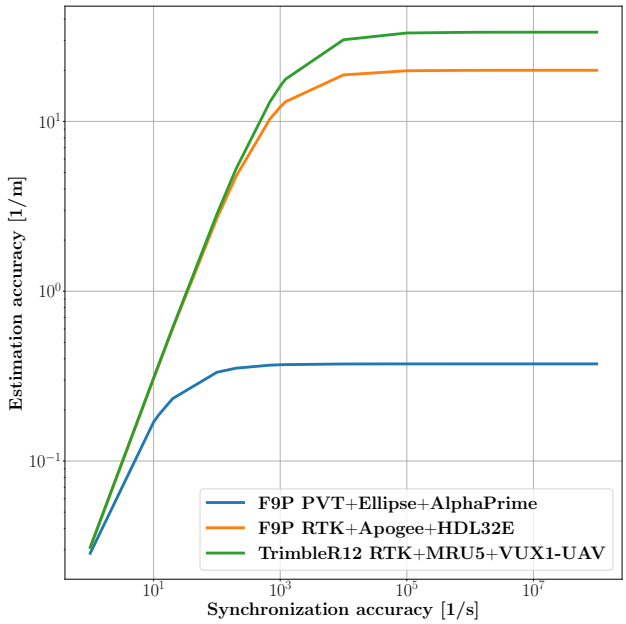


Figure A.4: Syncline model for UAV Fixed Wing with different sensor payloads

Table A.4: Minimum synchronization error τ_{crit}

Robot platform				
Sensor	USV	Car	Multi Rotor	Fixed Wing
F9P PVT	520ms	86ms	520ms	123ms
F9P RTK	3ms	577us	3ms	824us
Ellipse	9ms	4ms	663us	2ms
MRU5	130us	67us	9us	38us
AlphaPrime	11ms	5ms	2ms	3ms
Vux1	1ms	385us	333us	157us

A.6.2 Example 2: Underwater survey with USBL, MBE, INS and GNSS

In the second example, we consider a subsea survey system consisting of a surface vessel (SV) and an autonomous underwater Vehicle (AUV). The surface vessel has a GNSS, INS_{sv}, and Ultra-short baseline acoustic positioning system (USBL) receiver. The AUV has a USBL transponder, a INS_{auv} and a Multibeam Echosounder (MBE). This system is inspired by the authors in [13] The goal is to estimate global position of the AUVs MBE footprint $\boldsymbol{p}^e_{eo}$. This is achieved by performing direct georeferencing twice. First, the global position of the AUV $\boldsymbol{p}^e_{eb_{\text{auv}}}$ is estimated based on the GNSS measurements $\hat{\boldsymbol{p}}^e_{eb_{\text{sv}}}$, the INS_{sv} measurements $\hat{\boldsymbol{R}}_{nb_{\text{sv}}}$ and the USBL measurements $\hat{\boldsymbol{p}}_{b_{\text{sv}}b_{\text{auv}}}$, all taken at the surface vessel. This position estimate is fused with the INS_{auv} measurement $\hat{\boldsymbol{R}}_{nb_{\text{auv}}}$ and the range-and-bearing measurements from the MBE to the ocean floor $\hat{\boldsymbol{p}}^{b_{\text{auv}}}_{b_{\text{auv}}o}$.

As before, both an analytical model and the Syncline model are derived.

Analytical model

The state space consists of the position and attitude of the SV, the position and attitude of the AUV and the position of the MBE footprint.

$$\boldsymbol{X} = [\boldsymbol{p}^e_{eb_{\text{sv}}}, \boldsymbol{\Theta}_{nb_{\text{sv}}}, \boldsymbol{p}^e_{eb_{\text{auv}}}\boldsymbol{\Theta}_{nb_{\text{auv}}}, \boldsymbol{p}^e_{eo}]^\intercal. \tag{A.31}$$

The system has five sensors which are all timestamped with their respective clock. The synchronization error is defined as relative the MBE measurements. The GNSS, INS_{sv} and USBL measurements of the SV are combined into a single virtual position sensor for the AUV based on direct georeferencing. Internal to the virtual sensor, the synchronization error is modeled as relative to the USBL measurement.

The GNSS and INS measurements are derived in (A.20) and (A.22), respectively. The USBL measurements is a vector from the receiver on the SV to the transponder on the AUV, as follows:

$$\boldsymbol{y}_{\text{USBL}} = \hat{\boldsymbol{p}}^{b_{\text{sv}}}_{b_{urx}b_{\text{utp}}} \tag{A.32}$$

$$\hat{\boldsymbol{p}}^{b_{\text{sv}}}_{b_{urx}b_{\text{utp}}} = \boldsymbol{p}^{b_{\text{sv}}}_{b_{urx}b_{\text{utp}}} + \boldsymbol{\epsilon}^{\text{usbl}}_p. \tag{A.33}$$

The virtual AUV position measurement is defined as follows:

$$\boldsymbol{y}_{\text{AUVPOS}} = \hat{\boldsymbol{p}}^{e}_{eb_{\text{utp}}} \tag{A.34}$$

$$\begin{aligned}\hat{\boldsymbol{p}}^{e}_{eb_{\text{utp}}} &= \hat{\boldsymbol{p}}^{e}_{eb_g} + \boldsymbol{R}_{en}\hat{\boldsymbol{R}}_{nb_{\text{sv}}}(\boldsymbol{p}^{b_{\text{sv}}}_{b_g b_{urx}} + \hat{\boldsymbol{p}}^{b_{\text{sv}}}_{b_{urx} b_{\text{utp}}}) \\ &\quad + \boldsymbol{R}_{eb_{auv}}\left(\boldsymbol{v}^{b_{auv}}_{eb_{auv}} + \boldsymbol{S}(\boldsymbol{\omega}^{b_{auv}}_{nb_{auv}})\boldsymbol{p}^{b_{auv}}_{b_{utp} b_{auv}}\right)\mu^{\text{auvpos}}_{\text{sync}}.\end{aligned} \tag{A.35}$$

The first term is the measured position of the GNSS receiver at the SV. The second term is the measured orientation of the SV combined with the mounting lever arms for the sensors on the SV. The last term is the sync-induced error caused by the movement of the AUV during the timestamping offset between the virtual AUV position measurement and the AUV MBE measurement.

The INS_{auv} is modeled as follows

$$\boldsymbol{y}_{\text{INS}_{\text{auv}}}(t) = \hat{\boldsymbol{R}}_{nb_{\text{auv}}} \tag{A.36}$$

$$\hat{\boldsymbol{R}}_{nb_{\text{auv}}} = \boldsymbol{R}_{nb_{\text{auv}}} + \boldsymbol{R}_{nb_{\text{auv}}}\boldsymbol{S}(\boldsymbol{\omega}^{b_r}_{nb_r})\mu^{\text{ins}_{\text{sv}}}_{\text{sync}} + \boldsymbol{\epsilon}^{\text{ins}_{\text{sv}}}_{\Theta} \tag{A.37}$$

This is similar to (A.22). The error in the attitude measurement is a combination of the sync-induced error and the intrinsic sensor noise.

The MBE measurement is analogous to the LiDAR measurement in (A.24) in Example 1, and outputs the position vector $\hat{\boldsymbol{p}}^{b_{\text{auv}}}_{b_{mbe} b_o}$ as a range and a bearing as described in (A.26). The global position of the footprint can now be expresses in terms of the virtual AUV position measurement, the INS_{auv} measurement and the MBE measurement

$$\begin{aligned}\hat{\boldsymbol{p}}^{e}_{eo}(t_k) &= \hat{\boldsymbol{p}}^{e}_{eb_g} + \boldsymbol{R}_{en}\hat{\boldsymbol{R}}_{nb_{\text{sv}}}(\boldsymbol{p}^{b_{\text{sv}}}_{b_g b_{urx}} + \hat{\boldsymbol{p}}^{b_{\text{sv}}}_{b_{urx} b_{\text{utp}}}) \\ &\quad + \boldsymbol{R}_{en}\hat{\boldsymbol{R}}_{nb_{\text{auv}}}(\boldsymbol{p}^{b_{\text{auv}}}_{b_{\text{utp}} b_l} + \hat{\boldsymbol{p}}^{b_{\text{auv}}}_{b_{mbe} b_o}).\end{aligned} \tag{A.38}$$

Syncline model

The Syncline model is easily extended to account for the dynamics of both the SV and the AUV as follows

$$\delta^{*}_{\text{sync}}(\tau) = (v^{\text{sv}}_{\max} + v^{\text{auv}}_{\max} + d^{\text{sv}}\omega^{\text{sv}}_{\max} + d^{\text{auv}}\omega^{\text{auv}}_{\max}) \cdot \tau. \tag{A.39}$$

This is the sum of the two Synclines, one for the direct georeferencing of the AUV from the SV, and one for the direct georeferencing of the seabed from the AUV.

The sensor-induced errors are expressed as follows:

$$\begin{aligned}\delta^{*}_{\text{sensor}} &= \sigma_{p_{\text{sv}}} + \sigma_{r_{\text{sv}}} + (\sigma_{\Theta_{\text{sv}}} + \sigma_{u_{\text{sv}}}) \cdot d_{\text{sv}} \\ &\quad + \sigma_{p_{\text{auv}}} + \sigma r_{\text{auv}} + (\sigma_{\Theta_{\text{auv}}} + \sigma_{u_{\text{auv}}}) \cdot d_{\text{auv}}.\end{aligned} \tag{A.40}$$

The Syncline model is plotted in Fig. A.5 for both a large SV and a small SV performing subsea survey together with an AUV. The AUV is assumed to be at

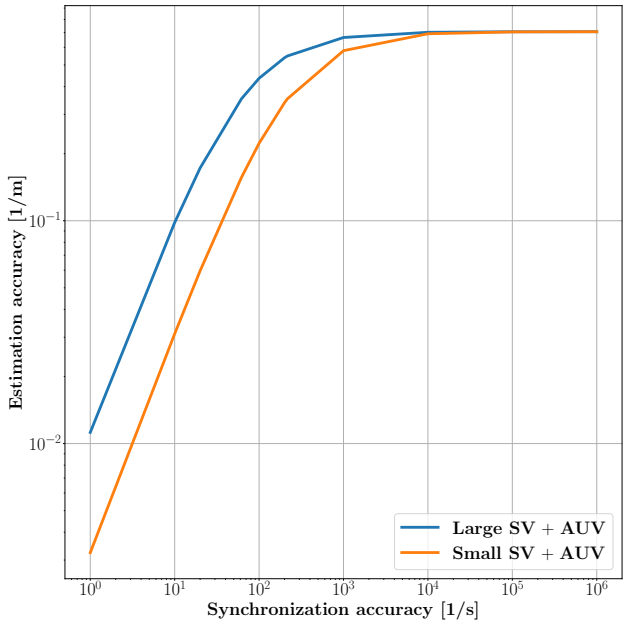


Figure A.5: Synclines for underwater survey with SV and AUV using Trimble R12 RTK GNSS, MRU5 INS, Sensodyne Gyro USBL7000 and Sonic 2026 MBE

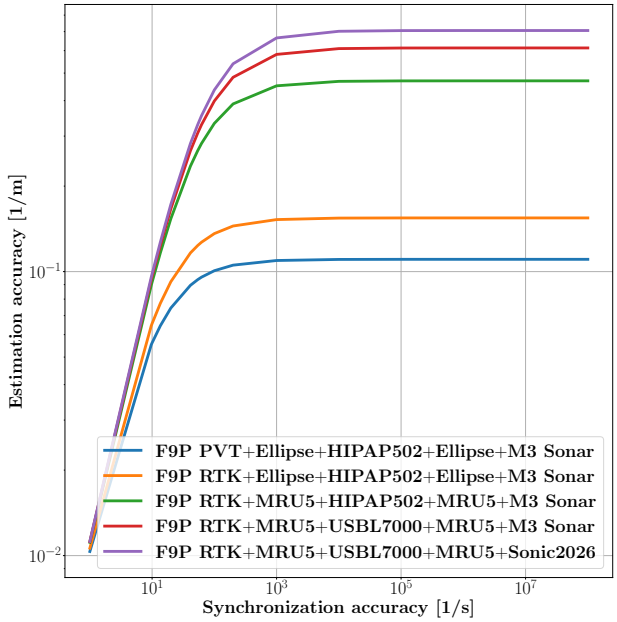


Figure A.6: Syncline model for Large SV and AUV with different sensor payloads

a 1000m depth and 30m above the sea floor. Both systems has roofs at around 1.67m. This is expected since they are equipped the same sensor payload and are at the same distance from the object of interest. The system with large SV has a longer roof which is due to the fact that it has slower dynamics and thus less sensitivity to synchronization errors. For the large SV $\tau_{crit} = 16$ ms while for the small SV $\tau_{crit} = 4$ ms.

In Fig. A.6, the Synclines for a system with a large SV and a AUV are compared for four different sensor payloads. Clearly, replacing the GNSS receiver with a better one does comparatively little with the performance compared to replacing the INS or USBL. Table A.5 reports the critical synchronization per sensor. This illustrates sensitivity of each sensor to synchronization error. For instance, the uBlox F9P PVT which only has around 1.5 m accuracy is not sensitive to synchronization error on such slow moving robot platforms as SVs and AUVs. The MRU5 which can estimate the robot attitude with an accuracy of 0.002° is much more sensitive, even on these slow platforms.

In Fig. A.7 the Syncline model is evaluated against a simulator based on the analytical model derived in (A.38). Again, there is no significant deviation between the Syncline model and the analytical simulation.

Table A.5: Critical synchronization error

	Robot platform		
τ_{crit}	Large SV	Small SV	AUV
uBlox F9P PVT	1039.230ms	649.519ms	-
uBlox F9P RTK	6.928ms	4.330ms	-
SBG Ellipse	52.754ms	14.217ms	19.335ms
MRU5	0.746ms	0.201ms	0.273ms
HIPAP502	18.521ms	4.991ms	9.713ms
USBL7000	12.368ms	3.333ms	6.727ms
Sonic2026	137.071ms	36.940ms	50.385ms
M3 Sonar	221.857ms	59.790ms	82.776ms

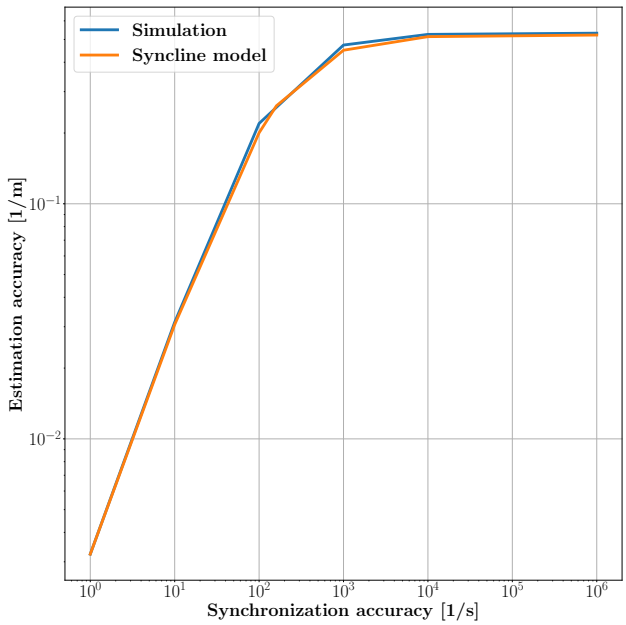


Figure A.7: Evaluation of Syncline model for Small SV and AUV against a simulator. The sensors used are uBlox F9P RTK, MRU5, HIPAP520 and Sonic 2026

A.7 Conclusions

The Syncline model is a simple visual model which allows system designers to explore the interaction between errors introduced by lacking sensor synchronization and errors introduced by intrinsic sensor noise. Synclines are derived for a number of different robot platforms and sensor payloads. The model is then evaluated against a mobile robot simulator for two different sensor fusion algorithms. The results are promising showing that the Syncline model is accurate for direct georeferencing and mapping applications.

Bibliography

[1] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik and R. Siegwart, 'The euroc micro aerial vehicle datasets,' *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.

[2] B. Yu, W. Hu, L. Xu, J. Tang, S. Liu and Y. Zhu, 'Building the computing system for autonomous micromobility vehicles: Design constraints and architectural optimizations,' in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 1067–1081. DOI: 10.1109/MICRO50266.2020.00089.

[3] J. M. Hansen, T. I. Fossen and T. Arne Johansen, 'Nonlinear observer design for gnss-aided inertial navigation systems with time-delayed gnss measurements,' *Control Engineering Practice*, vol. 60, pp. 39–50, 2017.

[4] E. Olson, 'A passive solution to the sensor synchronization problem,' in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010, pp. 1059–1064. DOI: 10.1109/IROS.2010.5650579.

[5] I. Skog and P. Händel, 'Effects of time synchronization errors in gnss-aided ins,' Jun. 2008, pp. 82–88, ISBN: 978-1-4244-1536-6. DOI: 10.1109/PLANS.2008.4570010.

[6] J. Mogul, P. D. L. Mills, U. Windl, J. Stone and J. Brittenson, *Pulse-Per-Second API for UNIX-like Operating Systems, Version 1.0*, RFC 2783, Mar. 2000. DOI: 10.17487/RFC2783. [Online]. Available: https://www.rfc-editor.org/info/rfc2783.

[7] 'Ieee standard for a precision clock synchronization protocol for networked measurement and control systems,' *IEEE Std 1588-2019 (Revision ofIEEE Std 1588-2008)*, pp. 1–499, 2020. DOI: 10.1109/IEEESTD.2020.9120376.

[8] D. Mills, 'Internet time synchronization: The network time protocol,' *IEEE Transactions on Communications*, vol. 39, no. 10, pp. 1482–1493, 1991. DOI: 10.1109/26.103043.

[9] *An232b-04 data throughput,latency and handshaking*, https://www.kernel.org/doc/html/v4.10/driver-api/index.html, 2006.

[10] A. Matveev and A. Savkin, 'The problem of state estimation via asynchronous communication channels with irregular transmission times,' *IEEE Transactions on Automatic Control*, vol. 48, no. 4, pp. 670–676, 2003. DOI: 10.1109/TAC.2003.809771.

[11] S. Williams, A. Waterman and D. Patterson, 'Roofline: An insightful visual performance model for floating-point programs and multicore architectures,' Feb. 2009.

[12] S. Krishnan, Z. Wan, K. Bhardwaj, P. Whatmough, A. Faust, G.-Y. Wei, D. Brooks and V. J. Reddi, 'The sky is not the limit: A visual performance model for cyber-physical co-design in autonomous machines,' *IEEE Computer Architecture Letters*, vol. 19, no. 1, pp. 38–42, 2020. DOI: 10.1109/LCA.2020.2981022.

[13] B. Jalving and E. Berglund, *Time referencing in offshore survey systems*, 2006. [Online]. Available: http://hdl.handle.net/20.500.12242/1930.

Paper B

Beyond the Threaded Programming Model on Real-Time Operating Systems

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Peter Donovan
UC Berkeley
Berkeley, USA

Fuzail Shakir
UC Berkeley
Berkeley, USA

Milica Orlandić
NTNU
Trondheim, Norway

Edward A. Lee
UC Berkeley
Berkeley, USA

Shaokai Lin
UC Berkeley
Berkeley, USA

Efsane Soyer
UC Berkeley
Berkeley, USA

Torleiv Håland Bryne
NTNU
Trondheim, Norway

Marten Lohstroh
UC Berkeley
Berkeley, USA

Published in
Proceedings of Cyber-Physical Systems and Internet of Things Week 2023

Abstract

The use of a real-time operating system (RTOS) raises the abstraction level for embedded systems design when compared to traditional bare-metal programming, resulting in simpler and more reusable application code. Modern RTOSes for resource-constrained platforms, like Zephyr and FreeRTOS, also offer threading support, but this kind of shared memory concurrency is a poor fit for expressing the reactive and interactive behaviors that are common in embedded systems. To address this, alternative concurrency models like the actor model or communicating sequential processes have been proposed. While those alternatives enable reactive design patterns, they fail to deliver determinism and do not address timing. This makes it difficult to verify that implemented behavior is as intended and impossible to specify timing constraints in a portable way. This makes it hard to create reusable library components out of common embedded design patterns, forcing developers to keep reinventing the wheel for each application and each platform. In this paper, we introduce the embedded target of Lingua Franca (LF) as a means to move beyond the threaded programming model provided by RTOSes and improve the state of the art in embedded programming. LF is based on the reactor model of computation, which is reactive, deterministic, and timed, providing a means to express concurrency and timing in a platform-independent way. We compare the performance of LF versus threaded C code—both running on Zephyr—in terms of response time, timing precision, throughput, and memory footprint.

B.1 Introduction

Microcontrollers are ubiquitous in embedded applications, such as consumer electronics, industrial automation, robotics, and automotive systems. Such applications are concurrent, event-driven, and in many cases, time-sensitive. Traditionally, these systems have been programmed in C, based on a limited shared-memory concurrency model offered by interrupts. However, such programs have long been known to be notoriously difficult to get right [1, 2]. This has contributed to serious bugs and vulnerabilities, sometimes with catastrophic results [3–5].

Real-time operating systems (RTOSes) offer a richer version of the shared-memory concurrency model through *threads*. Threads enable the user to decompose their application into multiple concurrent activities. The RTOS offers low-level primitives for controlling the scheduling, synchronization, timing, and communication between threads. Just like interrupts, the threaded programming model admits nondeterminism [6] and leaves the error-prone job of synchronizing the threads' accesses to shared variables to the programmer [7]. A concurrency primitive that popular operating systems like Zephyr [8] provide as an alternative to threads is *callbacks*, which are invoked from interrupt service routines [9]. Callbacks do not address the problem of nondeterminism either, but they do create the problem of control flow inversion [2], which makes programs much harder to understand [10]. The RTOS introduces a notion of time, which is useful when designing time-aware applications. However, the timing is too imprecise for many applications. The RTOS will use regular interrupts, called *systicks*, to measure

time and provide timing abstractions. The interval between the systicks is user-configurable but typically in the order of milliseconds. For many applications, this level of accuracy is not sufficient [11]. Developers can resort to implementing the timed behavior through callbacks, but this will expose them to the aforementioned problems.

In this paper, we propose Lingua Franca (LF) [12] as a means to enable intuitive concurrent, reactive, and time-aware programming of microcontrollers. LF offers lightweight concurrent objects called reactors, which can be composed in a network. Reactors do not share state but communicate through message passing via named ports. Deterministic concurrency is ensured on the basis of the topology of the reactor network. LF has a semantic notion of time that enables specification of arbitrary timing behavior of a program. Being a coordination language, not a general-purpose programming language, LF lets the programmer express computation in a target language of their choice. The Zephyr support implemented and evaluated in this paper is based on the C target. We compare the implementation of typical patterns in embedded system software using the timed reactor-based concurrency model against achieving the same kinds of behavior with threads.

The remainder of the paper is organized as follows. Section B.2 briefly introduces the reactor-oriented paradigm followed by LF and its suitability for embedded system design. Section B.3 takes a look at related work. Section B.4 introduces the Zephyr support layer we developed for LF. In Section B.5 we compare the performance of LF on Zephyr versus threaded C code running directly on Zephyr. Conclusions are provided in Section B.6.

B.2 Embedded Programming in Lingua Franca

In the reactor-oriented programming paradigm followed by LF, concurrency is exposed by the programmer through the composition of stateful components called *reactors* that encapsulate event-triggered *reactions*, leaving the task of concurrency management to the LF compiler and runtime. Programmers need not deal with locks, semaphores, or other low-level synchronization mechanisms. Any two simultaneously triggered reactions that do not have any data dependencies between them may execute in arbitrary order (or in parallel, on a multicore system) without execution order affecting the effects of those reactions. It is the runtime system, not the programmer, that is in charge of ensuring that data dependencies are observed. This is what makes LF programs deterministic by default.

When events are not logically simultaneous, the LF runtime guarantees that they will appear to be processed in order according to their positions, called *tags*, on a totally ordered logical timeline. Through its correspondence to physical time, as measured by the system clock, this logical timeline also determines the program's real-time behavior. Events with well-defined logical times are produced by hooking up reactors to *timers* that produce events periodically, or by scheduling events in reaction code via *actions*. LF's timing constructs can be parameterized to let the runtime control the offsets, periods, and spacing between events. Lo-

gical delays and deadlines can be used to specify timing constraints and to handle possible timing violations.

LF separates a declarative specification of a program's coordination fabric from the program's imperative code, which is contained in reaction bodies and written in a target language of choice. The LF toolchain capitalizes on this by providing automatically synthesized interactive diagrams in the IDE. In the diagrams, reactors are shown as rounded boxes. Small black triangles denote ports, and reactions are represented by chevrons. Reactions can be triggered by events originating from upstream ports or actions, connected via dashed lines. There are also distinguished startup and shutdown triggers that, respectively, have events at the very first and very last time instant of program execution. Dashed lines also connect a reaction to its *effects*, which are downstream ports or actions that it can produce events on. Events produced on ports are always instantaneous; those produced on actions are always scheduled to occur at a future time.

To express modal behaviors, where the software has to switch between different modes of operation, LF also has syntax for grouping a reactor's elements in mutually exclusive regions called *modes*. A modal reactor has a single initial mode and can have several other modes. A transition from one mode to another is initiated in reaction code and requires the reaction to specify the target mode among its effects. A mode can be entered via a reset transition (the default) or a history transition. The former initializes all timers and resets any state marked to be reset; the latter resumes operation where it left off when the mode was last exited. Modes and transitions are rendered visually in the diagrams as highlighted areas with arcs between them. The initial mode is marked by a thicker border.

The LF runtime can be compared to an RTOS kernel. Like an RTOS, it supports message-passing and context-switching between concurrently executing components. The message-passing is done by setting up direct references from input ports to the output ports they depend on. Ports carry immutable data by default which enables zero-copy communication. There are two types of context-switches performed by the LF runtime. The first, referred to as *reaction-switching*, involves scheduling a new reaction at the same tag a the previously executed reaction. The second type of context-switch requires advancing the logical time of the program. This is referred to as *logical time advancement*. It might include putting the system into low-power sleep to wait for the physical time to reach that particular value.

B.2.1 Embedded design patterns

In the following sections we will show how three common design patterns in embedded systems can be expressed easily using Lingua Franca.

Button debouncing

When a button is pressed, it may generate multiple spurious open and close transitions. Debouncing is the process of filtering out transitions that are so close in time that they likely are not due to an actual button press or release. LF's timed

```
target C;

reactor ButtonDebounce {
    preamble {=
        void * button_press_action; // Pointer to physical action
        void gpio_pin_isr() {
            lf_schedule(button_press_action, 0);
        }
    =}

    physical action button_press(0, 10 msec)

    reaction(startup) -> button_press {=
        // Setup GPIO pin and register ISR
        ...
        // Make physical action available externally
        button_press_action = button_press;
    =}

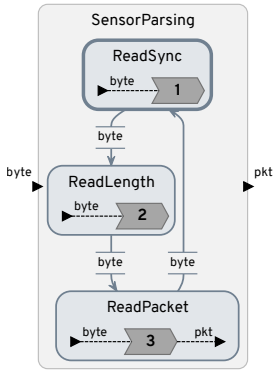
    reaction (button_press) {=
        // Handle
    =}
}
```

Figure B.1: Button debouncing in Lingua Franca

semantics makes such patterns simple to express. To capture asynchronous events, like button presses, and pin them to LF's logical timeline, a *physical* action is used, which can be scheduled from an external thread or ISR. The tag of such an event is derived not from the runtime's current logical time (as an event scheduled on a *logical* action would), but from the systems's physical time. An action can be assigned a minimum spacing, which will make the scheduler reject events scheduled too close to one another. Fig. B.1 shows how this is expressed in the C target of Lingua Franca. The physical action *button_press* has a minimum spacing of 10 ms, and the runtime system handles the debouncing.

Sensor parsing

Consider the common application of parsing an incoming packet from an external sensor connected through a digital communication interface. Figs. B.2b and B.2a show the implementation and diagram of a generic packet parser reactor in LF. Parsing a stream of bytes is conveniently expressed as a state machine. LF has native support for encoding state machines through modal reactors. The *SensorParsing* reactor has an input port for incoming bytes and an output port for the parsed packet. How the reactor responds to data on its input port depends on its current mode. In the initial mode, *ReadSync*, the incoming bytes are compared against the known sync ID of the packet. Upon finding the sync bytes, the reactor performs a mode change to *ReadLength*, where it reads the length field of the packet. When the length has been decoded, the reactor moves to the *ReadPacket* mode, reads the packet, and outputs the result on the *pkt* output port.



(a) Diagram

```
1  reactor SensorParser {
2    input byte:char
3    output pkt:sensor_pkt_t;
4
5    initial mode ReadSync {
6      reaction(byte) -> ReadLength {=
7        // Search for sync-byte ...
8        lf_set_mode(ReadLength);
9      =}
10   }
11
12   mode ReadLength {
13     reaction(byte) -> ReadPacket {=
14       // Read length field ...
15       lf_set_mode(ReadPacket);
16     =}
17   }
18
19   mode ReadPacket {
20     reaction(byte) -> pkt {=
21       // Read packet into p ...
22       lf_set(pkt, p);
23     =}
24   }
25 }
```

(b) Source code

Figure B.2: A sensor packet parser in Lingua Franca

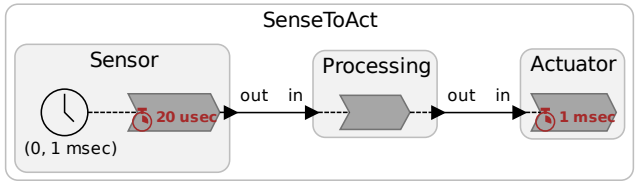


Figure B.3: Tight control loop application in Lingua Franca

Control loops

Another common design pattern is that of tight control loops in, for example, robotics and control. Consider the LF program depicted Fig. B.3. It consists of a *Sensor* reactor that reads from a sensor every 1 ms. The sensor values are passed downstream to the *Processing* reactor, which implements a control algorithm. Finally, the control set points calculated by *Processing* are applied to the actuators in the *Actuator* reactor.

Notice that there are two deadlines. First, the *Sensor* reaction is associated with a 20 us deadline. This is a way of expressing a bound on how much jitter we accept in our sensor readings. Likewise, the *Actuator* reactor has a deadline of 1 ms. This corresponds with the period of the control loop and expresses that the overhead starting from obtaining a reading from *Sensor* up to right before

invoking *Actuator* should be less than 1 ms. The timed semantics of LF provides an intuitive way to express such control loops and their timing requirements.

B.3 Related Work

There is a rich body of work exploring ways of raising the abstraction level for embedded software design. SynchronVM [9] is a virtual machine running on top of Zephyr OS or ChibiOS targeting microcontrollers. The underlying concurrency model is based on message passing and resembles communicating sequential processes (CSP) [13]. In SynchronVM one can specify a time window in which a synchronous message passing should occur. The time window informs the scheduling of the processes but does not have a well defined semantics. The underlying model of computation, CSP, is fundamentally nondeterministic as it allows processes to wait for events on multiple channels and synchronize with the first available.

MicroPython [14] is a project bringing the interpreted and garbage-collected high-level language Python to microcontrollers. The MicroPython interpreter supports REPL (Read-Evaluate-Print-Loop) interaction, which greatly simplifies prototyping. However, MicroPython requires FLASH memory on the order of 256KB, and its garbage collection and dynamic typing make it unsuited for real-time and safety-critical applications.

Medusa [15] is a concurrent programming language based on the Actor model [16]. Medusa is an extension of Python and runs an Owl Python runtime for microcontrollers. The Owl runtime has a code size of 38KB, and each Actor requires 100B of RAM. In Medusa, the hardware is also modeled as actors, and the peripherals communicate with the rest of the system through message passing performed by the interrupt service routines. Medusa does not have a semantic notion of time, and the underlying actor model is nondeterministic.

xC [17] is a C-based programming language designed for the Xcore [18] CPU architecture from XMOS. xC also has CSP as its underlying model, but it targets a CPU architecture with hardware support for processes and rendezvous. As such, there is no runtime providing a message-passing API. xC has language primitives representing hardware timers, which provide sleeping and waiting. The Xcore architecture is timing-predictable, and the tools support worst-case execution time analysis.

B.4 Design and Implementation

Fig. B.4 shows the software stack supporting LF on embedded systems. The Zephyr OS [8] forms the foundation of this stack. This was chosen over a bare-metal target for multiple reasons. First of all, the APIs of the Zephyr kernel and drivers are platform independent, and, at the time of writing, Zephyr supports more than 400 boards. Zephyr also comes with many useful drivers for communication protocols like Bluetooth Low Energy (BLE), Ethernet, USB, and CAN buses. This

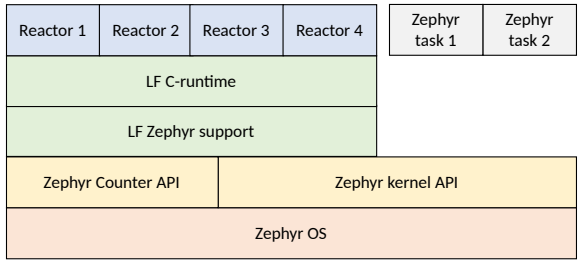


Figure B.4: Lingua France runtime stack

makes Zephyr an attractive framework even for single-threaded applications. Using Zephyr also enables simple integration of LF programs with existing libraries like TCP/IP stacks or clock synchronization implementations. These can simply execute in separate threads and be scheduled by the Zephyr kernel. Lastly, for running LF on multicore platforms, a threading library is needed in order to make use of all the cores. Zephyr implements a POSIX-compliant threading library.

In this paper we focus on the single-threaded LF C runtime. Like any LF target port, this one must implement a few key functions to (1) enter and leave critical sections; (2) sleep for a duration or until interrupted asynchronously; and (3) read a monotonically increasing physical clock. A critical section is created by disabling all interrupts; this functionality is exposed by the kernel API. Sleeping can be achieved in various ways, but the Zephyr kernel provides the function k_sleep which puts the calling thread to sleep for a number of clock ticks. A physical clock can be read using $k_cycle_get_32$. However, on many platforms, this clock, which underlies the kernel services, is a low-frequency 32KHz clock. This might not be sufficient for high-precision control or highly dynamic robots [11]. Most platforms will also have high-frequency clocks which can drive hardware timers. Such hardware timers can be used through the platform-independent Counter API. The Counter API is not part of the kernel API and might not be implemented for all platforms.

For multi-threaded support, a full POSIX-like implementation must be provided. Zephyr does implement the POSIX API, but again the granularity of the timing services is too coarse. Here, we use the Counter API again to provide high-precision sleeps.

For platforms that do not have hardware timers or do not implement the Counter API, the Zephyr kernel's timing primitives are used. This drastically reduces precision, but it enables testing and running LF programs on QEMU emulations [19], which do not simulate hardware timers.

B.5 Evaluation

The evaluation was done on a NRF52832 development kit from Nordic Semiconductor [20]. It has an Arm Cortex M4 microcontroller with 64KB RAM and 512KB

	Runtime	Reactor	Reaction	Timer	Action	Mode	Port(In/Out)	Event
Code	9.8KB(1KB)	96B	62B	88B	174B	36B	32B/16B	-
Data	266B(170B)	72B	80B	60B	72B	28B	104B/40B	32B

Table B.1: LF code and data size (B is bytes)

flash clocked at 64MHz. Benchmarks were compiled with the west build tool version 0.14.0 using Zephyr version 3.2.0.

B.5.1 Memory footprint

Memory footprint is a critical property for most microcontroller applications. In the following, we provide a detailed evaluation of both the flash and RAM footprints of LF.

Code size

The top row of Table B.1 shows the code size of different LF components. The runtime itself consumes less than 10KB and can be further reduced by 3KB if the program has no modal reactors. The 1KB in the parenthesis is the implementation of the Zephyr Counter driver. The Zephyr kernel adds around 19KB which makes a total of less than 30KB. For comparison, Medusa [15] requires a total of 38KB, Espruino [21] and MicroPython [14] requires 100KB-200KB and SynchronVM [9] requires 32KB.

A minimal reactor with just a single reaction triggered at startup only requires 96B. Adding reactions, timers and modes all add less than 100B of overhead. The user code in the reaction bodies, as well as the SDK components they depend on, will add to this.

As we will see next, the limiting factor for platforms like the NRF52 is not the size of the program memory but rather the size of RAM.

Data size

The bottom row of Table B.1 shows the RAM footprint of LF. It consists of three parts. First, there is the constant size of the runtime, which only amounts to 265B plus 170B for the Counter implementation. Then there is the memory footprint for each LF component like reactions, actions and ports. This is known at compile-time but is currently being allocated dynamically in the initialization routine. Last, we have the events being communicated between the reactors over the ports through the event queue. These events are allocated as needed at run-time. When an event has been consumed, it is recycled to avoid memory fragmentation. Due to the expressiveness of the Reactor model of computation, there is, in general, no upper bound to the size of the event queue. There exist syntactically valid pathological LF programs for which the event queue can grow without bound. For instance, consider the reactor *Unbounded* in Fig. B.5a. Each time the action a triggers, two events will be added to the event queue. The first triggers a at the

```
1  target C;
2
3  reactor Unbounded {
4      logical action a;
5      logical action b
6      state i:int(0)
7
8      reaction(a,b) -> a,b {=
9          lf_schedule(a,0);
10         lf_schedule(b,++self->i);
11     =}
12 }
```

(a) LF program with unbounded event queue

```
1 target C;
2
3 reactor Bounded {
4     timer t1(0, 10 msec)
5     timer t2(0, 50 msec)
6     reaction(t1) {= =}
7     reaction(t2) {= =}
8     reaction(t1,t2) {= =}
9 }
```

(b) LF program with bounded event queue

Figure B.5: Memory requirements for LF programs

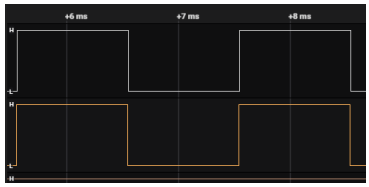


Figure B.6: Logical analyzer dump

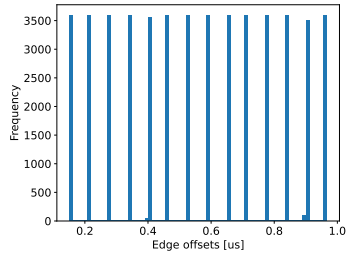


Figure B.7: Actuator driving precision

same tag, while the second triggers *b* at a future tag. Only events triggering *a* will ever be handled and the events triggering *b* will accumulate. Such a program is said to exhibit a *stuttering Zeno* condition [22].

However, there also exist classes of "safe" LF programs for which upper bounds on the event queue can be computed. Consider for instance the reactor *Bounded* in Fig. B.5b. It is a simple reactor with two periodic timers and three reactions triggered by them. The size of the event queue for this reactor will never exceed two. At any time during execution, there is one event per timer on the event queue. In fact, any LF program that uses only timers to drive logical time forward will have a statically computable bound on the size of its event queue.

B.5.2 Timing precision

One of the advantages of using LF for embedded applications is its timed semantics. An implementation of LF has good timing precision if the physical time at which a reaction is executed is close to the logical time at which it was scheduled to execute.

Square-wave generator

To test the timing precision, the NRF52 was configured to generate a 1 KHz square wave on an output pin using hardware timers. We created a simple LF program to generate the same signal using a timer and a reaction that toggles a GPIO pin.

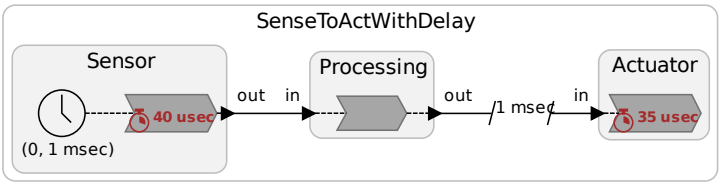


Figure B.8: Tight control loop with after-delay

Fig. B.6 shows the two resulting square waves as measured by a Salaea Logic Pro logic analyzer. Fig. B.7 shows a histogram of the offset between the LF square wave and the hardware generated square wave. Clearly, the LF runtime can deliver precise and repeatable timing. All observed offsets were less than 1 μs. The multi-modality of the distribution is due to inaccuracies associated with waking up from sleep. This could be improved by targeting a lower-level timer API; however, such an approach would not be platform-agnostic. An important observation is that this timing precision can be affected by independent long-running reactions due to barrier synchronization at each time tag. We are working on new scheduling strategies and language primitives to improve timing precision.

Tight control loops with load

Consider again the SenseToAct example from Fig. B.3 in Section B.2.1. An interesting performance metric is the load a system can sustain while still meeting its deadlines. By introducing busy-waits in the reaction bodies, different workloads can be emulated. Our experiments show that the runtime overhead for SenseToAct is only 70 μs. This leaves 93% of the 1 ms period for actual computation.

An important observation is that the semantics of an LF deadline only puts an *upper bound* on the lag of a reaction invokation. In fact, for Fig. B.3 there is a full 1 ms interval in which the *Actuator* reaction can legally be invoked. To introduce a *lower* bound on the invokation time we can use logical delays. In Fig. B.8 we have introduced a 1 ms logical delay on the connection between *Processing* and *Actuator*. This means that any output produced by *Processing* will be visible at *Actuator* one *logical* ms later. This, together with the deadline, specifies an interval of 35 μs in which it is acceptable to invoke the *Actuator* reaction. This is a portable way of explicitly specifying bounds on jitter. Notice also that this will coincide with the next evaluation of *Sensor*, so we have, in effect, made a pipeline.

With this configuration, we can execute *Processing* for 885 μs, without missing any deadlines. This yields a utilization of 88.5%. Notice that both the utilization is lower and the deadlines are higher. The reason for this is that the logical delay on the connection is syntactic sugar for a reactor consisting of two reactions and a logical action, which introduces some additional overhead [23].

The execution of this program without any deadline misses means that the runtime claims that timing is not being violated. To verify this, we instrumented

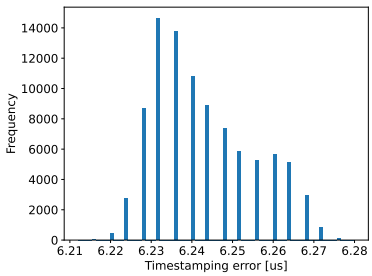


Figure B.9: Physical action precision

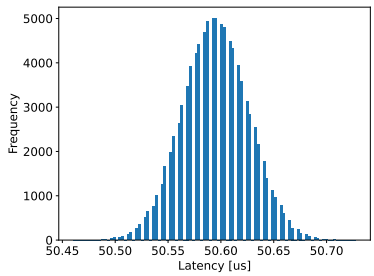


Figure B.10: Physical action latency

hardware timers to generate events on GPIO pins and compared with events generated from within the reaction bodies of *Sensor* and *Actuator*.

The results show offsets of around 28 μs and 35 μs for sensing and actuating, respectively. This is well below their deadlines. *Sensor* is lagging more than *Actuator*, this is expected as their reactions are logically simultaneous. On single-core platforms, simultaneous reactions are ordered based on earliest deadline first scheduling.

Asynchronous events

The ability to react to asynchronous events in a timely manner is crucial for many embedded systems. For instance, a sensor might signal that it has made a new sample by creating a transition on a pin. There are two important timing properties of a reaction to such an event. (1) The precision with which the system *timestamps* the occurrence of the event and (2) *latency* from occurrence to being handled by the system.

Fig. B.9 shows the latency between the invokation of an interrupt handler that schedules a physical action and LF's timestamping of the resulting event. The low jitter indicates that a constant offset of around 6.22 μs could be subtracted from the timestamp associated with the physical action.

Fig. B.10 shows the latency from asynchronously scheduling a physical action until a reaction triggered by the physical action is invoked. The latency is normally distributed with a mean of 50 μs.

The reported overheads are in addition to the interrupt latency of the underlying platform.

B.5.3 Performance

To assess the general performance of LF on microcontrollers, we run a relevant subset of the Thread-Metric and Savina benchmarks against a baseline implementation using Zephyr's thread APIs. At a fundamental level, LF's scheduling and context-switching overhead is compared to Zephyr's overhead of message-

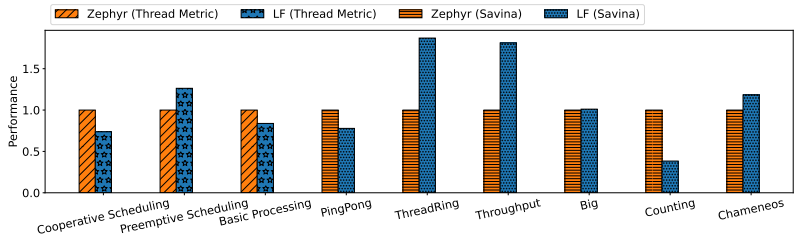


Figure B.11: Results of Thread-Metric and Savina benchmark suites for Lingua Franca and a threaded implementation targeting Zephyr kernel API

passing and context-switching. This is evaluated for multiple different program topologies and workloads. In the following, we will use the term *actor* to refer to a concurrent component. In the threaded baseline, it corresponds to a thread, and in the LF implementation, it corresponds to a reactor.

Thread-Metric

Thread-Metric [24] is a benchmark for measuring performance of RTOSes. It consists of seven benchmarks that measure overheads associated with different RTOS services like message passing, synchronization, scheduling, interrupt latency and memory allocation. LF does not expose low-level APIs for interrupts, memory allocation and message buffers to the user. In this evaluation, we have thus excluded Message Processing, Synchronization Processing, Interrupt Processing, and Memory Allocation.

The left part of Fig. B.11 shows the results of the remaining Thread-Metric benchmarks normalized to the threaded baseline. Performance is measured as the number of iterations through the program during a fixed period. In Cooperative Scheduling, LF achieves 74% of the performance of Zephyr threads.

We measured the cost of a Zephyr thread yield to be 8 μs while LF's reaction-switching and logical time advancement take 4 μs and 29 μs respectively. This means that the LF implementation would need eight low-overhead reaction switches to make up for the relatively expensive time advancement. Cooperative Scheduling only consists of five actors and thus renders LF's performance worse than the baseline.

In the Preemptive Scheduling benchmark, LF outperforms threads by 26%. The reason is the long chain of preemptions occurring when the lowest priority thread executes first. In LF this is not a problem as there is no priority-based scheduling, rather, the topology of the reactor network imposes well-defined restrictions on scheduling. In LF, the preemption is translated into reaction-switching followed by a logical time advancement in each iteration. The threaded baseline must perform twice as many context-switches as reaction-switches in the LF version, due to the priority-based preemption.

Basic Processing only involves a single actor, in a while loop, performing a long-running computation before incrementing a counter. LF is outperformed as

it performs logical time advancement after each iteration.

Savina

The Savina benchmark suite is an actor-oriented benchmark suite [25]. The Savina benchmarks focus on compute-intensive rather than IO-intensive applications and put emphasis on message-passing overhead. We evaluate LF on a subset of the Savina benchmarks called the micro benchmarks. The right portion of Fig. B.11 shows the result normalized to the Zephyr baseline.

PingPong only consists of two actors sending a message back and forth. The Zephyr implementation uses a message queue [26], which incurs around 8 μs overhead for each communication. In LF the same communication only takes around 6 μs. However, LF advances logical time after each round of back and forth. This takes almost 30μs. In other words, LF performs better when there are long chains of reactions between each logical time advancement. Counting suffers from the same problem. While PingPong has three reactions per logical tag, Counting only has two. Moreover, the threaded implementation of Counting allows for two iterations per context-switch where PingPong requires a context-switch between each ping and pong. This is because we are using a message queue of size 1 which can buffer one element and thus enable two elements communicated per context-switch.

Throughput and ThreadRing are examples at the other extreme. In Throughput, a single producer sends a message to 60 consumers, each performing some floating-point arithmetic on the received data. In LF, this constitutes a chain of 60 reactions per logical time advancement. The Zephyr baseline performs a normal context-switch for each communication. ThreadRing is similar only there is a a chain of consumers rather than the one-to-many topology of Throughput. Again the low reaction-switching overhead of LF enables a 87% and 81% improvement over Zephyr, respectively.

Big is a many-to-many benchmark where each actor sends a ping to a random actor and waits for a response. Due to limitations in RAM we only use 10 actors. This constitutes a chain of reactions just long enough to make up for the expensive logical time advancement.

Chameneos measures how contention on a shared resource affects the performance. In this benchmark, a large number of creatures called chameneos send messages to a single mall, which pairs up two chameneos for a meeting. The Zephyr threading API and the LF runtime both allow the mall to process messages in batches without context-switching between each message. This lets both implementations attain good performance because the mall processes a large volume of messages in comparison to the other actors in the program.

B.6 Conclusion

We have introduced the embedded target of Lingua Franca, which brings deterministic concurrency and portable specification of timing behaviors to resource-constrained platforms like microcontrollers. The embedded target is built on top of the popular Zephyr RTOS. The runtime is evaluated in terms of memory footprint, timing precision and general performance against baselines using Zephyr's shared memory concurrency primitives directly. We show that the LF runtime adds little overhead and in several cases outperforms the baseline.

Bibliography

[1] W. Wulf and M. Shaw, 'Global variable considered harmful,' *SIGPLAN Not.*, vol. 8, no. 2, pp. 28–34, Feb. 1973, ISSN: 0362-1340. DOI: 10.1145/953353.953355. [Online]. Available: https://doi.org/10.1145/953353.953355.

[2] P. Haller and M. Odersky, 'Event-Based Programming Without Inversion of Control,' in *Modular Programming Languages*, D. E. Lightfoot and C. Szyperski, Eds., ser. Lecture Notes in Computer Science, Berlin, Heidelberg: Springer, 2006, pp. 4–22, ISBN: 978-3-540-40928-1. DOI: 10.1007/11860990_2.

[3] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham and S. Savage, 'Experimental security analysis of a modern automobile,' in *2010 IEEE Symposium on Security and Privacy*, 2010, pp. 447–462. DOI: 10.1109/SP.2010.34.

[4] P. Koopman, 'A case study of Toyota unintended acceleration and software safety,' *Presentation. Sept*, 2014.

[5] N. G. Leveson and C. S. Turner, 'An investigation of the Therac-25 accidents,' *Computer*, vol. 26, no. 7, pp. 18–41, 1993.

[6] E. A. Lee, 'The problem with threads,' *Computer*, vol. 39, no. 5, pp. 33–42, 2006.

[7] W. Xiong, S. Park, J. Zhang, Y. Zhou and Z. Ma, 'Ad hoc synchronization considered harmful,' in *Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'10, Vancouver, BC, Canada: USENIX Association, 2010, pp. 163–176.

[8] T. L. Foundation., *Zephyr RTOS*, https://zephyrproject.org/, 2022.

[9] A. Sarkar, B. J. Svensson and M. Sheeran, *Synchron – an API and runtime for embedded systems*, 2022. DOI: 10.48550/ARXIV.2205.03262. [Online]. Available: https://arxiv.org/abs/2205.03262.

[10] T. Mikkonen and A. Taivalsaari, 'Web applications – spaghetti code for the 21st century,' in *2008 Sixth international conference on software engineering research, management and applications*, IEEE, 2008, pp. 319–328.

[11] E. R. Jellum, T. H. Bryne, T. A. Johansen and M. Orlandic, 'The syncline model–analyzing the impact of time synchronization in sensor fusion,' *arXiv preprint arXiv:2209.01136*, 2022.

[12] M. Lohstroh, C. Menard, S. Bateni and E. A. Lee, 'Toward a lingua franca for deterministic concurrent systems,' *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 4, May 2021, ISSN: 1539-9087. DOI: 10.1145/3448128. [Online]. Available: https://doi.org/10.1145/3448128.

[13] C. A. R. Hoare, 'Communicating sequential processes,' *Commun. ACM*, vol. 21, no. 8, pp. 666–677, Aug. 1978, ISSN: 0001-0782. DOI: 10.1145/359576.359585. [Online]. Available: https://doi.org/10.1145/359576.359585.

[14] G. Robotics, *MicroPython - Python for microcontrollers*, micropython.org, 2022.

[15] T. W. Barr and S. Rixner, 'Medusa: Managing concurrency and communication in embedded systems,' in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIX ATC'14, Philadelphia, PA: USENIX Association, 2014, pp. 439–450, ISBN: 9781931971102.

[16] G. Agha and C. Hewitt, 'Actors: A conceptual foundation for concurrent object-oriented programming,' in *Research Directions in Object-Oriented Programming*. Cambridge, MA, USA: MIT Press, 1987, pp. 49–74, ISBN: 0262192640.

[17] D. Watt, *Programming XC on XMOS devices*. XMOS Limited, 2009.

[18] D. May, 'The XMOS architecture and XS1 chips,' *IEEE Micro*, vol. 32, no. 6, pp. 28–37, 2012.

[19] F. Bellard, 'Qemu, a fast and portable dynamic translator.,' in *USENIX annual technical conference, FREENIX Track*, Califor-nia, USA, vol. 41, 2005, pp. 10–5555.

[20] N. Semiconductor, *nRF52 DK*, https://www.nordicsemi.com/Products/Development-hardware/nRF52-DK, 2022.

[21] G. F. Williams, *Making Things Smart: Easy Embedded JavaScript Programming for Making Everyday Objects into Intelligent Machines*. Maker Media, Inc., 2017.

[22] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems - A Cyber-Physical Systems Approach*, Second. Cambridge, MA, USA: MIT Press, 2017. [Online]. Available: http://LeeSeshia.org.

[23] H. M. F. Lohstroh, *Reactors: A deterministic model of concurrent computation for reactive systems*. University of California, Berkeley, 2020.

[24] iSotEE, *Thread Metric Test Suite*, https://github.com/iSotEE/thread_metric_test_suite, 2020.

[25] S. M. Imam and V. Sarkar, 'Savina - an actor benchmark suite: Enabling empirical evaluation of actor libraries,' in *Proceedings of the 4th International Workshop on Programming Based on Actors Agents & Decentralized Control*, 2014, pp. 67–80.

[26] Zephyr, *Message Queues*, `https://docs.zephyrproject.org/3.2.0/kernel/services/data_passing/message_queues.html`, 2022.

Paper C

Composable distributed real-time systems with deterministic network channels

Henrik Austad
SINTEF Digital
Trondheim, Norway

Sverre Hendseth
NTNU
Trondheim, Norway

Torleiv Håland Bryne
NTNU
Trondheim, Norway

Sigurd Mørkved Albrektsen
SINTEF Digital
Trondheim, Norway

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Geir Mathisen
NTNU
Trondheim, Norway

Kristoffer Nyborg Gregertsen
SINTEF Digital
Trondheim, Norway

Bjarne Emil Helvik
NTNU
Trondheim, Norway

Published in
Journal of Systems Architecture

Abstract

A system that needs to interact with the physical world in a timely manner is called a *real-time system*. When such a system is composed of multiple subsystems, or nodes, each of which is a geographically separate system, such a system of systems is called a *distributed* real-time system. The computation at each node must adhere to the timing requirements, and the connecting communication channels must never cause delays that trigger further timing violations. In this paper, we introduce *Deterministic Network Channels*, a network construct using Time Sensitive Networking QoS mechanisms that add reliable and deterministic communication for distributed tasks. Introducing such network channels as a construct allows designers to focus on higher-level primitives when building distributed systems. We describe our reference implementation and evaluate it by extending Timed C with network channels. A thorough performance evaluation to determine practical bounds for both Linux and TSN under heavy workloads and adverse network conditions to show how the proposed reference implementation performs in real-world scenarios. In our tests, we can synchronize two separate machines running commercial off-the-shelf hardware to within 15μs of each other under severe internal and external interference. All the software used in the experiments is available under open-source licenses and hosted on GitHub.

C.1 Introduction

In a real-time system, the correctness of a computation is not only dependent on its logical output, but also on the time at which the computation is ready. Meeting the temporal requirements of a system is a challenging task as most hardware architectures are not deterministic to time. Neither are most programming models that leave the programmer to fight temporal demons such as enforcing deadlines and handling timeouts on their own. Both require a deep understanding of both the problem domain and the hardware used for the system and often result in subtle, yet devastating timing bugs. A common technique is to configure timers to interrupt the program at specific points to prune away some of this non-deterministic timing. This increases overall complexity by exposing minute details about hardware capabilities, -configuration, and interrupts. It also shifts focus away from the core application being developed to the system on which the application runs.

The C programming language continues to be the most popular language for embedded systems. C is a small language, it has few keywords, a limited set of standard libraries, and, by allowing direct memory reference, exposes the underlying hardware to developers. This is a tremendous expressive power - and danger, causing many to consider C an unsafe language. What is more, as the design of C lends itself naturally to compilers and hardware, writing a *C implementation* (i.e., "compiler") is a relatively easy task, which is another reason why C is normally the first language supported on a new architecture. This makes C a common language for small and embedded systems.

Timed C is an extension of the C programming language developed at the KTH Royal Institute of Technology [1]. It adds timing primitives to the programming

language and thus abstracts away the configuring of timers and interrupts with a portable API. Timed C brings time as a "first-class member" and facilitates communication between local tasks. Currently, it does not support network transport, preventing Timed C from extending to distributed systems design. One reason for Timed C's "non-distributedness" is largely the network itself. The uncertainty in both delivery (i.e. packages being lost in transit) and delivery jitter (i.e. variance in transmission delay through the network) in ordinary packet-switched networks (PSN) must be properly addressed before PSN can be safely used in real-time systems.

As the world progresses towards "Industry 4.0" with Industrial IoT (IIoT) and Cyber-Physical Systems (CPS), the complexity and size of industrial systems continue to grow and expand. This shift to distributed architectures can effectively mitigate complexity as encapsulating each function in a separate node lends itself well to fault tolerance where critical functions are physically separated. However, with this distribution also comes increased complexity in managing the network connection between the nodes.

In this paper, we extend the Time Flow Graph formalism first described in Timed C with primitives for setting up and using deterministic *network channels*. We then describe a reference implementation and implement this on top of Time Sensitive Networking (TSN) streams and Linux before we show how this can be used to extend the capabilities of Timed C. This simplifies building scalable and composable distributed real-time systems (DRTS) in C. We then evaluate the real-time performance of such network channels running on a GNU/Linux system.

The rest of this paper is organized as follows: in Section C.2 we list the specific contributions made by this work. Section C.3 covers the background in real-time systems, TSN, and Timed C. Section C.4 presents our reference implementation, a federated architecture extension to Timed C. Section C.5 describes the experimental setup for our reference implementation. The results of these experiments are shown in Section C.6 before we conclude in C.7.

C.2 Contributions

In this work, we add support for deterministic network channels that provide guaranteed delivery, bounded latency, and ease of use. We have extended Timed C, a coordination framework for real-time tasks, such that it can be used to create DRTS which we have called *"Federated Timed C"*. In summary:

- C1: We have extended the Time Flow Graph formalism from Timed C to handle network channels for distributed systems and defined the requirements for composability.
- C2: We have extended Timed C by bringing network channels as a composable construct to tasks, enabling a distributed system to be designed as a Timed C program.
- C3: We have specified and implemented reference code with a reliable, determ-

inistic network transport construct, including bounded latency that allows reasoning about timing constraints in distributed systems.

C4: We have quantified the network transport latency and Linux's real-time capabilities to further strengthen the confidence in the latency bounds provided by the network channels.

The developed code is available under an open-source license at GitHub [2]with a preconfigured TimedC for also available [3]. Finally, all the tools used for generating interference are also available under open licenses. This is further discussed in Section C.5.2.

C.3 Background and Motivation

Broadly speaking, a Distributed Real-Time System (DRTS) must contain at least 3 components to be able to exhibit what we call "real-time behavior". First, each node in the system must be a capable real-time system. Second, the network connection between the nodes must be both reliable and deterministic in that no data should be lost nor should it be unduly delayed. Last, all nodes must share a common understanding of time.

C.3.1 Real-time Systems

The temporal requirements of the components of the real-time system are specified through deadlines. Depending on the consequence of missing a deadline, real-time systems are broadly divided into 3 categories: *a) Soft real-time*, where a missed deadline is not fatal but the value of the result declines rapidly; *b) Firm real-time*, where the value of a delayed result is worthless, but the system may yet recover; and *c) Hard real-time* systems where a missed deadline results in a total system failure. Where soft real-time systems need care and attention, hard real-time systems often require a combination of rigid and well-structured development processes, formal verification, thorough testing, and certified toolchains and operating systems.

A real-time system is typically composed of several modules or subsystems where each serves a distinct role. Not all parts may have the same real-time requirements, and requirements may not fall firmly into one category, but rather be "somewhere in between". It is quite common to have sections with soft to no real-time requirements whereas others have more stringent requirements. In such mixed-criticality cases, it is essential to have a clear separation of dependencies and execution. The system may run in multiple threads in the same process, as separate processes located on isolated CPU cores - or something in between. Common for all such cases is the need for a methodical design approach and a robust and deterministic run-time.

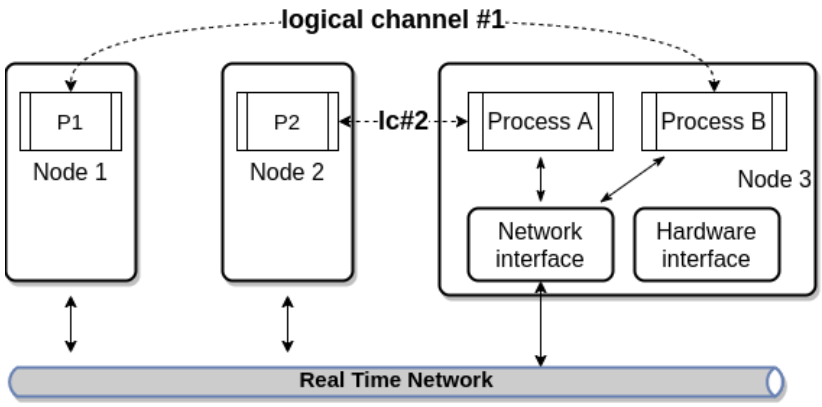


Figure C.1: Distributed real-time system with logical channels connecting nodes on 3 different hosts. In the expanded view of Node 3, we see that it contains two separate processes that both use the same network interface, but where the *logical channels* are distinctly between Process A and P2 and between process B and P1.

C.3.2 Distributed Real-Time Systems

DRTS is a "system of systems" where each subsystem itself is a real-time system. The subsystems are interconnected via a real-time network [4]. Figure C.1 shows a construed DRTS. In each node, there are one or more processes that run the node's part of the system. Between the nodes are *logical channels* through which the processes pass either control signals, data, or both. Depending on the abstraction level, these channels appear either as any other synchronization (or data sharing) primitive, a raw network socket, or something in between. These logical channels are ultimately handled by the network layer and are thus dependent upon the real-time behavior of the underlying real-time network.

There are several reasons to choose a distributed architecture for a real-time system. First of all, the physical layout of the system may be distributed and thus lend itself naturally to a distributed architecture. Second, we mitigate a major drawback of single-host architecture, namely *decreasing composability*. When a "single-host system" grows, it becomes increasingly more difficult to contain the side effects of CPU load, available memory, traffic on shared data buses, changes in scheduler timing, and so on. In other words, it is difficult to shield some parts of an application from one other on such a system. Attempting to handle this may in turn lead to increased complexity when new components are added, making the quest for subsystem shielding even more difficult.

Extending a distributed architecture can be straightforward as long as shared resources and data dependencies are avoided. It can be as simple as adding a new node to the system. Fault tolerance can also be easier to implement in a distributed architecture because different functions and resources are already cleanly

separated. This naturally lends itself to a design where critical functionality can easily be duplicated on multiple nodes.

The challenge lies in extending a DRTS with overlapping functionalities. The moment one node is dependent upon signals or data arriving from another node, the real-time behavior of the other nodes *and* the network becomes relevant. Thus, the delay *between* the nodes directly affects the system behavior.

C.3.3 Composability and Composable DRTS

A system is *composable* to a property if, once the property has been established for a subset of the nodes, the addition or removal of nodes does not affect the property [5]. For DRTS, the timing and network latency are relevant properties. In a non-composable system, the timing behavior of an individual component could change if new components were added to the system. An example of such a system is a multi-threaded program running on a single-core CPU managed by a non-real-time operating system. When each component is in a separate thread, any additions will change the timing. In general, the question of composability arises in systems with resource sharing. In the multi-threaded example CPU, memory and storage are other examples of shared resources. For a DRTS, the network must be managed as another shared resource.

A DRTS can only achieve composability over a shared medium if it is possible to properly shield critical network traffic from other, unrelated traffic. For some systems, the sensitivity to disturbance can be so low that *arrival guarantees* are adequate. For other applications, such as industrial protocols and control loops, higher demands are placed on the delivery latency provided by the network. A common technique for reducing the variance in the delay to a minimum is to use a time-triggered communication system [5] to properly shield critical traffic.

In general, you can differentiate between event-triggered and time-triggered communication protocols. In an event-triggered protocol, the control of the timing of the transactions lies with the individual nodes of the DRTS whereas in a time-triggered protocol, the control resides within the communication system. The available throughput is then often multiplexed in the time domain. Traditional PSNs are event-triggered systems, and must therefore have attributes that allow them to behave in a time-triggered manner.

C.3.4 Real-Time Networks and TSN

Unless each node has a dedicated link to every other node, the connecting fabric must provide a way to reduce interference from unrelated traffic. Time Sensitive Networking (TSN) is a set of IEEE standards that are a continuation of Audio/Video Bridging (AVB) [6]. TSN enables time-triggered communication over Ethernet and provides strict Quality-of-Service (QoS) guarantees for time-sensitive or critical traffic. Initially built on top of IEEE 802.3 Ethernet, QoS is governed by IEEE 802.1Q Bridges and Bridged Networks [7] standards. It can also use other packet-switched transport protocols such as 802.11n, MoCA v2, and ITU-T G9960.

TSN uses the concept of Bridges and End Stations to describe network entities that send streams of regular, periodic traffic from one Talker to one or more Listeners. Talkers and Listeners are both End-Stations, Bridges are network entities with 2 or more ports (i.e., switches and routers) that forward traffic through the network. A Talker first announces an available stream before one or more Listeners can subscribe by requesting the necessary capacity through the network. If the network can accommodate the request with available buffers and bandwidth, the reservation succeeds. The Listener then receives data with extremely low packet loss[1] and bounded latency. An administrative upper reservation bound is typically set to 75% of link capacity to ensure that Best Effort (BE) traffic can flow. Any reserved link capacity that is left unused can be freely used by any BE traffic class.

TSN uses shapers to form traffic, and the first shaper introduced was the Credit Based Shaper (CBS, [7, Ch. 35]) that provides a bounded end-to-end (E2E) latency of 2ms (for "class A") over a maximum of 7 network hops in a 100 Mbps network. Class B streams provide a 50ms bounded latency guarantee and include the possibility of 2 wireless links in the path. CBS is a class-based shaper and is designed for constant bitrate, periodic traffic. By actively working to reduce traffic bursts on all bridges, total network burstiness is managed. These guarantees can be given due to the admission control provided when reserving resources. 0The Time Aware Shaper (TAS) [9] provides time-deterministic messaging by employing Time Division Multiple Access (TDMA) through the *gate control lists*. Each item on the list specifies if the gates for the priority queues should be open or closed. By carefully aligning *gateOpen* events for only scheduled traffic across the path, TAS guarantees a 100 μs bounded latency (over 5 hops in a 1 Gbps network). For more sporadic traffic, TSN has a third shaper, the Asynchronous Traffic Shaper (ATS, [10]). ATS is based on the Urgency Based Scheduler [11], which does not require tight time synchronization of all bridges. ATS is thus more scalable than TAS and can handle sporadic traffic with less reservation overhead than CBS.

This comes as a contrast to other QoS schemes such as Integrated Services (IntServ, [12]) and Differential Services (DiffServ, [13]). The former is often called "Hard QoS" as it is based on reserving capacity for individual streams, and, as with TSN, a reservation may fail if inadequate resources are available. Due to the per-stream reservation, IntServ has scalability problems when the number of streams grows and the network increases in size. DiffServ is a class-based QoS and thus avoids many of the scalability problems IntServ faces, but at the cost of sacrificing QoS guarantees for individual streams. A combination of both was shown by Harju and Kivimaki [14] to provide both good QoS and scalability for large-scale IP-based networks.

This combination of IntServ and DiffServ is one of the approaches taken by IETF Deterministic Networking (DetNet, [15]). The goal of DetNet is to "provide a capability to carry specified unicast or multicast data flows for real-time applic-

[1] TSN guarantees no packet loss due to buffer congestion, but traffic can still be lost if the link itself is disrupted or the bridge becomes unavailable. For this, frame replication [8] can be used to create redundant paths.

ations with extremely low data loss rates and bounded latency within a network domain". Unlike TSN, DetNet operates on routable network traffic and does not provide explicit technology recommendations but instead, states desired behavior. A DetNet can be realized over a TSN but is not limited to TSN only. Nor is it required to be homogeneous in the sense that a DetNet can traverse both TSN and Multiprotocol Label Switching (MPLS) networks. DetNet targets large Wide Area Networks (WAN) with explicit real-time demands but does not aim to meet as high demands as TSN which can be as low as 100μs.

C.3.5 Precision Time Protocol

For a component in a system to be able to reason about information from any other component, a shared understanding of time is required. Comparing two sensor readings quickly becomes pointless when the capture time is unknown. This is normally solved in a single system by having a local source of time, e.g., a CPU clock. With a DRTS it is no longer possible to directly share a clock in this manner. In Lamport's seminal paper [16] about time and order of events in systems, he lay the foundation and need for time domains and network time protocols in distributed systems.

In 2002, IEEE published the first version of the Precision Time Protocol (PTP). One of the main goals of PTP was to enable an accurate time-keeping architecture for distributed systems operating in environments where access to high-quality time signals (e.g., atomic clock, Global Navigation Satellite System, GNSS) was either impossible or too expensive. PTP operates with the notion of *clocks* and treats both network infrastructure and endpoints as such. Network equipment is either *transparent clocks* or *boundary clocks* and endpoints are typically *ordinary clocks*. During normal operation, the clocks in the network select a single clock to act as the authoritative source of time, the *Grand Master (GM)*. The GM periodically sends updates, and all other clocks then update the offset and phase difference between their local clock and the GM clock. PTP specifies both a software- and a hardware-driven approach. In the software-driven case, the accuracy is greatly affected by how fast the system can react to new PTP messages. Any uncertainties in how tightly the local and remote times are coupled decrease the clock accuracy. Some network cards are capable of intercepting PTP sync messages and associating a local timestamp with incoming messages. This all but eliminates local variations in reacting to the messages and greatly improves the clock accuracy.

In a system where PTP is supported by both network switches and receiving hardware, the error between the GM and any clock is typically much less than 1μs, and with careful configuration, PTP is capable of even higher accuracy. As an example, Project White Rabbit [17] achieved *sub-nanosecond* accuracy by using FPGAs and Synchronous Ethernet (SyncE) to reduce the remaining jitter throughout the network. In 2008, PTP underwent a large revision PTPv2 is the version supported by most network infrastructures and endpoints today. 2019 saw a new update to what is known as PTPv2.1 [18]. PTPv2.1 is backward compatible with

v2 profiles, but v1 is not compatible with v2.

C.3.6 Linux, PREEMPT_RT and rt-tests

The Linux kernel [19] is a general-purpose operating system (GPOS) kernel, that, when bundled with system libraries, many of which are maintained by GNU [20], creates an open and free POSIX-compliant OS ("GNU/Linux"). PREEMPT_RT [21] started as a series of patches to modify the Linux kernel to create a deterministic real-time operating system (RTOS). It does this by making large portions of the kernel preemptable, most notably system calls (syscalls) and interrupt-handlers. By changing many of the interrupt handlers into kernel threads, they can be scheduled as regular threads. As we will see in Section C.6, using PREEMPT_RT can improve the determinism significantly when the system operates under heavy load.

`cyclictest` is part of the rt-test suite [22] and was originally developed to measure the accuracy of the reworked timer infrastructure in the Linux kernel. It has since proved to be a very effective tool for both profiling the real-time performance of systems as well as pinpointing troublesome drivers and applications since it can direct the Linux kernel tracing subsystem [23].

C.3.7 Timed C

There exist several frameworks, dedicated languages, and extensions that address one or several of the critical elements for DRTS. One such candidate is Timed C and due to its clean design and small size, is imminently suited to extend and experiment with.

Timed C is an extension to the C programming language developed at KTH [1, 24, 25]. It consists of a set of constructs and primitives that allows a programmer to specify the intended timing of a program directly. It can be seen as a way of making time itself a "first-class citizen" of C. Timed C is meant to be portable and generates code for both POSIX and FreeRTOS.

Timed C introduces the concept of *timing points*. Timing points allow the programmer to specify soft, firm, and hard real-time requirements. For example, the program should reach a certain point within a specified deadline. Semantically, timing points create a *logical* timeline for the program. Logical time only progresses at the timing points and program sequences between the timing points take zero logical time. When implemented as a physical system, the program will also exist on a *physical* timeline. A timeline violation occurs if the program arrives at a physical time that is greater than its associated logical time. A timeline violation can also be interpreted as a missed deadline.

The consequence of a missed deadline depends on the type of timing point, which represents the type of real-time guarantee it is associated with. A soft timing point does not change the program flow in case of a missed deadline but instead returns the amount of overshoot so that the application may deal with it. A firm timing point, on the other hand, interrupts the program flow when a deadline is missed and jumps directly to the timing point forcing the currently executing

```
1 int main(void) {
2     init();
3     sdelay(10, ms);
4     while (1) {
5         read_sensor();
6         process();
7         fdelay(20, ms);
8     }
9 }
```

Code listing C.1: Timed C program using `sdelay()` and `fdelay()` to insert timing points in a system that reads and processes a sensor value. After the initial setup the main loop repeats every 20 ms (50 t Hz)

code to be aborted [26, III. B]. A hard timing point will behave like a firm timing point, but instead of jumping to a timing point, it will move to a dedicated error-handling routine. The exact implementation of timing points is outlined in [26]. Currently, hard timing points are not enforced in Timed C on x86 architecture.

Listing C.1 shows a contrived example where timing points can be inserted into a system such that a sensor can be read regularly at 50Hz. If `process()` should exceed the available time, the program will be interrupted and the control flow can be diverted to a rescue routine. In Figure C.2 a timing diagram for initialization and two iterations of the loop can be found. `sdelay()` implements a soft timing point, `fdelay()` a firm.

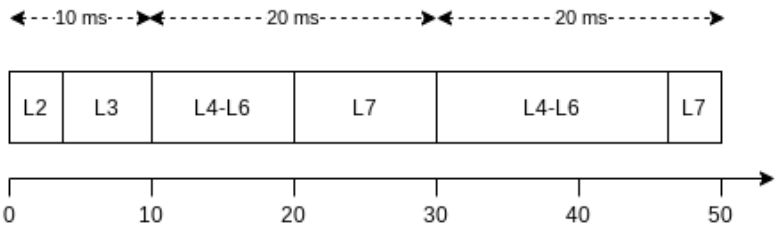


Figure C.2: Timing diagram of code in Listing C.1. Note that L3 and L7 are of variable length, if L2 completes after 4ms, then L3 adapts to spend 6 ms. The loop should run at 50Hz, which indicates a 20 ms timing point. In the first iteration, the lines L4-L6 consume 10ms, and in the second iteration, 16ms. L7 then adapts to ensure a full 20ms loop.

Timed C also contains the concurrent construct *task* which is a thin wrapper around the system multithreading library (pthreads in the case of Linux). Tasks can communicate through *fifochannels* and *lvchannels*. The former gives buffered non-blocking write and blocking read synchronization while the latter is a shared variable without synchronization. Listing C.2 shows a system with 2 parallel tasks communicating over a shared fifochannel. No run-time schedules the different tasks according to the specific timing points, instead Timed C relies on the underlying RTOS for scheduling. Tasks can be assigned to different scheduling policies;

```
1  int fifochannel(fifo);
2  task reader() {
3      spolicy(FIFO_RM);
4      int val;
5      while (1) {
6          cread(fifo, val);
7          printf("fifo: %d\n", val);
8      }
9  }
10
11 task writer() {
12     spolicy(EDF);
13     int w = 0;
14     while(1) {
15         cwrite(fifo, w);
16         fdelay(100, ms);
17     w++;
18     }
19 }
20
21 void main(){
22     cinit(fifo, 0);
23     reader();
24     writer();
25 }
```

Code listing C.2: Two periodic tasks with different scheduling policies exchanging data through a shared fifochannel.

the priority among the tasks is determined by static analysis of the timing points found in each task (see lines 3 and 12 in Listing C.2). Soft timing points are just translated into sleep statements, while firm timing points also start a countdown timer which would interrupt the program when the deadline is missed. Timed C is, as such, not a complete framework for implementing real-time systems. It is, however, a capable coordination framework in which one can implement systems with explicit timing constraints.

A formalism called Time Flow Graphs [26] is a way of representing a Timed C task. Timing points and the code fragments in between timing points are represented as nodes and the dependencies are represented as edges. A methodology for verifying the temporal properties specified in a Timed C program is also proposed. TFG is further discussed in Section C.4.1.

C.3.8 Related work

Ptides [27] is a programming model for distributed systems based on the relationship between model time and real-time. In Ptides, known bounds on network latency and execution times enable distributed synchronization with little communication. Lingua Franca [28] is a coordination framework for distributed real-time systems based on the Reactor model of computation (MoC) [29]. The Reactor model is a timed, discrete-event MoC. It includes a notion of logical time which is related to Timed C but has a more formal definition of time between synchroniza-

tion points, as well as the simultaneity of events. In Lingua Franca, software components, called reactors, communicate with time-tagged signals through named ports. The time tags are drawn from the logical timeline and logical time does not elapse during computation. Reactors react to events in time tag order; this ensures determinism at each logical instant. Distributed execution is achieved either with centralized coordination or decentralized coordination through Ptides. The distributed execution builds on sockets and while this is compatible with TSN, it leaves the challenging and error-prone task of correctly setting up the network for the user.

In "Temporal issues in Cyber-Physical Systems" [30], Broman et al. discuss how accurate clocks can and must be included in cyber-physical systems and how PTP can be used to accurately distribute time. Stanton [31] discusses how distributed coordination of tasks should target a future time instead of reacting to a message which then triggers an immediate action. This reduces the impact of network jitter but is vulnerable to inaccuracies in the time domain.

Gutiérrez et al. [32] showed by daisy chaining TSN nodes that the timeliness of the forwarding and shaping of TSN is well suited for running the control signal network for robotic systems. They did not test this with interfering traffic so no evaluation of stream protection was performed.

For large-scale industrial automation, the Open Platform Communication Unified Architecture (OPC-UA, [33]) is becoming the prevalent solution. The main focus of OPC-UA is device interoperability and is a device-centric, platform-independent architecture to integrate sensors and controllers. Although OPC-UA is primarily a centralized architecture, with amendment 14 [34] a publisher-subscriber ("Pub-Sub") is specified. This approach allows OPC-UA to scale to very large systems since most of the data can flow directly between the devices and not through a central aggregator. One drawback with OPC-UA has always been its complexity and as a system grows, it becomes increasingly difficult to configure the system to achieve desired latency whilst simultaneously not overloading the network and losing traffic as a result [35]. To improve the reliability and determinism of Pub-Sub, Bruckner et al. [36] investigated the usability of TSN and OPC-UA, and a recent joint effort by IEEE and IEC has started the standardization of TSN for Industrial Automation [37] where OPC-UA is part of the standardization effort. Recently the Open-Source Automation Development Lab (OSADL) has begun development to integrate TSN in the Publisher-subscribe profile [38] and provide a ready-made solution for industrial applications.

Another popular architecture is the Distributed Data Service (DDS, [39]), a data-centric, decentralized model that is purely publisher-subscribe. Each node can subscribe to a set of topics and the architecture orchestrates the delivery of data to the right recipients. The updated Robot Operation System (ROS2 [40]) has chosen DDS to handle the data handling. Agarwal et al. [41] used TSN to complement DDS and the simulations showed high message rates with very low latency for real-world data obtained from a wind farm. Their results show that TSN and DDS are a good match for real-world applications.

Communicating Sequential Processes (CSP) was first described by Tony Hoare in 1978 [42] and is a process algebra for specifying and verifying concurrency in systems by using message-passing and channels as synchronization primitives. Modeling concurrent systems suffer from a state space explosion when the size grows. To model and understand how these systems interact at the communication system level, a mathematical framework such as CSP is needed. CSP has successfully been used to verify critical systems such as avionics software at the International Space Station [43]. Messages and channels lend themselves naturally to distributed systems.

Whitney et al. [44] created a Go library called Gluster to support distributed applications in Go. Go is made for programming concurrent applications and is based on CSP. Gluster uses a Master/Worker model where a Master node schedules goroutines on the Worker nodes in the distributed system.

Both OPC-UA and DDS define a protocol format for exchanging data whereas Timed C, which we focus on in this paper, is more of a coordination framework for real-time tasks. In a sense, both Lingua Franca and Timed C could have used DDS or OPC-UA as the data integration protocol, and compared to our proposed network channel, both DDS and OPC-UA could in theory use this network channel as a transport layer. As this will add several layers of complexity for very little gain, we choose to connect Timed C and *netchannels* and evaluate the result.

To the best of our knowledge, no other work defines similar network channels with the same level of quantitative evaluation to demonstrate how deterministic networking can be used to build composable network primitives for distributed real-time systems.

C.4 Deterministic Network Channels and Federated Timed C

We extend Timed C with primitives for inter-task communication via TSN as building blocks to what we informally refer to as "Federated Timed C". These communication primitives are called *netchannels* as they are semantically similar to the existing *fifochannels*. While *fifochannels* can deliver time deterministic inter- and intra-process task communication, netchannels extends this to network task communication.

This section starts out by defining the formal semantics of both types of channels using Time Flow Graphs (TFG), before presenting the reference implementation for deterministic channels in Timed C.

C.4.1 Formalization

The Time Flow Graph (TFG) formalism introduced in [26] can be used to reason about the composability of Federated Timed C programs. In TFG a Timed C task is represented as a graph $G = (V, E)$ where the nodes $V = (P \cup F)$ is the set of timing points P and code fragments between timing points F. The edges E represent the

```
1 task sensor() {
2     int val;
3     while(1) {
4         val = sense();
5         chan_write(val);
6         fdelay(10, ms);
7     }
8 }
9
10 task processer() {
11     while(1) {
12         int val = chan_read();
13         process(val);
14     }
15 }
```

Code listing C.3: A simple Federated Timed C program composed of two tasks communicating over an abstract channel and can be realized as either fifochannel netchannel or IP channel

dependency relations between the nodes. A TFG can be analyzed at two levels of abstraction, at the program level or at the platform level. The highest level is the program level which lacks any information about the physical hardware on which it will execute. Analysis on the program level is purely in the logical time domain where code fragments have zero execution time. The second level is the platform level. It is a refinement of the program level where platform-dependent information about WCET of code fragments, release jitter, and trigger precision is included. Analysis on the platform level is on a refined logical timeline which we shall call the quasi-physical timeline. An operational semantics is also defined as a transition system.

A Federated Timed C program can be represented as a parallel composition of Timed C tasks communicating and synchronizing over asynchronous channels. We define the extended TFG $V_{ext} = (V \cup C) = (P \cup F \cup C)$, where C is a set of abstract communication channels. For Federated Timed C, $C = FC \cup NC \cup IC$, where FC is the set of fifochannels, NC is the set of netchannels and IC is the set of IP channels. IP channels are logical channels of best-effort traffic that connect two separate nodes and are not the primary target in Federated Timed C.[2] They are only added to benchmark the netchannels. Notice that lvchannels are not represented as they are not a message-passing construct, but rather a shared variable. The endpoint of a channel c_i is either a receiver or a sender.

Consider the Federated Timed C program in Listing C.3 and its corresponding TFG in Figure C.3. Channel endpoints $c \in C$ are represented by hexagons and the arrows denote the reader and writer. Timing points $p \in P$, are represented by circles and code fragments $f \in F$ by squares. The subscript on the components is a reference to the line number in Listing C.3.

We extend and redefine the program property for *arrival time* such that t_A :

[2] As is shown in Section C.6, using netchannels *without* stream protection can be detrimental to overall system performance-

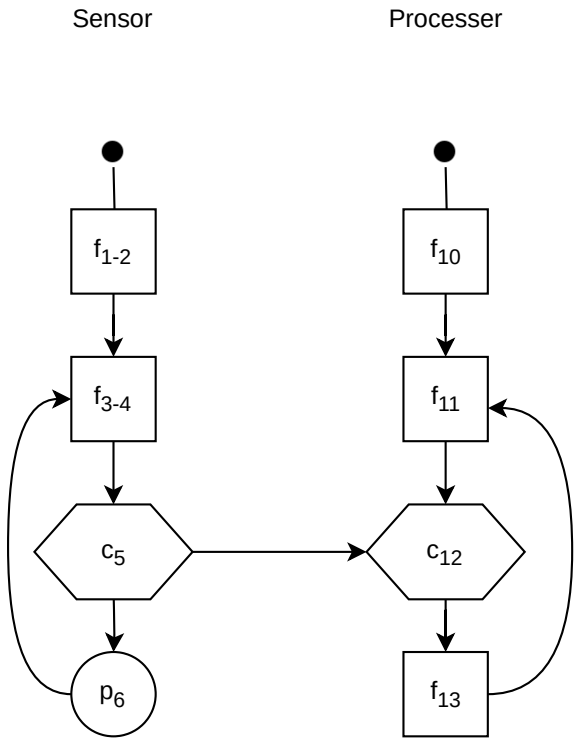


Figure C.3: TFG of Listing C.3 for Sensor and Processer communicating over an abstract channel.

$V_{ext} \rightarrow \mathbb{T}$. Here $\mathbb{T}$ represents time, it is either a relative time, i.e., a duration, or, in the case of t_A an absolute time. The arrival time of a node is the logical time when it *starts* executing. We also define the *release* time of a node as $t_R : V_{ext} \rightarrow \mathbb{T}$. The release time is the logical time when a node *finishes* executing. Lastly, we define the *latency* of a channel as $t_L : C \rightarrow \mathbb{T}$. The latency of a channel, $t_L(c)$ is the communication delay from the writer to the reader. From [26] $t_D(v)$ is the relative *deadline* of node v.

To formally define arrival and release times we also define the following functions.

- $pTP(v)$ is the set of previous/upstream timing points that has no other timing points between themselves, and the node v. Consider the TFG in Figure C.4. Here $pTP(v) = \{p_7, p_6\}$.
- $pC(v)$ is the set of upstream channels that has no other channels between themselves and the node v. Again consider Figure C.4, in this case, $pC(v) = c_8$. Notice that the cardinality of this set is never greater than one.

- $k : C \rightarrow \{writer, reader\}$ returns the type of channel end. In the case of Figure C.4 $k(c_8) = reader$.
- $o : C \rightarrow C$ returns the other connected channel end. In the case of Figure C.4 $o(c_9) = c_8$.

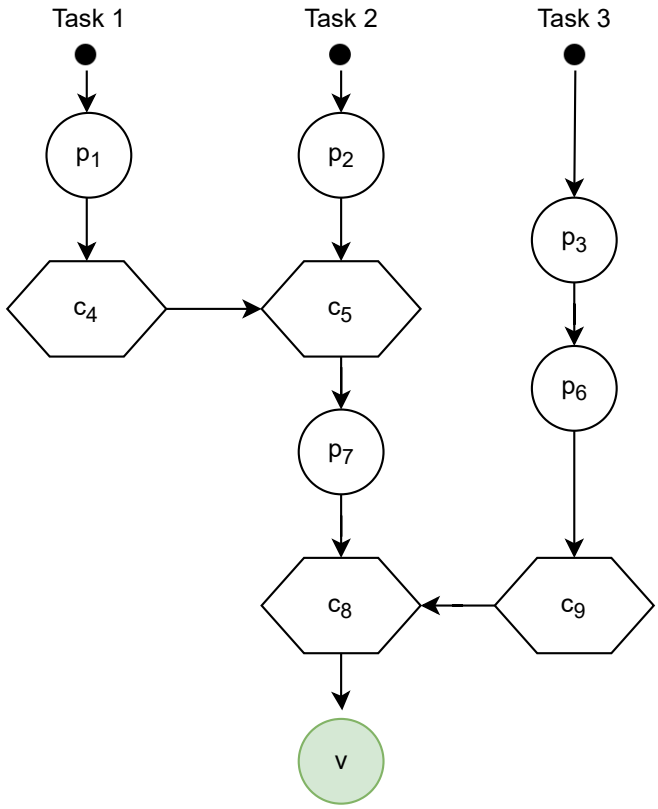


Figure C.4: Example TFG with multiple tasks, timing points, and channels.

The arrival time of a node is equal to the maximum of the release times of the set of upstream timing points and channels. This can be expressed formally as:

$$t_A(v) = \max\{t_R(u), \forall u \in (pTP(v) \cup pC(v))\} \tag{C.1}$$

The release time of a node depends on its type. The release time of a code fragment $t_R(f)$ and writer channel $t_R(c_w)$ is per definition equal to their respective arrival times. The release time of a timing point $t_R(p)$ is given in its function argument. E.g., $t_R(p_6) = 10$ ms in Figure C.3 and Listing C.3. The release time of a reader

Iteration	$t_A(c_5)$	$t_R(p_6)$	$t_A(c_{12})$	$t_R(c_{12})$
1	0	10	0	0
2	10	20	0	10
3	20	30	10	20

Table C.1: Arrival and release times for multiple iterations based on from Figure C.3 and Listing C.3

channel $t_R(c_r)$ is either its arrival time or the arrival time of its corresponding channel end plus the channel latency. It can be expressed as.

$$t_R(c_r) = \begin{cases} t_A(c_r), \text{ if } t_A(c_r) \geq t_A(o(c_r)) + t_L(c_r) \\ t_A(o(c_r)) + t_L(c_r), \text{ otherwise} \end{cases} \tag{C.2}$$

$$\forall c_r : k(c_r) \in \{reader\} \tag{C.3}$$

The channel latency depends on the channel type.

1. Fifochannels are modelled as without latency, i.e., $t_L(c \in FC) = 0$.
2. IP channels have unknown, possibly unbounded latency. $t_L(c \in IC) = t \in \mathbb{T}$
3. Netchannels have unknown but bounded latency. $t_L(c \in NC) = t \in \mathbb{T} : t \leq$ 2 ms.

With these additional constructs, we can reason about the logical timing of the Federated Timed C program in Figure C.3. We assume that $c_5, c_{12} \in FC$, i.e., the tasks are communicating through a fifochannel. The arrival and release times must be calculated at each iteration of the loop are calculated in Table C.1.

At the first iteration of the loop both *Sensor* and *Processer* will reach their respective channels, c_5 and c_{12} at logical time $t = 0$. They perform synchronous communication and are released without advancing logical time. *Processer* contains no timing points and reaches c_{12} in the second iteration while logical time is still 0. *Sensor* contains a timing point p_6 which incurs a 10 ms delay and will not reach c_5 until 10 logical ms. *Processer* blocks on c_{12} for 10 ms. In this way, the channel synchronizes the logical time of the two tasks.

A Federated Timed C program is composable to its logical timing if the timing is unchanged by introducing other, independent, Federated Timed C tasks. Formally, a program G_1 is composable when $t_A(v)$ and $t_R(v)$ are bounded or deterministic for all $v \in V$. Under these conditions. it can be composed with other programs, e.g., $G_1||G_2$ without changing its logical timing.

Per the definition, the only constructs introducing potentially variable timing properties are the channels. A program composed with only fifochannels is deterministic as fifochannels have bounded logical latency. Netchannels have strictly bounded latency and guaranteed delivery provided by the TSN QoS, and thus also satisfy this condition. IP channels, on the other hand, have best-effort delivery (i.e., no QoS from the network) and with a possibly unbounded latency, the channel latency is non-deterministic.

From this, we see that *without* bounded latency guarantees, which is only achievable by reserving transmission slots *and* buffer capacities, a network channel can never be composable. Conversely, by using QoS mechanisms such as TSN that shields critical traffic and provides bounded latency, network channels can be treated as composable components.

Note that composability is a property of our *model* of the program. In actual execution, the zero-delay execution of code fragments would naturally not hold. However, with a time-triggered architecture, such as an RTOS employing TDMA scheduling, execution times would remain deterministic and the overall program, composable.

C.4.2 Reference Implementation Overview

We now turn our attention to the implementation example, which is informally referred to as *netchannels* and when used in combination with Timed C becomes *Federated Timed C*. As previously stated, extending a real-time system with reliable networking support complicates code and configuration files (e.g., error handling, the configuration of network interfaces, correct addresses must be set, and QoS parameters specified). A *netchannel* is the implementation of a logical channel and is mapped to a TSN stream. Streams and channels are used somewhat interchangeably in the text. In essence, a *stream* indicates the actual TSN link, whereas a *channel* means the logical channel between two nodes that *may* be a TSN stream (Figure C.1 in Section C.3.2).

In Listing C.3 on page 94, we present a trivial example for a network channel which can be shown in Figure C.5. We now adapt this to two Timed C tasks communication using a network channel. In Table C.2 we have listed a partial API and C macros that allow us to write very clear and precise code from within a Timed C program.

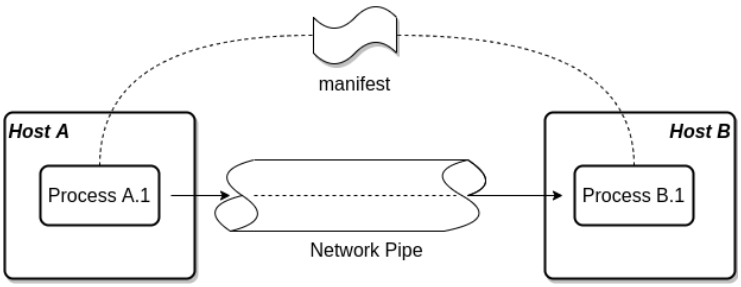


Figure C.5: Two hosts communicating through a network channel with a shared manifest describing the channel.

By adding simple primitives for a reliable channel that follows the same design principles as *fifochannel* in Timed C, our design makes it as easy to use a network channel as a normal fifo. The parameters are specified in a manifest file and

Macro Name	Corresponding C-functions	Comment
`NETFIFO_RX(chan)`	`pdu_create_standalone()`	Rx-end of a net-channel
`NETFIFO_TX(chan)`	`pdu_create_standalone()`	Tx-end of a net-channel
`CLEANUP()`	`nh_destroy_standalone()`	De-init and free all PDUs and nethandlers.
`WRITE(chan,val)`	`pdu_send_now()`	Write data to channel and return
`READ(chan,val)`	`pdu_read()`	Reading from the channel will block
`WRITE_WAIT(chan,val)`	`pdu_send_now_wait()`	Write and wait a pre-determined time
`READ_WAIT(chan,val)`	`pdu_read_wait()`	Read and wait a pre-determined time.

Table C.2: Brief FTC API summary, for simplicity, only parameters passed to the macros are listed. For full reference, see [2]. *chan* is the *manifest label*, which the macros use to inject a predictable variable name in the code alongside the appropriate function call. Listing C.4 describes a channel in a small manifest.

streams in the network must use globally unique *Stream ID* attributes. A single file can be used for the entire application without fear of "ID collisions". This not only avoids errors where two nodes use almost the same parameters for a stream, but it also opens up the possibility for automated tools to verify network configurations at compile time.

The network channel also has temporal delivery guarantees such that once a network stream has been established it reduces the problem known as the *Byzantine Generals Problem* [45], by ensuring that no frames are unexpectedly lost *during transit*. [3] The macros lend themselves to convenient and clean code, it is important to note that these are simple "glue-ins" for functions whose argument is inferred by the pre-processor and the manifest.

Network channels should be indistinguishable from *fifochannels* in that a sender *writes* to a channel and the receiving end performs a blocking *read*. The example of a DRTS in Figure C.1 shows a set of logical channels connecting the processes running on the different nodes. In Figure C.5 we see an example of the extension made to Timed C, where a network channel is described in a shared manifest and acts as such a logical channel. The channel is currently single-writer/single-reader, but nothing is preventing such a pipe from being multiple readers as it relies on multicast addressing and AVB/TSN stream reservation. Figure C.6 depicts the timing characteristics of such a logical channel where two hosts perform a simultaneous wakeup. This particular scenario is evaluated in Section C.6.5.

The code itself does not contain any dependencies to Timed C allowing it to be used as either a standalone system (example tools can be found in the repository [2]) or included in other frameworks. The core system is bundled as a set of header files and static libraries and made available through a clear C-API with an accompanying set of macros. In our example tools, we utilize the API provided by the macros as this resulted in the cleanest code and implementation from within Timed C.

[3] Granted, if parts of the network for some reason are removed, then data will be lost. We will not cover stream duplication and elimination in this paper.

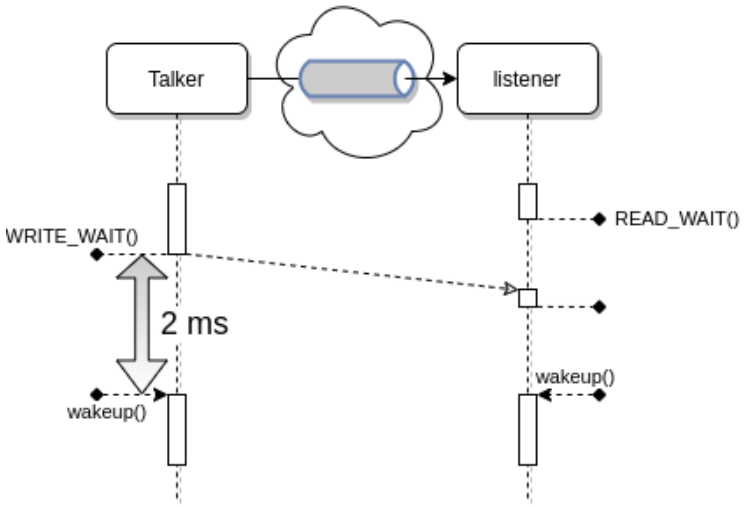


Figure C.6: Two hosts running simultaneous wakeup using `READ_WAIT()/WRITE_WAIT()`-pair with an AVB class A stream providing 2 ms upper bound on E2E delivery latency.

For stream protection, the SRP client code from AvNU's OpenAVB project [46] can be found in the srp/ subfolder. The code has been slightly modified to allow for multiple streams and easier integration with our system but is otherwise unchanged. It links to a separate archive and it is marked in the repository where our reference implementation is available [2].

C.4.3 Detailed Architecture

To better explain the underlying architecture, it is useful to expand the example from Figure C.6 to include more tasks in each node. This shows how multiple logical channels are handled by a single socket pair between the two hosts. In Figure C.7, we see that Task A.1 uses the provided WRITE() macro to send data. For the task itself, everything else is abstracted away. Likewise, the listener (task B.1) uses READ() which is a blocking call where the caller waits for data on the declared network pipe. Until Task A.1 writes a value to the pipe, Task B.1 will wait. This pipe is described in the shared manifest (Listing C.4). Similarly, multiple tasks on Host A can communicate over *their own logical channels*.

Tasks can also synchronize execution in the temporal domain by ensuring that both tasks continue exactly at the same time using the WRITE_WAIT() and READ_WAIT() pair. For a class A channel, both tasks will wait for 2ms *after the value was initially captured* ensuring that the sender will wait for exactly 2ms and the receiver will wait for 2 ms - "network latency". This gives an intuitive primitive

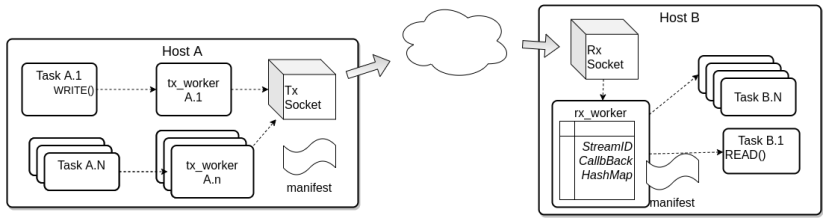


Figure C.7: `net_pipe` detailed architecture. To send a value from Task A.1 to B.1, A.1 first issues a `WRITE()` which feeds the data through a local pipe to a tx-worker. The data is then sent via the tx-socket to the corresponding Rx socket at Host B. B has a single Rx worker responsible for capturing all incoming traffic and forwarding it to the correct local task using the StreamID. Task B.1 receives the data by issuing a `READ()` which blocks on a local pipe awaiting incoming data.

for synchronizing two tasks without complex logic. By pairing up two such channels, it is trivial to construct a rendezvous mechanism where nodes wait until both are present before continuing at the same time. Looking back at the requirements for composability in Section C.4.1, we see that this channel pair is also composable. In Section C.6 we evaluate the accuracy of the simultaneous wakeup.

We split the system into 3 parts: a) core, b) writer, and c) reader.

Core

The core orchestrates the flow of data leaving one writer and data entering destined for a reader. The core is not meant to be used directly by a Timed C task but rather provides the needed infrastructure for the sender and receiver. Core data, such as active network card, PTP clock file descriptor, etc are kept by a `nethandler` container. The core handles resource management which is instrumental in how we can offer a set of simple macros. It also contains a linked list of all active outgoing and incoming channels and finally a map that connects a stream_id to the correct listener task. This last part is how individual streams are handled and is described in Section C.4.3.

Each network channel is further described internally by a struct detailing (network) address, stream_id, stream reservation handler (from AvNU's OpenAVB project, [46]), callback function for incoming data and buffer space for the latest frame traversing the channel. It also contains a Linux *pipe* pair used to communicate data to/from Timed C tasks and this is where READ/WRITE terminates. The shared *manifest* is used to describe each network channel and configure it with the correct values.

The manifest seen in Listing C.4 is an array of `struct net_fifo` which contains:

dst Destination address, a multicast address associated with a specific stream.

stream_id The unique identifier for the stream.

```
struct net_fifo net_fifo_chans[] = {{
    /* DEFAULT_MCAST */
    .dst       = {0x01, 0x00, 0x5E,
                  0x01, 0x11, 0x42},
    .stream_id = 42,
    .class     = CLASS_A,
    .size      =  8,
    .freq      = 50,
    .name      = "mcast42"
}};
```

Code listing C.4: Manifest for a system with a single pipe sending 8 bytes of data every 20 ms (50Hz). The label "mcast42" is used with both macros and functions to indicate which entry to use from the manifest and later identify the correct object in the functions.

class The AVB QoS class sets the expected observation interval and maximum E2E latency.

size The maximum amount of data each packet contains. In this particular example, we plan to send a single `uint64_t` value, which is 8 bytes.

freq How often data is sent This should be either the frequency of periodic data or an upper bound on how often sporadic data will be sent through this channel.

name a unique moniker used to find the correct entry in the table and also by the macros to inject variables and functions into the code (to properly hook into the core of Federated Timed C).

Writer

For a task to send data, it needs to declare a NETFIFO_TX (see Listing C.5). The label used corresponds to an entry in the manifest. During the setup process, each declared channel creates and configures its outgoing socket, and creates a pipe pair where the writer part is made available to the task and the reader-end is handed to a dedicated thread. This thread is what allows us to implement the same abstraction as Timed C's `fifochannel` since the thread will wait for data to arrive and handle all the logic of sending a correctly assembled avtp-frame. If tasked with using stream reservation, proper values are declared via SRP/MRP and announced to the network during the setup phase. Finally, to take advantage of dedicated hardware queues and the Credit Based Shaper, we set the socket priority such that Linux and the mqprio Qdisc can direct the traffic to the correct queue. Our test system uses an Intel I210 Network controller which has 4 tx-queues 2 of which support Credit Based Shaper. A prerequisite for using traffic steering is that Linux Qdiscs are configured (this is shown in Listing C.14 in Section C.10).

```
#include "manifest.h"
const int LOOPS = 1000000;
task writer() {
        NETFIFO_TX(mcast42);
        for (uint64_t i = 0; i < LOOPS; i++) {
            WRITE_WAIT(mcast42, &i);
            sdelay(20);
        }
        CLEANUP();
}
```

Code listing C.5: Code for talker based on Figure C.6.

```
#include "manifest.h"
task reader() {
    NETFIFO_RX(mcast42);
    uint64_t d = 0;
    while (1) {
        READ_WAIT(mcast42, &d);
        /* use d */
    }
    CLEANUP();
}
```

Code listing C.6: Code for listener example based on Figure C.6

The talker based on the sequence in Figure C.6 is shown in Listing C.5. The talker sends an incrementing value every 20 ms (50Hz) using the channel described by the manifest in Listing C.4. Behind the scenes, the core assembles an outgoing frame using the experimental type tag from IEEE 1722 [47], sets the presentation_time to the current PTP timestamp, and updates all relevant fields before sending the frame. It also uses the PTP timestampfrom when the frame was created and sleeps for a total of 2 ms (or 50 ms in the case of Class B streams). The current version of netchannel supports Class A and B, TSN's Scheduled Traffic which provides a 100 μs E2E bound is listed as future work. The final CLEANUP() ensures that streams are unannounced (removed from the network) and that is memory freed.

Reader

The receiving end of the channel in Listing C.5 is shown in Listing C.6. When declaring a pipe to be a receiver (using `NETFIFO_RX()`), the core machinery uses the `mcast42` label to retrieve the fields from the manifest and configure the receiver accordingly. This declares to the network stack that data should be received and delivered to the socket and configures a callback function for the corresponding *StreamID* so that data for this stream is delivered to the correct *fifo*. When the client code calls READ_WAIT(), it blocks in the correct queue until a frame with the expected StreamID arrives.

When a new frame is received for this stream, it is forwarded to the reader. If

`READ_WAIT()` is used, the full PTP capture timestamp is reconstructed. The reader schedules a wakeup at the same time as the writer. In this way, the writer knows that the data *will* be received, and *when* the receiver will continue forward. By using the `WRITE_WAIT()/READ_WAIT()` pair, both tasks will continue forward together at the same time.

C.4.4 Automatic Stream Reservation

From the manifest, the network subsystem extracts the required values to reserve bandwidth correctly all the way from sender to receiver. A talker (writer) first announces available streams and waits for new subscribers (readers) to connect. Note: once announced, the system waits for at least one listener to subscribe before continuing. This is so that the task knows that *someone* is receiving the data. Likewise, a listener waits for a matching announcement before subscribing.

C.4.5 Description of Protocol Format

We use a basic AVTP common data header [47, p. 4.4.3] without any extra subheaders. The type is marked as `EF_STREAM`[4] to signal to any other AVB-capable device on the network that this an experimental stream and most likely non-standard AVB traffic. The manifest specifies the amount of data that will be copied into the `stream_data_payload` from the fifo used by the core of federated Timed C. AVB is an L2 protocol, which means that everything is included directly in the payload field an ethernet frame (with VLAN extension). The protocol format is described in depth in the transport protocol definition [47, Sec. 4.4.4.1]

C.4.6 Integration with Timed C

The core of *netchannels* is a small set of functions and macros that form the API. The code is contained in an archive and can be easily linked with any C/C++ application. Our modification to Timed C limits itself to adding relevant headers and libraries to the build system for Timed C/KTC. For convenience, a fork of KTC with added libraries and headers is published to github [3, branch: net_chan].

- A set of header files exposing required API and constants.
- 2 static libraries libtimedc_avtp.a and libmrp.a (the latter being a slightly reworked client library for AvNu's MRP service to connect and configure the mrpd daemon)
- Add references to timedc_avtp and MRP in Ktc.pm, the configuration file for the KTC compiler
- Add example code for writer and reader to demonstrate how *netchannels* can be used from within Timed C.

[4] In truth, we use `AVTP_SUBTYPE_TIMEDC` which is a `#define` to the experimental value.

The total changeset to KTC is minimal [3] and it also shows that including *netchannels* to any other software system is fairly easy. In this paper, we have evaluated this in Timed C, so this is what we focus on for the rest of the paper.

C.5 Experimental Setup and Evaluation

In this section, we evaluate the reference implementation of our network channel concept and show how accurate this system can be using GNU/Linux and contemporary x86 hardware.

Figure C.8 shows the test scenario where the test machines were connected via 2 switches (Cisco Catalyst 3650 24PDM-S, with AVB, enabled). Additionally, two extra systems were used to generate a tunable load that could consume all available network bandwidth between the 2 core switches (bridges). This was done to investigate the core contribution of composable network channels so that they will not be affected by adverse network conditions. If TSN QoS guarantees perform as expected, any induced network load should not lead to excessive delay variations and dropped frames.

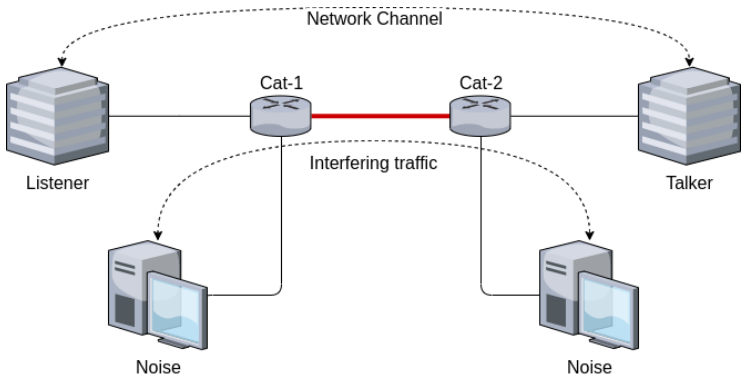


Figure C.8: Test system setup w/network setup. Critical traffic flows between Talker and Listener through a network channel. Additional noise is generated and introduced on the same physical link between the two core switches.

Table C.3 describes the test machines used in the experiment, the only difference being the amount of available memory. As the scenarios are not memory-bound, this is not a relevant difference. Both were configured to run a standard Debian Linux kernel (v5.10.0-11-amd64) and a custom kernel with PREEMPT_RT enabled (v5.16.2-rt19).

C.5.1 Test scenarios

We used a slightly modified version of the talker and listener code presented in Listing C.5 and C.6. We enabled logging and needed to be able to adjust real-time

Test Machines	
Motherboard	ASUS P11C-I
CPU	Intel Xeon E-2224
Freq	3.40 GHz
Memory	16/32 GB
Storage	WD Black SN750 PCIe v3
NIC	Dual I210AT
OS	Debian 11
kernel	v5.16.2-rt19 & 5.10.0

Table C.3: Test-machines HW/SW configuration, Talker and Listeners are close to identical in HW configuration, Talker have 32GB RAM.

measures and SRP behavior. The complete code can be found in Listing C.11 and C.12 (C.9). File I/O is a notorious problem for real-time applications since there are no safe ways to read or write to file in a critical path. Both the IO stack in the OS as well as a storage medium can introduce large, unpredictable delays. Instead, we exploited the available memory in our test machine and cached all the results in memory before finally writing it to disk at the end of the test run. We also allocated and locked all required memory before starting the profiling to avoid page faults during testing.

During testing, we logged the RMS error for the PTP clock reported by `ptp4l`. This allowed us to compound the PTP clock uncertainty with measurement noise from the test application used. If the PTP accuracy is poor (high RMS), we would have less confidence in the distributed system's accuracy.

C.5.2 Test tools

To quantify the e2e latency and wakeup latency, the code samples shown in Listings C.5 and C.6 are not well suited to profile the performance. The core code has instrumentation hooks that allow us to capture timestamps in a CSV format.

For every frame leaving or entering the system, timestamps were captured and saved to a buffer and written to a file *after* the test-run had completed. In particular, we captured when an outgoing frame was created (in `pdu_send()` used by `WRITE(), WRITE_WAIT()` macros) when the execution was passed to the Rx worker blocking on `recv_msg()` and finally we logged when the task itself was woken and selected to run (when we return from `READ(), READ_WAIT()`).

The logs from both sender and receiver were then merged during post-processing to reveal E2E latency and detect dropped frames. E2E was computed from the time the frame left the application to when it was received. This includes delay in the network stack, scheduling jitter, and network transmission delay.

Note: both `WRITE_WAIT()` and `READ_WAIT()` uses `CLOCK_MONOTONIC` to suspend for the necessary time; using the PTP clock directly is not possible in `clock_nanosleep()`. `CLOCK_MONOTONIC` is a strictly increasing clock meaning that even when the local

```
make allyesconfig
for i in $(seq 1 100); do
    make clean ;
    make -j32 all > /dev/null;
done
```

Code listing C.7: Steps to induce local load. The effect of the interference can be observed in Table C.5 and C.6 where the effectiveness of real-time measures is also demonstrated.

```
cyclictest --duration=3600 -m -S -p90 \
           --policy=rr -i200 -h500000 -q
```

Code listing C.8: cyclictest command options used to measure the real-time response of the kernel task scheduler and wakeup machinery.

clock is adjusted for drift, it will never move backward. This provides a more stable clock reference, especially when sleeping for relatively short durations. Initial investigation showed that the relative error between `CLOCK_MONOTONIC` and PTP time was not significant for the duration of the sleep cycle as this was less than 2 ms and once `ptp4l` had stabilized, clock adjustments were done in small steps.

Local interference - CPU and File I/O

To simulate a heavily loaded system with a mixture of high CPU utilization and heavy file I/O interrupts, we ran a multi-threaded compilation job of a large software system. Our systems have a total of 4 physical cores, we started a total of 32 threads that would read one or more files, and run through various CPU-intensive compilation stages before writing the result back to disk.

Our experience shows that running a highly parallel compilation task will saturate both the CPU and create a high level of file I/O interrupts and be a highly effective "real-time demonstrator". The effect is readily measured and will efficiently expose missing real-time measures (e.g., task priorities, task isolation, memory locking, interrupt shielding). We downloaded the latest Linux and ran Listing C.7 while our test application was running.

We also used cyclictest with the following configuration switches to establish a baseline. Results were captured for both vanilla and PREEMPT_RT kernel with the different load scenarios. The results are presented in Section C.6.2 and were used to guide the decision for which kernel to use in the test scenarios.

Network noise generator

To induce a suitable amount of network noise that would trigger a worst-case scenario for (un)protected flows, we created a small tool, `noisegen` [48], that would saturate any link with UDP traffic of a configurable size. For an unprotec-

```
1  src $> ./noisegen -I 192.168.10.1 -s 1470 -p 30
2 sink $> ./noisegen -I 192.168.10.1 -s 1470 -r > log
```

Code listing C.9: Running noisegen to introduce cross-link traffic ref. Figure C.8

ted stream, any high-rate interfering stream will trigger dropped frames through the network. From an AVB perspective, large frames will induce the highest interference. For our network, UDP packages with a payload of 1470 bytes proved to cause the highest ratio of dropped unprotected frames. By default, the generator saturates the network completely, but it is also possible to reduce transmitted noise to approx. 20% of link utilization or enter a periodic cycle where it will fill a link for N seconds before suspending for another N seconds. (This latter mode is what we see in Figure C.10.) The code was run by specifying the target, payload size, and period (second granularity). The effective bandwidth was logged by the receiver every second, which meant that it did not correlate perfectly with the logs from the Timed C profiling application. The purpose was to induce a network load and quantify the effect of AVB QoS capabilities. We can never perfectly replicate interfering traffic in a network, so any observed jitter, delay, and dropped frames must be captured by the AVB client (our Timed C distributed system). The second granularity allows us to visually adjust the induced load with the plots of the network but will not be used in any rigorous analysis of system performance.

With this setting, an average of 67.700 UDP packets/sec with 1470 bytes payload was sent back-to-back. Accounting for header size, checksums, and interframe gaps, this corresponded to 83.1% utilization of a 1 Gbps link. On a smaller network without VLAN trunking and service protection, this same setting resulted in 98.87% utilization. For unprotected streams, these conditions caused excessive frame drops. In both networks, unprotected traffic suffered severe packet loss with close to 100% dropped traffic. With a set of 0 received frames, it is not possible to draw any conclusions regarding E2E delay, jitter, or packet delivery. Instead, we used the cyclic mode to periodically apply noise and better illustrate the effect of stream protection.

C.5.3 3rd party tools

SRP daemon setup

TSN uses the Stream Reservation Protocol (SRP, [7, Ch. 34]) to dynamically reserve resources for a path between Talker and Listener. The AvNU Alliance has implemented a server through its OpenAVB initiative to handle these configuration messages that run as a standalone service [46].

The SRP daemon has not been modified, but we have extracted the MRP-client code and included it in our project to handle the interface between our application and the `mrpd` daemon. Originally written as a standalone binary, we had to perform some slight modifications to make it into a linkable library, the change is available under srp/ in our code repository [2]. To keep the separation

```
chrt --rr 30 ptp4l -i enp2s0 \
     --step_threshold=1 -f gPTP.cfg \
     --socket_priority=4
```

Code listing C.10: PTP Setup

clean, the SRP client code is now compiled into a separate library (`libmrp.a`) with corresponding headers residing under `include/srp` in the project catalog.

LinuxPTP - Timing setup and accuracy

On each machine, linuxptp [49] has been configured to run in 802.1AS mode. This is provided by a pre-made configuration files for the PTP daemon `ptp4l`. No changes to PTP priority fields were needed for the clients, causing one of the switches to act as GM for the network. As the network ran in isolation, this meant that the absolute time compared to a global reference source such as GNSS was wrong, but *internally* in the time domain, all nodes agreed on the same source - which ultimately is what matters. `ptp4l` was run with SCHED_RR and priority 30 to remove most of the interference from other tasks, no other measures were taken to shield it. To reduce the impact of the network noise system running *locally* on the test machines, `ptp4l` was configured to use socket priority 4 in Listing C.10. Setting up the network to provide a dedicated strict priority queue for ptp4l is described further in Listing C.14.

Linux Real-Time and Network Configuration

In short, "real-time measures" are all the small and large steps taken to improve the system responsiveness and predictability as experienced by its real-time tasks.

To avoid unpredictable delays due to major page faults, all memory was locked in memory. This is done using `mlockall(MCL_CURRENT|MCL_FUTURE)` which ensures that dynamic memory will not be written to swap. It is important to note that this does not page the memory *in*, so care must be taken to touch all memory before the critical sections begin.

Another common source of execution jitter is the CPUs' aggressive measures to enter lower cstates. This is done to save power and the deeper a CPU transitions, the longer it will take to return. This introduces variations in the order of 100s of μs.

Our test machines have 4 *physical* cores and with the help of HyperThreading (HT) can run up to 8 threads in "parallel". Rather crudely put, HT provides extra virtual cores and will quickly change which thread is executing by swapping a set of hardware registers whenever the executing thread stalls on a memory access. This means that HT can provide several delays where the length is dependent upon the execution path of an unrelated thread. To avoid this, HT has been disabled in BIOS.

To reduce the effect of interference from other tasks and interrupts, a shielded set of 2 cores was created in which the application itself was run. All other movable tasks were moved away from these cores to reduce scheduler interference. It further used SCHED_RR scheduler and priority 80 to out-rank any standard kernel thread and avoid task preemption. All interrupt handlers that could be moved were further affined to other cores by adjusting the IRQ affinity. This is particularly useful when using `preempt_rt` as several interrupt handlers are moved to dedicated kernel threads.

To manage the network traffic, a mqprio qdisc was attached to the I210 network card and configured to use the default AVB VLAN parameters before a Credit Based Shaper Qdisc was attached to Tx-0 and assigned socket priority 3. This is shown in Listing C.14. The design and operation of Linux Qdisc are complex and outside the scope of this paper.

A full example of local steps taken to improve RT behavior is presented in C.10.

C.5.4 Running the Test Scenarios

Once ready, we profiled both kernels to determine the accuracy of a standard and a "real-time tweaked" system. We tested these kernels under different scenarios:

1. *idle*: No other tasks were running other than standard background tasks.
2. *CPU Load*: A local task spawning multiple threads that all induced high CPU load and high IO activity (Section C.5.2).
3. *Network noise*: High network load on the shared link causing the switches to drop unprotected frames (Section C.5.2).

We then applied a) no measures, b) real-time measures (further described in Section C.5.3) c) real-time measures *and* performed stream reservation to protect the network stream. The latter was handled by the core part and the mrpd client library.

By splitting the load and measures this way, we aimed to quantify the different capabilities of the kernels and the real-time measures available in Linux. We also wanted to determine how TSN and stream reservation performed in practice under various loads.

Finally, two longer tests were run with the PREEMPT_RT kernel and all RT measures enabled. The first ran without enabling stream reservation, the latter with the stream being protected by the network as it moved from talker to listener. In addition, heavy network cross-traffic with a worst-case frame size was added to both these long-running tests to cause as much external interference to the traffic as possible.

C.6 Results

A key requirement for composability is to shield one part of a system from whatever happens in another part. This is done using traditional mechanisms available in Linux such as real-time priorities, CPU shielding, and memory. Expanding from this, a network channel should not be affected by unrelated traffic. The goal of using deterministic network channels as a composable building block is to be sure that traffic is not only delivered but delivered with a *bounded latency* regardless of other traffic. To evaluate this, we designed an experiment where the two test machines should wake up *exactly* at the same time and continue simultaneously with a cooperative task. In our case, this was merely logging the timestamp for later comparison but could be any task requiring some form of synchronicity. The simultaneous wakeup accuracy is therefore the main objective of the tests and should be independent of both local interference and network noise. We ran multiple tests profiling both gPTP accuracy as reported by `ptp4l`, network traversal time ("E2E-latency"), and wakeup accuracy using READ_WAIT() and WRITE_WAIT(). We tested the system as shown in Figure C.8. The entire set of logs obtained and used to generate the tables and figures are made available [50].

C.6.1 PTP Accuracy

For the timestamps captured during testing to be of any value, we must start by quantifying the PTP accuracy. Normally a system will slave its system clock (CLOCK_REALTIME) to the PHC using phc2sys but using the system clock for wakeup can be dangerous when it is being continuously adjusted. Instead, we read the timestamp directly from the PTP Hardware Clock (PHC) to remove one extra service from the test and remove a potential source of errors. It is generally recommended to use CLOCK_MONOTONIC, and we then must use 2 different clock sources; using the PHC is therefore not much extra complexity and saves us an extra source of error.

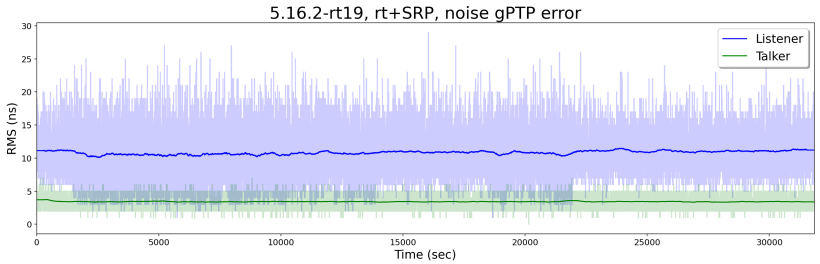


Figure C.9: gPTP performance evaluation, Cat-2 selected as PTP GM. RMS values as reported from gPTP on both hosts during excessive network interference. A 10-minute length, simple moving average (SMA) window is overlaid on each dataset.

Figure C.9 shows the PTP RMS accuracy reported by `ptp4l` for a long run

(8hr 50min) with heavy network cross traffic in the setup depicted in Figure C.8. `ptp4l` was run as specified in Section C.5.3 on a Linux 5.16.2-rt19 kernel. The performance was representative of other measured high-load scenarios, we show the result for the PREEMPT_RT kernel used for the final tests. More details can be found in Table C.4. The core switches have the AVB option enabled which means PTP sync messages are timestamped on both ingress and egress. Compared to the wakeup accuracy (covered next), this is 3 orders of magnitude better and thus more than sufficient for our tests.

	Listener	**Talker**
duration	8 hr 50min	
max	29 ns	7 ns
avg	10.8 ns	3.4 ns
σ	3.3 ns	0.8 ns

Table C.4: gPTP performance summary for a long test, data represents local time error from the master clock (PTP GM). GM selected by the network to run on switch closets to talker (Cat-2).

C.6.2 Kernel Wakeup Accuracy

To test the real-time capabilities of the selected kernels, we ran cyclictest for 1 hour during heavy CPU and IO load (massively parallel make of the Linux kernel source code). This is a common strategy for testing the performance of new RT kernels and configurations. Except for running cyclictest with real-time priority, no other real-time measures were taken. The results are presented in Table C.5.

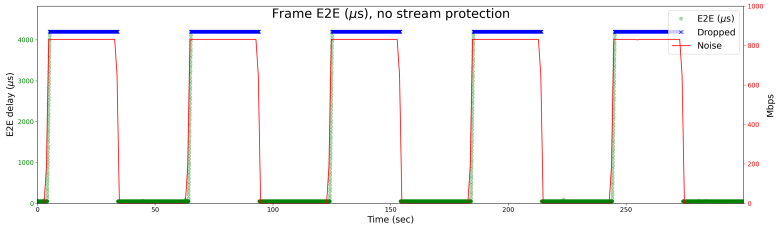
Not only is the maximum latency reduced by a factor of 2 but the standard deviation is also significantly lower yielding improved determinism for the PREEMPT_RT kernel. With these results in mind, we felt comfortable moving forward using the PREMPT_RT kernel. These results also yield lower bounds on the accuracy of our system regardless of network performance on a Linux-based system. *E.g., we cannot expect to achieve better wakeup accuracy than 16μs.*

	5.10.0	**5.16.2-rt19**
samples	71 796 433	71 999 889
max	3092 μs	16 μs
avg	3.8 μs	1.1 μs
σ	24.5 μs	0.3 μs

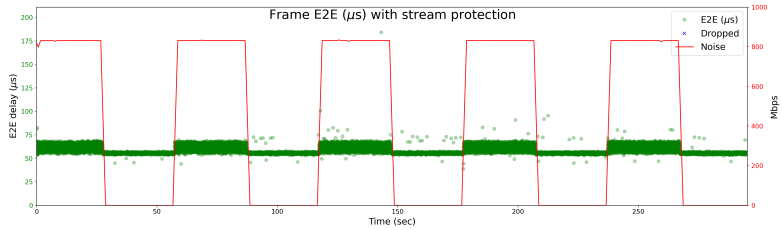
Table C.5: Cyclictest performance summary, task wakeup accuracy from `clock_nanosleep()`, 60-minute summary during heavy CPU load, aggregated results from all 4 threads. Wakeup deviation reported from cyclictest scaled to μs.

C.6.3 Package Transmission Delay

As the machines are running with `ptp4l` and clock accuracy is within 30ns, we can safely use the PTP timestamps for when a frame was created at WRITE()/WRITE_WAIT() and received to gauge the total delay. The delay measured not only the network delay but also the network stack at both ends. In Table C.6 we present the values for 5.16.2-rt19 with *no* network noise and no stream protection. The first test was without any real-time measures, the second using all the measures described in Section C.5.3.



(a) Frame E2E latency *without* stream protection, blue cross indicates dropped frame. 7240 of 15001 frames were lost. Delay increases steadily towards 4ms at which point the bridge's queues are overwhelmed



(b) Frame E2E latency *with* stream protection. 0 lost frames, the largest delay is 185μs.

Figure C.10: Frame E2E delay in a 5-minute test, variable network interference in 30-sec cycles. Blue crosses indicate lost frames, and green circles represent E2E delay for frames. The right y-axis (red) shows the consumed bandwidth of interfering traffic. See Table C.7 for details.

We see a clear improvement using real-time measures such as IRQ affinity, task shielding, real-time priorities, and locking memory to avoid page faults.

C.6.4 E2E Delay with Network Noise

As shown in Figure C.8, we can apply cross-link noise to the system to evaluate the effect of reserving network capacity. This experiment is an almost exact replica of the package transmission delay except that we now add network interference. Having established the benefit of using real-time measures, we now use this for all tests and demonstrate how stream reservation helps avoid package loss and reduce jitter. In Figure C.10 we see a clear effect of protecting the stream with SRP.

	5.16.2-rt19	
E2E Delay	No RT	With RT
Idle		
max	697.4 μs	95.8 μs
min	134.7 μs	39.3 μs
avg	471.0 μs	56.5 μs
σ	51.3 μs	1.0 μs
CPU Load		
max	256.7 μs	131.6 μs
min	69.2 μs	59.1 μs
avg	108.4 μs	87.5 μs
σ	9.2 μs	3.8 μs

Table C.6: E2E package latency of 50Hz stream, the system running without neither network interference nor SRP to reserve network resources. The top half shows an otherwise idle system, and the lower section shows delay values when running a CPU-intensive load.

Running without protection, we observe a near 100% frame drop in Figure C.10a when the noise generator saturates the network. Using stream protection in Figure C.10b, we can see that *no* frames from the critical stream are lost, yet we notice an increase in E2E delay jitter with approximately 20μs. This is consistent with worst-case interference of large frames for a shared link[5] and internal forwarding delay on the bridge. Table C.7 provides additional details where especially the max delay and standard deviation demonstrate the efficacy of stream protection in this case.

	No protection	**With SRP**
max	4196.9 μs	184.3 μs
min	39.1 μs	39.0 μs
avg	126.2 μs	58.9 μs
σ	437.8 μs	4.3 μs
loss	48.3%	0 %

Table C.7: E2E delay, Linux v5.16.2-rt19, the system is idle with periodic network interference (30-sec cycle of 0 to approximately 830Mbps traffic). The first column shows delays and packet loss without any network protection schemes; the second column shows results when SRP is used to reserve network resources. Figure C.10 provide a graph view of same dataset

[5] A frame of 1470 bytes + headers will consume 12μs in transmission delay

	Listener	Talker	Relative error
max	-17.4 μs	-7.9 μs	15.3 μs
avg	-2.3 μs	-2.1 μs	0.25 μs
σ	0.70 μs	0.38 μs	0.80 μs

Table C.8: E2E summary using RT+SRP, a heavy CPU load on both machines and variable network saturation (5 min noise period). For Listener and Talker, a negative error means waking up *after* targeted time. For relative error, values are absolute (which unit wakes up before the other is not as relevant as how far apart they wake)

C.6.5 Network Triggered Wakeup Accuracy

The final and most important metric to evaluate for our network channel is the coordinated wakeup accuracy. As described in Section C.4, `WRITE_WAIT()` and `READ_WAIT()` will let each node continue at the same time; the goal is to have the talker and listener tasks waking up simultaneously regardless of local load and network noise. Where Figure C.10 shows the necessity of using SRP to protect critical traffic, we now look at the complete service, the accuracy of two systems running a coordinated wakeup with `WRITE_WAIT()` and `READ_WAIT()`. Figure C.11 shows the aggregated wakeup error between the two machines, Table C.8 shows the maximum *relative* error in our setup.

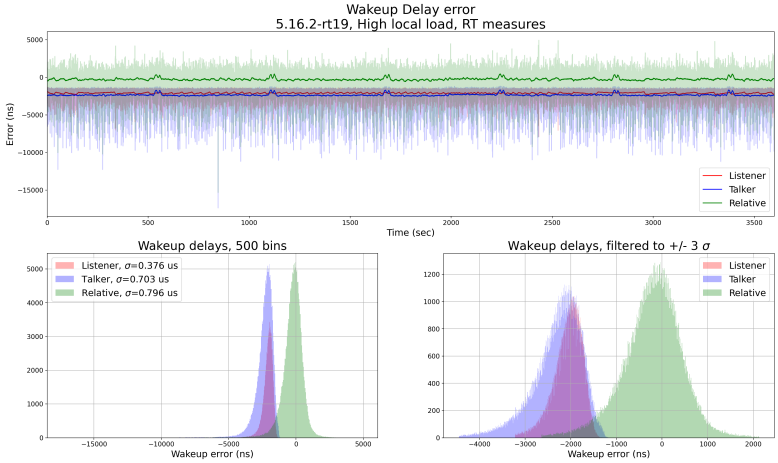


Figure C.11: Delay wakeup accuracy, RT+SRP measures under heavy CPU load and interfering traffic (Kernel 5.16.2-rt19). A negative value means wakeup *after* target. The lower right figure is the same as the left but restricted to +/- 3σ.

The result in Table C.8 shows that a shared signal between two hosts on a network can synchronize the execution flow to within 15.3 μs under heavy load and high network interference. When we compare this with the results presented

in Table C.5, we see that we are within the bounds found by the de-facto Linux real-time performance measuring toolkit.

Oliveira et al. [51] provide a thorough evaluation of PREEMPT_RT on comparable hardware to our test setup. Their load scenarios are more detailed than what we used in our evaluation, yet we see that for heavy loads, they present similar numbers. Similarly, for ARM-based devices, Adam et al. evaluate several flavors of Raspberry Pi (RPi) and kernel versions using cyclictest. The comprehensive summary ([52, Table 2]) they provide shows max latency numbers well below 100μs for most of the models tested.

C.6.6 Final Remarks

Although the results obtained during testing show a clear advantage of using protected streams, the amount of available hardware constrained the size of our experimental setup. We also only ran a single stream between the talker and listener to reduce the possible sources of errors. Due to the number of systems available for testing, both end stations, and network bridges, we have not been able to accurately quantify the effect of multiple reserved streams. The same limit applies to the number of network bridges between a talker and listeners.

C.7 Conclusion

In this paper, we have presented network channels, an extension to Timed C and Time Flow Graph that allows us to design, verify and implement robust distributed real-time systems. This has been rooted in a formal definition of TFG. A tool such as TFG is useful when describing large and complex systems, and by extending TFG and Timed C, to the network realm, it becomes possible to describe large, distributed systems with the same tools. For a component such as a network channel to be considered *composable*, the amount of temporal shielding the network can provide must be comparable to the real-time demands. For TSN using stream class A, an upper bound of 2ms E2E latency is specified in the standard which is what we have evaluated in this paper. Whereas TSN also provides a 100 μs upper bound using the time-aware scheduler, this has not been evaluated in this work as we did not have access to supportive network equipment. A TSN network such as we have used in this paper can therefore only provide composable channels for systems that allow *up to* 2ms delays.

A reference implementation has been described and evaluated on Linux. We saw that PREEMPT_RT greatly improves system determinism and that SRP and TSN ensure that no packets are lost regardless of network load. By using PTP, we can accurately reason about total transmission delay through both the network and the local network stack on both ends. The accuracy of the PTP clock synchronized during our test runs shows that PTP is 3 orders of magnitude more accurate than scheduler wakeup, allowing us to be confident that PTP timestamps can be used to synchronize task wakeups.

We showed that our framework for network channels can share signals across a network domain and synchronize the execution of a distributed system both during adverse network conditions and extreme local loads. We show experimentally that the construct fulfills the requirements for composability for systems that accept less than a 2 ms delay. The API presents itself as clear and simple, leading to a clean design of DRTS. We demonstrated this inside a Timed C example program, and as noted in the beginning, including it in any other C program is trivial.

Finally, we discuss the limitations of our experimental setup and how a future experiment could be constructed to further strengthen the results.

C.8 Future work

We see maximum E2E delays of more than 100μs in the system, but we suspect that a large portion of this is due to variance in the delay through the Linux network stack. Another interesting avenue is to look into Linux's Express Data Path (XDP) and move most of the network processing to user space to further reduce network delay and jitter.

As the current revision of the network channel uses class A and B from AVB, a logical next step is to enable the Time Aware Shaper for scheduled traffic and add support for sporadic traffic via the Asynchronous Traffic Shaper.

With the limits discussed in Section C.6.6, we see a clear opportunity for running the test scenario with multiple reserved streams as well as adding additional network bridges. It would also be interesting to create multiple streams flowing both ways between the systems acting as talker and listener. This should show how well our reference implementation scales with the number of logical channels as well as adding further pressure on the network bridges.

Finally, as the shared manifest is a single, central place that specifies all streams for the DRTS, combining this with a known network topology to determine routing and time slots for TAS is a necessary next step.

C.9 Code examples

```
#include <stdio.h>
#include <cilktc.h>
#include <timedc_avtp.h>
#include "manifest.h"

static bool running = false;
void sighandler(int signum)
{
    printf("%s(): Got signal (%d), closing\n",
    __func__, signum);
    fflush(stdout);
    running = false;
}

task writer()
{
    printf("%s(): getting ready\n", __func__);
    NETFIFO_TX(mcast42);

    for (uint64_t i = 0; i < LOOPS && running; i++)
    {
    WRITE_WAIT(mcast42, &i);
    if (!(i%100))
        printf("%lu: written\n", i);
    sdelay(20, ms);
    }

    uint64_t stop = -1;
    WRITE_WAIT(mcast42, &stop);
    printf("Magic stop marker written\n");

    CLEANUP();
    return NULL;
}

void main()
{
    nf_set_nic(NIC);
    printf("Run for %d, using %s\n", LOOPS, NIC);

    printf("Using SRP\n");
    nf_use_srp();
    nf_set_logfile("netfifo_talker_rt_srp.csv");
    nf_log_delay();

    running = true;
    signal(SIGINT, sighandler);
    writer();
}
```

Code listing C.11: Talker example with profiling

```c
#include <stdio.h>
#include <cilktc.h>
#include <timedc_avtp.h>
#include "manifest.h"

task reader()
{
    NETFIFO_RX(mcast42);
    /* Default, terminal value, unless changed by
     * remote, will terminate loop. */
    uint64_t d = -1;
    while (1) {
    READ_WAIT(mcast42, &d);
    if (!(d%100))
        printf("Counter received! -> %lu\n", d);

    if (d == -1) {
        printf("Magic terminator, stopping\n");
        break;
    }
    }
    CLEANUP();
    return NULL;
}

void main()
{
    printf("Using %s\n", NIC);
    nf_set_nic(NIC);
    nf_use_srp();
    nf_set_logfile("netfifo_listener_rt_srp.csv");
    nf_log_delay();
    reader();
}
```

Code listing C.12: Listener example with profiling

```c
#pragma once
#define NIC "enp2s0.2"

#define LOOP_HZ 50
#define IT_MIN (50*60)
#define IT_HR (IT_MIN * 60)
#define LOOPS (5 * IT_HR)

#include <timedc_avtp.h>
struct net_fifo net_fifo_chans[] = {{
    /* DEFAULT_MCAST */
    .dst       = {0x01, 0x00, 0x5E,
                  0x01, 0x11, 0x42},
    .stream_id = 42,
    .class     = CLASS_A,
    .size      =  8,
    .freq      = 50,
    .name      = "mcast42"
}};
```

Code listing C.13: Manifest for profiling code

C.10 Linux RT Setup

```
tc qdisc replace dev enp2s0 parent root mqprio \
    num_tc 4 \
    map 3 3 1 0 2 2 2 2 2 2 2 2 2 2 2 2 \
    queues 1@0 1@1 1@2 1@3 hw 0
tc qdisc add dev enp2s0 parent 8001:1 cbs \
    idleslope 20000 sendslope -980000 \
    hicredit 30 locredit -1470 offload 1
ip link add link enp2s0 name enp2s0.2 \
    type vlan id 2 egress-qos-map 2:2 3:3
ip link set enp2s0.2 up
cset shield --cpu 2-3
cset shield -s -p $$

# move network interrupts to reduce
# interference from non-related network load
for irq in $(cat /proc/interrupts | \
    grep -E '(enp1s0|enp2s0$|enp2s0-TxRx-0|enp2s0-TxRx-1)'| \
    cut -d ':' -f1);
do
    echo "Setting affinty for irq ${irq} to: 1"
    echo "1" > /proc/irq/${irq}/smp_affinity
done

for irq in $(cat /proc/interrupts | \
    grep -E '(enp2s0-TxRx-2|enp2s0-TxRx-3)'| \
    cut -d ':' -f1);
do
    echo "Setting affinty for irq ${irq} to: 14"
    echo "1" > /proc/irq/${irq}/smp_affinity
done
```

Code listing C.14: Linux setup of Qdisc and shielding of 2 cores

Bibliography

[1] S. Natarajan and D. Broman, 'Timed c: An extension to the c programming language for real-time systems,' in *24th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS 2018)*, [ed] Pellizzoni, R, IEEE, 2018, pp. 227–239, ISBN: 978-1-5386-5295-4. DOI: 10.1109/RTAS.2018.00031. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-235153.

[2] H. Austad, *Netchannels*, Jul. 2022. [Online]. Available: https://github.com/henrikau/net_chan.

[3] H. Austad, *Ktc fork with pre-built netchannel extension included*, Jul. 2022. [Online]. Available: https://github.com/henrikau/ktc/tree/net_chan.

[4] K. Erciyes, *Distributed Real-Time Systems, Theory and Practice*, 1st ed. Springer International Publishing, 2019, p. 341, ISBN: 978-3-030-22569-8. DOI: 10.1007/978-3-030-22570-4.

[5] H. Kopetz, *Real-Time Systems - Design Principles for Distributed Embedded Applications*, ser. Real-Time Systems Series. Springer, 2011, ISBN: 978-1-4419-8236-0. DOI: 10.1007/978-1-4419-8237-7. [Online]. Available: https://doi.org/10.1007/978-1-4419-8237-7.

[6] 'Standard for local and metropolitan area networks–audio video bridging (avb) systems,' *IEEE Std 802.1BA-2011*, pp. 1–45, 2011. DOI: 10.1109/IEEESTD.2011.6032690.

[7] 'Standard for local and metropolitan area network–bridges and bridged networks,' *IEEE Std 802.1Q-2018 (Revision of IEEE Std 802.1Q-2014)*, pp. 1–1993, 2018. DOI: 10.1109/IEEESTD.2018.8403927.

[8] 'Standard for local and metropolitan area networks–frame replication and elimination for reliability,' *IEEE Std 802.1CB-2017*, pp. 1–102, 2017. DOI: 10.1109/IEEESTD.2017.8091139.

[9] 'Standard for local and metropolitan area networks – bridges and bridged networks - amendment 25: Enhancements for scheduled traffic,' *IEEE Std 802.1Qbv-2015 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, and IEEE Std 802.1Q-2014/Cor 1-2015)*, pp. 1–57, 2016. DOI: 10.1109/IEEESTD.2016.8613095.

[10] IEEE, 'Asynchronous traffic shaping,' *IEEE Std 802.1Qcr-2020*, pp. 1–151, 2020. DOI: 10.1109/IEEESTD.2020.9253013.

[11] J. Specht and S. Samii, 'Urgency-based scheduler for time-sensitive switched ethernet networks,' in *2016 28th Euromicro Conference on Real-Time Systems (ECRTS)*, 2016, pp. 75–85. DOI: 10.1109/ECRTS.2016.27.

[12] R. T. Braden, D. D. D. Clark and S. Shenker, *Integrated Services in the Internet Architecture: an Overview*, RFC 1633, Jun. 1994. DOI: 10.17487/RFC1633. [Online]. Available: https://www.rfc-editor.org/info/rfc1633.

[13] D. L. Black, Z. Wang, M. A. Carlson, W. Weiss, E. B. Davies and S. L. Blake, *An Architecture for Differentiated Services*, RFC 2475, Dec. 1998. DOI: 10.17487/RFC2475. [Online]. Available: https://www.rfc-editor.org/info/rfc2475.

[14] J. Harju and P. Kivimaki, 'Co-operation and comparison of diffserv and intserv: Performance measurements,' in *Proceedings 25th Annual IEEE Conference on Local Computer Networks. LCN 2000*, 2000, pp. 177–186. DOI: 10.1109/LCN.2000.891025.

[15] N. Finn, P. Thubert, B. Varga and J. Farkas, *Deterministic Networking Architecture*, RFC 8655, Oct. 2019. DOI: 10.17487/RFC8655. [Online]. Available: https://www.rfc-editor.org/info/rfc8655.

[16] L. Lamport, 'Time, clocks, and the ordering of events in a distributed system,' *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978, ISSN: 0001-0782. DOI: 10.1145/359545.359563. [Online]. Available: https://doi.org/10.1145/359545.359563.

[17] M. Lipiński, T. Włostowski, J. Serrano and P. Alvarez, 'White rabbit: A ptp application for robust sub-nanosecond synchronization,' in *2011 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication*, 2011, pp. 25–30. DOI: 10.1109/ISPCS.2011.6070148.

[18] 'Standard for a precision clock synchronization protocol for networked measurement and control systems,' *IEEE Std 1588-2019 (Revision of IEEE Std 1588-2008)*, pp. 1–499, 2020. DOI: 10.1109/IEEESTD.2020.9120376.

[19] K. Maintainers, *The linux kernel*, Mar. 2022. [Online]. Available: https://www.kernel.org/doc/html/latest/.

[20] GNU, *The gnu c library (glibc)*, Mar. 2022. [Online]. Available: https://www.gnu.org/software/libc/.

[21] T. Gleixner, *Real-time linux history*, Mar. 2022. [Online]. Available: https://wiki.linuxfoundation.org/realtime/rtl/blog#preempt-rt_history.

[22] T. Gleixner, S. Rostedt and J. Kacur, *Rt-tests*, Mar. 2022. [Online]. Available: https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/rt-tests.

[23] T. Gleixner, S. Rostedt and J. Kacur, *Cyclictest latency debugging with ftrce*, Mar. 2022. [Online]. Available: https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclictest/tracing.

[24] S. Natarajan, 'Programming language primitives and tools for integrated real-time systems development,' QC 20210517, PhD thesis, KTH Royal Institute of Technology, 2021, ISBN: 978-91-7873-875-5. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-294315.

[25] S. Natarajan, M. Nasri, D. Broman, B. B. Brandenburg and G. Nelissen, 'From code to weakly hard constraints: A pragmatic end-to-end toolchain for timed c,' in *Proceedings - Real-Time Systems Symposium*, QC 20200702, Institute of Electrical and Electronics Engineers Inc., 2019, pp. 167–180, ISBN: 9781728144030. DOI: 10.1109/RTSS46320.2019.00025. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-274075.

[26] S. Natarajan and D. Broman, 'Temporal property-based testing of a timed c compiler using time-flow graph semantics,' in *Proceedings 2020 Forum on specification & Design Languages (FDL)*., [ed] Alain Girault, Institute of Electrical and Electronics Engineers (IEEE), 2020, pp. 1–8. DOI: 10.1109/FDL50818.2020.9232935. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-292191.

[27] P. Derler, T. Feng, E. Lee, S. Matic, H. Patel, Y. Zhao and J. Zou, 'Ptides: A programming model for distributed real-time embedded systems,' Apr. 2008.

[28] M. Lohstroh, C. Menard, S. Bateni and E. Lee, 'Toward a lingua franca for deterministic concurrent systems,' *ACM Transactions on Embedded Computing Systems*, vol. 20, pp. 1–27, May 2021. DOI: 10.1145/3448128.

[29] M. Lohstroh, I. Romeo, A. Goens, P. Derler, J. Castrillón, E. Lee and A. Vincentelli, 'Reactors: A deterministic model for composable reactive systems,' in. Feb. 2020, pp. 59–85, ISBN: 978-3-030-41130-5. DOI: 10.1007/978-3-030-41131-2_4.

[30] D. Broman, P. Derler and J. Eidson, 'Temporal issues in cyber-physical systems,' *Journal of the Indian Institute of Science*, vol. 93, no. 3, pp. 389–402, 2013.

[31] K. B. Stanton, 'Distributing deterministic, accurate time for tightly coordinated network and software applications: Ieee 802.1as, the tsn profile of ptp,' *IEEE Communications Standards Magazine*, vol. 2, no. 2, pp. 34–40, 2018. DOI: 10.1109/MCOMSTD.2018.1700086.

[32] C. S. V. Gutiérrez, L. U. S. Juan, I. Z. Ugarte and V. M. Vilches, 'Time-sensitive networking for robotics,' *CoRR*, vol. abs/1804.07643, 2018. arXiv: 1804.07643. [Online]. Available: http://arxiv.org/abs/1804.07643.

[33] OPCFoundation, *Unified Architecture*.

[34] OPCFoundation, *OPC 10000-14 Unified Architecture Part 14 Pub Sub*, https://reference.opcfoundation.org/Core/Part14/.

[35] S. Cavalieri and F. Chiacchio, 'Analysis of OPC UA performances,' *Computer Standards & Interfaces*, vol. 36, pp. 165–177, Nov. 2013. DOI: 10.1016/j.csi.2013.06.004.

[36] D. Bruckner, M.-P. Stănică, R. Blair, S. Schriegel, S. Kehrer, M. Seewald and T. Sauter, 'An introduction to opc ua tsn for industrial communication systems,' *Proceedings of the IEEE*, vol. 107, no. 6, pp. 1121–1131, 2019. DOI: 10.1109/JPROC.2018.2888703.

[37] *IEC/IEEE 60802 TSN Profile for Industrial Automation* |, https://1.ieee802.org/tsn/iec-ieee-60802/.

[38] O. S. A. D. L. contributors, *OPC UA PubSub over TSN: OSADL - Open Source Automation Development Lab eG*, https://www.osadl.org/OPC-UA-PubSub-over-TSN.opcua-tsn.0.html.

[39] I. Object Management Group, *Data distribution service*, Jan. 2021. [Online]. Available: https://www.dds-foundation.org/.

[40] *ROS: Home*, https://www.ros.org/.

[41] T. Agarwal, P. Niknejad, M. R. Barzegaran and L. Vanfretti, 'Multi-Level Time-Sensitive Networking (TSN) Using the Data Distribution Services (DDS) for Synchronized Three-Phase Measurement Data Transfer,' *IEEE Access*, vol. 7, pp. 131 407–131 417, 2019, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2939497.

[42] C. A. R. Hoare, 'Communicating sequential processes,' *Commun. ACM*, vol. 21, no. 8, pp. 666–677, Aug. 1978, ISSN: 0001-0782. DOI: 10.1145/359576.359585. [Online]. Available: https://doi.org/10.1145/359576.359585.

[43] B. Buth, J. Peleska and H. Shi, 'Combining methods for the livelock analysis of a fault-tolerant system,' in *International Conference on Algebraic Methodology and Software Technology*, Springer, 1999, pp. 124–139.

[44] J. Whitney, C. Gifford and M. Pantoja, 'Distributed execution of communicating sequential process-style concurrency: Golang case study,' *The Journal of Supercomputing*, vol. 75, Mar. 2019. DOI: 10.1007/s11227-018-2649-2.

[45] L. Lamport, R. Shostak and M. Pease, 'The byzantine generals problem,' *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, Jul. 1982, ISSN: 0164-0925. DOI: 10.1145/357172.357176. [Online]. Available: https://doi.org/10.1145/357172.357176.

[46] Avnu, *Openavnu git repository*, Jan. 2022. [Online]. Available: https://github.com/Avnu/OpenAvnu.

[47] 'Standard for a transport protocol for time-sensitive applications in bridged local area networks,' *IEEE Std 1722-2016 (Revision of IEEE Std 1722-2011)*, pp. 1–233, 2016. DOI: 10.1109/IEEESTD.2016.7782716.

[48] H. Austad, *Tool repository*, Apr. 2022. [Online]. Available: https://github.com/henrikau/tools.

[49] Richard Cochran, *The linux ptp project*, version v3.1, 20th Dec. 2021. [Online]. Available: https://linuxptp.sourceforge.net.

[50] H. Austad, *Logfiles from testruns, netchannels*, Aug. 2022. [Online]. Available: https://lethe.austad.us/fs/netchan_22.tgz.

[51] D. B. d. Oliveira, D. Casini, R. S. d. Oliveira and T. Cucinotta, 'Demystifying the Real-Time Linux Scheduling Latency,' in *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*, M. Völp, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), ISSN: 1868-8969, vol. 165, Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020, 9:1–9:23, ISBN: 978-3-95977-152-8. DOI: 10.4230/LIPIcs.ECRTS.2020.9. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2020/12372 (visited on 26/11/2022).

[52] G. K. Adam, N. Petrellis and L. T. Doulos, 'Performance Assessment of Linux Kernels with PREEMPT_rt on ARM-Based Embedded Devices,' en, *Electronics*, vol. 10, no. 11, p. 1331, Jan. 2021, Number: 11 Publisher: Multidisciplinary Digital Publishing Institute, ISSN: 2079-9292. DOI: 10.3390/electronics10111331. [Online]. Available: https://www.mdpi.com/2079-9292/10/11/1331 (visited on 26/11/2022).

Paper D

InterPRET: A Time-predictable Multicore Processor

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Peter Donovan
UC Berkeley
Berkeley, USA

Edward Wang
MIT
Boston, USA

Edward A. Lee
UC Berkeley
Berkeley, USA

Shaokai Lin
UC Berkeley
Berkeley, USA

Chadlia Jerad
University of Manouba
Manouba, Tunisia

Marten Lohstroh
UC Berkeley
Berkeley, USA

Martin Schoeberl
DTU
Lyngby, Denmark

Published in
Proceedings of Cyber-Physical Systems and Internet of Things Week 2023

Abstract

With the end of Moore's law and the breakdown of Dennard scaling, multicore processors are the standard way to continue improving performance while reducing Size, Weight and Power (SWaP). However, this performance is typically achieved at the cost of repeatability and predictability. Precision-timed (PRET) architectures have been shown to deliver high performance without sacrificing predictability. In this paper, we introduce InterPRET: an architecture consisting of FlexPRET cores interconnected via the S4NOC network-on-chip. Both the processor cores and the network-on-chip are time-predictable, yielding an end-to-end time-predictable architecture suitable for real-time systems.

D.1 Introduction

To meet their timing requirements, real-time system engineers can follow at least three different strategies. The first and arguably the most common strategy is to use general-purpose computer architectures that are optimized for average-case performance. These architectures employ optimization techniques like memory caching, branch prediction, and out-of-order execution, which trades timing predictability for average-case performance. To ensure that all real-time deadlines are met, measurements of execution times to find the worst-observed execution time is combined with over-provisioning of computing resources.

The second strategy is to implement the computation as a synchronous digital circuit, either on an FPGA or as an ASIC. This gives very precise control over timing at the granularity of the clock frequency driving the circuit. However, for modern real-time systems, which perform various computations on different workloads, implementations completely based on synchronous digital circuits are unrealistically complex and inefficient.

In this paper, we argue for a third strategy, which is to rely on timing-predictable computer architectures, also known as PRET machines [1]. PRET machines are stripped of optimization techniques that sacrifice timing predictability for average-case performance. To achieve performance similar to general-purpose processors [2], PRET machines often employ fine-grained multithreading using barrel register files. PRET machines offer such fine-grained control over timing that they are capable of implementing real-time interfaces in software. XMOS, the company behind the only commercially available PRET machine that we know of, refers to this capability as a "software-defined" System-on-Chip (SoC) [3].

As we reach the end of Dennard scaling [4], single-thread performance is no longer seeing much improvement. Computer architects are instead looking to multicore designs to deliver increased performance. This poses significant challenges for real-time systems designers. While multicore architectures may improve average-case performance, they often negatively impact worst-case execution time (WCET) or make it very difficult to reason about [5]. Communication between cores is typically implemented through shared memory. The cores typically do not share memory directly, but maintain a local cache of the shared memory, with a

cache coherence protocol ensuring that data retrieved from local caches is consistent across caches. A consequence of this design is that a write in one core may invalidate a cache line in another, forcing a subsequent read to fetch a new cache line from a larger and slower memory. This makes it difficult to predict how long it takes retrieve a value from memory. In other words, it renders tasks *non-composable* [6] in the sense that tasks running on different cores can affect each other's timing, even if no data dependencies exist between the tasks. This is one of the reasons why real-time systems engineers resist the introduction of multicore architectures into safety-critical systems [7].

A network-on-chip (NoC) is an alternative approach to inter-core communication based on *message passing*. Because message passing does not require a complex memory hierarchy, it is a better candidate for achieving composability. In general, we can distinguish between *event-triggered* and *time-triggered* NoCs. The former is more flexible and has a lower average-case latency. However, it is not composable. The latter is based on time-division multiplexing (TDM) where each communication link has statically allocated slots. This removes the potential for conflicts on shared resources and thus the need for buffering and complicated flow control.

In this paper, we introduce InterPRET (i.e., the **Inter**connected **PRET** machine), a multicore PRET machine based on message passing. InterPRET consists of a set of FlexPRET CPUs [8] interconnected by the time-predictable S4NOC [9, 10]. InterPRET is synthesized to an FPGA and evaluated in terms of resource utilization, timing predictability, and NoC throughput.

The paper is organized as follows. The following section presents related work. Section D.3 provides background into FlexPRET and S4NOC. Section D.4 presents the hardware architecture of InterPRET. Section D.5 evaluates InterPRET, and Section D.6 concludes the paper.

D.2 Related Work

The XCore architecture from XMOS [11] is a commercially available multicore PRET machine. Like InterPRET, the XCore is based on fine-grained multithreading to remove variability in the execution time of instructions. It has a NoC, called XConnect, which uses credit-based control flow. The S4NOC, being based on a static schedule, does not need this type of flow control.

T-CREST [12] was a European research project that aimed to develop a new generation of time-predictable systems based on multicore processors. A T-CREST multicore consists of Patmos processor cores [13] connected via the Argo NoC [14]. The Argo NoC is a TDM-based NoC similar to S4NOC, but with the routing information in the header and a default packet length of three words. The main difference between T-CREST and InterPRET is that the T-CREST cores share external memory. Therefore, they compete for that memory, which is resolved by a time-division multiple access protocol. InterPRET uses local memories for instructions and data and communicates via the NoC only.

MERASA [15] is a time-predictable multicore architecture that guarantees temporal isolation of a single hard real-time task from up to three non-real-time tasks on each core. MERASA contains both scratchpad memories and a cache hierarchy. This enables both timing predictability and high performance. It is less flexible than the InterPRET, which can support multiple hard real-time tasks per core. Furthermore, the NoC-based communication makes InterPRET more scalable.

Vicuna [16] is a time-predictable vector processor for highly parallel real-time applications. Vicuna has a single-instruction multiple-data architecture. Vicuna implements the integer and fixed-point instructions of the RISC-V vector extension. As such, Vicuna is not a stand-alone processor and can only act as a co-processor. InterPRET, on the other hand, is in itself a complete system. A single core of the InterPRET could use Vicuna as a co-processor.

MultiPRET [17] is a multicore PRET machine based on the FlexPRET. Inter-core communication is achieved through shared memory over a TDMA bus. For configurations with tens of cores, the shared memory becomes a bottleneck and a NoC-based architecture, like the InterPRET, is preferable.

The Reduced Complexity Many-Core project from the University of Augsburg adapts RISC-V architectures and includes the PaterNoster [18] NoC. PaterNoster uses packets with single-word flits and a network interface with a single FIFO queue connected to the memory stage of a RISC-V processor. The RISC-V instruction set has been extended with transmit and receive instructions that block until a free slot is available or a packet is found, respectively. Our NoC uses a similar architecture with TDM-based scheduling and a network interface (NI) that can be accessed with normal load and store instructions.

D.3 Background

The InterPRET architecture is based on two open-source projects: (1) The RISC-V PRET machine FlexPRET [8], and (2) the real-time NoC S4NOC [9]. In this section we provide background on those two projects.

D.3.1 FlexPRET

The FlexPRET CPU architecture is the third generation of multi-threaded PRET machines designed at UC Berkeley [2] and was the result of Michael Zimmer's Ph.D. thesis [8]. It is a five stage in-order processor implementing the base 32-bit RISC-V ISA (RV32I). FlexPRET has no branch predictor and uses fine-grained multithreading with a barrel register file to avoid pipeline bubbles. Compared to earlier-generation PRET machines, it has a more flexible thread scheduler which enables round-robin scheduling of threads configured as *hard-real time threads* (HRTTs) and opportunistic scheduling of threads configured as *soft real-time thread* (SRTTs).

The FlexPRET introduces three timing instructions for controlling the timing of each thread: (1) `delay_until` halts execution until some absolute point in the

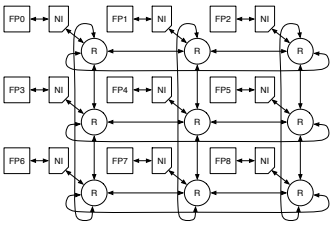


Figure D.1: A bi-torus configuration of an S4NOC with FlexPRET cores FPn, network interfaces NI, and routers R.

future; (2) `wait_until` halts until an interrupt, with the possibility of a timeout; and (3) `interrupt_expire` schedules an interrupt to occur at an absolute point in time.

D.3.2 The Network-on-Chip

The S4NOC [9] is designed to be used in real-time systems. It guarantees a maximum latency and a minimum bandwidth for all channels. S4NOC consists of a network of routers, connected with four ports in a bidirectional torus. The fifth port of the router is the local port and is connected to the network interface (NI). As shown in Figure D.1, we connect a FlexPRET to the processor interface of each NI. The S4NOC connects each FlexPRET core (FPn) through a network interface (NI) to the local port of a router (R).

Access to shared resources in S4NOC, e.g., router and links, are statically scheduled based on TDM. Each slot in the schedule is a single clock cycle during which any given resource of the router (link, multiplexer, output register) may be used by a single packet. The packet moves from the injection point at the sender NI through the network of routers until the destination NI in a pipelined fashion, one hop each clock cycle.

The S4NOC routes packets from a sending core to a receiving core. A packet is a single word of 32 bits. As all traffic is statically scheduled, there are no conflicts on any shared resources, such as links or crossbars. Without conflicts, there is no need for buffering in the routers, flow control between routers, or credit-based flow control between the NIs. The schedule is computed offline and stored in the routers. The schedule contains for each clock cycle the directions of the 5 4:1 multiplexers, which is 10 bits per clock cycle. Therefore, the size of the schedule table is negligible. An all-to-all core communication graph for N cores has $N \times (N-1)$ virtual channels. As an example of a 3×3 multicore, the NoC has 72 virtual channels. The scheduler can implement these virtual channels with a 10-slot TDM schedule. This schedule is only 2 slots longer than the minimum theoretical period needed by the 8 outgoing and 8 incoming virtual channels at each core.

The NI is split into two components: a standard bus interface for the processor and the buffering to decouple between the processor read/write interface and the NoC router's local port. The processor interface maps the ready/valid-based

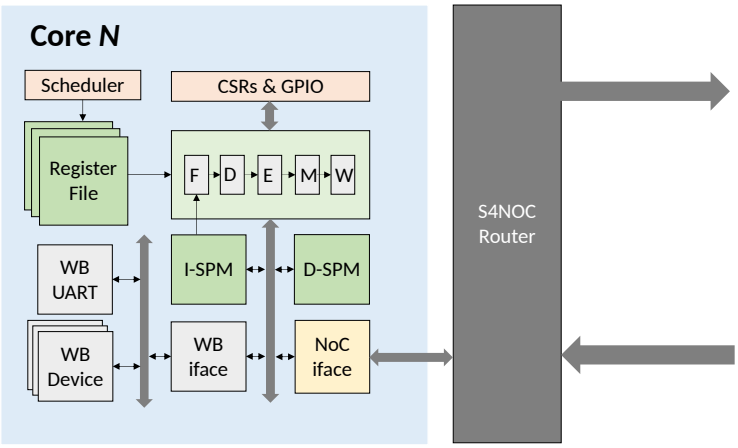


Figure D.2: A single FlexPRET core part of the InterPRET

receive and transmit queues to memory-mapped IO ports. Those ports may block if there is no data to read or space to send. In addition to those two ports, the NI provides status bits to poll on those conditions to avoid blocking operations.

D.4 InterPRET

InterPRET is a configurable multicore architecture. It consists of a number of FlexPRET cores interconnected by the S4NOC. This yields an end-to-end timing-predictable message-passing-based multicore. Figure D.2 shows how a single core of the InterPRET is connected to the rest of the SoC through the NoC. Each FlexPRET core supports hardware multithreading through a barrel register file where each hardware thread has its own set of registers. The NoC interface is memory-mapped together with the data scratchpad memory (D-SPM), the instruction scratchpad memory (I-SPM), and the Wishbone interface.

The NoC is supported by a simple header files library that exposes an API to send data to other cores and receive data from them. The CPU is connected to the NI of the NoC, which contains a buffer that might be full or empty. A timeout is therefore passed to the send and receive functions.

Listing D.1 shows an InterPRET program using the NoC library to facilitate inter-core communication. Both cores will store the same program in their I-SPM and start execution from the same `main`. Using a hardwired register, each core can jump to its respective main function, `main0` and `main1`. The program implements a ping-pong pattern where Core 0 sends a value to Core 1 and Core 1 responds with the same value. The non-blocking NoC API is used which means that for this program to work, the messages must have arrived by the time `noc_receive` is called (Lines 8 and 17). This is ensured by the calls to `delay_until` which time synchronize

```
1  void main0(void) { // Program executed on Core 0
2      uint32_t count = 0, release_time = START_TIME;
3      delay_until(release_time);
4      while(true) {
5          noc_send(1, ++count, NON_BLOCKING);
6          release_time += PERIOD;
7          delay_until(release_time + SEND_LATENCY);
8          noc_receive(&count, NON_BLOCKING);
9          release_time += PERIOD;
10         delay_until(release_time);
11     }
12 }
13 void main1(void) { // Program executed on Core 1
14     uint32_t count, release_time = START_TIME;
15     delay_until(release_time + SEND_LATENCY);
16     while(true) {
17         noc_receive(&count, NON_BLOCKING);
18         release_time += PERIOD;
19         delay_until(release_time);
20         noc_send(0, ++count, NON_BLOCKING);
21         release_time += PERIOD;
22         delay_until(release_time + SEND_LATENCY);
23     }
24 }
25 void main(void) {  // Entry point for both cores
26     switch(read_coreid()) {
27         case 0: main0(); break;
28         case 1: main1(); break;
29     }
30 }
```

Code listing D.1: C code for PingPong on InterPRET

the two cores. This program will be further analyzed in Section 5.

D.4.1 Predictable Timing

Edwards et al. distinguish between repeatable timing and predictable timing [19]. A sequence of instructions is said to be *repeatable* if, given the same inputs, every correct execution of the same sequence will lead to the same observable state. *Predictable* timing is a repeatable property that can be determined in finite time from a specification.

A single FlexPRET core achieves repeatable timing through fine-grained multithreading, which enables temporal isolation and constant instruction latencies, and the use of scratchpad memories. However, it is also the programmer's job to use the hardware resources appropriately to preserve this property. For example, the hardware threads are required to be scheduled at a constant rate (i.e., with a constant *scheduling frequency*).

FlexPRET can deliver repeatable behavior, but can it deliver predictable timing? Timing analyses typically involve two dimensions: the path problem, which is the challenge of finding the longest path in a program, and the state problem, which is the challenge of finding the least processor state that yields the longest execution time [20]. Since HRTTs are guaranteed a constant scheduling frequency

and constant latencies of instructions [8, 21], the use of FlexPRET obviates the need to perform complex modeling of the microarchitecture. In other words, the timing analysis for FlexPRET is more tractable because it is solely concerned with the path problem.

The timing predictability of InterPRET can be attributed to its time-predictable subsystems. Given programs mapped to the FlexPRET cores and a set of inputs, it is possible to derive the execution times of programs running on the FlexPRET cores and the communication times of the S4NOC. These quantities can be used to further calculate end-to-end latencies between cores. Computing a *precise* end-to-end latency depends on a detailed analysis of the aforementioned path problem and an analysis of the TDM schedule. On the other hand, computing *upper bounds* on the end-to-end latencies is a simpler and more composable form of the timing analysis problem. The *worst-case* execution time of programs and the *worst-case* communication latency of the S4NOC can be derived *independently* to calculate the bound of the *worst-case* end-to-end latency.

D.4.2 Software-defined Devices

PRET machines like InterPRET support the implementation of timing critical functionalities, such as IO devices, purely in software. We call such programs Software-defined Devices or *SDDs* for short. We envision a suite of open-source SDDs that can be picked and placed on the different HRTTs of an InterPRET. This is in line with XMOS' vision of a software-defined SoC. At the time of writing, we have implemented one such device, *sdd_uart*, which is a high-speed serial device. The *sdd_uart* runs in a single HRTT and uses the timing instruction *delay_until* to wake up at the right moment to read or write serial port data using a GPIO pin. When the thread is sleeping, SRTTs can use the HRTT's unused cycles.

The *sdd_uart* interacts with the rest of the InterPRET through two circular buffers. Data to be transmitted can be enqueued to the TX buffer, and received data from outside will be enqueued by the UART thread to the RX buffer.

D.4.3 Hardware Devices

There might be other reasons, besides timing predictability, to introduce devices in an SoC. For instance, to perform compute-intensive tasks like digital signal processing or cryptography. For these cases, using HRTTs might not be sufficient and a hardware accelerator might be required. The FlexPRET core is, for this purpose, extended with a Wishbone (WB) bus [22] to which hardware devices can be connected. The WB bus cannot be connected directly to the FlexPRET's bus interface because access latency to WB devices is not constant. The FlexPRET expects single-cycle writes and two-cycle reads with no stalling. The solution is to create a memory-mapped WB master device with predictable access time. The WB master will, in turn, be connected to the WB bus and communicate with the WB devices on behalf of the CPU.

	LUTs	LUTRAMs	FFs	BRAMs
FlexPRET	2526	160	2010	18
S4NOC	175	209	327	0
WB bus	22	0	38	0
WB UART	109	16	95	0

Table D.1: Resource utilization for different components of the InterPRET, for a single core.

	LUTs	LUTRAMs	FFs	BRAMs
4-core	11402 (1.0x)	1120 (1.0x)	9780 (1.0x)	74 (1.0x)
9-core	27346 (2.4x)	3412 (3.0x)	24271 (2.5x)	166.5 (2.3x)
16-core	50554 (4.4x)	8284 (7.4x)	47315 (4.8x)	296 (4.0x)

Table D.2: Resource utilization for different InterPRET configurations.

D.4.4 Bootloader

To enable loading different programs into the instruction memory, a boot procedure is needed. The FlexPRET cores each have a small bootloader hardwired into the first section of the instruction memories. Each core also has a hardware UART device connected to the WB bus. The bootloader receives the application, wrapped in a simple protocol, over this UART device. The application is written, word-by-word, into the I-SPM. After complete reception of the program, the bootloader jumps to the beginning of the newly-written application and starts executing it. Each FlexPRET core of the InterPRET has its UART device connected to the same top-level pins and executes an identical bootloader. The result is that the same program is loaded into each I-SPM of all cores simultaneously.

D.5 Evaluation

The evaluated our InterPRET implementation through benchmarks and case studies using the Zedboard FPGA, which has a XC7Z020 SoC FPGA with 630 KB on-chip memory and 13,300 logic slices.

D.5.1 Resource Utilization

To evaluate the resource utilization, we synthesize InterPRETs of various sizes targeting a XCKU035. Note that we target a different FPGA for resource utilization so that we can synthesize larger multicores. The FlexPRET cores of the InterPRET are configured with 16 KB I-SPM and D-SPM, the round-robin scheduler, four hardware threads, and without a hardware multiplier. The S4NOC is configured with a 32-bit data word size and NI buffer size of two.

Table D.1 shows the resource utilization of the different components of an InterPRET. The numbers reported are the utilization for a single core of an InterPRET. The S4NOC is configured as a 2x2 grid and we divide the resource utilization by four. The NoC has low resource utilization compared to the FlexPRET, except for the LUTRAMs. These are used for the network interface buffers, which make up the backbone of the NoC.

Table D.2 shows how the resource utilization of InterPRET scales with the number of cores. The usage of combinatorial logic (i.e., number of LUTs) and memory (BRAMs) scale linearly with the size of the InterPRET. That is to say, if the number of cores is doubled, so will roughly the utilization of these resources. However, the usage of distributed RAM (LUTRAMs) and registers (FFs) scales super-linearly. This is mainly due to buffering at each NI of the S4NOC. Each NI contains, besides shared input and output FIFOs, one decoupling buffer for each other node in the system. This means that a 2x2 configuration will have 4 NIs each with three buffers, while a 3x3 configuration will have 9 NIs each with eight buffers.

D.5.2 Latency, Jitter, and Bandwidth

To measure NoC sending and receiving latency, we implemented send and receive functions in assembly and called them from a simple C program to approximate a realistic amount of software overhead. In simulation, we measured communication latencies ranging from 35 to 44 cycles, inclusive, when sending data from core 0 to core 1, where InterPRET is configured with four cores and each core has a single HRTT that is scheduled in every clock cycle. Although we anticipate that in typical programs it will be infeasible to predict the communication latency exactly, it is straightforward to compute the jitter: The difference between the maximum latency and the minimum latency is 9 cycles because a 6-cycle busy-wait loop on the receiving side contributes 5 cycles of variability while the 5-cycle TDM schedule contributes 4 cycles of variability.

It is possible to achieve a high bandwidth without repeated software-level ready/valid checks if both the sender and receiver run in HRTTs that produce (respectively, consume) data at equal rates, such that no reads from an empty FIFO nor writes to a full FIFO are attempted. In this case, the rate at which data can be sent over the NoC after initiating communication is limited only by the TDM schedule and by the rate at which software can produce the data. Because the TDM schedule is five cycles long on a 4-core InterPRET, a new word can be sent to each receiving core as frequently as every five clock cycles using an unrolled loop. Our implementation uses a main loop with an unrolled inner loop that sends 16 words, followed by a remainder loop. This allows it to send 500 words in 2744 clock cycles. This results in a bandwidth of 1 word per 5.49 clock cycles, which is close to the maximum NoC bandwidth of 1 word per 5 clock cycles for a single channel. On an FPGA running at 50 MHz, this translates to 291.5 Mbit/s, while the upper bound imposed by the TDM schedule is 320 Mbit/s.

	20 words	100 words	500 words
4-core	165.8 Mbit/s	258.9 Mbit/s	291.5 Mbit/s
9-core	114.7 Mbit/s	148.3 Mbit/s	157.5 Mbit/s
16-core	67.5 Mbit/s	80.2 Mbit/s	83.4 Mbit/s

Table D.3: Measured message-passing bandwidth for different InterPRET configurations.

Similar calculations apply to the other bandwidths reported in Table D.3. Note that the maximum achievable bandwidth between any given pair of cores decreases as the number of cores increases. The TDM schedule on a 9-core InterPRET is 10 cycles long, yielding an upper bound of 160 Mbit/s, while the TDM schedule on a 16-core InterPRET is 19 cycles long, yielding an upper bound of 84.2 Mbit/s.

D.5.3 Case Study 1: Multicore PingPong

To demonstrate the time-predictability of InterPRET, we again turn to Listing D.1 from Section D.4. In this program, `noc_send()` and `noc_receive()` both take a timeout argument set to `NON_BLOCKING`. This means that the function will return immediately, with or without data. In order for this program to work the sender and receiver must be synchronized. Due to the timing predictability of FlexPRET this can easily be achieved without additional communication. The use of `delay_until()` enables both cores to synchronize using *time*. When receiving, the core will add an offset, `SEND_LATENCY`, to the call to `delay_until()`. This offset enables the sending core to wake up before the receiving core to account for the communication latency. As we shall see, this offset is statically computable due to the end-to-end time-predictability of the InterPRET.

In the following, we will compute a feasible value for `SEND_LATENCY`, referred to as L_{send}, ensuring no lost messages. The value can be expressed as

$$L_{send} = L^{tx}_{max} + L^{noc}_{max} - L^{rx}_{min}. \tag{D.1}$$

where L^{tx}_{max} is the maximum latency at the sender side, i.e., the WCET from when `noc_send()` is called to when the message and destination address are enqueued to the NI buffer of the NoC. L^{noc}_{max} is the worst-case message latency between the NIs of two cores communicating through the NoC. Finally, L^{rx}_{min} is the minimum latency at the receiver side, i.e., the best case execution time from when `noc_receive()` is called to when the NI buffer is read.

We can express L^{tx}_{max} as a function of c^{tx}_{max}, the worst-case number of cycles allocated to the sender thread to send a message, N_t the number of HRTTs, and p the clock period of the CPU.

$$L^{tx}_{max} = c^{tx}_{max} \times p \times N_t \tag{D.2}$$

The value of c^{tx}_{max} can be derived based on the program since, based on the configuration, each instruction consumes a known number of cycles on a FlexPRET HRTT. *E.g.,* on FlexPRET with 4 HRTTs, an HRTT typically spends a single cycle executing an instruction.

L^{rx}_{min} is expressed similarly. For a quad-core InterPRET with a round-robin scheduler and 4 HRTTs we get $c^{tx}_{max} = 40$ and $c^{rx}_{min} = 20$, yielding $L^{tx}_{max} = 3200ns$ and $L^{rx}_{min} = 1600ns$.

The worst case message latency between two cores, L^{noc}_{max}, is computed using the following formula extracted from [10]:

$$L^{noc}_{max} = (2 \times l + s + h) \times p \tag{D.3}$$

where l is the NI latency experienced at both ends of the communication, s is the TDM schedule length, and h is the maximum number of hops over the NoC. For our quad-core configuration we have $h = 2$, $s = 5$ $l = 2$ and $p = 20\,ns$, yielding $L^{noc}_{max} = 180ns$.

Finally, we can compute $L_{send} = 1780ns$ based on Eq. D.2. This value was confirmed experimentally on the FPGA.

D.5.4 Case Study 2: Software-defined UART

To demonstrate InterPRET's capabilities as a software-defined SoC, we developed *sdd_uart*, a software-defined UART. On the InterPRET, we use *sdd_uart* to implement a printing service on one of the HRTTs on Core 0. Other threads in the system can schedule the prints by enqueuing data to a circular buffer which is read by the *sdd_uart*-thread. If the thread is on another core, it can send the data to be printed via the NoC. A simple protocol is used where the number of bytes the thread wants to print is written to the buffer before the actual string of characters. The maximum achievable baud rate is limited by the *critical path* between driving the GPIO pin for two consecutive bits. We tested *sdd_uart* running on the FPGA clocked at 50MHz. It was connected through a USB-UART adapter to a PC and could reliably send data at baud rate of 270000. The timing of the UART transmission was also verified with a logic analyzer.

D.6 Conclusion

For future real-time systems, we need time-predictable and high-performance computing platforms. We presented InterPRET, a multicore processor consisting of interconnected FlexPRET cores for time-predictable execution of real-time programs. To make the use of InterPRET practical, we believe that a time-aware message-passing programming model is key. As future work, we plan to make InterPRET programmable using Lingua Franca [23]—a reactor-oriented coordination language that provides deterministic-by-default concurrency and a semantic

notion of time. The Chisel implementation of InterPRET is open source and available on GitHub[1].

[1] https://www.github.com/lf-lang/interpret

Bibliography

[1] S. A. Edwards and E. A. Lee, 'The case for the precision timed (PRET) machine,' in *Design Automation Conference (DAC)*, 2007.

[2] E. Lee, J. Reineke and M. Zimmer, 'Abstract PRET machines,' in *2017 IEEE Real-Time Systems Symposium (RTSS)*, Dec. 2017, pp. 1–11.

[3] XMOS, *Xmos announces software-defined soc platform now compatible with risc-v*, Online Multimedia, Dec. 2022. [Online]. Available: https://www.xmos.ai/xmos-announces-software-defined-soc-platform-now-compatible-with-risc-v/.

[4] R. Dennard, F. Gaensslen, H.-N. Yu, V. Rideout, E. Bassous and A. LeBlanc, 'Design of ion-implanted mosfet's with very small physical dimensions,' *IEEE Journal of Solid-State Circuits*, vol. 9, no. 5, pp. 256–268, 1974. DOI: 10.1109/JSSC.1974.1050511.

[5] J. Nowotsch and M. Paulitsch, 'Leveraging multi-core computing architectures in avionics,' in *2012 Ninth European Dependable Computing Conference*, 2012, pp. 132–143. DOI: 10.1109/EDCC.2012.27.

[6] H. Kopetz, *Real-Time Systems : Design Principles for Distributed Embedded Applications*. Springer, 1997.

[7] J. P. Cerrolaza, R. Obermaisser, J. Abella, F. J. Cazorla, K. Grüttner, I. Agirre, H. Ahmadian and I. Allende, 'Multi-core devices for safety-critical systems: A survey,' *ACM Comput. Surv.*, vol. 53, no. 4, Aug. 2020, ISSN: 0360-0300. DOI: 10.1145/3398665. [Online]. Available: https://doi.org/10.1145/3398665.

[8] M. Zimmer, D. Broman, C. Shaver and E. A. Lee, 'FlexPRET: A processor platform for mixed-criticality systems,' in *Real-Time and Embedded Technology and Application Symposium (RTAS)*, 2014. [Online]. Available: http://chess.eecs.berkeley.edu/pubs/1048.html.

[9] M. Schoeberl, F. Brandner, J. Sparsø and E. Kasapaki, 'A statically scheduled time-division-multiplexed network-on-chip for real-time systems,' in *Proceedings of the 6th International Symposium on Networks-on-Chip (NOCS)*, Lyngby, Denmark: IEEE, May 2012, pp. 152–160. DOI: 10.1109/NOCS.2012.25.

[10] M. Schoeberl, L. Pezzarossa and J. Sparsø, 'S4noc: A minimalistic network-on-chip for real-time multicores,' in *Proceedings of the 12th International Workshop on Network on Chip Architectures*, ACM, Oct. 2019, 8:1–8:6. DOI: 10.1145/3356045.3360714.

[11] D. May, 'The XMOS architecture and XS1 chips,' *IEEE Micro*, vol. 32, no. 6, pp. 28–37, 2012.

[12] M. Schoeberl, S. Abbaspour, B. Akesson, N. Audsley, R. Capasso, J. Garside, K. Goossens, S. Goossens, S. Hansen, R. Heckmann, S. Hepp, B. Huber, A. Jordan, E. Kasapaki, J. Knoop, Y. Li, D. Prokesch, W. Puffitsch, P. Puschner, A. Rocha, C. Silva, J. Sparsø and A. Tocchi, 'T-CREST: Time-predictable multi-core architecture for embedded systems,' *Journal of Systems Architecture*, vol. 61, no. 9, pp. 449–471, 2015, ISSN: 1383-7621. DOI: 10.1016/j.sysarc.2015.04.002.

[13] M. Schoeberl, W. Puffitsch, S. Hepp, B. Huber and D. Prokesch, 'Patmos: A time-predictable microprocessor,' *Real-Time Systems*, vol. 54(2), pp. 389–423, Apr. 2018, ISSN: 1573-1383.

[14] E. Kasapaki, M. Schoeberl, R. B. Sørensen, C. T. Müller, K. Goossens and J. Sparsø, 'Argo: A real-time network-on-chip architecture with an efficient GALS implementation,' *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 24, pp. 479–492, 2016. DOI: 10.1109/TVLSI.2015.2405614.

[15] M. Paolieri, J. Mische, S. Metzlaff, M. Gerdes, E. Quiñones, S. Uhrig, T. Ungerer and F. J. Cazorla, 'A hard real-time capable multi-core smt processor,' *ACM Trans. Embed. Comput. Syst.*, vol. 12, no. 3, Apr. 2013, ISSN: 1539-9087. DOI: 10.1145/2442116.2442129. [Online]. Available: https://doi.org/10.1145/2442116.2442129.

[16] M. Platzer and P. Puschner, 'Vicuna: A Timing-Predictable RISC-V Vector Coprocessor for Scalable Parallel Computation,' in *33rd Euromicro Conference on Real-Time Systems (ECRTS 2021)*, B. B. Brandenburg, Ed., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 196, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 1:1–1:18, ISBN: 978-3-95977-192-4. DOI: 10.4230/LIPIcs.ECRTS.2021.1. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2021/13932.

[17] N. Hili, A. Girault and E. Jenn, 'Worst-case reaction time optimization on deterministic multi-core architectures with synchronous languages,' in *2019 IEEE 25th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, IEEE, 2019, pp. 1–11.

[18] J. Mische and T. Ungerer, 'Low power flitwise routing in an unidirectional torus with minimal buffering,' in *Proceedings of the Fifth International Workshop on Network on Chip Architectures*, ser. NoCArc '12, Van-

couver, British Columbia, Canada: ACM, 2012, pp. 63–68, ISBN: 978-1-4503-1540-1. DOI: 10.1145/2401716.2401730.

[19] S. A. Edwards, S. Kim, E. A. Lee, I. Liu, H. Patel and M. Schoeberl, 'A disruptive computer design idea: Architectures with repeatable timing,' in *International Conference on Computer Design (ICCD)*, IEEE, 2009, pp. 54–59. DOI: 10.1109/ICCD.2009.5413177.

[20] S. A. Seshia and J. Kotker, 'Gametime: A toolkit for timing analysis of software,' in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS),*, 2011.

[21] M. Zimmer, 'Predictable processors for mixed-criticality systems and precision-timed i/o,' PhD thesis, EECS Department, University of California, Berkeley, Aug. 2015. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-181.html.

[22] OpenCores, *Wishbone bus specifications*, Online Multimedia, Mar. 2019. [Online]. Available: https://opencores.org/projects/wishbone.

[23] M. Lohstroh, M. Schoeberl, A. Goens, A. Wasicek, C. Gill, M. Sirjani and E. A. Lee, 'Actors revisited for time-critical systems,' in *Proceedings of the 56th Annual Design Automation Conference 2019*, ser. DAC '19, Las Vegas, NV, USA: ACM, Jun. 2019, 152:1–152:4, ISBN: 978-1-4503-6725-7. DOI: 10.1145/3316781.3323469.

Paper E

fpga-tidbits: Rapid Prototyping of FPGA Accelerators in Chisel

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Yaman Umuruglu
Trondheim, Norway

Milica Orlandić
NTNU
Trondheim, Norway

Martin Schoeberl
DTU
Lyngby, Denmark

To appear in
2023 26th Euromicro Conference on Digital System Design (DSD)

Abstract

With increasingly complex workloads and the end of Dennard scaling, the need for heterogeneous computing is becoming apparent. SoC FPGAs (System-on-chip Field-Programmable Gate Arrays) are a promising solution to this need. They combine the versatility of CPUs and the reconfigurability and high performance of FPGAs. SoC FPGAs have received a great deal of attention in recent years from both academia and chipmakers. However, the task of hardware-software codesign, posed by these platforms, remains challenging. This is partly due to the lack of vendor-neutral abstractions for building and evaluating designs. Chisel is a promising hardware construction language based on the idea of writing hardware generators. In this paper, we present `fpga-tidbits`, an open-source, vendor-neutral Chisel library for rapid prototyping of accelerators for SoC FPGAs.

E.1 Introduction

Computer architectures can roughly be divided into four categories. (1) General-purpose CPUs, (2) specialized processing units like GPUs (Graphic Processing Unit), (3) ASICs (Application Specific Integrated Circuits), and (4) FPGAs (Field Programmable Gate Arrays). Computing has, since its inception, been dominated by general-purpose CPUs, but a shift towards the other categories is under way [1]. This is driven by the increasing demand for computation combined with the end of Dennard-scaling [2]. GPUs, initially designed for the efficient parallel rendering of pixels on computer monitors, proved to be very efficient at implementing deep learning inference and training. However, a lot of applications with more complex control-flow and custom data types do not map efficiently to a GPU. In those cases, an ASIC might be appropriate. An ASIC can achieve high energy efficiency by hand tailoring an accelerator with a control path, data path, and memory hierarchy customized for a specific application. However, designing an ASIC is time-consuming and has high non-recurrent engineering costs. FPGAs give the customizability of an ASIC with the versatility of a CPU. The accelerator can be customized to the application, but the FPGA fabric is still reprogrammable. This paves the way for rapid prototyping and a short time-to-market. The reconfigurability of the FPGA comes at the cost of clock speed, power and area. A rule of thumb is that for the same design, ASICs achieve 10× the speed while FPGAs consume 10× the area and 10× the power.

An SoC FPGA combines embedded CPUs with FPGAs, yielding a versatile and customizable platform with high performance. The two major chipmakers Intel and AMD, have each recently acquired a major FPGA company [3] and are now heavily invested in SoC FPGAs.

FPGA accelerators typically have very similar interfaces to the host CPUs often consisting of (1) memory-mapped control and status registers (CSR), (2) memory ports for requesting and receiving data and (3) general-purpose IO connected to the IO pins of the chip. The accelerators often make use of a lot of the same components as well. For example, direct memory access (DMA) for accessing off-chip memories, various memory like random access memories (RAM), queues, content

addressable memories (CAM), and caches, as well as stream components like forks and filters. Despite these commonalities, there is a lack of vendor-independent abstractions. Choosing the vendor of the SoC FPGA will have a large effect on the design process. This is unlike the software space where different vendors agree on a common abstraction of the CPUs, namely the instruction set architecture.

Accelerators are today typically designed at the register-transfer level (RTL) using traditional hardware description languages (HDLs) such as VHDL and Verilog. However, originally these languages were designed for circuit simulation and not circuit design. Therefore, these languages carry a lot of legacy and non-synthesizable code constructs. Moreover, they lack features of modern programming languages like polymorphism and abstract types. Chisel is a recent hardware construction language [4] implemented as a domain-specific language in Scala. This gives Chisel programmers access to all the Scala language features and enables abstracting away boilerplate hardware with the Scala type system and class hierarchy. A prominent example is Chipyard [5], which is a complete SoC-design framework based on Scala and Chisel.

This paper presents `fpga-tidbits`, a framework built on Chisel that enables rapid prototyping of hardware accelerators for hardware-software codesign on SoC FPGAs. `fpga-tidbits` is focused on open-source tools like Verilator [6] and uses the power of Chisel to create a library of hardware components portable across multiple platforms. `fpga-tidbits` also generates necessary software drivers and memory interfaces, enabling rapid and agile development of FPGA accelerators. A set of case studies on using `fpga-tidbits` is also presented.

E.2 Background

E.2.1 Hardware-Software Codesign on SoC FPGAs

Designing hardware accelerators for SoC FPGAs is a challenging endeavor. A simplified design methodology employed by the authors is illustrated in Fig. E.1. The first step, *Partitioning*, consists in identifying the computational bottlenecks of an application and establishing a partitioning between hardware and software.

In the second step, *High-level modeling*, both the FPGA and the SW part of the algorithm are modeled in a high-level language. The FPGA part will serve later as a functional "golden model" for the FPGA implementation.

In the third stage, *FPGA implementation*, the hardware accelerator is implemented using either an HDL or high-level synthesis (HLS). HLS is the process of automatically generating hardware descriptions from high-level programming languages such as C, OpenCL or MATLAB. While HLS can reduce design time and increase productivity, RTL design offers better fine-grained control and more optimization opportunities for the generated hardware design.

In the fourth stage, *Accelerator architecture*, the overall architecture of the accelerator is decided. The number and types of memory ports, as well as the control and status interface to the CPUs, have to be decided.

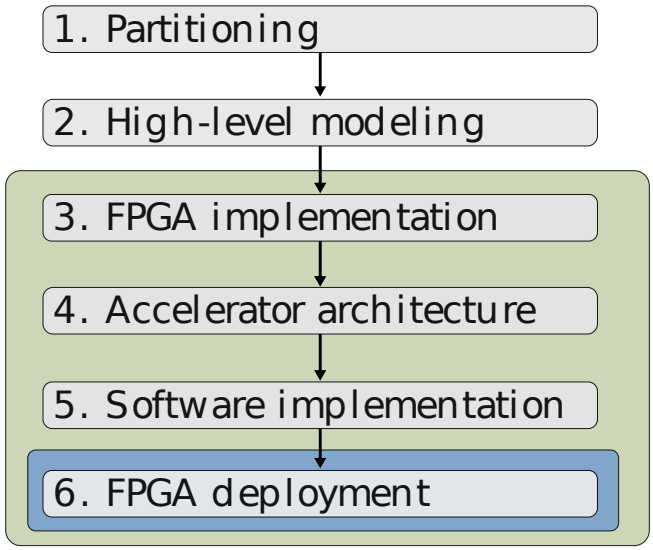


Figure E.1: Design methodology for hardware-software codesign. The green box encapsulates vendor-dependent steps when using the standard development methodology. The blue box encapsulates vendor-dependent steps using fpga-tidbits

In the fifth stage, *Software implementation*, the software side of the hardware-software (HW/SW) codesign is implemented. This part is largely concerned with the interaction between SW and HW through shared memory and control and status registers.

In the last stage, *FPGA Deployment*, the hardware design and the software is deployed onto the actual platform.

Ideally, all stages but the last, are independent of the FPGA vendor and the exact details of the platform. However, in our experience, these details "creep" upward and is present also at the *FPGA Implementation* stage. Partly this is because the design process is iterative, not linear. For instance, it might be desirable to evaluate an initial accelerator design with software in the loop to verify some assumptions about the memory interface between software and hardware. Using state-of-the-art tools requires committing to an SoC FPGA and the associated vendor tools. This results in a lack of reusability of hardware designs across projects unless the same platforms from the same vendor are used. Proprietary tools have the additional disadvantage that they may be discontinued or drop their support for platforms.

E.2.2 Chisel

Even with recent advances in HLS, most hardware design is done at the RTL using one of the two major HDLs, Verilog and VHDL. However, the semantics of Verilog and VHDL are intended for circuit simulation [7] and carry a lot of legacy and non-synthesizable code constructs. The traditional HDLs also lack modern features for managing complexity like polymorphism, parameterized, and abstract types. When designing non-trivial circuits, like a system-on-chip, the traditional HDLs are often used together with scripting languages to *generate* different configurations of HDL modules.

Chisel [4] is a domain-specific language embedded in Scala, a popular programming language that runs on the Java Virtual Machine. It consists of a set of Scala libraries defining hardware data types as well as a compiler emitting an intermediate representation called FIRRTL [8]. In Chisel, a circuit is a directed graph of nodes that represent hardware operations, literals, and updatable elements. Nodes can be grouped into modules which adds hierarchy and exposes an interface to the outside.

Chisel is a hardware *construction* language. This means that it removes the need for an additional scripting language to generate complex configurations of modules. Parameterized types can be used to define circuits and operations that work on a range of hardware data types. Abstract data types can also be used with e.g., overloading of operators. In general, Chisel allows interleaving the design of hardware elements and operations (done with Chisel syntax) with the complex logic for conditional instantiation and wiring of the elements, i.e. writing hardware generators, (done with Scala syntax).

E.3 Related Work

Hardware-software codesign has received a lot of attention in recent years [9], and there is a rich body of work on the topic. In this section, we will highlight a few other hardware-software codesign frameworks of interest. A comparison between `fpga-tidbits` and the related work is summarized in Table E.1.

E.3.1 OpenCL

Open Computing Language (OpenCL) [10] by the Khronos Group is a vendor-neutral framework for writing programs that executes on heterogeneous platforms such as SoC FPGAs. It provides a uniform programming model for CPUs, GPUs and FPGAs based on the C programming language. In the context of SoC FPGAs, OpenCL provides a convenient abstraction for HW/SW codesign since both hardware and software are expressed in the same programming model, this makes it easier to experiment with the partitioning between HW and SW. The boilerplate hardware and software facilitating the communication are generated by the OpenCL compiler. OpenCL is a HLS framework.

E.3.2 SYCL

SYCL [11] is a more recent, higher-level programming model built on top of OpenCL. While OpenCL is based on C and requires that each kernel is written in a separate file, SYCL uses C++ and allows a single-source model. SYCL also abstracts away much of the low-level device management and memory-handling required in OpenCL.

E.3.3 FPGA shells

FPGA shells [1] is a Chisel library intended for synthesizing different configurations of Rocket Chip SoCs [12] to different FPGA platforms. FPGA shells provides so-called overlays for different peripheral devices both on and off-chip like UARTs, SPIs, LEDs, and buttons. Each supported FPGA platform has a *shell*, which the user extends and instantiates their design (typically a CPU core) and peripherals inside it. FPGA shells provide the user with more abstractions of hardware devices than `fpga-tidbits`, but it does not provide co-simulation, software driver generation, or HLS design entry.

E.3.4 Chipyard

Chipyard is an open-source framework for designing and testing customizable SoCs built on top of Chisel [5]. It provides pre-built SoC templates that can be customized for different use cases. Chipyard is focused on the integration of coarser-grain SoC components as opposed to the fine-grained FPGA building blocks offered

[1] https://github.com/chipsalliance/rocket-chip-fpga-shells

by `fpga-tidbits`. Chipyard includes the FireSim [13] project which is a platform for cycle-accurate simulation of Chipyard SoCs on FPGA clusters in the cloud. FireSim provides similar functionality to `fpga-tidbits` but is focused on the simulation and evaluation of CPU designs rather than hardware-software codesign.

E.3.5 Vitis Unified Software Platform

Vitis is a unified software platform for HW/SW codesign on AMD/Xilinx hardware [14]. Vitis bundles a series of tools that previously were distributed individually like Vivado, Vivado HLS, SDSoC, and Petalinux tools. Vitis provides an SW-centric workflow where hardware designs can be compiled and linked with the Vitis v++ compiler. Vitis supports both HDL and HLS for describing the FPGA designs and C, and C++ for the software components. Vitis also supports OpenCL. The communication and synchronization between HW and SW is abstracted by the Xilinx Runtime library (XRT) which provides a hardware-independent API to the accelerators.

	Vendor neutral	Component library	HW design level	SW driver generation
OpenCL	✓	✓	HLS	✓
SYCL	✓	✓	HLS	✓
FPGA Shells	✓	✗	RTL	✗
Chipyard	✓	✓	RTL	✗
Vitis	✗	✓	HLS, RTL	✓
fpga-tidbits	✓	✓	HLS, RTL	✓

Table E.1: Comparison of fpga-tidbits and the related works

E.4 fpga-tidbits

`fpga-tidbits` is a Chisel library for rapid prototyping of portable accelerators for SoC FPGAs. It is vendor-agnostic and the user can defer the selection of the target platform until the last stage in the development cycle (see Fig. E.1). It embraces open-source and uses tools like Verilator [6] to enable end-to-near-end open-source hardware-software codesign. `fpga-tidbits` has three main functions. (1) Providing a platform-independent abstraction on which portable accelerators can be designed. (2) Providing a generic, reusable library of key components like memories, queues, and data movers. (3) Providing a simulation environment where the accelerators can be simulated together with host software.

Listing E.1 shows a trivial accelerator called `HelloAccel` with a host interface `HelloAccelIO`. On line 8, a simple top-level IO with two inputs and a single output (Lines 2-4) is instantiated. All IO signals defined in `HelloAccelIO` are compiled by

```
1 class HelloAccelIO(numMemPorts: Int, p: PlatformWrapperParams) extends
      GenericAcceleratorIF(numMemPorts, p) {
2   val op1 = Input(UInt(32.W))
3   val op2 = Input(UInt(32.W))
4   val res = Output(UInt(32.W))
5 }
6 class HelloAccel(p: PlatformWrapperParams) extends GenericAccelerator(p) {
7   def numMemPorts=0
8   val io = IO(new HelloAccelIO(numMemPorts, p))
9   io.signature := makeDefaultSignature()
10  io.res := io.op1 + io.op2
11 }
```

Code listing E.1: A simple adder accelerator in fpga-tidbits

`fpga-tidbits` to control and status registers (CSRs) which can be written or read from software. In `HelloAccel` the host software can write two values into the CSRs `op1` and `op2`, whose sum is produced by the accelerator and written to the output register `res`.

When compiling `HelloAccel` from Listing E.1, `fpga-tidbits` will generate platform-independent C and C++ drivers. This enables hardware-software codesign without committing to a vendor or a platform.

Listing E.2 shows the generated C++ driver for `HelloAccel`. Notice that the top-level IO of `HelloAccel` can be accessed through the functions defined on lines 14-17.

To construct an instance of a HelloAccel driver (lines 3-9), a `WrapperRegDriver` object is needed. This is the object which facilitates the actual transactions with the hardware design. Different target platforms must be paired with an implementation of `WrapperRegDriver`.

Listing E.3 shows the abstract `WrapperRegDriver` class. It defines a mandatory interface for reading and writing CSRs (lines 21-22). An optional interface for handling shared memory between the host CPU and the accelerator (lines 6-9). It also defines an optional interface for attaching and detaching the accelerator (lines 17-18). All target platforms, like the ZedBoard [15] or Verilator [6], must be paired with a subclass of `WrapperRegDriver` which provides implementations of these functions.

Listing E.4 shows an example of a user-written application program interacting with `HelloAccel`. Notice that this program contains no assumptions on what FPGA platform this is running on. In fact, this program can be linked with any of the `PlatformWrapper`s. In our experience, it is beneficial to develop and debug with emulations of the accelerators for as long as possible. With `fpga-tidbits`, this means simply using the Verilator platform which enables an iterative development cycle without deploying to actual FPGA hardware.

For the rest of this section, we will introduce the different components of `fpga-tidbits`

```
class HelloAccel {
public:
  HelloAccel(WrapperRegDriver * platform) {
    m_platform = platform;
    attach();
    if(readReg(0) != 0x10fc2bdf)  {
      throw "Unexpected accelerator signature, is the correct bitfile loaded?";
    }
  }
  ~HelloAccel() {
    detach();
  }

  AccelReg get_res() {return readReg(1);}
  void set_op2(AccelReg value) {writeReg(2, value);}
  void set_op1(AccelReg value) {writeReg(3, value);}
  AccelReg get_signature() {return readReg(0);}

protected:
  WrapperRegDriver * m_platform;
  AccelReg readReg(unsigned int i) {return m_platform->readReg(i);}
  void writeReg(unsigned int i, AccelReg v) {m_platform->writeReg(i,v);}
  void attach() {m_platform->attach("HelloAccel");}
  void detach() {m_platform->detach();}
};
```

Code listing E.2: Auto-generated software drivers for the HelloAccel

```
class WrapperRegDriver
{
public:
  virtual ~WrapperRegDriver() {}
  // (optional) functions for host-accelerator buffer management
  virtual void copyBufferHostToAccel(void * hostBuffer, void * accelBuffer, unsigned int
       numBytes) {}
  virtual void copyBufferAccelToHost(void * accelBuffer, void * hostBuffer, unsigned int
       numBytes) {}
  virtual void * allocAccelBuffer(unsigned int numBytes) { throw "allocAccelBuffer not
       supported"; }
  virtual void deallocAccelBuffer(void * buffer) {}
  // return CPU-accessible address for a buffer returned from allocAccelBuffer
  // only makes sense for some (shared-memory) platforms
  // facilitates SW that takes advantage of cache coherency
  virtual void * phys2virt(void * accelBuffer) { throw "phys2virt not supported"; }
  virtual bool is_coherent() { return false; }

  // (optional) functions for accelerator attach-detach handling
  virtual void attach(const char * name) {}
  virtual void detach() {}

  // (mandatory) register access methods for the platform wrapper
  virtual void writeReg(unsigned int regInd, AccelReg regValue) = 0;
  virtual AccelReg readReg(unsigned int regInd) = 0;
  virtual std::string platformID() = 0;

};
```

Code listing E.3: The abstract WrapperRegDriver class

```
#include "ExampleAccel.hpp"
#include "platform.h"

int main(int argc, char **argv)
{
  WrapperRegDriver * platform = initPlatform();
  HelloAccel accel(platform);

  accel.set_op1(22);
  accel.set_op2(20);
  assert(accel.get_res() == 42);

  deinitPlatform(platform);

  return 0;
}
```

Code listing E.4: A simple program interacting with the accelerator

E.4.1 Generic Accelerator

The `GenericAccelerator` is an abstract Chisel module that should be extended by the top-level user accelerator as shown in Listing E.1. The `GenericAccelerator` defines a common interface to the host CPU. Top-level user IOs are compiled to CSRs, and shared memory ports are implemented by `GenericMemoryMasterPort`. We envision libraries of open-source `GenericAccelerators` which are easily integrated with existing software that can be deployed to various SoC FPGAs.

E.4.2 Platform Wrappers

The actual top-level module of the FPGA is referred to as the platform wrapper. The `PlatformWrapper` class is an abstract Chisel module that contains the `GenericAccelerator` and the register file. All the user IOs of the `GenericAccelerator` are compiled to CSRs housed in this register file. The abstract `PlatformWrapper` does not provide a connection between the generic memory ports of the `GenericAccelerator` and actual memory ports nor from any top-level host interface to the register file. This is provided in hardware-specific platform wrappers. The platform wrappers enable targeting both real hardware and virtual platforms for emulation.

Fig E.2 shows the `VerilatorWrapper`, which is a particular virtual platform wrapper. Here we see that the register file is contained within the abstract `PlatformWrapper` and connected to the user IO of the `GenericAccelerator`. The `VerilatorWrapper` also emulates a shared memory which is connected to the memory port of the `GenericAccelerator`. On actual platforms, the shared memory will exist outside of the `PlatformWrapper` and be accessible through its top-level IO. The `VerilatorWrapper` exposes two top-level host IOs, one for reading and writing the register file and another for reading and writing to shared memory. When compiled with Verilator, these will be directly accessible from C++. The auto-generated software drivers will use this interface to emulate reads- and writes to shared memory and the Register File from software. The `AXIPlatformWrapper` is another platform wrapper that is shared by platforms with

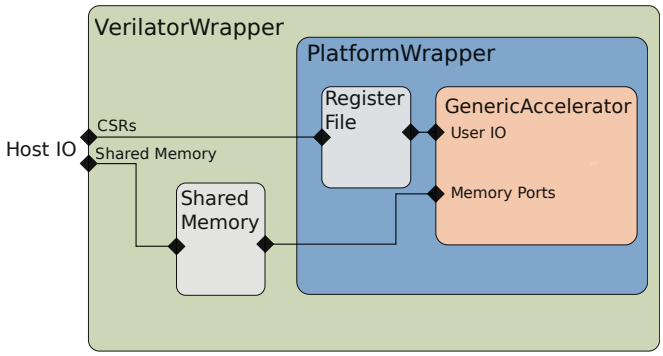


Figure E.2: The module hierarchy for a GenericAccelerator instantiated in the VerilatorWrapper in fpga-tidbits.

an AXI connection between the host CPUs and the FPGA fabric, like the Xilinx Zynqs. It is similar to the `VerilatorWrapper` however, it has a top-level AXI slave port where the CPU can read and write to the Register File and a top-level AXI master port where the accelerator can fetch data from shared memory.

`fpga-tidbits` also support PCIe-attached accelerators through the `WolverinePlatformWrapper` for Convey Computing's Wolverine WX690T platform. This could be further extended to other PCIe-attached accelerators such as AMD/Xilinx Alveo.

E.4.3 On-chip Memories

Efficient data storage management often proves to be the key challenge for achieving high performance and energy efficiency in accelerators. Chisel provides some primitives for memories such as `Vec`, `SyncReadMem`, and `Mem`. However, it is difficult to correctly infer different vendor-specific on-chip memories, such as Block RAMs (BRAMs) and UltraRAMs (URAMs), from these. `fpga-tidbits` provides a library of Chisel modules that are inferred to single-port BRAMs, simple dual-port BRAMs, and true dual-port BRAMs. Building on top of this capability, simple cache memories that combine on-chip memories for reuse with direct memory access for handling misses are also available

E.4.4 Off-chip Memory Access

Shared memory communication between FPGA accelerators and host CPUs is often a performance bottleneck. SoC FPGAs typically include hardened DRAM controllers accessible from both software and the FPGA fabric. Off-chip DRAMs can thus be used to communicate large chunks of data, e.g., video frames, between the CPU and the FPGA. `fpga-tidbits` abstracts away the connection from the FPGA to the DRAM controller behind an interface called the `GenericMemoryMasterPort`. This interface contains ports for read requests, read responses, write requests, write

responses, and write data. Each of these ports is of a `GenericMemoryRequest` type which carries a bundle of signals inspired by the AXI4. The different platform wrappers must provide the parameters deciding the widths of the different parts of the `GenericMemoryRequest`. Finally, for each platform, the `GenericMemoryMasterPort` of the accelerator must be connected to the actual DRAM controller. On Xilinx Zynq systems this is through the High Performace AXI port.

`fpga-tidbits` also contains software drivers for abstracting away the DRAM controller. The API of those drivers is shown on lines 6-14 in Listing E.3. This enables hardware-software cosimulation as we can emulate the off-chip DRAM for Verilator emulations of accelerators.

`fpga-tidbits` includes direct memory access (DMA) modules which writes and reads data to and from the `GenericMemoryMasterPort`. These DMA modules are not software-controlled, as is usual, rather they allow the accelerators to independently make requests to the shared memory.

E.4.5 Streams

FPGAs are suitable for accelerators based on the latency-insensitive design paradigm [16] and `fpga-tidbits` provides a set of stream components like, forks, filters and reducers to enable rapid prototyping of such streaming accelerators.

E.4.6 High-level Synthesis

Chisel's ability to integrate Verilog modules through what are called BlackBox interfaces opens the door to Chisel-centric integration of hardware modules written in HLS languages that compile to Verilog. `fpga-tidbits` provides preliminary support for Vitis HLS from AMD/Xilinx. The user describes their hardware module as a C or C++ function with function arguments annotated to indicate which protocol is desired. The user must then create a Chisel module extending the abstract `TemplatedHLSBlackBox` class. The interface of this module must match, both in signal names and datatypes, the interface inferred by Vitis HLS. `fpga-tidbits` provides scripts for conveniently invoking Vitis HLS and compiling the C/C++ files to Verilog as part of the Chisel compilation.

E.4.7 Hardware-Software Cosimulation

`fpga-tidbits` provides hardware-software cosimulation through the `VerilatorWrapper`. If an accelerator is instantiated within the `VerilatorWrapper`, a C++ emulation of the accelerator is generated. This emulation implements the `WrapperRegDriver` interface shown in Listing E.3. The user software application, which targets the `WrapperRegDriver` interface, can then be linked with this emulation. Each time the interface is used, e.g., by writing to a CSR of the accelerator, the emulation will run for 10 clock cycles. Busy-polling a CSR of the accelerator can thus be used to keep advancing the simulated time in the accelerator. Shared memory communication through off-chip DRAM is simulated by instantiating a memory in the

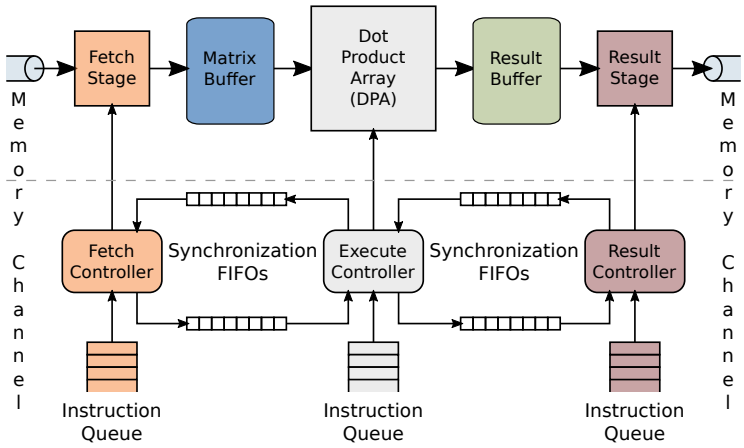


Figure E.3: The bit-serial matrix multiplication (BISMO) architecture is based on fpga-tidbits. Reproduced from [17].

`VerilatorWrapper` and providing implementations of the shared memory communication API (lines 6-8 in Listing E.3) which stores data in that memory.

E.5 Evaluation

We evaluate `fpga-tidbits` through a series of case studies on how it has been used in different academic projects.

E.5.1 Case Study 1: BISMO

One of the first projects utilizing `fpga-tidbits` was BISMO [17], a framework for generating bit-serial matrix multiplication accelerators on FPGAs. Bit-serial matrix multiplication accepts arbitrary-precision fixed-point operands whose bit-widths can vary at runtime. This is well suited for energy- and performance-sensitive applications that require different precisions at different times, such as inference on mixed-precision neural networks [18].

Figure E.3 provides an overview of the BISMO hardware architecture, consisting of three pipeline stages for data fetch, execution and result writeback. Each stage has its own controller, which handles stage operation and inter-stage synchronization according to the instructions it receives. The fetch and result stages, which handle off-chip memory communications, are implemented using `fpga-tidbits` DMA modules, while the rest of the datapath is built with on-chip memories and streaming components from the library. The key component for bit-serial matrix multiplication is the dot product array in the execution stage, which can be parametrized along several dimensions to control the resource utilization and

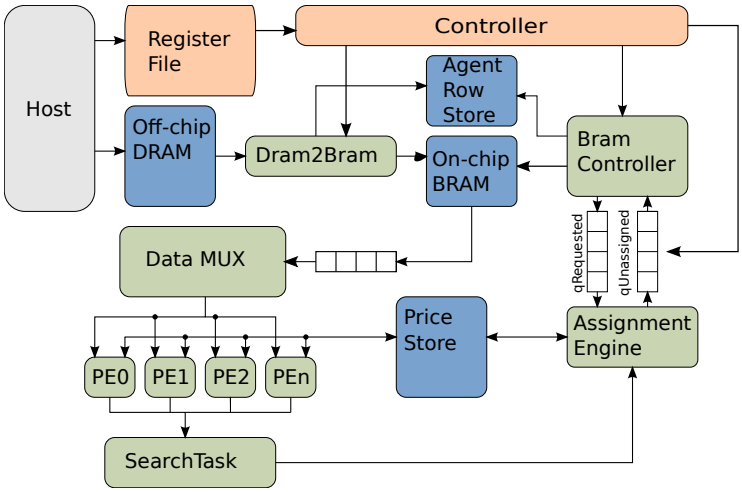


Figure E.4: The architecture of the Auction FPGA accelerator which is based on fpga-tidbits. The orange boxes represent the control and status subsystem. The blue boxes are different forms of memories. Lastly, the green boxes perform computation and control tasks. Reproduced from [19]

performance of the generated accelerator. The entire accelerator is derived from `GenericAccelerator`, enabling the generation of drivers that support the BISMO software orchestrating the runtime behavior.

E.5.2 Case Study 2: Auction Accelerator

In the second case study, we look at how `fpga-tidbits` was used to develop an accelerator for the classical auction algorithm proposed by Bersetkas [19]. The starting point for this project was a Matlab implementation of a multiple object tracking algorithm showing that the solving of large, sparse, assignment problems were the performance bottleneck.

Due to the large quantities of data to be communicated between the host CPU and the FPGA, a shared memory model was chosen. The accelerator receives two pointers to shared memory from the CPU through a register file interface. The first points to the start of the input problem and the other to the place where the results should be written.

Fig. E.4 shows the high-level architecture of the Auction FPGA accelerator. The host CPU communicates with the FPGA through the *Register File* which is generated by `fpga-tidbits` and the *Off-chip DRAM*. The design is organized around a DMA module *Dram2Bram* which fetches data from shared memory and compresses it and writes the compressed data to *On-chip BRAM*. The `StreamReader` and `StreamWriter` components from the streams package of `fpga-tidbits` is used in *Dram2Bram* to generate the read and write requests for the off-chip DRAM.

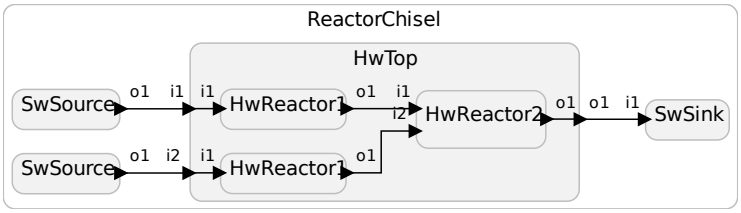


Figure E.5: Simple hardware-software codesign in Lingua Franca using reactor-chisel

The development of both hardware and software was mostly done using the `VerilatorWrapper`. This enabled rapid prototyping and verification. In the later stages of the project, the accelerator was instantiated in the `ZedboardWrapper` and deployed to a ZedBoard FPGA.

E.5.3 Case Study 3: Reactor-Chisel

`fpga-tidbits` also plays a central role in `Reactor-chisel`, an implementation of the reactor model-of-computation [20] in hardware. It is intended as a hardware target for the Lingua Franca [21] language to enable reactor-oriented HW/SW co-design. Lingua Franca already has support for running on the application processing units and the real-time processing units [22] available on the SoC FPGAs. `Reactor-chisel` will thus enable programming the entire SoC FPGA in a single model of computation (MoC).

Fig. E.5 shows a diagram representing a user-defined computation consisting of a set of interconnected *reactors* which communicate through timestamped messages over ports. The actual computation takes place in event-triggered *reactions*. This example has three hardware-reactors contained inside a *HwTop* reactor connected to three software reactors. The computation performed in the reactions within the hardware reactors is expressed in Chisel HDL or HLS while the computation within the software reactions is expressed in C or C++.

Lingua Franca has deterministic semantics for such a composition of reactors. The semantics is concerned with the order of invoking the reactions and messages communicated over the connections. For the software reactions, the semantics is through the use of a timestamp-ordered event queue. In the hardware reactions, it is preserved by only allowing the use of the synchronous subset of the Reactor MoC.

`fpga-tidbits` is used to generate the infrastructure for HW/SW communication and synchronization preserving the deterministic Reactor semantics. Fig. E.6 shows how, in `reactor-chisel`, the top-level hardware-reactor from Fig. E.5 is implemented as a `GenericAccelerator`. This means that we can design the hardware reactor runtime and hardware-software codesign without making any assumptions about which platform it should be hosted on. Indeed, one of the advantages of reactor-oriented HW/SW co-design is that the overall execution is determin-

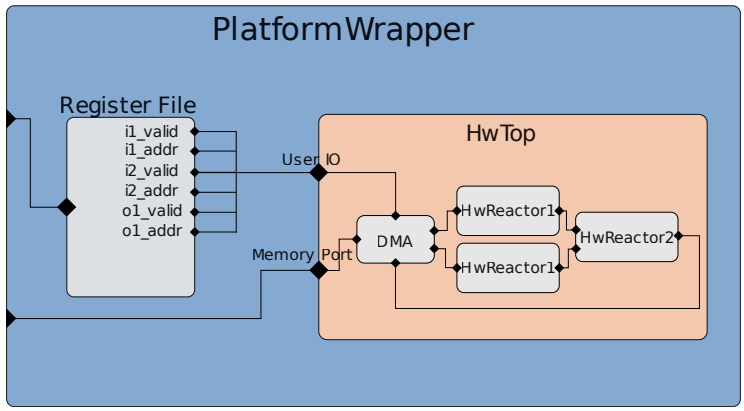


Figure E.6: Software interface for reactor-chisel based on fpgatidbits

istic and independent of the execution times which depends on which platform it executes on. This means that the Verilator wrapper can be used without impacting the behavior.

`Reactor-chisel` is based on shared memory communication between software and hardware reactors. For a software reactor to trigger a hardware reactor it will write pointers to the messages into the register file of the hardware reactor. This is indicated by each input port to `HwTop` in Fig. E.5 having two input ports in Fig. E.6, e.g., `i1` turns into `i1_valid` and `i1_addr`. The addresses are used by the `DMA` module, provided by `fpga-tidbits`, which fetches the messages from shared memory and forwards them to the hardware reactors. The output ports are handled in the same way with the messages being written directly to shared memory and the address and status being communicated through the register file.

`Reactor-chisel` is an open-source work-in-progress and hosted at GitHub. [2]

E.6 Future Work

Although having proved very useful in several projects, `fpga-tidbits` still has room for a lot of improvement. First, an abstraction for the IO pins of the FPGA is needed for interacting with off-chip devices, e.g., triggering a camera. Second, SoC FPGAs typically include several hardened devices like UARTs, SPIs, and Ethernet controllers which can be accessed from the FPGA, abstraction over such devices is needed. Finally, custom interfaces between the FPGA and CPUs such as the ACP ports on the AMD FPGAs also need abstractions.

[2] https://www.github.com/erlingrj/reactor-chisel

E.7 Acknowledgements

We would like to thank the program committee of DDVC for their thorough reviews which helped us clarify and improve the paper.

E.8 Conclusion

We have presented `fpga-tidbits`, a Chisel library for rapid prototyping of accelerators for SoC FPGAs. `fpga-tidbits` provides vendor-neutral abstractions for prototyping and evaluating HW/SW codesigns. `fpga-tidbits` is open-source and available at GitHub. [3]

[3] https://www.github.com/maltanar/fpga-tidbits

Bibliography

[1] W. J. Dally, Y. Turakhia and S. Han, 'Domain-specific hardware accelerators,' *Commun. ACM*, vol. 63, no. 7, pp. 48–57, Jun. 2020, ISSN: 0001-0782. DOI: 10.1145/3361682. [Online]. Available: https://doi.org/10.1145/3361682.

[2] M. B. Taylor, 'Is dark silicon useful? harnessing the four horsemen of the coming dark silicon apocalypse,' in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC '12, San Francisco, California: Association for Computing Machinery, 2012, pp. 1131–1136, ISBN: 9781450311991. DOI: 10.1145/2228360.2228567. [Online]. Available: https://doi.org/10.1145/2228360.2228567.

[3] J. Hou and J.-E. Schmitt, 'All change in edge computing: As amd buys xilinx and nvidia acquires arm, we ask two industry experts what this could mean for the vision sector.,' *Imaging and Machine Vision Europe*, no. 102, pp. 12–14, 2020.

[4] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek and K. Asanovic, 'Chisel: Constructing hardware in a Scala embedded language,' in *Design Automation Conference (DAC)*, ACM/EDAC/IEEE, 2012, pp. 1216–1225.

[5] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović and B. Nikolić, 'Chipyard: Integrated design, simulation, and implementation framework for custom socs,' *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020. DOI: 10.1109/MM.2020.2996616.

[6] Veripool, *Verilator manual*, https://www.veripool.org/wiki/verilator/Manual-verilator, 2020.

[7] *Verilog hdl background and history*, Accessed on July 11, 2023. [Online]. Available: https://digilent.com/reference/learn/fundamentals/digital-logic/verilog-hdl-background-and-history/start.

[8] P. S. Li, A. M. Izraelevitz and J. Bachrach, 'Specification for the firrtl language,' EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-9, Feb. 2016. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-9.html.

[9] J. Hennessy and D. Patterson, 'A new golden age for computer architecture: Domain-specific hardware/software co-design, enhanced,' in *ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018.

[10] A. Munshi, 'The opencl specification,' in *2009 IEEE Hot Chips 21 Symposium (HCS)*, IEEE, 2009, pp. 1–314.

[11] *Sycl*, Accessed on July 11, 2023. [Online]. Available: https://www.khronos.org/sycl/.

[12] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. A. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo and A. Waterman, 'The rocket chip generator,' EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, Apr. 2016.

[13] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, D. Lee, N. Pemberton, E. Amaro, C. Schmidt, A. Chopra, Q. Huang, K. Kovacs, B. Nikolic, R. Katz, J. Bachrach and K. Asanovic, 'Firesim: Fpga-accelerated cycle-exact scale-out system simulation in the public cloud,' in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 29–42. DOI: 10.1109/ISCA.2018.00014.

[14] AMD/Xilinx, *Vitis unified software platform documentation: Embedded software development*, Retrieved July 11, 2023. [Online]. Available: https://docs.xilinx.com/r/en-US/ug1400-vitis-embedded.

[15] Avnet, *ZedBoard HW Users Guide*, https://digilent.com/reference/_media/reference/programmable-logic/zedboard/zedboard_ug.pdf, 2021.

[16] L. Carloni, K. McMillan and A. Sangiovanni-Vincentelli, 'Theory of latency-insensitive design,' *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 9, pp. 1059–1076, 2001. DOI: 10.1109/43.945302.

[17] Y. Umuroglu, L. Rasnayake and M. Själander, 'Bismo: A scalable bit-serial matrix multiplication overlay for reconfigurable computing,' in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, 2018, pp. 307–3077. DOI: 10.1109/FPL.2018.00059.

[18] Y. Umuroglu and M. Jahre, 'Streamlined deployment for quantized neural networks,' *arXiv preprint arXiv:1709.04060*, 2017.

[19] E. Jellum, M. Orlandić, E. Brekke, T. Johansen and T. Bryne, 'Solving Sparse Assignment Problems on FPGAs,' *ACM Trans. Archit. Code Optim.*, vol. 19, no. 4, Dec. 2022, ISSN: 1544-3566. DOI: 10.1145/3546072. [Online]. Available: https://doi.org/10.1145/3546072.

[20] M. Lohstroh, M. Schoeberl, A. Goens, A. Wasicek, C. Gill, M. Sirjani and E. A. Lee, 'Actors revisited for time-critical systems,' in *Proceedings of the 56th Annual Design Automation Conference 2019*, ser. DAC '19, Las Vegas, NV, USA: ACM, Jun. 2019, 152:1–152:4, ISBN: 978-1-4503-6725-7. DOI: 10.1145/3316781.3323469.

[21] M. Lohstroh, C. Menard, S. Bateni and E. A. Lee, 'Toward a Lingua Franca for deterministic concurrent systems,' *ACM Transactions on Embedded Computing Systems (TECS), Special Issue on FDI'19*, vol. 20, no. 4, Article 36, May 2021. DOI: 10.1145/3448128.

[22] E. R. Jellum, S. Lin, P. Donovan, E. Soyer, F. Shakir, T. Bryne, M. Orlandic, M. Lohstroh and E. A. Lee, 'Beyond the Threaded Programming Model on Real-Time Operating Systems,' in *Fourth Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2023)*, F. Terraneo and D. Cattaneo, Eds., ser. Open Access Series in Informatics (OASIcs), vol. 108, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 3:1–3:13, ISBN: 978-3-95977-268-6. DOI: 10.4230/OASIcs.NG-RES.2023.3. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2023/17734.

Paper F

Solving Sparse Assignment Problems on FPGAs

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Milica Orlandić
NTNU
Trondheim, Norway

Edmund Brekke
NTNU
Trondheim, Norway

Tor Arne Johansen
NTNU
Trondheim, Norway

Torleiv Håland Bryne
NTNU
Trondheim, Norway

Published in
ACM Transactions on Architecture and Code Optimizations (TACO)

Abstract

The assignment problem is a fundamental optimization problem and a crucial part of many systems. For example, in multiple object tracking, the assignment problem is used to associate object detections with hypothetical target tracks and solving the assignment problem is one of the most compute-intensive tasks. To enable low-latency real-time implementations, efficient solutions to the assignment problem is required. In this work, we present Sparse and Speculative (SaS) Auction, a novel implementation of the popular Auction algorithm for FPGAs. Two novel optimizations are proposed. First of all, the pipeline width and depth are reduced by exploiting sparsity in the input problems. Secondly, dependency speculation is employed to enable a fully pipelined design and increase the throughput. Speedups as high as 50× are achieved relative to the state-of-the-art implementation for some input distributions. We evaluate the implementation both on randomly generated data sets and realistic data sets from multiple object tracking.

F.1 Introduction

The assignment problem is a special type of a linear programming problem, where an optimal one-to-one assignment between a set of *objects* and a set of *agents* is sought. The assignment problem is widely used, for instance, to model network traffic and optimizing routes [1], alignment of protein-protein interaction networks in bioinformatics [2], dynamic task allocation for robots [3–5] and computer vision tasks [6, 7]. It is formulated as maximum weight matching for bipartite graphs. The assignment problem is of particular importance in multiple object tracking (MOT) [8], where, at the data association stage, each object detection is to be associated with a single track. MOT is an NP-hard problem and the assignment problem is the main computational bottleneck [9, 10]. MOT is one of the enabling capabilities for autonomous robotics systems [11, 12] and efficient and scalable solutions are needed.

The Auction algorithm is an effective iterative solution to the assignment problem [13]. There exists multiple optimized implementations of the Auction algorithm for CPUs, GPUs [14], and FPGAs [15–17]. In [17], a FPGA implementation is proposed which claims 10× performance increase relative to a baseline CPU implementation, whereas a 7× performance increase is reported for the GPU implementation in [14]. However, the proposed implementations fail to address two critical properties of the assignment problems found in e.g. MOT:

1. The problems show a high degree of **sparsity**,
2. The assumption of **data dependencies** between consecutive iterations is often false.

In this paper we propose Sparse and Speculative (SaS) Auction, an accelerator generator for the auction algorithm targeting System-on-Chip (SoC) FPGAs. It takes advantage of the sparsity often found in the problems, which both reduces the area and depth of the processing pipeline. We also propose a data dependency speculation scheme which reduces the number of pipeline bubbles and improves

Table F.1: A simple assignment problem

	Object		
Agent	o_1	o_2	o_3
$Alice$	5	10	1
Bob	4	11	3
$Charlie$	5	14	8

the throughput. The implementation is evaluated on a SoC FPGA and the source code is available on Github [18].

The rest of the paper is organized as follows. In Section 2, a brief background into the assignment problem, the Auction algorithm and its application in MOT is given. In Section 3, related work is reviewed. In Section 4, the proposed implementation of the SaS Auction is introduced. In Section 5, the evaluation and results are discussed, before the papers is concluded in Section 6.

F.2 Background

F.2.1 The assignment problem

The goal of the assignment problem is to find an assignment between a set of objects O and a set of agents A which maximizes a reward function. The assignment problem is also referred to as maximum weight matching of weighted bipartite graphs and is solvable in polynomial time. The assignment problem is characterized by a reward matrix R of size $n \times m$ where r_{ij} is the reward for assigning object j to agent i. The solution to the assignment problem is then finding an optimal assignment matrix B where matrix element $b_{ij} = 1$ if the edge from object j to agent i is picked, and zero otherwise, i.e. b_{ij} is binary.

This optimization problem can be expressed as a linear program as follows

$$\max \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} r_{ij} \cdot b_{ij} \tag{F.1}$$

$$\text{subject to } \sum_{i=0}^{m-1} b_{ij} \leq 1, \forall j = 0, 1, \ldots n-1 \tag{F.2}$$

$$\sum_{j=0}^{n-1} b_{ij} = 1, \forall i = 0, 1, \ldots m-1. \tag{F.3}$$

Consider the simple assignment problem shown in Table F.1. Here the set of agents is $A = \{Alice, Bob, Charlie\}$ and the set of objects are $O = \{o_1, o_2, o_3\}$. The value r_{ij} represents the reward associated with assigning agent a_i to object o_j.

F.2.2 Methods for the assignment problem

Simplex methods

The assignment problem can be solved with well-studied algorithms from linear programming like the simplex method [19]. The simplex method starts by observing that the solution space of a linear programming problem is a polytope defined by the constraints of the linear program. Since the cost function is linear it can be shown that the optimum must lie on a vertex of this polytope. For the assignment problem, each possible mapping of agents to objects make up a vertex in a $(n+m)$-dimensional polytope, where n is the number of agents and m is the number of objects. The Simplex method starts by picking an initial vertex. It then traverses the edges, from vertex to vertex, until the optimum is found. In the worst case, the algorithm must visit all vertices, giving it a computational complexity equivalent of a brute force method with $O(2^N)$.

The assignment problem belongs to a special class of linear programming problems known as transportation problems. Transportation problems can be represented as graphs where the decision variables are the weights of the edges. Each decision variable only occurs twice in the constraints, once for the source node and once for the destination node. This additional structure is taken advantage in e.g. the Streamlined Simplex [20].

In recent year, enterprise optimization software like Gurobi [21] and CPLEX [22] have gained popularity. Here the assignment problem is solved as a linear integer programming problem using Simplex-like algorithms.

The Hungarian method

The Hungarian method was introduced by Khun [23] as a more efficient solution to the assignment problem than the traditional linear programming methods. In the Hungarian method, the assignment problem is modeled as a graph $G = (V, E)$ and a reward matrix R. Where $V = A \cup O$ is the set of vertices, $E = \{(a_i, o_j)\}_{a_i \in A, o_j \in O}$ is the set of edges and r_{ij} is the weight of edge (a_i, o_j). A matching M is a subset of E such that no edges in M share a common vertex. We call M a perfect matching if it matches all the vertices of V. Given a graph G and a matching M, an augmenting path is a path in G which starts and ends at an unmatched vertex and contains alternating edges from M and $E - M$. Augmenting paths can be used to iteratively find perfect matchings.

The assignment problem can thus be reformulated as finding the perfect matching with the maximum weight. Compared to the general class of transportation problems, the assignment problem has two additional features. Firstly, the weights are binary, i.e. either 1 or 0. Secondly, if a particular edge between agent a_i and object o_j has the weight 1, then all other edges connected to both a_i and o_j has the weight 0. This information is not utilized by the linear programming methods and makes them comparatively slow [13]. The Hungarian method solves the assignment problem in polynomial with a computational complexity of $O(N^3)$. The

Hungarian method is a primal-dual algorithm where the starting point is a feasible solution. It works by maintaining and iteratively updating two data structures. The first is a labeling l of G. A labeling is a function mapping the set of vertices to the real numbers. A valid labeling is defined as one where the the sum of the labels for two vertices is less than or equal to the weight of the edge connecting them. The second is a matching M. It can be shown that given a labeling l, and the equality subgraph G_l then a perfect matching M of G_l is also the maximum weight matching of G. The Hungarian method starts with a matching M and a valid labeling l. It then augments the matching until either it finds a perfect matching, or no augmenting path exists. In the latter case, it improves the labeling function l before going back to looking for an augmenting path.

The Hungarian method is not suitable for paralellization as both the labeling improvement and finding an augmenting path are inherently sequential operations.

LAPJV

The LAPJV algorithm was introduced by Jonker and Volgenant [24] as an improvement of the Hungarian method. While the Hungarian method works by finding any feasible augmenting path, LAPJV looks for the shortest augmenting path. This is found using Dijkstras shortest path algorithm.

The Auction algorithm

The Auction algorithm was proposed by Bertsekas [25] as a general solution to the assignment problem. The assignment problem is modeled as an auction and the reward r_{ij} is interpreted as how much agent a_i is willing to pay for the object o_j.

The Auction method is not a primal-dual algorithm like the Hungarian method and each iteration is not guaranteed to reduce the total cost. Optimality can be guaranteed if ϵ, the minimum raise, is less than a threshold. The algorithm is guaranteed to terminate if there exists a feasible solution. The relaxed constraint on monotonically improving solutions enables the Auction method to reach the optimum in fewer steps. Its computational complexity is $O(N^2 log(N))$ [26].

The Auction algorithm works by updating a price vector P which contains the current price for each object, and an assignment vector that stores information about which agents are assigned to which objects. The price vector is initialized to 0 and is increased as agents bid for objects. Each iteration consists of two phases, the bidding phase and the assignment phase. In the bidding phase, a subset of the agents $A_{bid} \subseteq A$ calculate bids for a subset of the objects $O_{bid} \subseteq O$. In the assignment phase, the highest bid is picked and the bidding agent is assigned to the object. There are two commonly used approaches to the bidding phase. The first alternative is referred to as the Jacobi variant. Here, all unassigned agents bid for a single unassigned object. I.e., $A_{bid} = A_{unassigned}$ and $O_{bid} = o_k$, where

$A_{unassigned}$ is the set of unassigned agents and o_k is a single unassigned object. Referring to the example in Table F.1, the first iteration of the Jacobi method would consist of Alice, Bob and Charlie bidding of object o_1. The second alternative is referred to as the Gauss-Seidel variant. Here, a single unassigned agent bids for all the unassigned objects. This means that $A_{bid} = a_k$ and $O_{bid} = O$. Here, a_k is a single unassigned agent. The first iteration of the Gauss-Seidel variant would consist of Alice bidding for objects o_1, o_2, o_3. The Jacobi variant can be implemented by assigning a processing element to each *agent* to compute the bids in parallell. The disadvantage of this approach is that as more agents are assigned, fewer of the processing elements are used. The Gauss-Seidel variant can be implemented by assigning a processing element to each *object*. It does not suffer from the same disadvantage as the Jacobi method as the number of objects being bid on remains constant as the algorithm proceeds. For the rest of this work, the Gauss-Seidel variant is used.

The winning bid each round is the bid for the object which has the highest benefit to the agent. The benefit of object j for agent i is the difference between the agents' reward $R(i, j)$ for the object o_j and the current price $P(j)$, computed as:

$$b(i, j) = R(i, j) - P(j). \tag{F.4}$$

When the object with the highest benefit is found, the agent decides its bid for that object. The bid is made such that the object still has the highest benefit to the agent. This is achieved by raising the bid for the object with an amount equal to the difference between the highest and the second-highest benefit, expressed as:

$$\text{raise} = b_{\max} - b_{\max-1}. \tag{F.5}$$

The pseudo-code for the Auction algorithm is presented in Figure F.1. Lines 1-3 initialize the arrays containing object prices and assignments and the queue containing the unassigned agents. Inside the loop, an unassigned agent is dequeued and agents' benefits for each object are computed. Line 7 extracts the index of the object with the highest benefit, while Line 8 gets the highest and second-highest benefit. Line 9 calculates the raise according to (F.5). Lines 14-17 unassigns any previously assigned agent. This proceeds until agents are either assigned or disregarded due to not being able to create a positive raise (lines 10-13). An optimized C++ implementation is published in [27].

```
1   prices[N] = {0}
2   objectAssignments[N] = {invalid}
3   unassigned.enqueue(0 to N)
4   while unassigned not empty do
5     currAgent = unassigned.dequeue()
6     benefit = reward[currAgent] - prices
7     object = benefit.indexOfMax()
8     highest = benefit.sorted().take(2)
9     raise = highest[0] - highest[1]
10    if raise < 0 then
```

```
11        continue
12      end if
13      bid = price[object] + raise
14      if objectAssignments[object] then
15        oldAgent = objectAssignments[object]
16        unassigned.enqueue(oldAgent)
17      end if
18      objectAssignments[object] = currAgent
19      prices[object] = bid
20    end while
```

Code listing F.1: Auction algorithm, Gauss-Seidel variant

Execution of the Auction algorithm

Fig. F.1 shows the execution of the four iterations needed to solve the assignment problem in Table F.1 using the Auction algorithm.

1. In the first iteration, *Alice* is bidding. Since the prices are all zero, the object benefits are equal to the object rewards. The most beneficial is o_2 marked with blue and the second most beneficial object is o_1 marked with green. The *raise* is the difference between the most and the second most beneficial object which is 5. After iteration 1 *Alice* is assigned to o_2 and the price is set to 5.
2. In the second iteration, Bob is bidding. His benefits are calculated using his rewards and the updated prices. His most beneficial object is o_2 and his raise is 2. Alice is thus unassigned from o_2 and replaced by Bob. The price is raised to 7.
3. In the third iteration, Charlie is bidding. His most beneficial object is o_3 which has a 1 unit more benefit than o_2. He thus bids 1 for o_3 and is assigned to it.
4. In the fourth and last iteration, Alice is bidding again. With the updated prices, o_1 is now the most beneficial object, 2 units more beneficial than o_2. She bids 2 for o_1 and is assigned to it. After iteration 4 there are no more unassigned agents and the algorithm terminates.

F.2.3 The assignment problem in MOT

In MOT, the goal is to estimate the position and velocity of multiple targets, or objects, based on sensor measurements from a single or multiple sensors. Probabilistic models are used to account for clutter (false positives) and misdetections (false negatives) in the sensor measurements. A type of Bayesian filter is usually used to predict *a priori* state estimates and combine them with sensor data into *a posteriori* state estimates [28].

Targets that are sufficiently close to each other are clustered together. In this manner, the complexity is lowered as the algorithm can be run independently

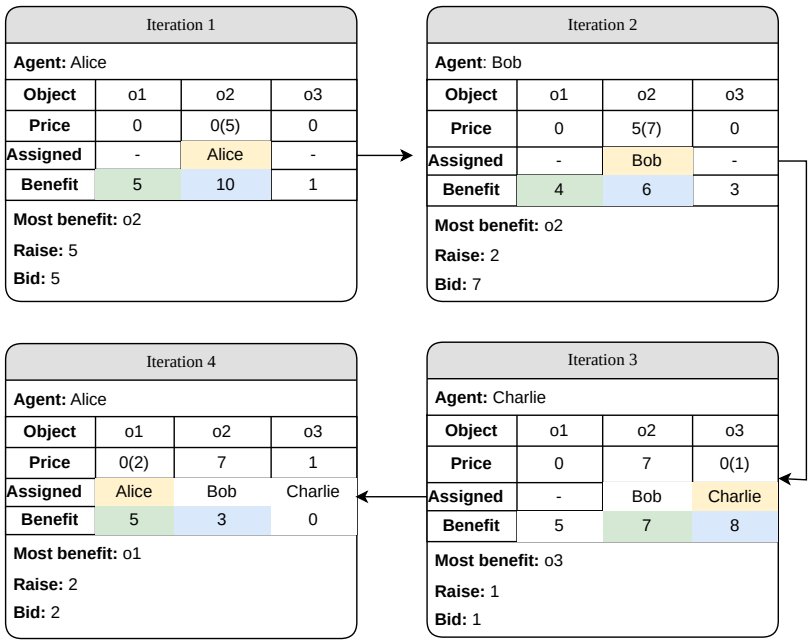


Figure E.1: Execution of the Gauss-Seidel Auction algorithm

on each cluster. A possible association between a measurement and a target is called a single-target hypothesis. The measurements within the range of a cluster will result in data association hypotheses for all the targets in the cluster. Finding the optimal data association hypothesis is modeled as an assignment problem. The number of hypotheses in a cluster can reach hundreds or even thousands. However, it is common that relatively few hypotheses contains the majority of the probability density mass. Validation gating is a common technique where hypotheses with probability below a certain threshold are set to zero. This can yield large and highly sparse assignment problems. In the Multiple Hypothesis Tracker (MHT) [8], a set of hypotheses are maintained over consecutive iterations. For each new set of measurements, the hypotheses give birth to child hypotheses that inherit the probability properties from their parents. To avoid an explosion of hypotheses, pruning is applied to remove the most unlikely candidates.

In recent years, a generalization of MHT known as the Poisson Multi-Bernoulli Mixture (PMBM) filter [29] has been proposed as a state-of-the-art approach to MOT. We evaluate SaS Auction on a set of assignment problems generated by a PMBM tracker. It is referred to as the MOT data set.

F.3 Related Work

In [14], Vascencelos et. al. proposes accelerating the Auction algorithm on a GPU claiming a 7x speedup relative to a baseline CPU implementation. The algorithm is divided into a bidding phase and an assignment phase. In the bidding phase, all agents bid on an object in parallel, whereas all objects find a valid bid and update their price in the assignment phase. Whilst achieving good performance for the big dense problems, the GPU is characterized by having high energy consumption, making it unfit for many embedded applications.

In [15, 16] Hung et. al. proposes an FPGA accelerator for solving the assignment problem. The architecture is based on a recurrent neural network but the performance scales poorly to big problems which can take multiple seconds to solve. In [17], Zhu et. al. proposes a hardware architecture for Auction algorithm accelerators on FPGAs. They claim a 10x speedup relative to a baseline CPU implementation. A simplified overview of the architecture is shown in Fig. F.2 . It consists of a pipeline with a configurable number of processing elements (PEs). In each iteration object-rewards for a single object is read from memory or the host PC through the I/O FIFO. The rewards are distributed amongst the PEs in the PE Array. Each PE contains a small LUTRAM where the object prices related to that PE are stored. For an architecture with N PEs capable of solving problems with up to M objects PE-k will store prices $P_{k+i\cdot N}$ for $k = 1 \dots M/N$. The resulting benefits are passed to the SearchTask module which finds the highest and second-highest benefit. The SearchTask is implemented as a tree of comparators and its depth scales with $\log_2(\mathrm{N})$. The two highest benefits are further passed to the AssignmentEngine which computes the raise. The AssignmentEngine contains the assignment array and unassigns the previous agent holding the highest bid for

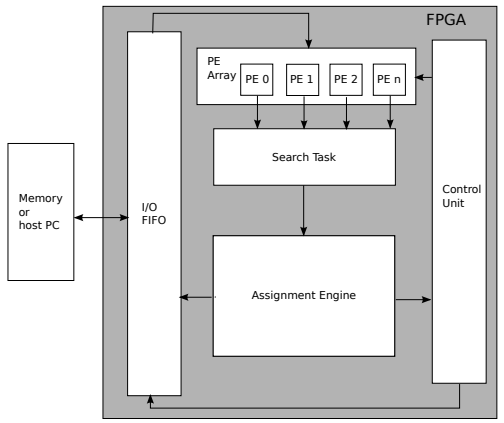


Figure F.2: Architecture of the accelerator of Zhu. et. al. Adapted from [17]

the object in question. The raise is then forwarded from AssignmentEngine to the ControlUnit which updates the price within the right PE. The I/O FIFO receives information of unassigned agents such that it can request its object rewards from memory. In the rest of this paper, we refer to the FPGA implementation by Zhu et. al. as state-of-the-art (SoA) Auction.

F.4 Sparse and Speculative (SaS) Auction

SaS Auction is a flexible FPGA accelerator that efficiently solves the assignment problem. There are multiple reasons for using an FPGA rather than a CPU or GPU for solving the assignment problem. Due to its ability to implement arbitrary datapaths, it has greater potential for taking advantage of sparsity. This can yields designs with lower latency and lower energy consumption. In robotic applications embedding target tracking the Auction algorithm will typically be on the critical path. Reducing the latency of the algorithm can thus allow the robot to navigate faster, Also, for mobile battery-driven autonomous robots, energy efficiency is paramount. FPGAs can achieve much higher energy efficiency than CPUs and GPUs [30]. Lastly, for some autonomous robots, like drones, the weight of the computational platform is an important parameter. In some cases, the weight can be the limiting factor for performance [31]. Due to reduced power consumption, using an FPGA could remove the need for a heavy heatsink.

In the following, the term *agent* is used to refer to the row of object-rewards associated with the agent. We use *agent-index* to refer to the index, or id, of the agent.

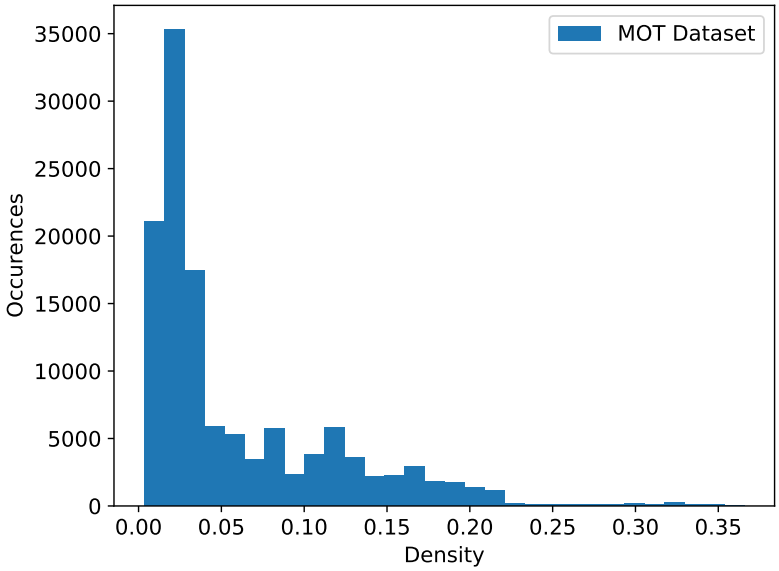


Figure F.3: Sparsity in MOT simulations

F.4.1 Sparse problems

An assignment problem is sparse if the majority of the elements in the reward matrix R are zero. If $r_{ij} = 0$, then object-j is never assigned to agent-i. We also refer to these as non-valid object rewards. Fig. F.3 shows the level of sparsity found in the assignment problems in the MOT data set. The x-axis shows the density which is the inverse of the sparsity. Clearly, these are highly sparse assignment problems. The vast majority of problems has less than 5% density.

Fig. F.4 shows how the depth of the SearchTask scales with the number of processing elements (PEs). Each level of the search tree is implemented in a separate pipeline stage. Over-dimensioning the accelerator by using too many PEs has a negative performance impact. The throughput is affected by the depth of the SearchTask as pipeline stalling takes place at the PE stage until the preceding agent has finished executing in the SearchTask. The number of PEs also impacts the area and the obtainable clock rate of the accelerator.

On the other hand, under-dimensioning the accelerator with too few PEs is also costly and reduces the throughput, since it makes each agent spans multiple pipeline stages.

Generally, the optimal number of PEs for a particular problem is the number of object rewards per agent. By taking advantage of sparsity, the optimal number of PEs can be greatly reduced and thus improving throughput and latency.

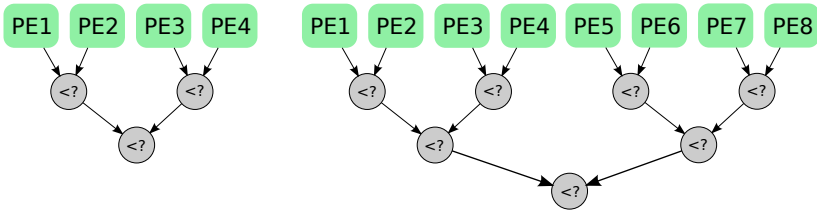


Figure F.4: SearchTask depth for 4 and 8 PEs

Representing Sparsity

There are multiple ways of representing sparse matrices. A simple tuple representation [32] was chosen, where the element and the column index are stored as a tuple. The original reward matrix R resides in the off-chip DRAM and each element of the reward matrix is represented by a 64-bit data word. The transformed vector C is the sparse representation of R and resides in the on-chip BRAM. For architecture with n PEs, each BRAM data word in C contains n tuples of object reward and column index as given:

$$c_i = (e_1, e_2, ..., e_n),$$
$$e_j = (\text{column}, \text{value}).$$

The number of bits used to represent a single object reward is configurable. The words of the transformed BRAM vector C are padded such that the first non-zero element of each row of the original DRAM matrix R always occupies the first element e_1 of a BRAM word c. This means that each BRAM word only contains rewards from the same row of the original matrix. A row of the original matrix may span several words of the transformed vector C. A separate array is used to map the row indices of the original matrix to indices of the transformed matrix.

In the following, an example of a transformation between a problem matrix R and its sparse representation C is given as follows:

$$R = \begin{bmatrix} 0 & 0 & 1 & 2 & 0 & 0 & 0 & 7 \\ 0 & 0 & 0 & 0 & 0 & 4 & 0 & 8 \\ 6 & 0 & 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 & 2 & 9 \\ 0 & 8 & 4 & 0 & 2 & 6 & 5 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 7 & 0 \end{bmatrix}$$

(a) Original reward matrix

$$C^T = \begin{bmatrix} (0,0),(5,1),(4,2),(0,7) \\ (0,0),(0,0),(2,4),(0,8) \\ (0,0),(0,0),(7,6),(1,2) \\ (0,0),(3,4),(1,2),(0,9) \\ (0,0),(0,0),(0,0),(6,8) \\ (5,4),(3,2),(2,6),(1,5) \\ (0,0),(0,0),(0,0),(1,7) \end{bmatrix}$$

(b) Transformed reward vector

Figure F.5

An architecture with 4 PEs is assumed for simplicity. This means that each BRAM word stores up to four valid, non-zero object rewards and their column indices. The reward matrix R is sparse with the majority of the rewards being zero. The bottom right corner has indices (0,0). Each element c_i in the transformed vector C consists of four tuples $e_1 \dots e_4$. The individual tuples consist of the element value and its column index in the original matrix R. The transformed vector C is not fully dense because the first element of each *row* in R is aligned to the beginning of a data *word* in C. This is done to efficiently read out an entire row of the original input matrix by reading a single BRAM data word. In the example, elements 1 and 2 of C (zero-indexed starting from the bottom) both belong to row 1 of R. When the number of valid elements in a row in R exceeds the number of PEs in our architecture the elements are spread out over multiple words. The row index of each element in C is stored in an array that maps each row of R to an element in C.

$$RowIndex = \begin{bmatrix} 0 & 1 & 3 & 4 & 5 & 6 \end{bmatrix}.$$

Cost of sparsity-aware processing

There are three major costs associated with using a sparse representation. First of all, the translation from dense to sparse representation must be performed. In the SaS Auction architecture, the translation is done in hardware and this incurs an area cost as opposed to the time cost associated with a software implementation. Secondly, a sparse representation removes the static relationship between PE and object price. In SoA Auction, PE0 always calculates the benefit of object 0 for the various agents. This means that each PE can store the prices locally and thus can all the prices be accessed in parallel at low cost. To achieve parallel lookup of all PEs, SaS Auction duplicates the BRAM with object prices for each PE. Lastly, sparse representation adds overhead because two values are needed for each element, the column index and the element value. The memory needed to store a sparse matrix is given by:

$$Mem_{sparse} = w_{reward} \cdot log_2(n_{max}) \cdot r_{sparse}, \tag{F.6}$$

where w_{reward} is the bits needed to represent a reward, n_{max} is the maximum problem size and r_{sparse} is the rate of non-zero elements. The memory requirements for a dense representation is as follows:

$$Mem_{dense} = w_{reward} \cdot n_{max}^2. \tag{F.7}$$

For an FPGA implementation, both Mem_{sparse}, n_{max} and w_{reward} are fixed at compile-time. For a given combination of these parameters, there is an upper bound on the size and density of a problem matrix that can be stored.

F.4.2 Dependency speculation

Each iteration of the Auction algorithm, consists of reading the object rewards for a specific agent and comparing them to the current object prices. An iteration

may result in a bid and thus change the state of the object prices. This means that unless the processing pipeline is stalled until the current agent is done, the next agent might use stale prices when calculating its bid. We define an **unresolved agent** as an agent that is currently computing a bid but has yet to commit it by updating the prices. There is a data dependency between agents a_1 and a_2 if the output of processing a_1 is dependent on whether it executes before or after a_2. Dependencies are dynamic, i.e. two agents might be dependent at some point during the processing and independent at another. Dependencies can be divided into two categories. A **strong dependency** occurs if both a_1 and a_2 has the same most-beneficial object. In this case, a_1 calculates an illegal bid if it does not wait for a_2 to commit its bid first. A **weak dependency** occurs when a_2 bids for the second most beneficial object to a_1. In this case, the order does not change which object a_1 bids for, but it affects the size of the bid. It makes a_1 bid more conservative which is equivalent to making a smaller step toward the global optimum. In [17], the dependencies are handled by stalling all agents in the PE stage until one agent computes its bid introducing expensive pipeline bubbles.

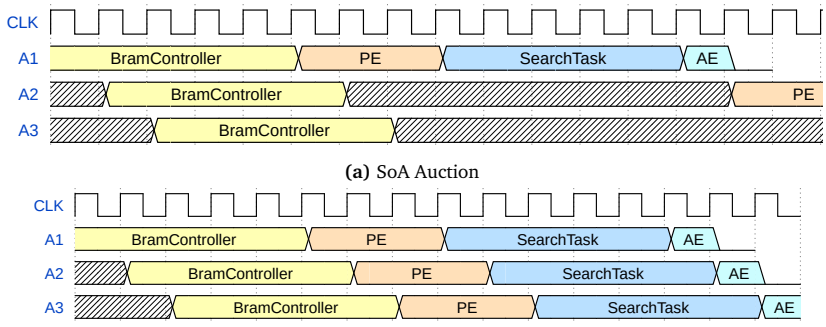
The processing stages for agent a_i through the pipeline of SoA Auction with 8 PEs are showed in Fig. F.6a. The components will be described in more detail in Section 4.3. The pipeline is stalled such that only one agent traverses the loop consisting of PEs, SearchTask and AssignmentEngine stages at the time. In the proposed SaS Auction, a basic static dependency speculation scheme is employed. It is always speculated that there are no strong dependencies between the agent currently at the PE stage and the unresolved agents which have not committed their bids yet. In the cases where there actually is a strong dependency, the agents bid is disregarded and it is added back in the unassigned queue. This is detected by comparing the bid to the current price. If it is lower then a misspeculation has occurred.

Fig. F.6b shows a timing diagram for SaS Auction, an implementation supporting dependency speculation, characterized by no pipeline bubbles which provides 9× increase in throughput compared to SoA auction implementation in [17]. The actual speedup is limited by the rate of misspeculations and pipeline bubbles naturally occurring because there are few unassigned agents left.

F.4.3 Hardware Architecture

The proposed SaS Auction is an optimized accelerator generator for efficiently solving the assignment problem. It is specifically optimized for the sparse problems found in e.g. MOT.

Fig. F.7 shows an overview over the SaS Auction hardware architecture and Fig. F.8 shows the speculative Auction algorithm represented as a flow-chart. A streaming dataflow architecture optimized for throughput is proposed. It is inspired by SoA Auction by Zhu et. al. The accelerator is characterized by n_{PE}, its number of PEs. This decides the width of the pipeline and BRAM interfaces. The accelerator reads the problem matrix from the off-chip DRAM into the on-chip



(a) SoA Auction

(b) SaS Auction

Figure F.6: Pipeline timing diagrams. AE is short for AssignmentEngine

BRAM. It also initializes the queue of unassigned agents. Then, it proceeds to perform the steps in the algorithm in Figure F.1. The accelerator reads one row of the problem matrix, finds the most beneficial object and calculates the bid. Then, it updates the price and assignment vector. This continues until there are no unassigned agents left. In the following each component is described briefly.

1. Register File provides the user interface of the accelerator. It is conventional AXI4-Lite based memory mapped control and status register file. The user software writes the memory address of the input problem matrix along with its dimensions into registers residing at predefined addresses. Starting and stopping the accelerator is also achieved through the register file.

2. Dram2Bram functions as a DMA that reads problem data from the off-chip DRAM, compresses it to the proposed sparse representation and writes it to the on-chip BRAM. The parsing and compressing is pipelined in two stages.

1. Non-valid, i.e. rewards that are zero, are filtered out. The valid rewards are entered into an register-based array, $ValidR$, corresponding to a single BRAM word.
2. When either the array is full, or a complete row of the input problem has been read, the array is written to the on-chip BRAM and the address at which it is stored is written to the AgentRowStore

The length of $ValidR$, and consequently the width of the on-chip BRAM interface, is equal to n_{PE}. This allows the rewards for all the PEs to be read out in a single cycle and.

3. qUnassignedAgents and **qRequestedAgents** are first-in-first-out (FIFO) queues between the AssignmentEngine and the BramController. They hold the unassigned agents and the currently executing agents respectively. BramContoller dequeues agent-indices from the qUnassignedAgents and enqueues them back on the qRequestedAgents after their rewards have been requested from the BRAM. When a bid arrives from the SearchTask, the AssignmentEngine dequeues from the

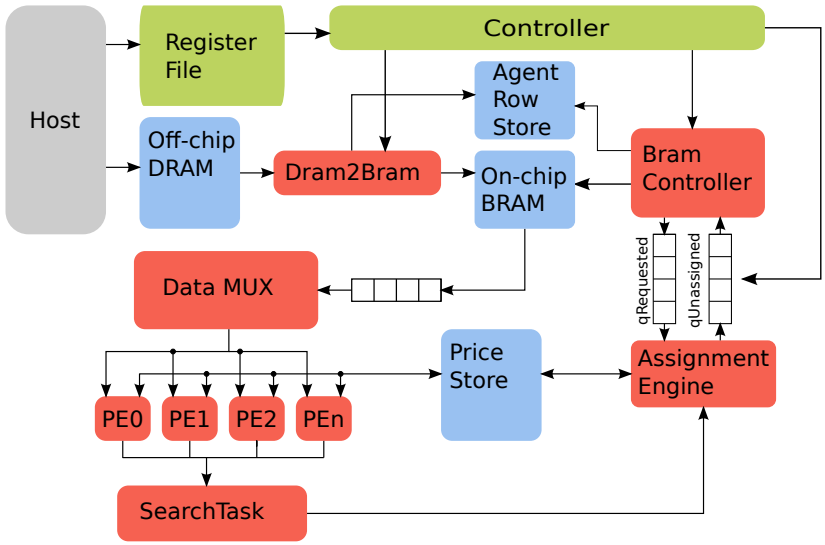


Figure F.7: High-level architecture of SaS Auction

qRequestedAgents and will potentially enqueue a new agent-index on the qUnassignedQueue.

4. AgentRowStore is a BRAM storing the mapping of the row index of the original input matrix to index of the on-chip BRAM data word containing the transformed version of that row. The Dram2Bram module writes to the AgentRowStore as it is parsing and compressing the input matrix. The AgentRowStore is read by the BramController before fetching rewards in the on-chip BRAM.

5. BramController controls the requesting and the parsing of the rewards from the BRAM. It is pipelined over multiple stages.

(1) Dequeue index of unassigned agent from qUnassigned.
(2-3) Read the BRAM address of the agent from AgentRowStore.
(4-5) Read rewards from BRAM and forward to DataMux.

6. PEs performs the actual computation as expressed in Eq. F.4. The PE stage is pipelined with the following stages.

1. Read current price of object from PriceStore.
2. Calculate benefit of object.
3. Forward benefit to SearchTask.

The PE performs static dependency speculation and always assumes that there is no dependencies between current agent and any unresolved agent. Any violation, which is caused by a strong dependency, will be caught in the AssignmentEngine.

7. PriceStore is the BRAM-based centralized storage for the object prices. Each

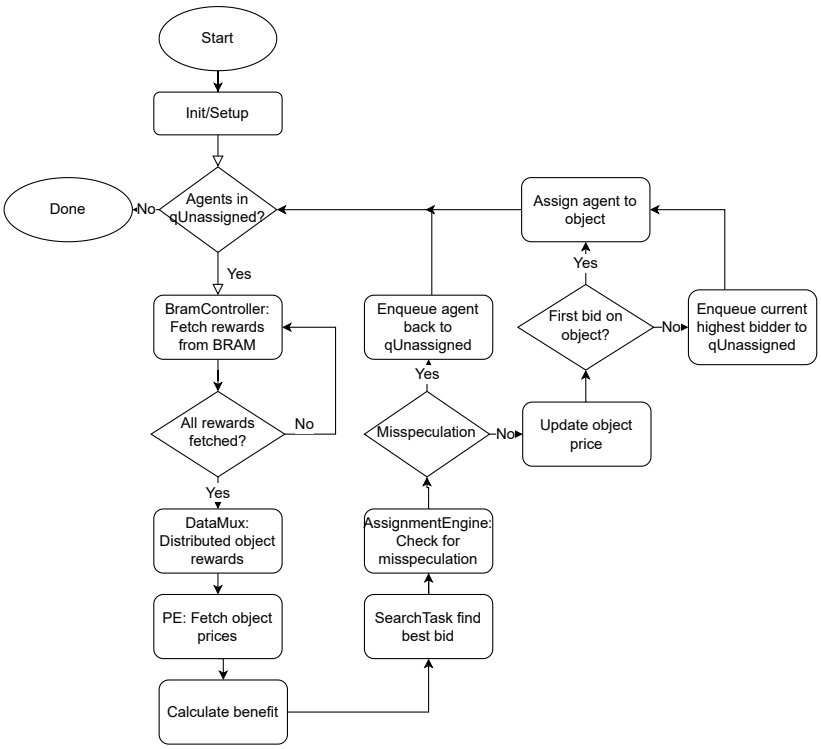


Figure F.8: Flowchart of SaS Auction

PE has a dedicated BRAM where all the prices are stored, i.e. the prices are duplicated for each PE. The write ports of all the duplicate BRAMs are connected to the AssignmentEngine which updates all the price copies after a bid. Each duplicate BRAM has a single read port which is connected to its PE. The AssignmentEngine also has a duplicate BRAM which is used resolve the dependency speculation.

8. SearchTask is illustrated in Fig. F.4 and is a tree of comparators that finds the highest and second-highest benefits, among the benefits outputted by the PE stage. Each level of the tree is implemented as a pipeline stage. The SearchTask can perform multiple iterations to support solving problems where the agents span multiple pipeline stages. When the highest and second-highest benefits are found, the SearchTask computes the bid as expressed in Eq. F.5.

9. AssignmentEngine is in charge of maintaining the prices, assignments and detecting and handling misspeculations. The AssignmentEngine is pipelined with the following stages.

1. Receive bid b for object o_j from the SearchTask and the corresponding agent-idx a_i from qRequestedAgents. Also read current price of the object $P(o_j)$ from the PriceStore. And the currently assigned agent $objectAssignments(o_j)$.
2. Resolve speculation by comparing $P(o_j)$ with b

(3a) If $b <= P(o_j)$, there has been a misspeculation caused by a strong dependency between a_j and a previous agent. In that case, a_i is enqueued back to qUnassignedAgents.

(3b) If $b > P(o_j)$, both the PriceStore and ObjectAssignments are updated. The previously assigned agent, which was requested in the first stage, is enqueued to qUnassignedAgents.

Weak dependencies are not detected or handled by the AssignmentEngine.

10. Controller performs control tasks. It fills up the qUnassignedAgents FIFO when the accelerator is started, detects the end of accelerator's operation and initiates writing the result back to DRAM.

Fig. F.8 shows a flowchart model of the pipeline. It is focused on the core processing pipeline which starts and ends with the qUnassigned. The interaction with off-chip DRAM, host software and transformation of input problems to sparse representations are abstracted away in the Init/Setup process.

F.4.4 Comparison to SoA Auction

The key differences between the SaS and the SoA Auction are as follows.

Sparsity

SoA Auction contains a simple memory interface which either can read the rewards from an off-chip main memory or an on chip BRAM. The latter necessitates that the SW has transferred the data to BRAM first. SaS Auction implements a

DMA in Dram2Bram which moves the data from off-chip DRAM to on-chip BRAM and compresses the data on the fly.

Price storage

With sparsity-aware processing the static mapping of object price to PE no longer exist. A object price might be needed by any PE. To enable single-cycle price-lookup for all PEs, SaS Auction stores a copy of the prices for each PE.

Speculation

SaS Auction implements speculative processing. While SoA Auction stalls execution in the PE stage, SaS Auction continues with possibly stale prices. The AssignmentEngine will detect a strong data dependency and reschedule the agent that used stale prices.

F.4.5 Performance model for speculation

In the following, a performance model for speculative execution in SaS Auction is derived.

Stalling cost

First, a cost model for the state-of-the-art implementation is derived. The cost of stalling execution at the PE stage depends on the number of clock cycles needed by the preceding agent to calculate and commit its bid. It can be expressed as:

$$\begin{aligned} C_{\text{stall}} &= d_{\text{PE}} + d_{\text{SearchTask}} + d_{\text{AssignmentEngine}} \\ C_{\text{stall}} &= 4 + log_2(n_{\text{PE}}), \end{aligned} \tag{F.8}$$

where d_{PE}, $d_{\text{SearchTask}}$ and $d_{\text{AssignmentEngine}}$ refer to the latency (or depth) of the respective modules, given in Table F.2. Note that these values are based on our implementation inspired by Zhu et. al. [17], where no data regarding the latencies of the original implementaion is available.

Clearly, the stalling cost is independent of the input problem dimension and only depends on the number of PEs. However, the total processing latency of one agent is also depends on the size of input problem dimension relative the number of PEs. Thus, the *relative* cost of stalling decreases the more underdimensioned the accelerator is.

Speculation cost

The speculation cost is defined as the additional number of clock cycles needed to process an agent if it is not stalled at the PE stage, but rather being speculatively processed. This cost is a stochastic variable as it depends on whether there is a data dependency between the agent and any unresolved agents. The goal is thus

Table F.2: Pipeline latencies

	Module		
PE	Search Task	Assignment Engine	Bram Controller
3	$2+\log_2(n_{\text{PE}})$	1	5

to find the *expected* value of the speculative cost. The misspeculation cost is first derived. It is the penalty of dispatching an agent from the PE stage if there is an unresolved strong dependency and is defined as:

$$C_{\text{miss}} = 1 + n_{\text{bubbles}} + C_{\text{stall}}. \tag{F.9}$$

The constant cost of C_{stall} is the worst-case expected cost of redoing the loop consisting of PE, SearchTask and AssignmentEngine stages once the agent is ready in the PE-stage again. The variable n_{bubbles} is the number of pipeline bubbles occurring between the misspeculation and the next instant the agent is ready in the PE-stage. It is estimated as:

$$n_{\text{bubbles}} = \max(0, d_{\text{PE}} - n_{\text{unassigned}}). \tag{F.10}$$

If $n_{\text{unassigned}}$ is larger than the number of clock cycles from the AssignmentEngine to the PE stage, there are no bubbles. The expected costs of stalling and speculating are expressed as:

$$\begin{aligned} E(\text{stall}) &= C_{\text{stall}} \\ E(\text{speculate}) &= p_{\text{miss}} \cdot C_{\text{miss}}, \end{aligned} \tag{F.11}$$

where p_{miss} is the probability of a misspeculation, i.e. the probability that the agent at the PE stage has a strong dependency on an unresolved agent.

Probability of misspeculation

To estimate the probability of a strong data dependencies, p_{miss}, some simplifications must be made. It is assumed that the most beneficial object o_{pick} for an agent a_k is a independent random variable drawn from a uniform distribution. This can be expressed as follows:

$$p(o_{pick} = o_i) = \frac{1}{m} \tag{F.12}$$

where m is the number of objects.

The probability of a strong dependency can thus be expressed as the probability of an agent picking the same object as any of the unresolved agents.

$$p_{\text{miss}}(n) = 1 - \left(\frac{m-1}{m}\right)^n, \tag{F.13}$$

where n is the number of unresolved agents and m is the number of objects. The actual number of unresolved agents n depends several factors.

First of all, it has an upper bound defined by the number of PEs in the accelerator. The number of PEs decides the number of pipeline stages between reading the prices in the PE stage and committing a bid in the AssignmentEngine. In an architecture with 8 PEs, there are 7 pipeline stages between PE and the commit operation.

Secondly, it also depends on how many pipeline stages each agent spans. In a scenario with 8 PEs and agents with 64 valid object rewards, the agent will span 8 pipeline stages. This means that when a new agent arrives at the PE stage, there will at most be one unresolved agents which would span all the 7 pipeline stages between the PE stage and the commit stage. Consider another scenario, also with 8 PEs, but this time with agents with at most 8 valid object rewards. This implies that each agent only spans a single pipeline stage. In this scenario, when a new agent arrives at the PE stage, there might be as many as 7 unresolved agents.

Lastly, it also depends on how many unassigned agents are left. When the agent of the previous scenario arrives at the PE stage. The number of unresolved agents could be anywhere from 0-7. The actual number depends on how full the pipeline is, which depends on how many unassigned agents are left.

Finally, using the derived probability of misspeculation p_{miss}, (F.11) is plotted in Fig. F.9 for an accelerator with 8 PEs with different input problem sizes. The grey horizontal line represents the cost of stalling which is constant and independent of the number of unresolved agents and the number of potential pipeline bubbles. The expected cost of speculating depends on both $n_{\text{unresolved}}$ and the potential number of bubbles n_{bubbles}. Thus, Fig. F.9 shows the *range* of expected cost of speculating. The maximum expected cost occurs in the case of an empty pipeline *behind* the agent in the PE stage. The minimum cost arises when the pipeline is full and there are enough unassigned agents such that $n_{\text{bubbles}} = 0$.

Clearly, for bigger problems the probability of misspeculation is very small and thus also the expected cost of speculation. Intuitively, increasing number of unresolved agents leads to increased chance of misspeculation and thus increased expected cost. Given the assumption of uniform probability distribution, a static speculation scheme is sufficient. In static speculation, as opposed to dynamic speculation, the pipeline always proceeds in the PE stage, regardless of the number of unresolved agents or the number of expected pipeline bubbles. Such a static speculation scheme implemented by SaS Auction.

F.5 Evaluation

The accelerator is implemented as a parameterizable hardware generator in the hardware description language Chisel [33]. The Zedboard [34] with a Xilinx Zynq XC7020 and off-chip DRAM is used as a target platform for evaluation. The Zynq is a SoC FPGA, which comprises a dual core ARM Cortex A9 and a Xilinx Virtex FPGA. The open source FPGA-tidbits project [35] is used to generate host and

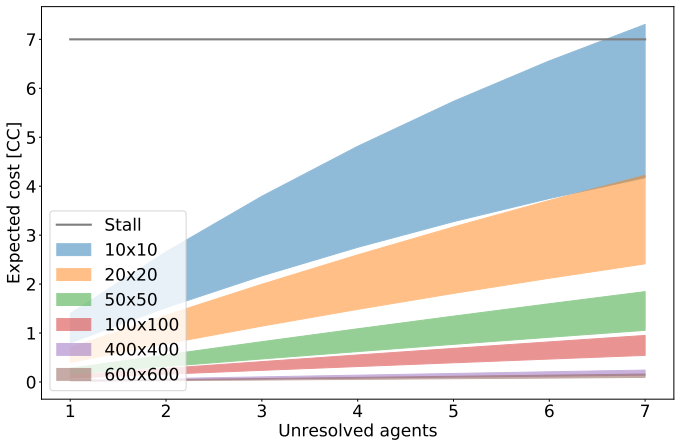


Figure F.9: Expected cost of speculating and stalling for different problem sizes

DRAM interfaces for the accelerator. The SW baselines execute on a dedicated i9-10900X CPU running at 1.2GHz. To filter out the non-determinism introduced by the operating system, each problem is run 5 times and the median execution time is used.

We evaluate three different SaS Auction designs and one SoA Auction against a C++ implementation and a version using the popular Gurobi framework.

We use two data sets for the evaluation. The first data set consists of randomly generated sparse assignment problems. The practice of using random number generators to create assignment problems is discussed in [36]. This data set consists of assignment problems ranging from small 10x10 problems up to 600x600 problems. For each size the sparsity is varied from only a few valid object rewards per agent up to fully dense.

The second data set is from a simulation of the state-of-the-art MOT algorithm introduced by Garcia et. al. in [29]. We have used different levels of validation gating to create problems with a greater range of sparsity. The algorithm maintains 100 tracks over 50 iterations, generating over 100 000 assignment problems.

The following implementations are evaluated.

- **C++ (SW)** is a single threaded C++ implementation of the Auction algorithm available at [27]. It was allocated a dedicated i9-10900X core.
- **Gurobi (SW)** is using the industrial Gurobi Optimizer [21]. It was also allocated a single dedicated i9-10900X core. Gurobi solves the assignment problem as an integer linear programming problem using the simplex method.
- **SoA xPE** is our implementation of the state-of-the-art where x is the number of PEs. It is based on the work of Zhu. et al. [17].

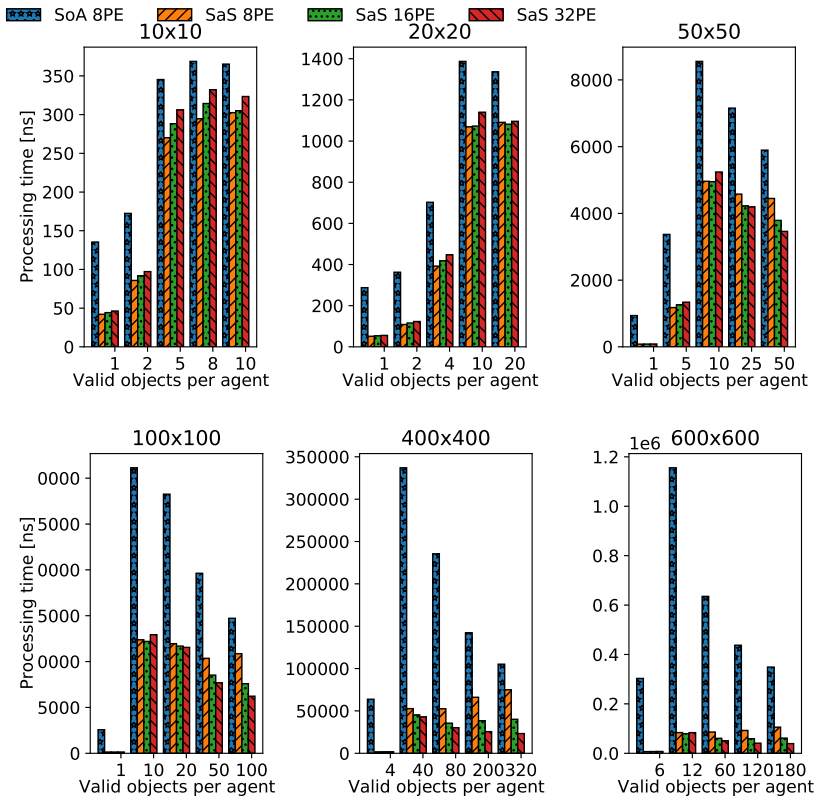


Figure F.10: *Core* processing time of SaS Auction and SoA Auction on a random spares data set

- **SaS** x**PE** is the Sparse and Speculative Auction implementation where x is the number of PEs.

F.5.1 Performance

Random sparse data set

Fig. F.10-F.12 shows the results of evaluating SaS and SoA Auction on the random sparse data set. Each evaluation is divided into six subplots. Each subplot shows problems with the same input dimensions, which are given in the subplot title. The y-axes denote the processing time in nanoseconds while the x-axes show the number of valid object-rewards per agent, i.e., the sparsity-level. There are five sparsity-levels in each subplot. E.g. in the 600x600 subplot there are results for problems with 6,12,60,120 and 180 valid object rewards per agent. For each

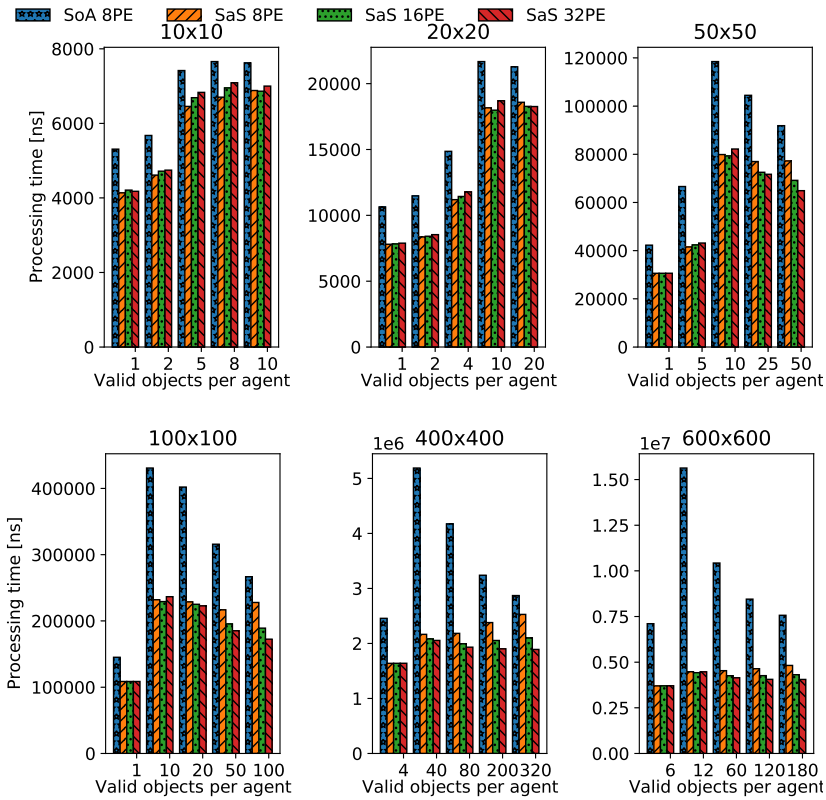


Figure F.11: *Total* processing time of SaS Auction and SoA Auction on a random spares data set

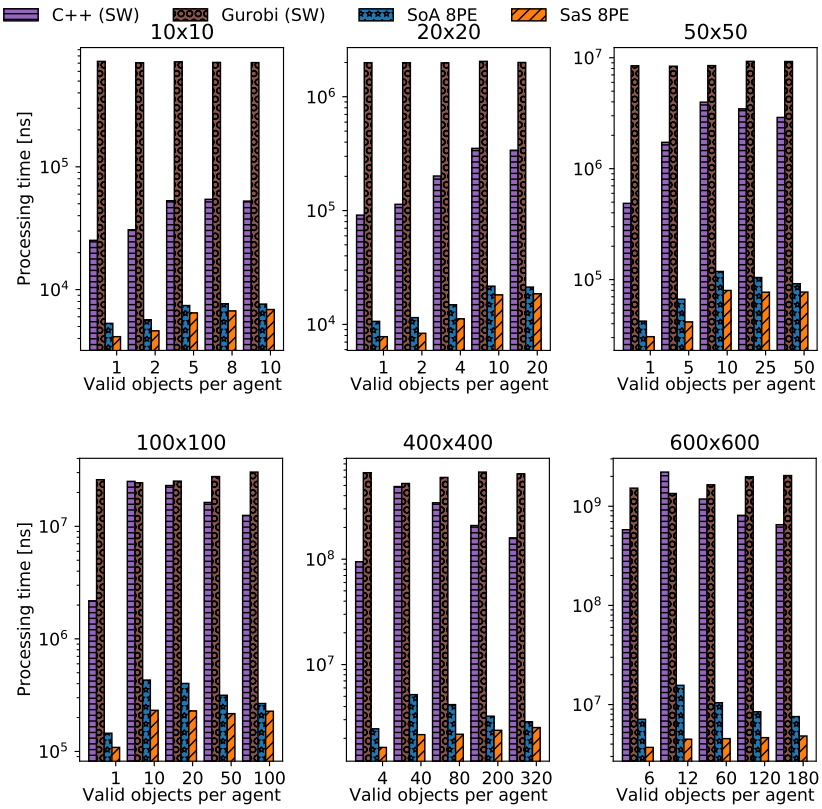


Figure F.12: *Total* processing time of Sas Auction and SoA Auction compared to a SW implementation

sparsity-level, the reported value is the average processing time for that particular sparsity-level.

In Fig. F.10 the *core* processing times of SaS and SoA Auction with different number of PEs is shown. The core processing time is the time spent doing actual computation, neglecting the time spent transferring the problem matrices from DRAM to the on-chip memory. The core processing time is measured by the accelerator itself which counts the cycles spent computing the solution. There are several things to note.

As expected, the potential speedup when taking advantage of speculation and sparsity-aware processing is big. As high as 50× for big sparse problems. It is also clear that the optimal number of PEs is related to the sparsity of the problem. The 50x50 subplot shows that SaS 8PE performs best while there are less than 8 valid objects per agent. Then SaS 16PE performs better until we have more than 16 valid objects per agent when SaS 32PE is best.

It is also clear that the speedup of SaS relative SoA is generally higher for sparse than for dense problems. For large and sparse input problems the potentially speedup is enormous. This is mostly due to the sparsity-aware processing. E.g. for a 600x600 problem each agent in SoA 8PE will span 75 pipeline stages, regardless of the sparsity. When the problem is fully dense, the only remaining speedup is due to the dependency speculation.

Fig. F.11 depicts the *total* processing times of SaS and SoA Auction. The total processing time includes the time spent transferring the rewards from DRAM to the on-chip-memory and the time spent writing the final results back. Clearly, the relative speedup of the SaS accelerators are limited by the fact that a major portion of the run-time is spent moving data from the DRAM to the on-chip BRAM. This effect is most visible for highly sparse problems. Consider the 600x600 problems with only 6 valid object rewards per agent. The *core* processing time of SoA 8PE is 50x higher than that of SaS 8PE. However, the *total* processing time is only 1.8x higher. The reason is that these highly sparse problems requires fewer iterations to solve than dense problems. This results in a situation where the core processing time only accounts for 2% of the total processing time.

Another interesting observation is that neither the core, nor the total, processing time is monotonically increasing as the problems gets denser. Rather, the processing time peaks at a relatively low density. Fig. F.13 shows the number of iterations needed by the C++ (SW) implementation to solve a data set of random sparse 50x50 problems. The x-axis shows the level of density and the y-axis shwos the number of iterations. Surprisingly, the processing time peaks at 22% density with an average core processing time of 7173 ns before monotonically decreasing as density increases. At 100% density the core processing time has decreased to an average of 4964 ns. The mechanism behind this is not clear to the authors.

Fig. F.12 compares the *total* processing time of SaS and SoA Auction with SW implementations. The y-axis now uses a logarithmic scale. As described in prior work, there are enormous potential for speedup. From 100x speedup for the small 10x10 problems to 1000x speedup for the sparse 600x600 problems. It is also clear

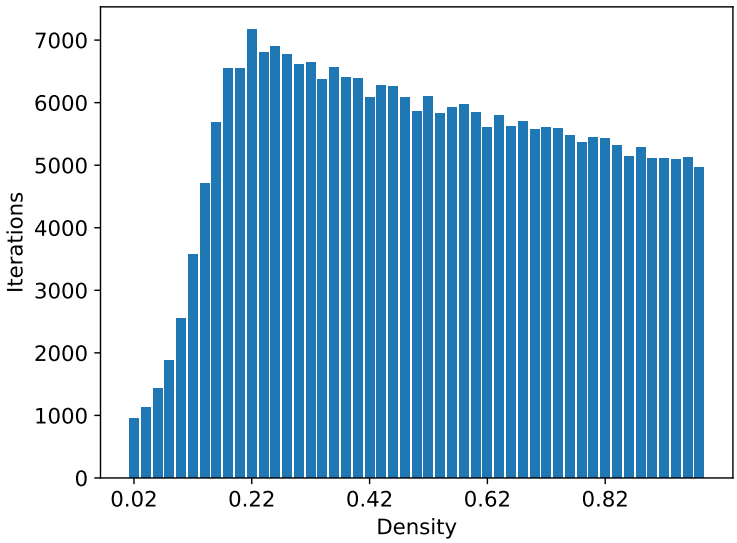


Figure F.13: Auction algorithm iterations for random sparse 50x50 problems

that a vanilla C++ auction algorithm implementation outperforms industrial optimization software based on linear programming. However, for larger problems the speedup is less significant.

MOT data set

In Fig. F.14 shows the results of evaluating SaS and SoA Auction on the MOT data set. There are a few subtleties which makes this plot different than the previous ones. Each subplot now shows problems with the same number of rows (i.e. agents). The subplots have titles like 16xY which means that the problems have 16 agents, however the number of objects vary from 1 to over 300. The x-axes shows the number of valid object-rewards per agent, this is no longer directly tied to the sparsity level since the number of non-valid object-rewards varies . The values on the x-axes now indicate limits. E.g. for the 16xY subplot, the first bar-group are the average processing time for problems with 1-8 valid object-rewards, the second is for 8-20 valid object-rewards and so on.

As expected, the processing time of SaS Auction increases as the number of valid object-rewards increases. This is in line with the results from the randomly sparse data set. Interestingly, this is not the case for SoA Auction, where the processing time does not seem to strongly correlate with the number of valid object-rewards. The reason is that the processing time of SoA Auction mostly only depend

on the input dimensions of the problem. The input distribution of the MOT problems are such that the agents seldom bid for the same objects. Thus, the sparsity-level is not an indicator of how many iterations are needed and thus to overall processing time. The SaS Auction, on the other hand, can take advantage of the sparsity and drastically reduce the core processing time.

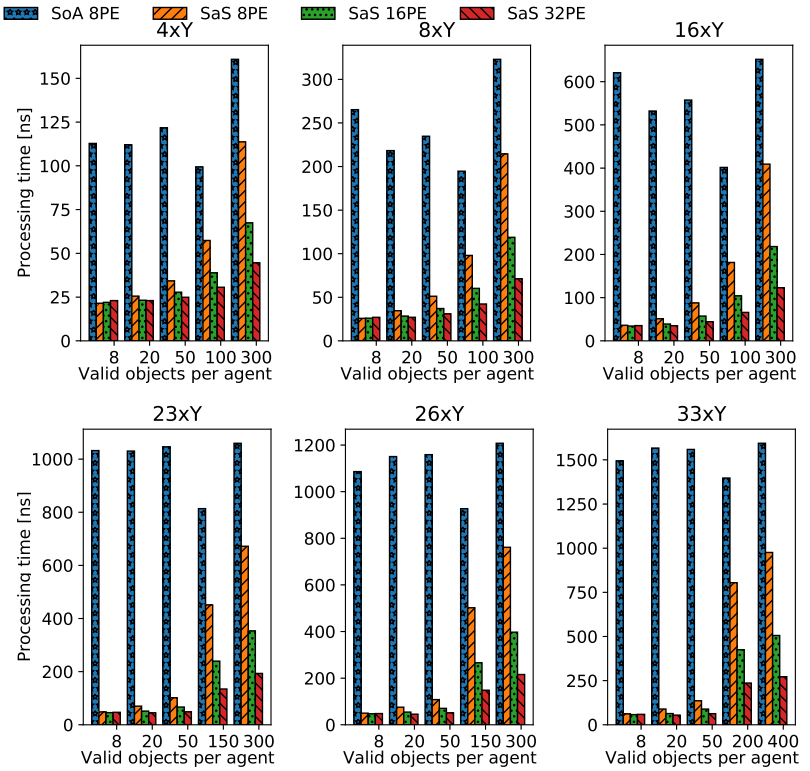


Figure F.14: *Core* processing time for SaS Auction and SoA Auctiob on the MOT data set

F.5.2 Performance model evaluation

Fig. F.15 shows the number of misspeculations for the SaS Auction accelerator when evaluating the random sparse data set. As expected, SaS 8PE consistently has fewer misspeculations due the fact that it has a lower bound on the maximum number of unresolved agents. In general the number of misspeculations increase with the number of valid object-rewards until the number exceeds the number of PEs.

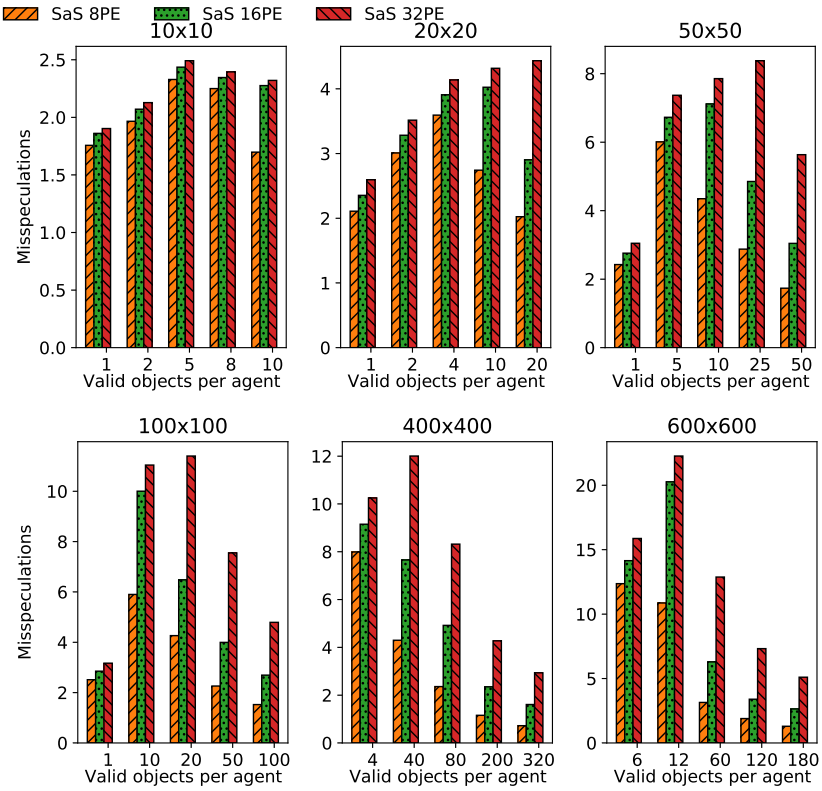


Figure F.15: Misspeculations observed during evaluation on the randomly generated sparse data set

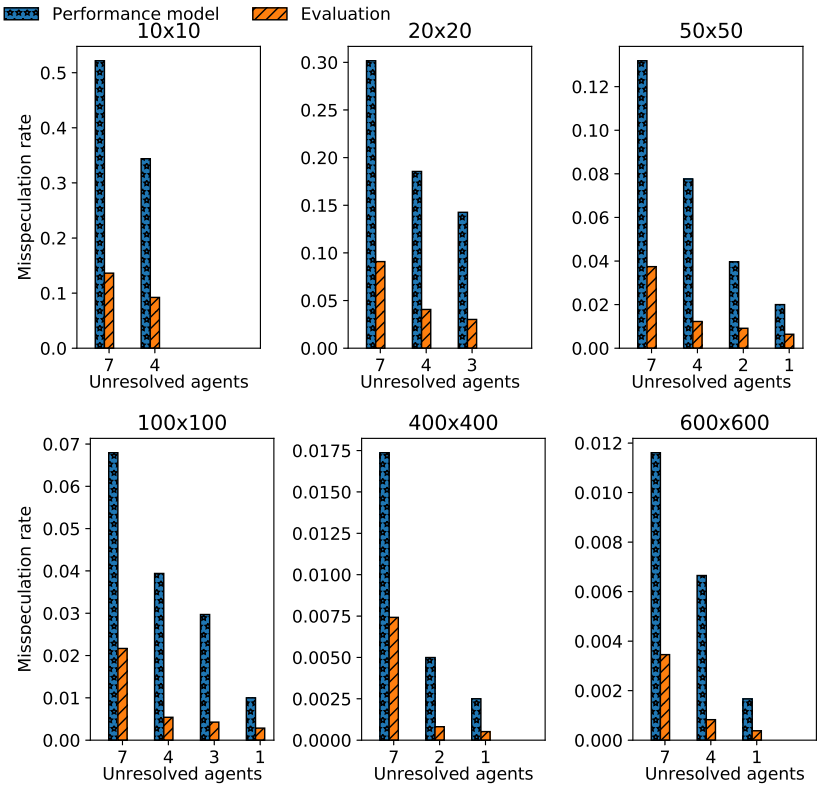


Figure F.16: Misspeculation rate predicted by the performance model versus observed during evaluation. Given as a function of the remaining unresolved agents.

Fig. F.16 compares the expected rate of misspeculations, derived in the performance model section, with the measured rate of misspeculation rate for the SaS 8PE accelerator. The y-axis shows the misspeculation rate and the x-axis show the number of unresolved agents. The number of unresolved agents are based on the sparsity-level of the input problem according to

$$\mathrm{a}_{\mathrm{span}} = \frac{n_{nonzero}}{n_{PE}} \tag{F.14}$$

$$\mathrm{unresolved} = \frac{\mathrm{max}_{\mathrm{unresolved}}}{\mathrm{a}_{\mathrm{span}}}. \tag{F.15}$$

Where $\mathrm{a}_{\mathrm{span}}$ is the agent span, i.e. how many pipeline stages each agent spans. The n_{nonzero} is the number of non-zero object rewards per agent, i.e. the sparsity level. Finally, $\mathrm{max}_{\mathrm{unresolved}}$ is the maximum number of unresolved agents possible. I.e. the latency from PE stage to commit. For SaS 8PE, $\mathrm{max}_{\mathrm{unresolved}} = 7$.

Clearly, the performance model is inaccurate and overestimates the misspeculation rate. The major reason for this is that the predicted misspeculation rate is based on the assumption that the pipeline is always full. I.e. for a 20x20 problem with 8 non-zero rewards per agent we assume that there always will be 7 unresolved agents in the pipeline. In reality, the pipeline is only full in the first part of the execution. As there are fewer and fewer unassigned agents left the pipeline will be less and less full until there is only one unassigned agent left. The execution can proceed for many iterations with only a single unassigned agents. During those last iterations a misspeculation is impossible. We thus conclude that the performance model gives overly conservative upper bound of the misspeculation rate.

F.5.3 Area

Table F.3 shows the resource utilization for the different implementations. SoA 8PE is our implementation of the accelerator based on the work of Zhu et. al, while Zhu 16PE are the numbers reported in [17]. The difference in BRAM usage stems from the fact that SoA 8PE stores the rewards on-chip while Zhu 16PE accesses the rewards from an off-chip memory. SoA 8PE needs 130 BRAMs to be able to solve 600x600 problems efficiently. A solution reading rewards from off-chip DRAM would be alot slower and not be fair comparison to SaS which stores the rewards, with sparse representation, on-chip. The difference in LUT and flip-flop usage likely stems from the fact that the SoA 8PE is based on SaS 8PE. The sparsity-aware and speculative optimizations were removed, but some overhead still remains.

We note that SaS 16PE uses 2.8x more flip-flops and 1.3x more LUTs than Zhu 16PE. This overhead is high, but expected. The Dram2Bram, PriceStore and on-chip storage of the rewards make up over a third of the flip-flop overhead. Moreover, the pipeline of SaS 16PE is considerably longer and more information

Table F.3: Resource utilization

	Accelerator				
	SoA 8PE	**Zhu 16PE**	**SaS 8PE**	**SaS 16PE**	**SaS 32PE**
Flip-flops	2966	1647	3177	4586	7414
LUT	2346	2265	2235	2927	4315
LUTRAM	57	540	121	229	432
BRAMs	134.5	6	79	83	91

must be passed through the pipeline to support sparsity-aware processing and speculation.

F.6 Conclusions

In this paper, Sparse and Speculative Auction, a novel implementation of the popular Auction algorithm for FPGAs, has been presented. It takes advantage of the high level of sparsity found in the assignment problems formulated by many algorithms. It employs both sparsity-aware processing and speculation and achieves up to 50x speed-up over the state-of-the-art for highly sparse problems. It is evaluated on randomly generated sparse problems and realistic problems from a MOT simulator.

Bibliography

[1] A. Ehsanfar and P. Grogan, 'Auction-based algorithms for routing and task scheduling in federated networks,' *Journal of Network and Systems Management*, vol. 28, pp. 1–27, Apr. 2020. DOI: 10.1007/s10922-019-09506-y.

[2] A. Elmsallati, C. Clark and J. Kalita, 'Global alignment of protein-protein interaction networks: A survey,' *IEEE/ACM Trans. Comput. Biol. Bioinformatics*, vol. 13, no. 4, pp. 689–705, Jul. 2016, ISSN: 1545-5963. DOI: 10.1109/TCBB.2015.2474391. [Online]. Available: https://doi.org/10.1109/TCBB.2015.2474391.

[3] Z. Zhang, J. Wang, D. Xu and Y. Meng, 'Task allocation of multi-auvs based on innovative auction algorithm,' in *2017 10th International Symposium on Computational Intelligence and Design (ISCID)*, vol. 2, 2017, pp. 83–88. DOI: 10.1109/ISCID.2017.231.

[4] X. Duan, H. Liu, H. Tang, Q. Cai, F. Zhang and X. Han, 'A novel hybrid auction algorithm for multi-uavs dynamic task assignment,' *IEEE Access*, vol. 8, pp. 86 207–86 222, 2020. DOI: 10.1109/ACCESS.2019.2959327.

[5] K. Xue, Z. Huang, P. Wang and Z. Xu, 'An exact algorithm for task allocation of multiple unmanned surface vehicles with minimum task time,' *Journal of Marine Science and Engineering*, vol. 9, no. 8, 2021, ISSN: 2077-1312. DOI: 10.3390/jmse9080907. [Online]. Available: https://www.mdpi.com/2077-1312/9/8/907.

[6] N. S. Hirata and F. D. Julca-Aguilar, 'Matching based ground-truth annotation for online handwritten mathematical expressions,' *Pattern Recogn.*, vol. 48, no. 3, pp. 837–848, Mar. 2015, ISSN: 0031-3203. DOI: 10.1016/j.patcog.2014.09.015. [Online]. Available: https://doi.org/10.1016/j.patcog.2014.09.015.

[7] W. Lian and L. Zhang, 'A concave optimization algorithm for matching partially overlapping point sets,' *Pattern Recognit.*, vol. 103, p. 107 322, 2020.

[8] R. Danchick and G. E. Newnam, 'Reformulating Reid's MHT method with generalised Murty K-best ranked linear assignment algorithm,' *IEE Proceedings - Radar, Sonar and Navigation*, vol. 153, pp. 13–22, Feb. 2006, ISSN: 1350-2395. DOI: 10.1049/ip-rsn:20050041.

[9] P. Gu, Z. Jing and L. Wu, 'Robust adaptive multi-target tracking with unknown measurement and process noise covariance matrices,' *IET Radar, Sonar & Navigation*, vol. 16, no. 4, pp. 735–747, 2022. DOI: https://doi.org/10.1049/rsn2.12216. eprint: https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/rsn2.12216. [Online]. Available: https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/rsn2.12216.

[10] M. Beard, B. T. Vo and B.-N. Vo, 'A solution for large-scale multi-object tracking,' *IEEE Transactions on Signal Processing*, vol. 68, pp. 2754–2769, 2020. DOI: 10.1109/TSP.2020.2986136.

[11] K. Li, X. Zhao, Z. Sun and M. Tan, 'Robust target detection, tracking and following for an indoor mobile robot,' in *2017 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2017, pp. 593–598. DOI: 10.1109/ROBIO.2017.8324481.

[12] C. Robin and S. Lacroix, 'Multi-robot target detection and tracking: Taxonomy and survey,' *Autonomous Robots*, vol. 40, Aug. 2015. DOI: 10.1007/s10514-015-9491-7.

[13] D. P. Bertsekas, 'The auction algorithm for assignment and other network flow problems: A tutorial,' *INFORMS Journal on Applied Analytics*, vol. 20, no. 4, pp. 133–149, 1990. DOI: 10.1287/inte.20.4.133. eprint: https://doi.org/10.1287/inte.20.4.133. [Online]. Available: https://doi.org/10.1287/inte.20.4.133.

[14] C. Vasconcelos and B. Rosenhahn, 'Bipartite graph matching computation on gpu,' Aug. 2009, pp. 42–55, ISBN: 978-3-642-03640-8. DOI: 10.1007/978-3-642-03641-5_4.

[15] D. L. Hung and Jun Wang, 'A fpga-based custom computing system for solving the assignment problem,' in *Proceedings. IEEE Symposium on FPGAs for Custom Computing Machines (Cat. No.98TB100251)*, 1998, pp. 298–299. DOI: 10.1109/FPGA.1998.707924.

[16] D. Hung and J. Wang, 'Digital hardware realization of a recurrent neural network for solving the assignment problem,' *Neurocomputing*, vol. 51, pp. 447–461, Apr. 2003. DOI: 10.1016/S0925-2312(02)00627-6.

[17] P. Zhu, C. Zhang, H. Li, R. C. Cheung and B. Hu, 'An fpga-based acceleration platform for auction algorithm,' May 2012. DOI: 10.1109/ISCAS.2012.6271395.

[18] E. R. Jellum, *Auction accelerator*, https://github.com/erlingrj/auction-accelerator, 2021.

[19] P. Wolfe, 'The simplex method for quadratic programming,' *Econometrica*, vol. 27, p. 170, 1959.

[20] D. Klingman and R. Russell, 'A streamlined simplex approach to the singly constrained transportation problem,' *Naval Research Logistics Quarterly*, vol. 25, no. 4, pp. 681–695, 1978.

[21] Gurobi, 'Gurobi optimizer reference manual, 2022,' *URL http://www. gurobi. com*, vol. 9.5, 2022.

[22] C. Bliek1ú, P. Bonami and A. Lodi, 'Solving mixed-integer quadratic programming problems with ibm-cplex: A progress report,' in *Proceedings of the twenty-sixth RAMP symposium*, 2014, pp. 16–17.

[23] H. W. Kuhn, 'The hungarian method for the assignment problem,' *Naval research logistics quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[24] R. Jonker and A. Volgenant, 'A shortest augmenting path algorithm for dense and sparse linear assignment problems,' *Computing*, vol. 38, pp. 325–340, 2005.

[25] D. P. Bertsekas, 'The auction algorithm: A distributed relaxation method for the assignment problem,' *Ann. Oper. Res.*, vol. 14, no. 1–4, pp. 105–123, Jun. 1988, ISSN: 0254-5330. DOI: 10.1007/BF02186476. [Online]. Available: https://doi.org/10.1007/BF02186476.

[26] B. Schwartz, 'A computational analysis of the auction algorithm,' *European Journal of Operational Research*, vol. 74, no. 1, pp. 161–169, 1994, ISSN: 0377-2217. DOI: https://doi.org/10.1016/0377-2217(94)90214-3. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0377221794902143.

[27] E. R. Jellum, *Auction algorithm c++*, https://github.com/erlingrj/auction-cpp, 2012.

[28] K. Granstrom, M. Baum and S. Reuter, *Extended object tracking: Introduction, overview and applications*, 2017. arXiv: 1604.00970 [cs.CV].

[29] A. F. Garcia-Fernandez, J. L. Williams, K. Granström and L. Svensson, 'Poisson multi-Bernoulli mixture filter: Direct derivation and implementation,' *IEEE Transactions on Aerospace and Electronic Systems*, pp. 1–1, 2018, ISSN: 0018-9251. DOI: 10.1109/TAES.2018.2805153.

[30] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, J. Zambreno and P. H. Jones, 'Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels,' in *2019 IEEE International Conference on Embedded Software and Systems (ICESS)*, 2019, pp. 1–8. DOI: 10.1109/ICESS.2019.8782524.

[31] S. Krishnan, Z. Wan, K. Bhardwaj, P. Whatmough, A. Faust, G.-Y. Wei, D. Brooks and V. J. Reddi, 'The sky is not the limit: A visual performance model for cyber-physical co-design in autonomous machines,' *IEEE Computer Architecture Letters*, vol. 19, no. 1, pp. 38–42, 2020. DOI: 10.1109/LCA.2020.2981022.

[32] Y. Saad, *Iterative Methods for Sparse Linear Systems*, Second. Society for Industrial and Applied Mathematics, 2003. DOI: 10.1137/1.9780898718003. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9780898718003. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9780898718003.

[33] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek and K. Asanović, 'Chisel: Constructing hardware in a scala embedded language,' in *DAC Design Automation Conference 2012*, Jun. 2012, pp. 1212–1221. DOI: 10.1145/2228360.2228584.

[34] Avnet, *ZedBoard HW Users Guide*, https://digilent.com/reference/_media/reference/programmable-logic/zedboard/zedboard_ug.pdf, 2021.

[35] Y. Umuroglu, *Fpga tidbits*, https://github.com/maltanar/fpga-tidbits, 2020.

[36] D. F. Crouse, 'On implementing 2d rectangular assignment algorithms,' *IEEE Transactions on Aerospace and Electronic Systems*, vol. 52, no. 4, pp. 1679–1696, 2016. DOI: 10.1109/TAES.2016.140952.

Paper G

Codesign of Reactor-oriented Hardware and Software for Cyber-Physical Systems

Erling Rennemo Jellum
NTNU
Trondheim, Norway

Martin Schoeberl
DTU
Lyngby, Denmark

Edward A. Lee
UC Berkeley
Berkeley, USA

Milica Orlandić
NTNU
Trondheim, Norway

Submitted for review to
ACM Transactions on Reconfigurable Technology and Systems

Abstract

Modern cyber-physical systems often make use of heterogeneous systems-on-chip with reconfigurable logic to provide adequate computing power and flexible I/O. However, modeling, verifying, and implementing the computations spanning CPUs and reconfigurable logic is still challenging. The hardware and software components are often designed by different teams and at different levels of abstraction, making it hard to reason about the resulting computation. We propose to lift both hardware and software design to the same level of abstraction by using the Lingua Franca coordination language. Lingua Franca is based on a sparse synchronous model that allows modeling concurrency and timing while keeping a sequential model for the actual computation. We define hardware reactors as a subset of the reactor model of computation underlying Lingua Franca. We also present and evaluate reactor-chisel, a hardware runtime implementing the semantics of hardware reactors, and an extension to the Lingua Franca compiler enabling reactor-oriented hardware-software codesign.

G.1 Introduction

With the release of the Zynq-7000 in 2011 by Xilinx and Altera's Cyclone V in 2012 a new field of computing was established. These so-called System-on-Chip Field Programmable Gate Arrays (SoC FPGAs) integrate multiple general-purpose and real-time CPUs, various peripherals, and reconfigurable logic (i.e., an FPGA) on the same silicon die. Installing small CPU cores in the FPGA logic, so-called soft cores, was already common. However, with the SoC FPGAs, the developers can access full-featured application processors capable of running various operating systems. The flexibility and performance offered by these platforms make them suitable for being at the heart of sensor fusion systems found in embedded systems in IoT, medicine, robotics, and space. Xilinx and Altera went on to be acquired by AMD and Intel, respectively, continuing the legacy of the SoC FPGA.

However, these platforms pose serious challenges concerning how the computations spanning CPUs and FPGA should be modeled, verified, and implemented. At the core of this challenge are the fundamental differences in computational substrate offered by the CPUs and the FPGA. The CPUs offer an interface to the programmer (or compilers) called the instruction set architecture (ISA), which consists of a set of registers and instructions and a deterministic model describing how the state of the machine evolves when executing any sequence of instructions. The most common programming languages used to specify the behavior of these CPUs, like C, C++, and Python, are built on a sequential and imperative model [1]. This is convenient because the ISA is a sequential model, making the compilation process more straightforward.

The FPGA, on the other hand, offers an interface in which each configurable logic block, memory resource, and switch box are configured by a so-called bitstream. The resulting network of logic blocks and memories will then represent a computation like the program of the CPU. The format of the bitstream is typically proprietary and closed. For all intents and purposes, we can say that the interface

offered by the FPGA is that of synchronous digital logic. In synchronous digital logic, instead of a sequence of instructions, we have a set of memory elements whose state changes are synchronized by a clock signal. Although the most common languages for modeling synchronous logic, like Verilog and VHDL, are based on discrete-event models, the usage of the languages to synthesize hardware is synchronous [2]. This synchronous usage is also reflected by a newer hardware language, Chisel [3], which even hides the clock from the developer.

There are three core differences between the algorithmic approach offered by the CPU and the synchronous approach offered by the FPGA:

(1) In the sequential model, there is a notion of order but not of time [4]. There is no portable way of specifying, e.g., in C or Python, that a certain instruction should be executed at an exact time in the future. Time is merely a side effect of a particular implementation. This is also matched by fundamental nondeterminism in the timing of the CPUs themselves [5]. This makes ordinary CPUs and the sequential model a poor fit for solving timing critical problems like sensor synchronization [6]. In the synchronous model, on the other hand, all computation occurs (conceptually) at an exact timestamp drawn from an external clock. If this external clock is periodic and the synchronous hypothesis[7] is valid, this timestamp can be given a physical interpretation.

(2) The sequential model also lacks a notion of concurrency. In C, there is no way to specify that two different functions should be executed concurrently. One has to execute before the other. The standard way of introducing concurrency is through threads; however, these have been shown to be an unsatisfactory abstraction for concurrency [8]. In the synchronous model, all computations triggered by the same global clock tick are concurrent, except when they have explicit dependencies.

(3) Finally, there is a fundamental difference in how the physical computing device is modeled. Jantsch makes the distinction between *hierarchy* and *abstraction* in models of computation [9]. Hierarchy is defined as the process of hiding information by partitioning a system into smaller parts. Conversely, abstraction is defined as representing a system in a new way, using new primitives. Sequential imperative programming languages can thus be seen as hierarchical models over the ISA since they do not fundamentally change the way computation is represented. However, the gap from a bitstream to a synchronous program is an abstraction. In this paper, we will introduce a model of computation for SoC FPGAs based on abstraction.

A heterogeneous system composed of sequential algorithms on the CPU and an accelerator in the form of synchronous digital logic on the FPGA is hard to model and thus to reason about and verify. In practice, most designers make informal use of models of computation to simplify this. They might, for instance, implement a sequential interface to the FPGA, where the software signals the accelerator to start by writing to a “start” register and polls a “finished” register until the accelerator is done. In this case, we can reason about the accelerator as if it were a C function. If concurrency between software and hardware is desired the

accelerator interface could draw inspiration from the process networks. Either using asynchronous inputs and outputs, similar to a Kahn Process Network [10], or rendezvous-based inputs and outputs, consistent with Communicating Sequential Processes [11].

We propose, instead, a formal approach where there is a well-defined model of computation underlying the language used to specify the computation spanning hardware and software. This merging of model and implementation leads to systems that are "correct-by-construction." We believe that such a model of computation should have the following properties:

A. Deterministic semantics.
B. Ability to model concurrency at arbitrary granularity.
C. Ability to model time.
D. Event-driven with the ability to handle asynchronous inputs.

This paper introduces hardware reactors, a subset of the reactor model of computation [12]. Reactor semantics is defined as the sequence of time-tagged events flowing between concurrent components called reactors. For a well-formed network of reactors, there is a unique legal sequence of such events (A). Reactors are concurrent objects, and the event-triggered reactions, encapsulating computation, have arbitrary granularity (B). The reactor model introduces time as a first-class citizen (C) and allows for limited non-determinism by incorporating asynchronous events (D).

We present `reactor-chisel`, a hardware runtime for implementing reactor networks spanning both the CPUs and the reconfigurable logic of an SoC FPGA. We extend the open-source Lingua Franca language [13] with both a Chisel and a Codesign target. This enables reactor-oriented codesign of hardware and software. We evaluate the implementation in terms of resource utilization, maximum clock frequency, latency and throughput on a set of simple test programs as well as a more realistic image processing example. The results are promising and higrevight areas that need improvements and future research.

G.2 Background

G.2.1 Models of computation

A model of computation (MoC) is a mathematical *abstraction* of a computing device [1]. The purpose of an abstraction is to hide information and provide a more convenient or economical way of specifying the intended behavior. There are multiple meta-models of MoCs, i.e., different ways of explaining what they are, i.e., their semantics, and how different MoCs can be compared. In the tagged signal [14] and Rugby [9] meta-models, MoCs are described in terms of how they can model concurrency and time. MoCs can also be explained in terms of data flow and control flow [15]. The data flow specifies how data is produced, communicated, and consumed, while the control flow deals with when, if, and how

different elements process data.

MoCs can prove very useful in designing complex systems as they can allow for mechanized reasoning about the possible behaviors of the resulting system without analyzing the implementation details. MoCs are essential for verification, where typically, a system specification is written in some specification language with a well-defined underlying MoC. This kind of methodology suffers from what is known as "round-trip engineering" [16]. There are now at least two artifacts representing the system, the specification, and the implementation, and keeping these artifacts synchronized becomes a challenge. Another approach followed in this work is to create design languages based on well-defined MoCs. This elegantly avoids the round-trip problem because the specification and the implementation are the same program.

G.2.2 Synchronous data flow

Synchronous data flow (SDF) belongs to the family of dataflow models of computation [17, 18]. Concurrent functional units called *actors* are connected by first-in-first-out channels. The unit of communication in dataflow models is the *token* which is read from and written to the channels by the actors. Actors perform computation in atomic quanta called firings where a number of tokens are *consumed* from the input channels and *produced* to the output channels. Since the firing is atomic, it is not enabled until all the tokens to be consumed are present at the input channels. SDF is a static dataflow model where the consumption and production rates are fixed and cannot change at runtime, for instance, based on the data carried by the tokens. An SDF graph can be represented as a set of balance equations, one for each channel. A non-zero solution to the balance equations is called a repetition vector and denotes a set of actor firings such that all actors fire and the SDF graph returns to its initial state. An SDF graph is *consistent* [19] if such a repetition vector exists. A consistent SDF graph can be executed indefinitely with bounded memory. The simplest form of SDF is the homogeneous SDF where each actor consumes a single token from each input channel and produces a single token to each output channel. SDF graphs are *deterministic* in the sense that the order in which a set of actor firings is performed does not affect the final state of the graph.

SDF models are suitable for hardware-software codesign as they map easily both to software and hardware. SDF models are often used to model and implement signal-processing applications where the data rates are static and predictable. However, modern cyber-physical systems are often reactive, meaning they include dynamic, unpredictable, and asynchronous inputs. This is hard to model using SDF.

G.2.3 Lingua Franca and the reactor model of computation

For such reactive systems, discrete-event MoCs like the reactor model [20] might be more appropriate. Reactors were formalized as part of Marten Lohstroh's Ph.D.

thesis [21]. Lingua Franca (LF) [13] is a polyglot coordination language based on reactor semantics [22]. To informally introduce the reactor model we will use the simple example Lingua Franca program and its auto-synthesized diagram in Fig G.1.

```
1  target C;
2  reactor Source {
3      timer t(0, 100 msec)
4      logical action a
5      physical action p
6      output out: int
7
8      reaction(t) -> out {=
9          // User C code here
10     =}
11     reaction(a) -> out {= =}
12     reaction(p) -> a, out {= =}
13 }
14 reactor Sink {
15     timer t(0, 50 msec)
16     input in: int
17     state s: int
18     reaction(t) {= =}
19     reaction(in) {= =}
20 }
21 main reactor {
22     source = new Source()
23     sink = new Sink()
24     source.out -> sink.in
25 }
```

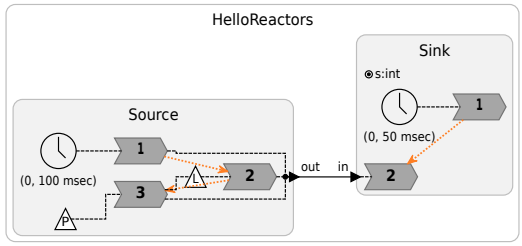


Figure G.1: An example Lingua Franca program using the C target

The program in Fig. G.1 consists of three reactors related by connections and containment. At the top of the containment hierarchy is the main reactor, which in this example contains two reactor instances, one `Source` reactor and one `Sink` reactor. Such *contained* reactors are referred to as child reactors, and the *containing* reactor as the parent reactor. A reactor is a stateful concurrent component represented as a rectangle with rounded corners in the diagrams. A reactor communicates with other reactors through typed *ports*; in our example, the `Source` reactor has an output port `out`, which is connected to an input port `in` at `Sink`. The fundamental unit of computation is the *reaction*. These are represented as dark grey chevrons and can be thought of as *event* handlers. An event is the unit of communication in the reactor model. It consists of a value and a timestamp, referred to as tags in the reactor model. Like in the synchronous model, a reaction invocation is logically instantaneous. Events originate from triggers, which include fixed period *timers*, represented in the diagrams as clocks, input ports, represented as small triangles on the borders of the reactor, and *actions*, represented as white triangles. Logical actions are denoted with an *L* and allow reactions to schedule future events at well-defined timestamps in the future. Physical actions, denoted with a *P*, allow asynchronous contexts, like an interrupt service routine, to schedule events into

the system. Reactions may also produce events to output ports or schedule future events via logical actions. The actual computation taking place in a reaction is specified in the reaction body, which is expressed in a target language between two brackets `{= ... =}`.

In SDF, actors are only fired when sufficient tokens are on the input channels. In the reactor model, reactions are only triggered when all observable events with the same tag have been produced. Reactions inside the same reactor can share state variables and are executed in a predefined order should they be triggered at the same tag. The reactor model is deterministic, assuming that the reaction bodies are deterministic.

The reactor model uses a superdense model of time and includes two timelines. The *logical* timeline can be compared to the simulated timeline of a VHDL circuit. The reactor model delivers deterministic computations relative to this logical timeline. However, the programs also exist on a real, *physical* timeline. There are four points of interaction between the physical and the logical timeline. (1) At startup, physical and logical time is synchronized. (2) Logical time can never exceed physical time. (3) Physical actions are scheduled relative to the current physical time, unlike logical actions, which are scheduled relative to the current logical time. (4) A *deadline* can be associated with a reaction. A deadline specifies the maximum difference between logical and physical time, i.e., the *lag*, allowed at the invocation of the reaction. The user must also specify a deadline-miss handler. The ability to model the composition of static and dynamic dataflow (through timers and logical actions) with asynchronous events (physical actions) and real-time constraints (deadlines) makes reactors an ideal model for cyber-physical systems. Bringing the key aspects of timing and concurrency into the semantics of the program also enables reusability and modularity of designs.

The reactor model also lends itself well to distributed execution. Distributed programs can coordinated centrally, with a scheme based on high-Level architecture or decentralized with a scheme based on Ptides [23]. In recent work, LF has been extended with support for modal behaviours [24]. It also has support for running on bare-metal microcontrollers and can function as an RTOS [25]. LF has been shown to provide high performance, comparable to other actor-oriented frameworks while introducing determinism [26].

G.3 Related Work

There have been multiple attempts at lifting hardware and software design to the same level of abstraction. In this section, we review a selection of notable related works.

G.3.1 OpenCL

Open Computing Language (OpenCL) [27] by the Khronos Group is a vendor-neutral framework for writing programs that execute on heterogeneous platforms

such as SoC FPGAs. It provides a uniform programming model for CPUs, GPUs, and FPGAs based on the C programming language. In the context of SoC FPGAs, OpenCL provides a convenient abstraction for hardware-software codesign because both hardware and software are expressed in the same programming model; this makes it easier to experiment with the partitioning between hardware and software. The OpenCL compiler generates the boilerplate hardware and software facilitating the communication. However, the programming model is very general and requires explicit synchronization of data across the various memories used by different OpenCL kernels. This allows for non-deterministic programs. OpenCL is a high-level synthesis (HLS) framework. HLS automatically generates hardware descriptions from high-level programming languages such as C, OpenCL, or MATLAB. While HLS can reduce design time and increase productivity, design at the register-transfer level (RTL) offers better fine-grained control and more optimization opportunities for the generated hardware design. Unlike reactor-chisel, HLS is based on sequential languages and cannot specify timing and concurrency [28].

G.3.2 SYCL

SYCL [29] is a more recent, higher-level programming model built on top of OpenCL. While OpenCL is based on C and requires that each kernel be written in a separate file, SYCL uses C++ and allows a single-source model. SYCL also abstracts away much of the low-level device management and memory handling required in OpenCL. However, it inherits OpenCL's limitations with respect to determinism, timing and concurrency.

G.3.3 Compaan/Laura

In Compaan/Laura [30] a computation is modeled in a MATLAB framework built on the Kahn Process Networks (KPN) model of computation. A KPN consists of a network of processes connected by first-in-first-out channels. The Compaan compiler will partition the KPN into a software part and a hardware part and the Laura compiler generates a VHDL description of the hardware processes and a runtime implementing the control flow. The software processes are implemented in C++. KPNs are closely related to the dataflow models that the hardware reactor model is based on. However, reactors are more expressive than KPNs. A reactor can have encapsulated data-dependent production of output events. Through the runtime infrastructure, downstream reactors will be stalled until output events are produced, or guaranteed to not occur. Using KPNs the data-dependent production logic must also be replicated in the downstream reactors so they can know whether the upstream will produce outputs or not.

G.3.4 ReconOS

ReconOS [31] extends the POSIX programming model into the FPGA domain. It makes it possible to design hardware threads executing in the FPGA logic com-

municating with software threads or other hardware threads using inter-process communication primitives like FIFOs and pipes. This significantly lowers the barrier of entry for programmers wanting to develop applications on SoC FPGAs. The key difference between reactor-chisel and ReconOS is the underlying model of computation. ReconOS is based on shared-memory concurrency which is a fundamentally non-deterministic model of computation. `Reactor-chisel` is based on timed and deterministic message-passing instead.

G.3.5 ReconROS

ReconROS [32] is another framework addressing the desire for a unified way of designing hardware and software for robotics applications. This is achieved by integrating Robot Operating System 2 (ROS2) into their existing ReconOS project. ROS2 is a middleware for robotics applications centered around a publish-subscribe protocol. While an improvement over shared memory concurrency, publish-subscribe is also fundamentally non-deterministic [33].

G.3.6 ForSyDe

ForSyDe was proposed in the Ph.D. thesis of Ingo Sander in 2003 [34]. Its core idea is to raise the abstraction level of software design based on models of computations. The designer specifies the intended behavior of their system as a hierarchical network of processes communication with tagged signals. Each process is a function mapping process inputs to process outputs. A core tenant in ForSyDe is the co-called separation of concerns, where the concerns are computation and communication. The computation is expressed as functions and the communication is defined by a model of computation (MoC). The ForSyDe library supports multiple MoCs like the synchronous MoC, KPN, and SDF. Our approach focuses on a single MoC and takes a different approach to the idea of a "separation of concerns". With Lingua Franca we use different language for expressing the concerns of communication and coordination, from the language used for expressing the computation.

G.3.7 Vitis Unified Software Platform

Vitis is a unified software platform for hardware-software codesign on AMD/Xilinx devices [35]. Vitis bundles a series of tools that previously were distributed individually like Vivado, Vivado HLS, SDSoC, and Petalinux tools. Vitis provides a software-centric workflow where hardware designs can be compiled and linked with the Vitis v++ compiler. Vitis supports both HDL and HLS for describing the FPGA designs and C, and C++ for the software components. Vitis also supports OpenCL. The communication and synchronization between hardware and software is abstracted by the Xilinx Runtime library (XRT) which provides a hardware-independent API to the accelerators. Compared to Lingua Franca and hardware reactors, Vitis can be viewed as an ad-hoc approach. It offers multiple paradigms

for hardware-software interaction but leaves it up to the developer to combine them in a predictable way.

G.3.8 AADL

Architecture Analysis and Design Language (AADL) is a modeling language for safety-critical embedded systems. AADL differs from the other frameworks discussed because it models both software components and the execution platform. AADL also lets you specify nonfunctional requirements like execution time or memory footprint. An AADL model is a hierarchy of components that has both a textual and graphical representation. A component models either a software or an execution platform entity. The software components are data, threads, thread groups, subprograms, and processes. The execution platform components are memory, busses, processors, devices, virtual processors, and virtual buses. Inheritance enables multiple implementations of the same component. Components may be organized into packages, which enables reusability. AADL is not built on a single concurrent MoC but allows the designer to implement a mixture of a shared variable MoC and a dataflow process network MoC. There are multiple tools, such as OSATE and RAMSES [36], which provide refinements and software synthesis from AADL models. Kaolin is a development process and tool for mapping computation expressed in AADL onto FPGAs [37]. As with ForSyDe, AADL offers a more heterogeneous approach, while we focus on delivering the semantics of a single deterministic model of computation.

G.3.9 SystemC

SystemC [38] is a system modeling language based on C++. It supports modeling both hardware and software and their interaction at multiple levels of abstraction. Like Lingua Franca and the reactor model, SystemC is also based on a discrete-event model. However, SystemC is mainly targeted at hardware-software co-simulation, not actual implementation.

G.3.10 Bluespec Codesign Language

The Bluespec Codesign Langauge (BCL) was developed as part of Myron King's Ph.D. thesis [39]. It is a language based on the guarded atomic actions model, which can be compiled into both hardware and software. A BCL program consists of explicitly declared state variables and a set of guarded atomic actions on these state variables. Compilation of such programs into an efficient finite state machine is performed using the Bluespec compiler [40]. BCL introduces the ability to generate efficient C++ modules implementing the computation specified with guarded atomic actions. Moreover, a program can be arbitrarily partitioned between hardware and software, and the compiler will generate logic for communication and synchronization. A key difference between BCL and the `reactor-chisel` is that

```
1  target Chisel
2  reactor Blinky {
3    @external
4    output led: {=Bool()=}
5    state count: {=UInt(32.W)=}
6    timer t(0, 500 msec)
7    reaction(t) -> led {=
8      lf_set(led, ~lf_get(out))
9      lf_write(count, lf_read(count)+1.U
           )
10   =}
11 }
12
13 main reactor {
14     blinky = new Blinky()
15 }
```

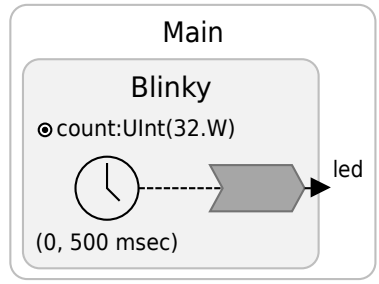


Figure G.2: A simple blinky example using the Chisel-target.

guarded atomic actions are a non-deterministic model of computation. Atomic actions enabled simultaneously can be applied in any order. This is argued to be an advantage because it gives the compiler more freedom. Additionally, while BCL introduces a new language for expressing computation, Lingua Franca only introduces a new syntax for coordination. The actual computation is expressed in a language already familiar to the user.

G.3.11 LEAP

LEAP is a FPGA runtime, aimed at making FPGA development easier and more familiar for software developers [41]. Much like an operating system, LEAP provides basic device abstractions, I/O and memory management services, hiding the complex details of the underlying FPGA hardware from the designer. LEAP consists of a set of Bluespec System Verilog libraries and a compilation toolchain. Our work differs from LEAP in two regards. First, LEAP is targeted at standalone FPGA designs and not HW/SW-codesign. Second, LEAP is based on the latency-insensitive paradigm which admits nondeterminism.

G.4 Reactor-oriented hardware-software codesign

In this section, we dive right into a set of programs exemplifying the capabilities of the proposed Chisel-target of Lingua Franca. The underlying model of computation is introduced later, in Section G.5 and the details of the implementation of `reactor-chisel` in Section G.6.

G.4.1 Blinky

The Hello World of embedded systems is the Blinky program. Fig. G.2 shows a Blinky program, written in the Chisel-target of Lingua Franca. The Blinky reactor

has an output port called `led` with datatype `Bool()`. It is marked "external," meaning that it is just routed directly to the top level of the design. A timer named `t` with a 500 millisecond period is defined as well as a state variable `count`. The reactor contains a single reaction triggered by `t`. The reaction functionality is defined by a reaction body between the curly brackets `{= ... =}`. The reaction body is a sequential circuit written in Chisel, which by default is enabled for a single clock cycle but might execute for an arbitrary number of clock cycles. A simple API is provided for reading values from the input ports, `lf_get(inputPort)`, reading from state variables, `lf_read(stateVar)`, writing to state variables, `lf_write(stateVar, value)`, and setting output port values, `lf_set(outputPort, value)`. There are also API functions for reading the current logical time, `lf_time_logical()`, and the current physical time, `lf_time_physical()`.

This program will be compiled into a Chisel design where a top-level pin is toggled every 500 logical milliseconds. The timers in Lingua Franca are *logical*, which means that they specify the order and simultaneity of events, not a precise physical time at which the events should occur. Since reactions with data dependencies execute in a deterministic order, and reactions might execute for an arbitrary number of cycles, the exact physical time that a reaction will be enabled is not necessarily deterministic. This subtle distinction between the logical timeline, as specified in Lingua Franca syntax, and the physical timeline which is dependent on both the hardware platform and the sequential circuits embedded in the reactions have implications for the appropriate level for granularity computations should be modeled with reactors.

G.4.2 Vector addition

Due to hardware runtime overhead, we should not expect circuits modeled at clock-cycle level granularity, e.g. with reactions triggered each clock-cycle, to be realizable in physical time. `Reactor-chisel` is more suitable for coarse-grained computations, where, for instance, an entire accelerator is embedded in a single reactor. Such a reactor, executing on the FPGA, can also be composed with reactors written in other targets without changing the semantics. This makes it ideal for targeting SoC FGPAs. For this purpose, we have added a "Codesign-target" to Lingua Franca which allows mixing software reactors written in C++ with hardware reactors written in Chisel.

Listing G.1 shows such a program spanning both the CPUs and the FPGA of a SoC FPGA. It is the classical VectorAdd example where the vector addition of two long vectors is off-loaded to the FPGA. The reactor called `Software` is to be run on the CPUs and will, every 10 milliseconds, output two 2048 element arrays to the `Accel` reactor, which is to be run on the FPGA. The `Accel` reactor has a single reaction triggered when there are tokens on both inputs. It reads out all elements from each port, sums them, and writes them to the output port. By setting the `reactionDone` wire, the reaction body can execute for an arbitrary number of cycles.

This program shows how easy it is to offload computation from the CPU to

```
1  target Codesign
2  reactor Software {
3      output op1: uint32_t[2048]
4      output op2: uint32_t[2048]
5      input res: uint32_t[2048]
6      timer t(0, 10 msec)
7
8      reaction(t) -> op1, op2 {=
9        // Cpp program writing two 2048 word arrays to each output port
10     =}
11     reaction(res) {=
12       // Cpp program using the result
13     =}
14 }
15 reactor Accel {
16     @array(length=2048)
17     input op1:{=UInt(32.W)=}
18     @array(length=2048)
19     input op2:{=UInt(32.W)=}
20     @array(length=2048)
21     output res:{=UInt(32.W)=}
22     reaction(op1, op2) -> res {=
23       val readPort1 = lf_get_array(op1, 0.U, 2048.U)
24       val readPort2 = lf_get_array(op2, 0.U, 2048.U)
25       val writePort = lf_set_array(res, 0.U, 2048.U)
26       val cnt = RegInit(0.U(32.W))
27       val fire = writePort.ready && readPort1.valid && readPort2.valid
28       when(fire) {
29         writePort.valid := true.B
30         readPort1.ready := true.B
31         readPort2.ready := true.B
32         writePort.bits.data := readPort1.bits.data + readPort2.bits.data
33         cnt := cnt + 1.U
34       }
35       reactionDone := cnt === 2048.U // Builtin signal
36     =}
37 }
38 main reactor {
39     sw = new Software()
40     @fpga
41     accel = new Accel()
42     sw.op1, sw.op2, accel.res -> accel.op1, accel.op2, sw.res
43 }
```

Code listing G.1: A Lingua Franca programming using the Codesign target that allows mixing C++ and Chisel

the FPGA. The `@array` attribute on Lines 15, 17, and 18 informs the compiler that there is an *array* of tokens being communicated. Connections between such ports are buffered in BRAM if it is between hardware reactors and off-chip DRAM if it is between a software reactor and a hardware reactor. `Reactor-chisel` includes direct memory access (DMA) modules that efficiently handle the interleaving of memory requests and responses for multiple ports. The Codesign target aims to abstract away all the communication and synchronization details between the CPU and the FPGA and leave the developer only with the task of implementing the actual computation going on in the event-triggered reactions.

G.4.3 Multi-sensor fusion

The `Accel` hardware accelerator in the `VectorAdd` program was purely reactive and had no locally originating tokens. While the Codesign target allows efficient modeling and implementation of such simple codesigns, it is also much more expressive.

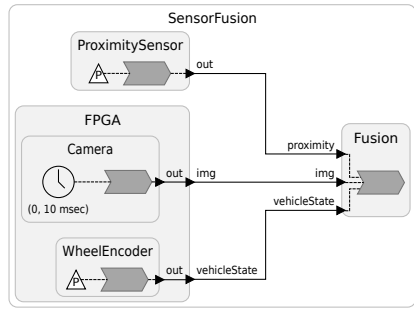


Figure G.3: A sensor fusion applications spanning FPGA and CPU

Imagine acquiring, preprocessing, and fusing messages from multiple sensors. Fig. G.3 shows a Lingua Franca diagram of a program fusing three sensors. A proximity sensor delivers sensor readings asynchronously through an interrupt service routine to the software, a camera is triggered and processed at 100Hz from the FPGA; as well as a wheel encoder sensor which also triggers asynchronously and is connected to the FPGA. Asynchronous events are modeled in Lingua Franca through the use of physical actions.

This Lingua Franca program has a semantic that is quite hard to achieve by using ad-hoc programming in C++ and VHDL. This program dictates that the `Fusion` reactor will always see sensor messages in their timestamp order. This is a crucial property for many sensor fusion systems. Out-of-order handling of sensor messages can, in the case of sensors measuring continuous quantities, e.g., an accelerometer or tracking of the position of features in an image, lead to serious estimation errors [42]. For sensors measuring discontinuous quantities, like a wheel encoder that indicates whether the vehicle is moving forward or backward and a proximity sensor telling whether an object is close to the vehicle or not, out-of-order handling of messages can lead to the system passing through inconsistent states [23].

Using `reactor-chisel`, control-logic is synthesized to the FPGA and the CPU such that they share a common notion of time through the exchange of timestamps. An event arising in the `WheelEncoder` will stall the software, preventing it from handling any other events with a later timestamp until the `WheelEncoder` event has propagated through the system and has been fully handled. This coordination is bidirectional, and the FPGA cannot handle events until dependent events in the software reactors are handled. This coordination mechanism will be further elaborated in

Section G.6.9

To the authors' knowledge, this kind of tight bi-directional coordination between computation in software and hardware is novel and unlocks a new way of doing hardware-software codesign.

G.5 Hardware reactors model of computation

In the same way that VHDL and Verilog define synthesizable subsets, we propose a synthesizable subset of Lohstroh's original reactor model [12]. This subset lends itself well to a dataflow-like implementation. In this section, we will give an informal description of this subset. When there might be ambiguity, we will refer to the components of the hardware reactor model as *hardware* reactors and *hardware* reactions; if there is no ambiguity, we will refer to them as reactors and reactions.

Hardware reactors are similar to ordinary reactors. They encapsulate timers, physical actions, state variables, ports, and event-triggered hardware reactions, as well as contained hardware reactors. The ports of hardware reactors can be connected through connections yielding a directed graph of hardware reactors which constitutes a program.

The fundamental unit of communication in the hardware reactor model is a timestamped *token*. Although similar to the events of the reactor model we refer to them as tokens since this is the terminology used in dataflow models and the hardware reactor subset is arguably closer to dataflow than discrete-event.

Hardware reactions are the fundamental components in our model, and they resemble actors in homogenous SDF. In the reactor model, reactions are triggered by events from *any* trigger; on the other hand, hardware reactions are triggered by the presence of a single token on *all* triggers and sources. In the reactor model, a reaction firing *may* produces an event to any output port marked as an effect. In the hardware reactor model, it produces a token to *all* output ports. This requires us to add the notion of an absent token. If a hardware reaction does not produce a token to a certain output port, then the runtime produces an absent token there.

Input ports "fork" tokens to all dependent hardware reactions. In the same way, output ports "join" tokens from all hardware reactions that have the port as an anti-dependency. Connections buffer tokens between ports.

There are four ultimate sources of tokens. Timers, built-in triggers like startup and shutdown, physical actions, and ports at the top-level hardware reactor, which is connected to external reactors, e.g., running in software. Every time one of the sources generates a token, all other sources must also generate a token, possibly an absent token. This means that each token source will output a sequence of tokens with identical tags. For timers, this sequence can be computed as a static schedule of a repeated hyper-period. For physical actions and inputs from external reactors, the absence or presence of tokens must be checked at each step in the hyperperiod. Tokens from physical actions and external reactors might also arrive at tags that are not present in the static schedule, in which case all the timers will fire with an absent token. Physical actions are restricted to only scheduling

tokens without any additional offset. Physical actions can thus be queued in a FIFO queue, which is cheaper in hardware than a sorted priority queue.

There is a subtle distinction between the logical timeline, defined by the tags of the tokens flowing between hardware reactors, and the physical timeline, defined by the clock signal driving the FPGA. The tags of the tokens can also be understood as *reaction indices*. They denote the order in which the reactions should be executed. A runtime should try to minimize the difference between the logical and the physical timeline, however, the semantics of the program are defined relative to the logical timeline, not the physical.

Lohstroh's original reactor model also includes logical actions, which are not included in the hardware reactor model. Logical actions are very expressive and hard to capture in static dataflow. Suppose a reaction declares a logical action as an effect. In that case, the reaction may schedule an arbitrary number of future events with timestamps only limited by a possible minimum spacing parameter. This means that any Lingua Franca program with a logical action might need an unbounded event queue. Logical actions can be thought of as equivalent to the `after` statement in VHDL which is not part of the synthesizable subset for the very same reason.

To summarize, our underlying model is close to homogenous SDF. A computation is thought of as a graph of reactions fired when there are tokens at all inputs. The computation is driven forward by the token sources generating sequences of tokens and absent tokens. At every tag where a token is present, all reactions in the program will be fired, possibly with absent tokens.

G.6 reactor-chisel

In the following section, we will explain how such a model can be implemented efficiently on a SoC FPGA. We introduce `reactor-chisel` which is a hardware runtime, meant for executing on the FPGA, implementing the semantics of the hardware reactors model and enabling the correct execution of the programs presented in Section G.4. We refer to `reactor-chisel` as a "runtime" because in function it resembles a runtime underlying languages such as Java, Python and Golang. However, due to the fundamental difference between FPGAs and CPUs, the implementation is fundamentally different. `Reactor-chisel` is organized as "glue logic" wrapping user-written modules. This glue logic is mainly concerned with abstracting away the communication between different modules. To ensure the deterministic composition of modules, a centralized scheduler module is also part of the runtime which distributes tokens to the user-written modules. The tokens act as a virtual clock signal, enabling them in a pre-defined order. Through control and status registers, the scheduler module also coordinates with a software runtime ensuring well-defined ordering of execution of software and hardware modules.

We will use the example program in Listing G.2 represented with a diagram in Fig. G.4 as a running example. This program contains most of the Lingua Franca

```
target Chisel

reactor Source {
  timer t(0, 1 usec)
  output outSingle: {=UInt(8.W)=}
  @array(length=8)
  output outArray: {=UInt(8.W)=}
  @external
  input inExt: {=UInt(1.W)=}
  state cnt: {=UInt(8.W)=}
  reaction(t) -> outSingle {=
    // User-written Chisel code here
  =}
  reaction(startup) inExt -> outArray {=
    // User-written Chisel code here
  =}
}
reactor Sink {
  @array(length=8)
  input inArray: {=UInt(8.W)=}
  input inSingle: {=UInt(8.W)=}
  @external
  output extOut: {=UInt(1.W)=}
  output out : {=UInt(32.W)=}
  physical action phy: {=UInt(8.W)=}
  reaction(inArray) {=
    // User-written Chisel code here
  =}
  reaction(phy, inArray) -> out {=
    // User-written Chisel code here
  =}
  reaction(shutdown) -> extOut {=
    // User-written Chisel code here
  =}
  contained = new Contained()
  inArray -> contained.inArray
}
reactor Contained {
  @array(length=8)
  input inArray: {=UInt(8.W)=}
  reaction(inArray) {=
    // User-written Chisel code here
  =}
}
main reactor {
  src = new Source()
  sink = new Sink()
  src.outSingle, src.outArray -> sink.inSingle, sink.inArray
}
```

Code listing G.2: An example program to illustrate the different Lingua Franca primitives supported by `reactor-chisel`

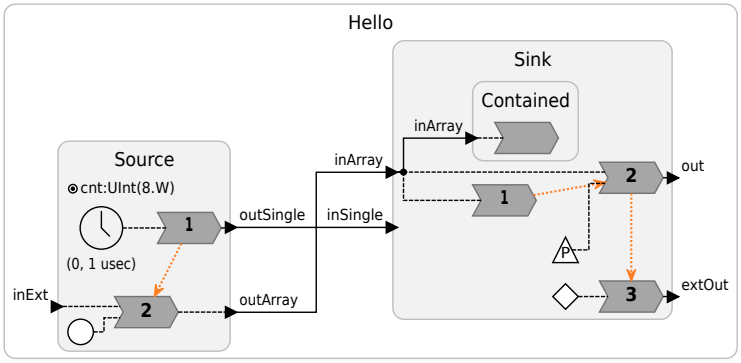


Figure G.4: The auto-generated diagram of the program in Listing G.2

primitives currently supported by `reactor-chisel`. When passing such a program to the Lingua Franca Compiler a hardware design using `reactor-chisel` as a library will be generated. In the following, we will introduce the different parts of this generated design.

G.6.1 Tokens

Tokens are the fundamental unit of communication in a hardware reactor program. All connections, dependencies, and anti-dependencies, indicated by solid and dashed lines in Fig G.4 will be compiled to interfaces that exchange tokens.

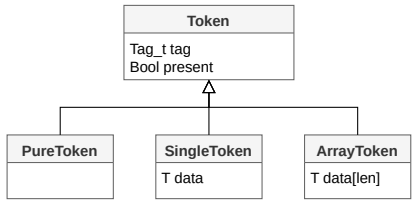


Figure G.5: The Chisel class hierarchy of Tokens

Fig. G.5 shows the class hierarchy of tokens in `reactor-chisel`. All tokens include a tag (i.e. a timestamp representing the logical time of the token) and a bit indicating whether this is an absent token or not. The default in `reactor-chisel` is to use a 32-bit signed integer in nanosecond resolution to represent time, this will not cause any issues unless the generated schedule has a hyperperiod that spans more than two seconds (more on hyperperiods later). The tag will be optimized away unless the user-written modules are explicitly reading the tag. In Fig. G.5, there are three types of tokens. Pure tokens carry no values and are used for triggers from timers, physical actions with no type and precedence ports between the reactions. Single tokens are used for tokens carrying a single value

of arbitrary Chisel type. Single tokens should only be used with narrow data types since they will be compiled into interfaces with bit widths sufficient to carry the entire data type. For bigger data types, such as an image frame, Array tokens are more appropriate. Array tokens encapsulate arrays of values that are buffered in random access memories (RAM) between reactors. This introduces additional read latency which depends on the type of RAM (either BRAM or DRAM).

G.6.2 Token interfaces



(a) Token read channels **(b)** Token write channels

Figure G.6: The four channels used to read and write tokens. We refer to `TokenReadMaster`, `TokenReadSlave`, `TokenWriteMaster` and `TokenWriteSlave` as *interfaces* which we display with rounded rectangles.

Tokens are communicated between reactions contained in various reactors. Fig. G.6 shows the four *channels* used for reading and writing tokens and the four different *interfaces* we compose out of these channels. A channel is a unidirected group of signals and an interface is a group of such channels. All channels, and thus also interfaces, are parameterized by the token type. `TokenReadMaster` and `TokenReadSlave` are the interfaces used to read, i.e. receive, tokens. They are identical except for the direction of the channels. A `TokenReadReq` channel is used to communicate a read request from the master to the slave. If the token is an `ArrayToken` the request includes an address and a read size. In the case of a `PureToken` or a `SingleToken`, this channel is unused. A `TokenReadResp` channel is used by the slave to respond to requests. In the case of `SingleTokens` and `PureTokens`, the slave will present any available token on the `TokenReadResp` channel, without the need for a prior request. `TokenWriteMaster` and `TokenWriteSlave` are the interfaces used to write, i.e. send, tokens. A `TokenWriteReq` channel is used by the master to request sending a token. It includes an address and a write size for `ArrayTokens`, whereas `SingleToken` and `PureToken` do not use this channel. A `TokenWriteDat` channel is used for transmitting the actual data within the token. All channels handshake with ready/valid signals. In addition to the channels, there is also a fire signal. It is used by the `TokenReadMaster` to signal that the firing of any connected component, typically a reaction, has been completed and that the tokens have been consumed. The fire signal is also used by the `TokenWriteMaster` to signal that the writer has finished writing the tokens.

G.6.3 Reaction

Reactions are the fundamental unit of computation. In the hardware reactor model, a computation is thought of as a directed graph of reactions. The programmer's

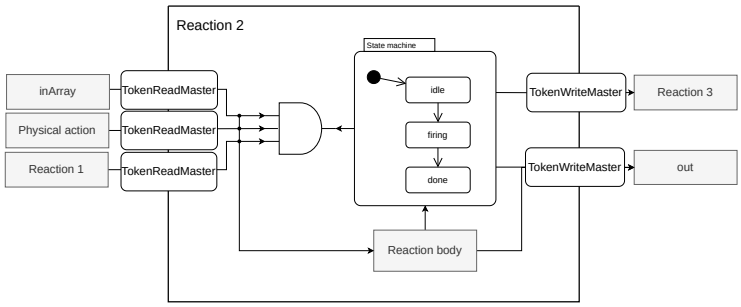
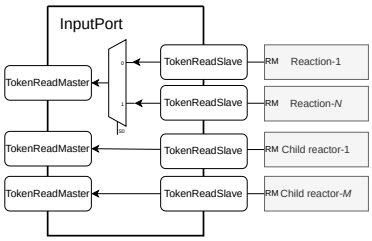


Figure G.7: The Chisel module implementing Reaction2 in the Sink reactor of Fig. G.4

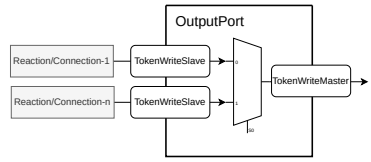
task is mainly that of designing the computations encapsulated by each reaction. These computations are also referred to as reaction-bodies and are expressed in a target language between the `{=...=}` brackets in the Lingua Franca programs, e.g. at Line 12 in Listing G.2. `Reactor-chisel` currently only supports reaction-bodies written in Chisel, but any hardware design language that compiles to Verilog, including various HLS languages, could be supported.

In `reactor-chisel`, a **`Reaction`** is a parameterized hardware module which, among other things, encapsulates the user-written reaction-body. A **`Reaction`** has a `TokenReadMaster` interfaces to each of its dependencies and a `TokenWriteMaster` for each anti-dependency. Possible dependencies are triggers (like timers, physical actions, and startup/shutdown), input ports of the parent reactor as well and precedence ports from other reactions within the same reactor with higher precedence. The possible anti-dependencies are the output ports of the parent reactor and precedence ports to reactions with lower precedence. The core feature of a **`Reaction`** is a finite state machine that controls when the user-written reaction-body shall be enabled.

Fig. G.7 shows Reaction 2 from the `Sink` reactor in Fig. G.4. Notice that this reaction has three dependencies, each connected through a `TokenReadMaster` interface. It is the port called `inArray`, a physical action as well as a precedence port from Reaction 1. The **`Reaction`** `body`, indicated in the Figure, is a user-written Chisel module and is copied directly from the Lingua Franca program. Everything else is part of the runtime and code-generated. In Listing G.2 all reaction bodies are empty. A state machine controls the execution. Whenever there are tokens present on all the input channels, and all the output channels are ready to accept tokens, the reaction will enter a state where the reaction body is enabled. It will execute until a special signal, called `reactionDone`, which defaults to “true” but can be overwritten from the reaction body, is asserted, at which point we will move to the “done” state. In the “done” state, all output channels that have not received any token receive an absent token.



(a) An InputPort multiplexes the read signals from the reactions within the same reactor and also forwarding read signals from reactions in child reactors.



(b) An OutputPort multiplexing the write signals from reactions and connections.

Figure G.8: InputPorts and OutputPorts are arbiters connecting reactions and contained reactors to the input and output ports of the parent reactor.

G.6.4 Ports

Ports are modules that enable reactions to communicate tokens. In `reactor-chisel`, each port is implemented by a set of `TokenReadMaster` or `TokenWriteMaster` interfaces, depending on whether it is an input or output port. An `InputPort` allows multiple reactions to read through a single `TokenReadMaster`, conversly, an `OutputPort` does the same for outputs. Contained reactors can also have their input ports connected to an input port of the parent, as is the case with the `Contained` reactor in Fig. G.4. This contained reactor will also be connected to the `InputPort`. The `InputPort` works like an arbiter allowing multiple reactions within the same reactor sharing a single `TokenReadMaster` interface. Reactions at different levels of the containment hierarchy will get parallel `TokenReadMaster` interfaces. Fig G.8a shows a generic `InputPort` module connecting N reactions from the same reactor and M contained reactors to the input port of the parent reactor. This results in $M+1$ parallel "read channels" through the input port. The number of read channels associated with an input port can only be decided by traversing down the containment hierarchy and finding all reactions connected to the input port in question. Fig. G.8b shows a generic `OutputPort` module. It multiplexes the `TokenWriteMaster` interfaces from reactions and connections. There is never more than a single "write channel". This is due to the reactor semantics that do not allow reactions at different levels of the containment hierarchy to connect to the same output port.

G.6.5 Connections

In `reactor-chisel`, a `Connection` is a memory, typically situated between an output port of one reactor and the input port of another. A `Connection` will expose a single `TokenWriteSlave`, typically connected to an output port, and several `TokenReadSlave` matching the number of read channels of the connected input port. Fig. G.9 shows a `Connection` connected to two reactors with a generic memory for storing tokens.

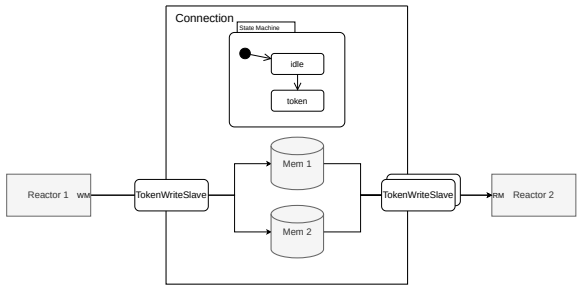


Figure G.9: A Connection is written to by the output port of a reactor and read from by the input port of another reactor. `RM` and `WM` are shorthand for TokenReadMaster and TokenWriteMaster

`Connections` are parameterized by the type of token it carries. `SingleTokens` are stored in flip-flops, while `ArrayTokens` uses BRAMs. A `Connection` can only have a single writer, but multiple readers. This is mirrored by the fact that input ports of reactors might have multiple read channels, while output ports only can have a single write channel. In Fig. G.9 two duplicate memories are written to by an output port, while being read from, independently, by two input ports. A simple state machine transitions on the fire signals on the `TokenWriteSlave` and `TokenReadSlaves`.

G.6.6 State variables

In Lingua Franca, state variables are persistent data associated with a reactor. They retain their value between reaction invocations, and they can be read and written from all reactions in the reactor. This is unlike any sequential elements instantiated in the reaction bodies, which will be reset between each reaction invocation. In `reactor-chisel` state variables are implemented as memories with read and write ports connected to all the reactions within the same reactor. Due to the mutual exclusion between these reactions, all of these ports can be multiplexed onto a single read and a single write port connected to the actual memory. In our current implementation, state variables are stored in flip-flops.

G.6.7 Reactor

As mentioned, it is the reactions that are the fundamental unit of computation and where the main control logic is synthesized. Reactors are best thought of as containers that allow for multiple reactions sharing access to the same state variables. Fig. G.10 shows the generated `Sink` reactor from Fig. G.4. It contains three reaction modules each connected through an `TokenReadMaster`, to a set of `InputPorts` and `Connection` modules. Note that the direction of the arrow indicates the master-slave relationship, not the flow of tokens. E.g. the arrow from the reactor `Contained` to the input port `inArray` indicates that `Contained` has a `TokenReadMaster` interface which is

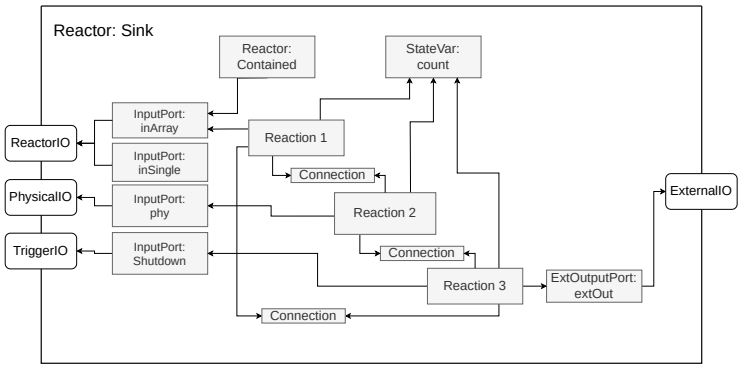


Figure G.10: The code-generated implementation of Sink from Fig. G.4

connected to a `TokenReadSlave` interface at `inArray`. The tokens flow in the opposite direction. All reactions are connected with read and write ports to the state variable `count`. The precedence relationship, shown with dashed orange lines in Fig. G.4, are implemented through the `Connection` modules separating the reactions. Notice also the `Connection` from **`Reaction 3`** to **`Reaction 1`**. This ensures that **`Reaction 1`** does not start processing its next token until **`Reaction 3`** is done with the first. This connection is initialized with a single pure token.

A reaction will fire when there are tokens available on all of its `TokenReadMaster` interfaces. When it fires, the user-written Chisel design will execute until it is done. Any write port not used by the user-written design will have an absent token written to it. This will include the generated precedence connections. In this way, the reactions within the same reactor are executed strictly in order.

The `InputPort` modules are connected to one of three IO bundles (shown in rounded rectangles at the border of the module). `ReactorIO` contains all the actual Lingua Franca input and output ports, e.g. `inArray` and `inSingle`. `PhysicalIO` contains the physical actions, in this case, the `phy` dependency. Finally `TriggerIO` contains all timers and built-in triggers like startup and shutdown. There is also an `ExternalIO` which contains ports annotated with `external`. The `ExternalIO` bundle allows reactions to directly read and write to wires routed to the top level of the design.

G.6.8 Triggers

Ultimately, tokens either arrive through top-level input ports, from reactors running in software, or from a module called `TriggerGenerator` shown in Fig. G.11. This module exposes a `timerTriggers` interface and a `phyTriggers` interface. Both are vectors of `TokenWriteMaster` interfaces, containing one interface per timer and physical action in the program. `TriggerGenerator` also accepts scheduling of physical actions through its `phySchedule` interface which is a vector of `TokenWriteSlave` interfaces, one per physical action. The purpose of the `TriggerGenerator` is to output sequences of

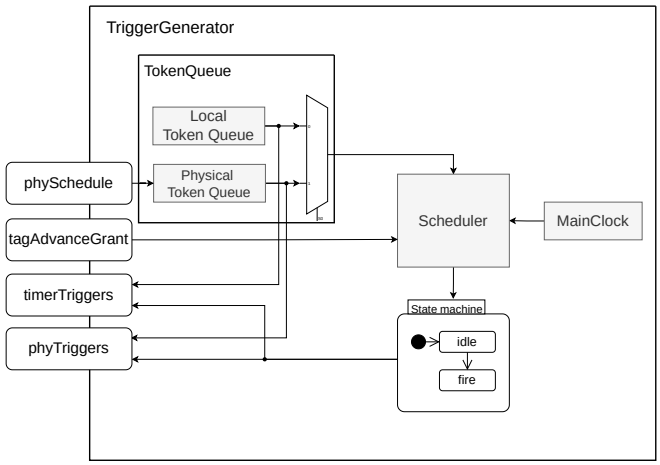


Figure G.11: The TriggerGenerator module in reactor-chisel

tokens on the `timerTriggers` and `phyTriqggers` interfaces that correspond to the timers in the Lingua Franca program and the asynchronous scheduling of physical actions. This is achieved through the two "queues" in `TokenQueue`, one for physical actions (`PhysicalTokenQueue`), and one for timers and built-in triggers (`LocalTokenQueue`). Both expose an output signal called the Next Event Tag (NET) which is the tag of their next token, as well as a *trigger vector* indicating which triggers are present at this tag. The token queue with the lower NET is forwarded to the outputs of the `TokenQueue` module. The `Scheduler` module contains the firing logic. It accepts a NET from the `TokenQueue`, an external Tag Advance Grant (TAG), and the current time from the `MainClock`. In the case of standalone FPGA designs based on the Chisel-target, the TAG is hardwired to a value greater than any tag (a forever tag). When the `Scheduler` decides to fire, the trigger vector from the `TokenQueue` is forwarded to the `timerTriggers` and `phyTriggers` interfaces and the `TokenQueue` can move to the next step in schedule.

The `PhysicalTokenQueue` is a first-in-first-out (FIFO) queue implemented using the `Queue` primitive from the Chisel3 standard library. The `LocalTokenQueue` is technically not a queue, but rather two fixed schedules assembled at compile-time. A hyperperiod is computed based on the period and offsets of the timers in the program. The schedules last a single hyperperiod, containing a trigger vector for each step. The *initial* schedule contains the trigger vectors to be outputted on the very first run through the hyperperiod. Due to the possibility of timers with no period, this round might be different than the rest, encoded in the *periodic* schedule. There can also be a trigger vector associated with the termination of the program if there are any reactions triggered by the built-in shutdown trigger.

The semantics of a set of interconnected hardware reactors are defined with respect to the tags of the tokens communicated between them. This is ultimately

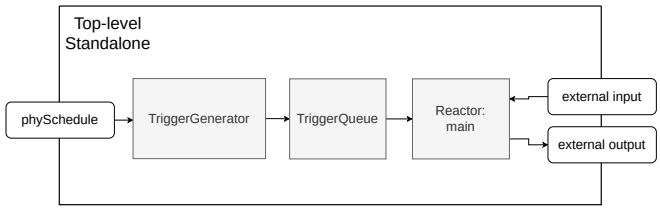


Figure G.12: The top-level of a standalone Lingua Franca program executing in hardware.

made possible by the `TriggerGenerator` module which synchronizes the creation of tokens.

Fig. G.12 brings everything together for "standalone" programs, i.e. programs that run exclusively in hardware. It shows how the `TriggerGenerator` is connected through a `TriggerQueue` to the main reactor. The trigger queue is simply a queue for the tokens and gives some flexibility in the face of bursty behavior. The `TokenWriteSlave` interfaces associated with `phySchedules` is routed to the very top level of the design together with any port marked as `external`.

G.6.9 Hardware-software codesign

So far, we have only discussed how `reactor-chisel` enables designing FPGA accelerators with reactor semantics. However, one of the key benefits of `reactor-chisel` is that it enables deterministic coordination with external reactors, e.g., running on the CPUs of an SoC FPGA. We leverage the open-source project fpga-tidbits [43] for creating the platform-independent communication channels between hardware and software using either control and status registers (CSR) or shared memory.

Consider again the `VectorAdd` program in Listing G.1. This program consists of two software reactors, `Software` and **`main`**, and one hardware reactor, `Accel`. Data flowing on the connections between hardware reactors and software reactors are either sent through memory-mapped CSRs synthesized to the FPGA or through shared memory. Ports carrying single tokens will use CSRs while ports carrying array tokens use the shared memory. For instance, in the `VectorAdd` program in Listing G.1, all the ports carry array tokens. `Reactor-chisel` includes DMA modules which lets the reactions access the tokens in off-chip DRAM as if they were ordinary array tokens buffered in on-chip BRAM. If there are multiple array connections from software to hardware, `reactor-chisel` will multiplex the memory requests to off-chip DRAM onto a single physical memory interface.

The execution of reactors in software and hardware must be synchronized to correctly deliver reactor semantics. We employ a subset of the coordination scheme used in distributed Lingua Franca programs [33]. Three signals are needed between the hardware reactors on the FPGA and the software reactors on the CPU as seen in Fig. G.13. (1) The Next Event Tag (NET) signal informs the CPU of

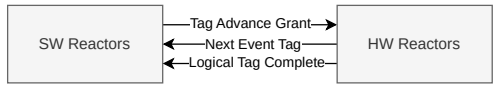


Figure G.13: Coordination signals between software reactors and hardware reactors

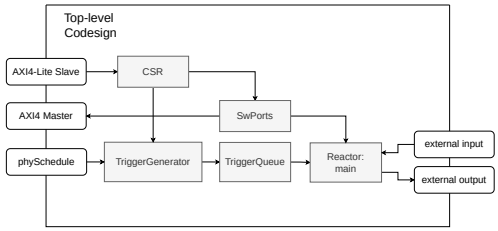


Figure G.14: The top-level FPGA module generated when using the Codesign-target.

the tag of the next token on the token queue in the FPGA. (2) The Logical Tag Complete (LTC) signal informs the CPU of the tag of the latest completed token in the FPGA. (3) The Tag Advance Grant (TAG) is sent from the CPU to the FPGA and grants the FPGA permission to advance its time until a specific tag.

This coordination interface is implemented through a set of CSRs. The `TriggerGenerator` introduced in Section G.6.8 handles the control logic from the FPGA side. It does not release the next token until it has received a TAG from the software that is greater than its NET. Moreover, as a token is released, the tag is queued such that it can be dequeued and written to the LTC status register when it has fully propagated through the hardware reactions.

Fig. G.14 shows the generated top-level design on the FPGA when using the Codesign-target of Lingua Franca. The CSRs are read and written to, by the software reactors, through an AXI4Lite slave interface. Array connections are facilitated through shared memory, accessed through an AXI4 master interface. External inputs and outputs can be connected to other designs on the FPGA, or directly to IO pins. Finally, scheduling of physical actions can be done from external FPGA designs which in turn can be connected to IO pins.

On the software side, we synthesize a special reactor in charge of communicating and synchronizing with the hardware reactors. Fig. G.15 shows the synthesized reactor for the `VectorAdd` program in Listing G.1. The reactor called `_FpgaWrapper` is generated based on the reactor `Accel` from Listing G.1. Notice that it duplicates all of its ports. There are three reactions. Reaction 1 handles the startup (the startup trigger is represented by a white circle), this involves programming the bitstream of `Accel` onto the FPGA and initializing the shared memory regions used for hardware-software communication. Reaction 2 is triggered by the input port and contains most of the logic. It forwards data from the `Software` reactor by writing

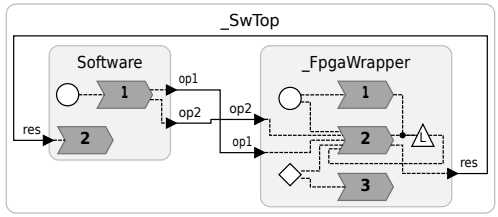
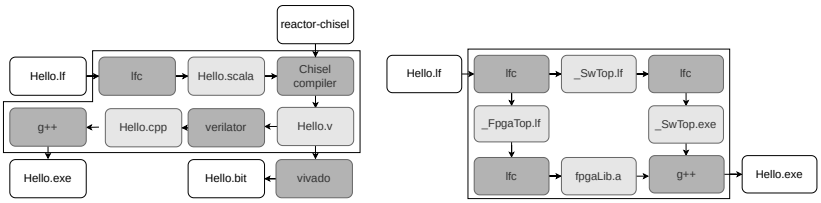


Figure G.15: Diagram of synthesized software reactor for Listing G.1, handling communication and synchronization with the hardware reactors.

it to shared memory and passing the address to the `Accel` reactor through a CSR. Data coming from the `Accel` reactor, through the `res` port, is copied from the FPGA memory region and outputted over the duplicate `res` port. This reaction is also triggered by a logical action which is scheduled whenever there are tokens originating from the hardware reactors. Finally, Reaction 3 is triggered by shutdown (represented by a white diamond). Here we do clean-up and free any allocated memory. This synthesized reactor relies on fpga-tidbits for communicating with the FPGA or doing co-simulation of hardware and software.

G.6.10 Compilation



(a) Compilation of Lingua Franca programs for the Chisel target

(b) Iterative compilation of Lingua Franca programs for the Codesign target

Figure G.16: Compiling Lingua Franca designs for SoC FPGA with the Chisel-target and the Codesign-target

To generate these circuits, we extend the Lingua Franca Compiler (`lfc`) with two new target languages. The Chisel-target supports generating code for hardware reactors written in Chisel. The Codesign-target enables the compilation of Lingua Franca programs consisting of both software reactors written in C++ and hardware reactors written in Chisel.

Fig. G.16 shows the steps involved in compiling Lingua Franca programs for SoC FPGAs. white rectangles are the initial input and final outputs, grey rectangles are intermediate artifacts, dark grey rectangles are compilers and tools, and the

large white rectangle encircles all the steps performed automatically when invoking `lfc`. First, we look at compiling standalone circuits shown in Fig G.16a. We begin in the top left corner with the program `Hello.lf` passed to `lfc`. This step transforms an input Lingua Franca program into a Chisel program (`Hello.scala`). This generated Chisel program is then compiled, together with the `reactor-chisel` runtime, by the Chisel compiler, which eventually emits a Verilog description (`Hello.v`). This Verilog description can either pass through `verilator`, generating a C++ emulation of the circuit (`Hello.cpp`) before a C++ compiler like `g++` can generate an executable (`Hello.exe`). This is the default behavior of our toolchain. A single click turns the Lingua Franca program into an executable emulation of the circuit. When the user eventually wants to target real hardware, the generated Verilog must be passed through a proprietary tool like `Vivado`, which will result in a bitstream (`Hello.bit`) that can be programmed onto an FPGA.

The compilation steps for the Codesign-target are more complicated and shown in Fig. G.16b. We start in the top left corner with `Hello.lf`, but this time, it uses the Codesign target and includes software and hardware reactors. When `lfc` is passed such a program, it will generate two new projects. The hardware reactors will be extracted to a new project organized in `_FpgaTop.lf`; this uses the normal Chisel-target and is passed through `lfc` again through a process similar to the one in Fig G.16a. However, it outputs a static library (`fpgaLib.a`), not an executable. The software reactors are put in a separate project with the reactor synthesized for hardware-software communication, shown in Fig. G.15. This is organized in `_SwTop.lf` which uses the Lingua Franca C++ target. This file is passed through `lfc` again and finally linked with `fpgaLib.a` to produce the final executable. If we are targeting emulation, then the final executable includes the verilator emulation of the circuit. If we are targeting a real FPGA, the final executable only includes the drivers for interacting with the FPGA.

We make use of fpga-tidbits to enable this seamless transition between emulation in verilator and deployment on real hardware.

G.7 Evaluation

We evaluate `reactor-chisel` on a set of test programs in terms of resource utilization, latency, and throughput. We synthesize our designs using Vivado 2021.2 targeting the xc7z020clg484-1 part, with a 5 nanosecond clock period. The benchmarks are available online together with documentation at Github [1]

G.7.1 Image processing example

We begin with a more qualitative evaluation of a realistic real-time image processing example. Fig. G.17 is auto-generated from the LF program which are using the Codesign target. It consists of two top-level reactors. `CameraDriver` executes in

[1] https://github.com/erlingrj/reactor-chisel-benchmarks

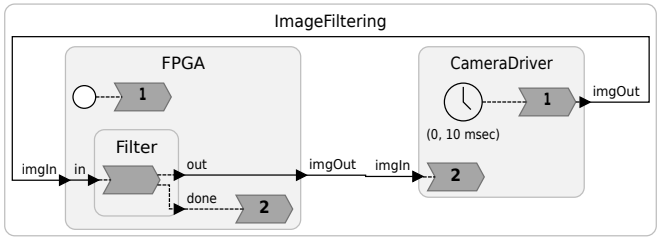


Figure G.17: A simple HW/SW image processing application

software and is triggered every 10 milliseconds. It fetches a new image from the camera, in our implementation, it just reads an image from the filesystem, and writes it to an output port connected to the `FPGA` reactor which executes on the FPGA. The `FPGA` reactor contains another reactor called `Filter` which performs the image processing algorithm. There are also two other reactions which are used for instrumentation. For simplicity, we chose a grayscale filter algorithm which we split over two pipeline stages.

We have also made a handwritten implementation of the same application in C++ and Chisel. It uses fpga-tidbits for generating CSRs, DMAs and software drivers. Arguably, this is a fair comparison since `reactor-chisel` itself is based on fpga-tidbits. Thus both versions will experience similar overheads from the implementation of DMAs and CSRs which is outside the scope of this work. The same grayscale filter module is used in both examples. Only the coordination fabric is different.

	Handwritten	`Reactor-chisel`
LUTs	835	1087
Flip Flops	1018	1285
Fmax (MHz)	221	216
Latency (clock cycles)	576024	614423
HW loc	130	77
SW loc	87	49

Table G.1: Comparing a handwritten image-processing accelerator with a reactor-chisel based implementation

A quantitative comparison is done in Table. G.1. Clearly, `reactor-chisel` introduces an overhead. Both in terms of resource utilization, critical path and runtime execution. However, we do believe that it is an acceptable tradeoff for reduced complexity of the overall project. We see a significant decrease in the number of lines of code in both hardware and software components.

The key advantage of Lingua Franca and `reactor-chisel` is that it brings

the entire computation, spanning CPUs and FPGA into a single program. Since Lingua Franca is a declarative language, synthesizing diagrams like the one in Fig. G.17 is possible. This is tremendously helpful in understanding the semantics of the computation. Moreover, the timed semantics and the static topology of the reactors open the door for interesting compiler optimizations. For instance, in Fig. G.17, we can easily infer, at compile time, that the two connections will carry an image frame every 10 millisecond. If there are more connections, the compiler could figure out which to multiplex onto the same AXI ports and which to give dedicated ports by identifying at which tags the different connections might carry tokens. Such compiler optimizations are left for future work.

G.7.2 Resource utilization

Test	LUTs	Flip flops	BRAMs (Kb)	Fmax (MHz)
ArrayConnection	137	137	18	317.06
Hierarchy	65	117	0	323.73
PhysicalAction	139	171	0	224.92
PhysicalAction2	153	193	0	251.32
PhysicalAction10	210	215	0	239.92
Reactions1	62	119	0	319.18
Reactions2	79	128	0	309.21
Reactions3	91	134	0	328.84
Reactions10	165	180	0	303.49
Timers1	62	119	0	319.18
Timers2	127	140	0	315.96
Timers3	146	178	0	319.80
Timers10	375	304	0	286.86

Table G.2: Resource utilization for standalone circuits using the Chisel-target

Table G.2 shows the resource utilization for a set of programs using the Chisel-target, aimed for standalone execution on the FPGA, i.e., no synchronization with CPU or off-chip DRAM. The test programs include minimal reaction bodies that only do enough to avoid being optimized away. We are thus trying to measure solely the overhead by the runtime. The names of the tests are meant to illustrate what they are measuring. For instance, `ReactionN` measures the overhead of N reactions.

The `Reactions1` test represents the smallest program we can envision. It thus gives an idea of the baseline overhead introduced by the runtime. Going from one to two reactions in the same reactor costs about 20 LUTs and 10 flip-flops. The resource cost is not linear and a further increase from two to ten reactions only incurs a 12 LUT and 7 flip-flop cost, on average. The addition of more reactions

does not have a substantial effect on the critical path.

Increasing the number of timers in a program is slightly more expensive. On average, adding a timer to the design costs 31 LUTs and 20 flip-flops. It is more expensive to go from a single timer to a two timers due to optimizations performed for single-timer designs.

The `PhysicalAction` test shows the cost of introducing physical actions. As described in Section G.6.8, physical actions are scheduled onto a separate token queue, whose head is compared against the head of the ordinary token queue. Adding a single physical action adds 77 LUTs and 52 flip-flops, compared to a program with a single timer. Adding more physical actions is cheaper.

In the `Hierarchy`, we introduce an empty wrapper reactor around the program from `Reactions1`. There is virtually no additional cost showing that the hierarchy introduced by reactors is a zero-cost abstraction.

Finally, in `ArrayConnection` we have a test program consisting of a source and a sink reactor that communicates array tokens. The array token includes 2048 elements of 32-bit each. The results show that an array connection with one writer and one reader adds around 60 LUTs and 54 flip-flops.

Test	LUTs	Flip flops	BRAMs (Kb)	Fmax (MHz)
ArrayConnections1	497	661	0	240.85
ArrayConnections2	814	891	0	239.52
SingleConnection1	308	520	0	253.94
SingleConnection2	320	596	0	257.80
WithLocalEvents	421	576	0	234.58
WithoutLocalEvents	308	520	0	253.94

Table G.3: Resource utilization for hardware-software designs using the Codesign-target

Table G.3 shows the resource utilization of programs using the Codesign-target. This means that the resulting designs include DMA modules, control-and-status registers (CSRs), AXI Lite slave for giving access to the CSRs, AXI Master ports to main memory.

In `SingleConnection1` and `Singleconnection2` we observe the baseline overhead for the Codesign-target in the order of 300 LUTs and 500 flip-flops. This is mainly consumed by the CSRs, which account for 86% of the LUTs and 69% of the flip-flops. Adding another single token connection from software to hardware adds, 12 LUTs and 76 flip-flops. Again, this is almost exclusively spent on CSRs.

In `WithLocalEvents` and `WithoutLocalEvents` we look at the cost of going from a purely reactive hardware reactor which receives all its tokens from software, to a hardware reactor that also generates tokens through a timer. This costs an additional 113 LUTs and 56 flip-flops. This is mainly used in the read/write logic of the CSRs and the `TriggerGenerator`.

`ArrayConnections1` and `ArrayConnections2` show the overhead of array connections between hardware and software. Relative to a single token connection, an array connection costs 189 LUTs and 141 flip-flops. Going from a single array connection to two array connections adds 317 LUTs and 230 flip-flops. For every array connection between hardware and software a DMA module is created. If there are more than two, then a multi-channel memory system is created which allows multiple read requests and read responses to be multiplexed onto a single memory port. All of these components are part of the fpga-tidbits project

In general, the communication across hardware and software adds some overhead, both in terms of resources and maximum clock frequency. This would also be the case for ad-hoc designs using e.g. AMD/Xilinx IP blocks for CSRs, AXI Lite slave, AXI4 Master and DMAs. We leave it for future works to investigate how the underlying framework, i.e. fpga-tidbits, can be improved to reduce resource utilization.

G.7.3 Latency

Test	Latency (clock cycles)
ArrayConnectionLatency1	5
ArrayConnectionLatency2	260
ArrayConnectionLatency3	515
MutexLatency	4
TenStagePipeline	27
TimerLatency1	0

Table G.4: Reaction invokation latency for programs using the Chisel-target.

Table G.4 shows the results of the latency benchmarks which measure the reaction invocation latencies for various program topologies. It is measured by comparing the logical and physical time, i.e., the lag, at the time the reaction is triggered. These benchmarks use the Chisel-target, meaning that we are measuring latencies on the FPGA fabric.

`TimerLatency1` shows that the invocation latency for a single reaction triggered by a timer is zero clock cycles. We achieve this by releasing tokens from the token queue a few clock cycles too early to account for the pipeline stages. This is a static optimization. Reactions triggered deeper in the containment hierarchy will have more pipeline stages between them and the TriggerGenerator and will thus experience different invocation latencies. This highlights the subtle difference between the logical and physical time points at which these reactions are invoked.

`MutexLatency` shows that a second reaction in the reactor, triggered by the same timer, will experience an additional four clock cycles latency due to mutual exclusion. The first reaction executes in a single cycle and there are three additional

cycles of overhead.

`ArrayConnectionLatency1` shows that it takes five cycles between an upstream reaction writing a single value to an array connection and a downstream reaction is triggered and has fetched the value. In `ArrayConnectionLatency2` the upstream reaction writes 256 words to the array connection, here the downstream is triggered after 260 clock cycles. Finally, in `ArrayConnectionLatency3` the upstream reaction writes 256 words and the downstream reads out all 256 words. Here the latency is 515 clock cycles. This highlights the semantics of array connections. The upstream must complete all of its writes before the downstream can start reading. Thus the reading does not start until 256 cycles have passed. We are working on a FIFO token that can get around this issue.

Lastly, in `TenStagePipeline` we have a pipeline of ten reactors, each containing a single reaction executing in a single clock cycle. We measure the time needed to traverse from the first to the last stage to 27 clock cycles. This is four cycles per hop, consistent with the result in `MutexLatency`

G.7.4 Throughput

Test	Throughput (clock cycles per event)
DeepPipe	7
ShallowPipe	7
TenParallelPipes	7

Table G.5: Token throughput for programs using the Chisel-target.

Our throughput benchmarks measure the number of clock cycles spent per single token for various designs in the Chisel target. We do this by generating tokens at the highest frequency, i.e., a timer with a single clock cycle period, and measuring how many clock cycles are needed to process a fixed amount of tokens. The processing consists of pipelines of reactors, and each reactor contains a single reaction executed in a single cycle. In `DeepPipe`, `ShallowPipe`, and `TenParallelPipes` we show that in such a pipeline design, we can handle a new token every seven clock cycles. This delay is mainly caused by two factors. First, the state machine that controls the execution of a reaction wastes one cycle before enabling the reaction and another after. If a reaction body executes one clock cycle, then the total execution will be three clock cycles. The second contributing factor is that a reaction is not enabled until its immediate downstream has completed execution. This is to protect the memory in the connections from concurrent access from upstream and downstream. Both of these overheads are addressable through further optimizations. We could allow the upstream to start execution but block it if it tries writing to its output port. We are also working on the concept of a FIFO token that resembles array tokens. However, FIFO tokens are buffered in the connec-

tions using FIFOs instead of RAM. This would enable the downstream reaction to be triggered when the first write occurs, rather than the completion of the upstream. Such optimizations are left for future work.

G.8 Conclusion

We have presented the hardware reactor model, a subset of the reactor model of computation and a convenient abstraction level for doing hardware-software codesign for cyber-physical systems. We have also presented `reactor-chisel`, a Chisel implementation of a hardware runtime delivering the semantics of the hardware reactor model. We have also extended the polyglot coordination language Lingua Franca with a Chisel-target and a Codesign-target, enabling hardware-software to codesign on the reactor level. We have shown several example programs illustrating the ease of which hardware-software codesign is performed in the Codesign-target of Lingua Franca. We evaluated our methodology on a set of benchmarks showing relatively low resource utilization and promising latency and throughput numbers. The work is open-source and available on Github [2]

[2] https://github.com/erlingrj/reactor-chisel

Bibliography

[1] A. Jantsch, *Modeling Embedded Systems and SoCs - Concurrency and Time in Models of Computation*. Morgan Kaufmann, 2003.

[2] A. Benveniste and G. Berry, 'The synchronous approach to reactive and real-time systems,' *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1270–1282, 1991.

[3] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek and K. Asanovic, 'Chisel: Constructing hardware in a Scala embedded language,' in *Design Automation Conference (DAC)*, ACM/EDAC/IEEE, 2012, pp. 1216–1225.

[4] E. A. Lee, 'Computing needs time,' *Communications of the ACM*, vol. 52, no. 5, pp. 70–79, 2009, ISSN: UCB/EECS-2009-30. DOI: 10.1145/1506409.1506426.

[5] E. R. Jellum, S. Lin, P. Donovan, C. Jerad, E. Wang, M. Lohstroh, E. A. Lee and M. Schoeberl, 'InterPRET: A Time-predictable Multicore Processor,' in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, ser. CPS-IoT Week '23, San Antonio, TX, USA: Association for Computing Machinery, 2023, pp. 331–336, ISBN: 9798400700491. DOI: 10.1145/3576914.3587497. [Online]. Available: https://doi.org/10.1145/3576914.3587497.

[6] E. R. Jellum, T. H. Bryne, T. A. Johansen and M. Orlandic, 'The Syncline Model - Analyzing the Impact of Time Synchronization in Sensor Fusion,' in *2022 IEEE Conference on Control Technology and Applications (CCTA)*, 2022, pp. 1446–1453. DOI: 10.1109/CCTA49430.2022.9966179.

[7] G. Berry and G. Gonthier, 'The Esterel synchronous programming language: Design, semantics, implementation,' *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.17.5606.

[8] E. A. Lee, 'The problem with threads,' *Computer*, vol. 39, no. 5, pp. 33–42, 2006. DOI: 10.1109/MC.2006.180.

[9] A. Jantsch and I. Sander, 'Models of computation and languages for embedded system design,' *Computers and Digital Techniques, IEE Proceedings -*, vol. 152, pp. 114–129, Apr. 2005. DOI: 10.1049/ip-cdt:20045098.

[10] G. Kahn, 'The semantics of a simple language for parallel programming,' in *Proc. of the IFIP Congress 74*, North-Holland Publishing Co., 1974, pp. 471–475.

[11] C. A. R. Hoare, 'Communicating sequential processes,' *Communications of the ACM*, vol. 21, no. 8, pp. 666–677, 1978.

[12] M. Lohstroh, Í. Í. Romeo, A. Goens, P. Derler, J. Castrillon, E. A. Lee and A. Sangiovanni-Vincentelli, 'Reactors: A deterministic model for composable reactive systems,' in *Cyber Physical Systems. Model-Based Design*, R. Chamberlain, M. Edin Grimheden and W. Taha, Eds., Cham: Springer International Publishing, 2020, pp. 59–85, ISBN: 978-3-030-41131-2.

[13] M. Lohstroh, E. Lee, S. Bateni, C. Menard, P. Donovan, C. Fournier, H. S. (Wong, A. Schulz-Rosengarten, E. R. Jellum, H. Kim, M. Weber, S. Lin and A. Rengarajan, *Lingua Franca*. [Online]. Available: https://github.com/lf-lang/lingua-franca.

[14] E. A. Lee and A. Sangiovanni-Vincentelli, 'A framework for comparing models of computation,' *IEEE Transactions on Computer-Aided Design of Circuits and Systems*, vol. 17, no. 12, pp. 1217–1229, 1998. [Online]. Available: http://ptolemy.eecs.berkeley.edu/publications/papers/98/framework/.

[15] J. Castrillon, K. Desnos, A. Goens and C. Menard, 'Dataflow models of computation for programming heterogeneous multicores,' in *Handbook of Computer Architecture*, A. Chattopadhyay, Ed. Singapore: Springer Nature Singapore, 2022, pp. 1–40, ISBN: 978-981-15-6401-7. DOI: 10.1007/978-981-15-6401-7_45-2. [Online]. Available: https://doi.org/10.1007/978-981-15-6401-7_45-2.

[16] Wikipedia, *Round-trip engineering — Wikipedia, the free encyclopedia*, http://en.wikipedia.org/w/index.php?title=Round-trip%20engineering&oldid=1172040452, [Online; accessed 19-September-2023], 2023.

[17] J. B. Dennis, 'First version data flow procedure language,' MIT Laboratory for Computer Science, Report MAC TM61, 1974.

[18] E. A. Lee and D. G. Messerschmitt, 'Synchronous data flow,' *Proceedings of the IEEE*, vol. 75, no. 9, pp. 1235–1245, 1987. DOI: 10.1109/PROC.1987.13876.

[19] E. A. Lee and D. G. Messerschmitt, 'Static scheduling of synchronous data flow programs for digital signal processing,' *IEEE Transactions on Computers*, vol. C-36, no. 1, pp. 24–35, 1987. DOI: 10.1109/TC.1987.5009446.

[20] M. Lohstroh, M. Schoeberl, A. Goens, A. Wasicek, C. Gill, M. Sirjani and E. A. Lee, 'Actors revisited for time-critical systems,' in *Proceedings of the 56th Annual Design Automation Conference 2019, DAC 2019, Las Vegas, NV, USA, June 02-06, 2019*, 2019, 152:1–152:4.

[21] M. Lohstroh, 'Reactors: A deterministic model of concurrent computation for reactive systems,' PhD thesis, EECS Department, University of California, Berkeley, Dec. 2020. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2020/EECS-2020-235.html.

[22] M. Lohstroh, C. Menard, S. Bateni and E. A. Lee, 'Toward a Lingua Franca for deterministic concurrent systems,' *ACM Transactions on Embedded Computing Systems (TECS), Special Issue on FDL'19*, vol. 20, no. 4, Article 36, May 2021. DOI: 10.1145/3448128.

[23] S. Bateni, M. Lohstroh, H. S. Wong, H. Kim, S. Lin, C. Menard and E. A. Lee, 'Risk and mitigation of nondeterminism in distributed cyber-physical systems,' in *Proceedings of the 21st ACM-IEEE International Conference on Formal Methods and Models for System Design*, ser. MEMOCODE '23, <conf-loc>, <city>Hamburg</city>, <country>Germany</country>, </conf-loc>: Association for Computing Machinery, 2023, pp. 1–11, ISBN: 9798400703188. DOI: 10.1145/3610579.3613219. [Online]. Available: https://doi.org/10.1145/3610579.3613219.

[24] A. Schulz-Rosengarten, R. von Hanxleden, M. Lohstroh, E. A. Lee and S. Bateni, 'Polyglot modal models through lingua franca,' in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, 2023, pp. 337–342.

[25] E. R. Jellum, S. Lin, P. Donovan, E. Soyer, F. Shakir, T. Bryne, M. Orlandic, M. Lohstroh and E. A. Lee, 'Beyond the Threaded Programming Model on Real-Time Operating Systems,' in *Fourth Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2023)*, F. Terraneo and D. Cattaneo, Eds., ser. Open Access Series in Informatics (OASIcs), vol. 108, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 3:1–3:13, ISBN: 978-3-95977-268-6. DOI: 10.4230/OASIcs.NG-RES.2023.3. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2023/17734.

[26] C. Menard, M. Lohstroh, S. Bateni, M. Chorlian, A. Deng, P. Donovan, C. Fournier, S. Lin, F. Suchert, T. Tanneberger, H. Kim, J. Castrillon and E. A. Lee, 'High-performance deterministic concurrency using lingua franca,' *ACM Trans. Archit. Code Optim.*, Aug. 2023, Just Accepted, ISSN: 1544-3566. DOI: 10.1145/3617687. [Online]. Available: https://doi.org/10.1145/3617687.

[27] A. Munshi, 'The opencl specification,' in *2009 IEEE Hot Chips 21 Symposium (HCS)*, IEEE, 2009, pp. 1–314.

[28] S. Edwards, 'The challenges of synthesizing hardware from c-like languages,' *IEEE Design & Test of Computers*, vol. 23, no. 5, pp. 375–386, 2006. DOI: 10.1109/MDT.2006.134.

[29] *Sycl*, Accessed on July 11, 2023. [Online]. Available: https://www.khronos.org/sycl/.

[30] T. Stefanov, C. Zissulescu, A. Turjan, B. Kienhuis and E. F. Deprettere, 'System design using khan process networks: The compaan/laura approach,' *Proceedings Design, Automation and Test in Europe Conference and Exhibition*, vol. 1, 340–345 Vol.1, 2004.

[31] A. Agne, M. Happe, A. Keller, E. Lübbers, B. Plattner, M. Platzner and C. Plessl, 'Reconos: An operating system approach for reconfigurable computing,' *IEEE Micro*, vol. 34, no. 1, pp. 60–71, 2014. DOI: 10.1109/MM.2013.110.

[32] C. Lienen, M. Platzner and B. Rinner, 'Reconros: Flexible hardware acceleration for ros2 applications,' in *2020 International Conference on Field-Programmable Technology (ICFPT)*, 2020, pp. 268–276. DOI: 10.1109/ICFPT51103.2020.00046.

[33] S. Bateni, M. Lohstroh, H. S. Wong, R. Tabish, H. Kim, S. Lin, C. Menard, C. Liu and E. A. Lee, 'Xronos: Predictable coordination for safety-critical distributed embedded systems,' *arXiv preprint arXiv:2207.09555*, 2022.

[34] I. Sander, 'System modeling and design refinement in ForSyDe,' Thesis, 2003.

[35] AMD/Xilinx, *Vitis unified software platform documentation: Embedded software development*, Retrieved July 11, 2023. [Online]. Available: https://docs.xilinx.com/r/en-US/ug1400-vitis-embedded.

[36] E. Borde, S. Rahmoun, F. Cadoret, L. Pautet, F. Singhoff and P. Dissaux, 'Architecture models refinement for fine grain timing analysis of embedded systems,' in *2014 25nd IEEE International Symposium on Rapid System Prototyping*, 2014, pp. 44–50. DOI: 10.1109/RSP.2014.6966691.

[37] D. Blouin, G. Ochoa-Ruiz, Y. Eustache and J.-P. Diguet, 'Kaolin: A system-level aadl tool for fpga design reuse, upgrade and migration,' in *2015 NASA/ESA Conference on Adaptive Hardware and Systems (AHS)*, 2015, pp. 1–8. DOI: 10.1109/AHS.2015.7231166.

[38] 'Ieee standard for standard systemc language reference manual,' *IEEE Std. 1666-20011*, 2012.

[39] M. D. King *et al.*, 'A methodology for hardware-software codesign,' PhD thesis, Massachusetts Institute of Technology, 2013.

[40] Arvind, R. S. Nikhil, D. Rosenband and N. Dave, 'High-level synthesis: An essential ingredient for designing complex asics,' in *International Conference on Computer Aided Design (ICCAD)*, 2004.

[41] K. Fleming and M. Adler, 'The leap fpga operating system,' *FPGAs for software programmers*, pp. 245–258, 2016.

[42] H. Grip, 'Suviving an in-flight anomaly: What happened on ingenuity's sixth flight,' *NASA Mars Helicopter Blog*, 2021. [Online]. Available: https://mars.nasa.gov/technology/helicopter/status/305/surviving-an-in-flight-anomaly-what-happened-on-ingenuitys-sixth-flight/.

[43] E. R. Jellum, Y. Umuruglu, M. Orlandic and M. Schoeberl, 'fpga-tidbits: Rapid Prototyping of FPGA Accelerators in Chisel,' in *To appear in: 2023 26th Euromicro Conference on Digital System Design (DSD)*.

Part III

Concluding remarks

Chapter 1

Conclusion

This thesis contains contributions to the problem of achieving both predictable and high-performance computing in the context of CPS. The contributions can naturally be divided by the compute substrate that they target. Papers B, D and C target general-purpose CPUs and Papers E, F, and G target SoC FPGAs. Paper A does not target a particular compute substrate but provides an analysis of one aspect of predictability, namely timestamping. In this chapter, we provide a summary of the contributions from the papers presented in Part II organized around the research questions from Part I Section 1.2

RQ1: What is the Importance of Predictable Timestamping in CPS?

Errors introduced by poor predictability can outweigh the errors introduced by sensor noise. Particularly when it comes to sensor timestamping for multiple sensor fusion systems. The Syncline model, presented in Paper A, reveals how state estimation errors are a sum of errors introduced by sensor noise and errors introduced by time-synchronization inaccuracies. The faster the dynamics of the physical system being sensed, the greater the requirements for time-synchronization accuracy.

RQ2: How Can We Enable Predictable Computing on General-Purpose CPUs?

By using languages with a predictable MoC. Languages and middlewares most commonly used lack both a semantic notion of time and parallelism. The fact that many computing devices do not offer deterministic timing or parallelism does not mean that our programming languages should not offer these abstractions. Languages based on "logical time" are promising because they can deliver both high performance and predictability. In Paper B, we extend Lingua Franca, a timed and

concurrent language, to hundreds of resource-constrained platforms by building on timing and concurrency abstractions from the Zephyr RTOS. In Paper C, we extend the Timed C language, another programming language with a semantic notion of time and concurrency, to support deterministic networking using TSN. In Paper D we develop a multicore CPU that has timing semantics in its instruction set.

By using deterministic networking technologies. In particular, the TSN standard will enable the next generation of CPS to utilize distributed computing without sacrificing predictability. Languages based on logical time are a promising solution to the problem of how the distributed computation, as a whole, should be expressed. In Paper C we show how this can be achieved in Timed C.

By developing time-predictable multicore CPU architectures. Computer architectures intended for CPS should be designed with predictability in mind. Multicore CPUs are necessary to run the compute-intensive workloads of the next-generation CPS, but they do not need to be as unpredictable as they are today. Fine-grained multithreading should be used in the pipeline of each individual core and time-triggered NoCs should be used for inter-core communication. InterPRET, presented in Paper D, is an example of such a multicore. It demonstrates that high-performance and predictability can be achieved together. InterPRET does introduce new challenges, especially with respect to how such NoC-based multicores should be programmed.

RQ3: How Can We Enable Predictable Computing on SoC FPGAs?

By creating powerful abstractions for accelerator design. SoC FPGAs are promising for CPS as they are intrinsically more predictable than most general-purpose CPUs. The challenge is to provide convenient abstractions to program them. In Paper E, we present `fpga-tidbits`, a powerful framework for developing FPGA accelerators in Chisel. In Paper F, we show how `fpga-tidbits` can be used to develop a high-performance accelerator.

By extending logical time to the FPGA. FPGA accelerators are typically designed in a synchronous language like VHDL where the logic at each clock cycle is explicitly laid out. With logical time we can define a more coarse-grained clock, at the accelerator-level. This can provide a clearer picture of the behavior resulting from the composition of multiple accelerators. In Paper G we present `reactor-chisel` and the Chisel-target of Lingua Franca which enables FPGA design using logical time.

By coordinating accelerators and software according to predictable MoCs.

This coordination is typically done ad-hoc. For instance, a single-threaded program (i.e. the software), might start an accelerator and busy-poll a status register until the accelerator is done. This coordination effectively turns the accelerator into a sequential component and the resulting computation can be reasoned about within the Sequential MoC. In Paper G, we propose to extend the reactor model of computation to hardware-software coordination. We present a "Codesign-target" for Lingua Franca which enables the programming of software for the CPU and accelerators for the FPGA in the same program.

Chapter 2

Future Work

The work conducted as part of this thesis touches on a range of topics and opens up several opportunities for future research.

Distributed systems with resource-constrained systems

The work in Paper B on supporting Lingua Franca on microcontrollers opens up the opportunity for predictable distributed real-time systems consisting of microcontrollers and microprocessors. This would require porting the federated infrastructure of Lingua Franca to the Zephyr platform.

Programming models for single-core and multi-core PRET machines

The InterPRET, introduced in Paper D, opens up several interesting research questions. The question of which abstraction level such machines should be programmed on remains open. XMOS is based on CSP and MultiPRET uses the synchronous language ForeC. We hypothesize that Lingua Franca and the reactor model are more suited. We also propose research into software-defined devices. These can be thought of as hardware circuits written in software. This is possible using PRET machines because they, like synchronous digital circuits, have deterministic timing.

Reactor-oriented high-level synthesis

The hardware reactor model of computation, introduced in Paper G, would be a promising abstraction-level for doing HLS. It would enable keeping the C-based frontend for the developers while at the same time introducing the possibility to model concurrency and timing.

Reactor-oriented dynamic partial reconfiguration

Dynamic partial reconfiguration is a promising emerging FPGA technology. It enables, at run-time, to reprogram parts of the FPGA fabric. However, reliably in-

corporating such dynamicity into a hardware-software system is challenging. The reactor model of computation is an interesting candidate for modeling this. There is already support for modal behaviours [105] and run-time mutations of the reactor graph are under development. It enables deterministic and controlled usage of this powerful feature.

Compiler optimizations

Doing hardware-software codesign at the reactor-level means that information about the communication between hardware and software is readily available in Lingua Franca syntax. By analyzing the Lingua Franca code answer such as: "What is the bandwidth of this connection from a software reactor and a hardware reactor?", and "Are these two connections ever going to carry data simultaneously?". This type of information can be used to perform very useful compiler optimizations such as deciding how many and what kinds of physical ports between the FPGA and the CPUs to use.

Bibliography

[1] R. Rajkumar, I. Lee, L. Sha and J. Stankovic, 'Cyber-physical systems: The next computing revolution,' in *Proceedings of the 47th Design Automation Conference*, ser. DAC '10, Anaheim, California: Association for Computing Machinery, 2010, pp. 731–736, ISBN: 9781450300025. DOI: 10.1145/1837274.1837461. [Online]. Available: https://doi.org/10.1145/1837274.1837461.

[2] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.

[3] S. Baruah, M. Bertogna and G. Buttazzo, *Multiprocessor scheduling for real-time systems*. Springer, 2015.

[4] E. R. Jellum, T. H. Bryne, T. A. Johansen and M. Orlandic, 'The Syncline Model - Analyzing the Impact of Time Synchronization in Sensor Fusion,' in *2022 IEEE Conference on Control Technology and Applications (CCTA)*, 2022, pp. 1446–1453. DOI: 10.1109/CCTA49430.2022.9966179.

[5] E. R. Jellum, S. Lin, P. Donovan, E. Soyer, F. Shakir, T. Bryne, M. Orlandic, M. Lohstroh and E. A. Lee, 'Beyond the Threaded Programming Model on Real-Time Operating Systems,' in *Fourth Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2023)*, F. Terraneo and D. Cattaneo, Eds., ser. Open Access Series in Informatics (OASIcs), vol. 108, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 3:1–3:13, ISBN: 978-3-95977-268-6. DOI: 10.4230/OASIcs.NG-RES.2023.3. [Online]. Available: https://drops.dagstuhl.de/opus/volltexte/2023/17734.

[6] E. R. Jellum, S. Lin, P. Donovan, C. Jerad, E. Wang, M. Lohstroh, E. A. Lee and M. Schoeberl, 'InterPRET: A Time-predictable Multicore Processor,' in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, ser. CPS-IoT Week '23, San Antonio, TX, USA: Association for Computing Machinery, 2023, pp. 331–336, ISBN: 9798400700491. DOI: 10.1145/3576914.3587497. [Online]. Available: https://doi.org/10.1145/3576914.3587497.

[7] E. R. Jellum, Y. Umuruglu, M. Orlandic and M. Schoeberl, 'fpga-tidbits: Rapid Prototyping of FPGA Accelerators in Chisel,' in *To appear in: 2023 26th Euromicro Conference on Digital System Design (DSD)*.

[8] E. Jellum, M. Orlandić, E. Brekke, T. Johansen and T. Bryne, 'Solving Sparse Assignment Problems on FPGAs,' *ACM Trans. Archit. Code Optim.*, vol. 19, no. 4, Dec. 2022, ISSN: 1544-3566. DOI: 10.1145/3546072. [Online]. Available: https://doi.org/10.1145/3546072.

[9] H. Austad, E. R. Jellum, S. Hendseth, G. Mathisen, T. H. Bryne, K. N. Gregertsen, S. M. Albrektsen and B. E. Helvik, 'Composable distributed real-time systems with deterministic network channels,' *Journal of Systems Architecture*, vol. 137, p. 102 853, 2023, ISSN: 1383-7621. DOI: https://doi.org/10.1016/j.sysarc.2023.102853. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762123000322.

[10] E. R. Jellum, M. Schoeberl, E. Lee and M. Orlandic, 'Codesign of Reactor-oriented Hardware and Software for Cyber-Physical Systems,' *Submitted to: ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 2023.

[11] S. Mitchell and L. Tzu, *Tao Te Ching: A New English Version*. HarperCollins, 2009, ISBN: 9780061807398. [Online]. Available: https://books.google.no/books?id=esUrAbMWAa4C.

[12] B. Jowett *et al.*, *The dialogues of Plato*. Scribner, Armstrong, 1873, vol. 4.

[13] I. Kant, *Prolegomena to any future metaphysics that will be able to present itself as a science*. Manchester University Press, 1966.

[14] E. A. Lee, *Plato and the Nerd — The Creative Partnership of Humans and Technology*. MIT Press, 2017.

[15] E. A. Lee, 'The past, present, and future of cyber-physical systems: A focus on models,' *Sensors*, vol. 15, no. 3, pp. 4837–4869, 2015. DOI: 10.3390/s150304837. [Online]. Available: http://www.mdpi.com/1424-8220/15/3/4837.

[16] A. Jantsch, *Modeling Embedded Systems and SoC's: Concurrency and Time in Models of Computation*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2003, ISBN: 1558609253.

[17] E. A. Lee and A. Sangiovanni-Vincentelli, 'A framework for comparing models of computation,' *IEEE Transactions on Computer-Aided Design of Circuits and Systems*, vol. 17, no. 12, pp. 1217–1229, 1998. [Online]. Available: http://ptolemy.eecs.berkeley.edu/publications/papers/98/framework/.

[18] D. Grund, J. Reineke and R. Wilhelm, 'A template for predictability definitions with supporting evidence,' in *Bringing Theory to Practice: Predictability and Performance in Embedded Systems*, P. Lucas, L. Thiele, B. Triquet, T. Ungerer and R. Wilhelm, Eds., ser. OpenAccess Series in Informatics (OASIcs), vol. 18, Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2011, pp. 22–31, ISBN: 978-3-939897-28-6. DOI: http://dx.doi.org/10.4230/OASIcs.PPES.2011.22.

[19] L. Thiele and R. Wilhelm, 'Design for timing predictability,' *Real-Time Systems*, vol. 28, no. 2-3, pp. 157–177, 2004.

[20] R. Kirner and P. Puschner, 'Time-predictable computing,' in *IFIP International Workshop on Software Technolgies for Embedded and Ubiquitous Systems*, Springer, 2010, pp. 23–34.

[21] R. Baheti and H. Gill, 'Cyber-physical systems,' *The impact of control technology*, vol. 12, no. 1, pp. 161–166, 2011.

[22] J. M. Hansen, T. I. Fossen and T. Arne Johansen, 'Nonlinear observer design for gnss-aided inertial navigation systems with time-delayed gnss measurements,' *Control Engineering Practice*, vol. 60, pp. 39–50, 2017.

[23] K. Granstrom, M. Baum and S. Reuter, *Extended object tracking: Introduction, overview and applications*, 2017. arXiv: 1604.00970 [cs.CV].

[24] C. Robin and S. Lacroix, 'Multi-robot target detection and tracking: Taxonomy and survey,' *Autonomous Robots*, vol. 40, Aug. 2015. DOI: 10.1007/s10514-015-9491-7.

[25] P. Gu, Z. Jing and L. Wu, 'Robust adaptive multi-target tracking with unknown measurement and process noise covariance matrices,' *IET Radar, Sonar & Navigation*, vol. 16, no. 4, pp. 735–747, 2022. DOI: https://doi.org/10.1049/rsn2.12216. eprint: https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/rsn2.12216. [Online]. Available: https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/rsn2.12216.

[26] E. A. Lee, 'Computing needs time,' *Communications of the ACM*, vol. 52, no. 5, pp. 70–79, 2009, ISSN: UCB/EECS-2009-30. DOI: 10.1145/1506409.1506426.

[27] H. Kopetz, *Real-Time Systems*. Boston, MA, USA: Kluwer Academic, 1997.

[28] A. Cervin, B. Lincoln, J. Eker, K.-E. Årzén and G. Buttazzo, 'The jitter margin and its application in the design of real-time control systems,' in *Proceedings of the 10th International Conference on Real-Time and Embedded Computing Systems and Applications*, 2004.

[29] A. Cervin, 'Stability and worst-case performance analysis of sampled-data control systems with input and output jitter,' in *2012 American Control Conference (ACC)*, 2012, pp. 3760–3765. DOI: 10.1109/ACC.2012.6315304.

[30] K. Smeds and X. Lu, 'Effect of sampling jitter and control jitter on positioning error in motion control systems,' *Precision Engineering*, vol. 36, no. 2, pp. 175–192, 2012.

[31] M. Lohstroh, E. Lee, S. Bateni, C. Menard, P. Donovan, C. Fournier, H. S. (Wong, A. Schulz-Rosengarten, E. R. Jellum, H. Kim, M. Weber, S. Lin and A. Rengarajan, *Lingua Franca*. [Online]. Available: https://github.com/lf-lang/lingua-franca.

[32] A. M. Turing, 'On computable numbers with an application to the entscheidungsproblem,' *Proceedings of the London Mathematical Society*, vol. 42, pp. 230–265, 1936.

[33] T. Gibson-Robinson, P. Armstrong, A. Boulgakov and A. Roscoe, 'FDR3 — A Modern Refinement Checker for CSP,' in *Tools and Algorithms for the Construction and Analysis of Systems*, E. Ábrahám and K. Havelund, Eds., ser. Lecture Notes in Computer Science, vol. 8413, 2014, pp. 187–201.

[34] A. H. Reynisson, M. Sirjani, L. Aceto, M. Cimini, A. Jafari, A. Ingólfsdóttir and S. H. Sigurdarson, 'Modelling and simulation of asynchronous real-time systems using timed rebeca,' *Science of Computer Programming*, vol. 89, pp. 41–68, 2014.

[35] G. Agha, *ACTORS: A Model of Concurrent Computation in Distributed Systems*, ser. The MIT Press Series in Artificial Intelligence. Cambridge, MA: MIT Press, 1986.

[36] A. Jantsch, *Modeling Embedded Systems and SoCs - Concurrency and Time in Models of Computation*. Morgan Kaufmann, 2003.

[37] C. A. R. Hoare, 'Communicating sequential processes,' *Communications of the ACM*, vol. 21, no. 8, pp. 666–677, 1978.

[38] J. Galletly, *Occam-2*, 2nd. University College London Press, 1996, ISBN: 1857283627.

[39] J. D. Ichbiah, B. Krieg-Brueckner, B. A. Wichmann, J. G. Barnes, O. Roubine and J.-C. Heliard, 'Rationale for the design of the ada programming language,' *ACM Sigplan notices*, vol. 14, no. 6b, pp. 1–261, 1979.

[40] A. A. Donovan and B. W. Kernighan, *The Go programming language*. Addison-Wesley Professional, 2015.

[41] D. Watt, *Programming XC on XMOS devices*. XMOS Limited, 2009.

[42] M. Aubury, I. Page, G. Randall, J. Saul and R. Watts, 'Handel-c language reference guide,' *Computing Laboratory. Oxford University, UK*, vol. 12, 1996.

[43] A. Benveniste and G. Berry, 'The synchronous approach to reactive and real-time systems,' *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1270–1282, 1991.

[44] S. A. Edwards, *The specification and execution of heterogeneous synchronous reactive systems*. University of California, Berkeley, 1997.

[45] G. Berry, *The Constructive Semantics of Pure Esterel*, Draft Version 3. Unpublished, 1999. [Online]. Available: `http://www-sop.inria.fr/meije/esterel/doc/main-papers.html`.

[46] N. Halbwachs, P. Caspi, P. Raymond and D. Pilaud, 'The synchronous data flow programming language LUSTRE,' *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1319, 1991.

[47] A. Benveniste and P. Le Guernic, 'Hybrid dynamical systems theory and the SIGNAL language,' *IEEE Tr. on Automatic Control*, vol. 35, no. 5, pp. 525–546, 1990.

[48] E. Yip, A. Girault, P. S. Roop and M. Biglari-Abhari, 'Synchronous deterministic parallel programming for multi-cores with forec,' *ACM Trans. Program. Lang. Syst.*, vol. 45, no. 2, Jun. 2023, ISSN: 0164-0925. DOI: 10.1145/3591594. [Online]. Available: https://doi.org/10.1145/3591594.

[49] R. von Hanxleden, 'SyncCharts in C—A proposal for light-weight deterministic concurrency,' in *ACM Embedded Software Conference (EMSOFT)*, 2009, pp. 11–16. DOI: 10.1145/1629335.1629366.

[50] J. B. Dennis, 'First version data flow procedure language,' MIT Laboratory for Computer Science, Report MAC TM61, 1974.

[51] G. Kahn, 'The semantics of a simple language for parallel programming,' in *Information Processing 74: Proceedings of the IFIP Congress 74*, J. L. Rosenfeld, Ed., IFIP, North-Holland Publishing Co., Aug. 1974, pp. 471–475.

[52] E. A. Lee, 'A coupled hardware and software architecture for programmable digital signal processors,' University of California, Report UCB/ERL M86/54, Jun. 1986. [Online]. Available: http://ptolemy.eecs.berkeley.edu/publications/papers/86/LeePhDThesis/.

[53] G. Kahn, 'The semantics of a simple language for parallel programming,' in *Proc. of the IFIP Congress 74*, North-Holland Publishing Co., 1974, pp. 471–475.

[54] J. Eker and J. W. Janneck, 'Cal language report: Specification of the cal actor language,' University of California, Berkeley, CA, Report Technical Memorandum No. UCB/ERL M03/48, Dec. 2003. [Online]. Available: http://ptolemy.eecs.berkeley.edu/papers/03/Cal/index.htm.

[55] T. Stefanov, C. Zissulescu, A. Turjan, B. Kienhuis and E. F. Deprettere, 'System design using khan process networks: The compaan/laura approach,' *Proceedings Design, Automation and Test in Europe Conference and Exhibition*, vol. 1, 340–345 Vol.1, 2004.

[56] J. Kodosky, J. MacCrisken and G. Rymar, 'Visual programming using structured data flow,' in *IEEE Workshop on Visual Languages*, IEEE Computer Society Press, 1991, pp. 34–39.

[57] C. Hewitt, 'Viewing control structures as patterns of passing messages,' *Journal of Artificial Intelligence*, vol. 8, no. 3, pp. 323–363, 1977.

[58] J. Armstrong, R. Virding, C. Wikström and M. Williams, *Concurrent programming in Erlang*, Second. Prentice Hall, 1996, ISBN: ISBN 0-13-508301-X.

[59] R. K. Dybvig, *The Scheme programming language*. Mit Press, 2009.

[60] R. Roestenburg, R. Bakker and R. Williams, *Akka In Action*. Manning Publications Co., 2016, ISBN: 1617291013.

[61] M. Samek, 'Quantum programming for embedded systems: Toward a hassle-free multithreading,'

[62] C. L. Liu and J. W. Layland, 'Scheduling algorithms for multiprogramming in a hard-real-time environment,' *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973, ISSN: 0004-5411. DOI: 10.1145/321738.321743. [Online]. Available: https://doi.org/10.1145/321738.321743.

[63] S. Baruah, V. Bonifaci, A. Marchetti-Spaccamela, L. Stougie and A. Wiese, 'A generalized parallel task model for recurrent real-time processes,' in *2012 IEEE 33rd Real-Time Systems Symposium*, IEEE, 2012, pp. 63–72.

[64] C. M. Kirsch and A. Sokolova, 'The logical execution time paradigm,' in *Advances in Real-Time Systems*, Springer, 2012, pp. 103–120.

[65] T. A. Henzinger, B. Horowitz and C. M. Kirsch, 'Giotto: A time-triggered language for embedded programming,' in *EMSOFT 2001*, vol. LNCS 2211, Springer-Verlag, 2001, pp. 166–184.

[66] AUTOSAR, 'Specification of timing extensions,' *AUTOSAR CP Release 4.4.0*, Oct. 2018.

[67] M. Lohstroh, E. A. Lee, S. A. Edwards and D. Broman, 'Logical time for reactive software,' in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, 2023, pp. 313–318.

[68] J. Hui, K. J. Edwards and S. A. Edwards, 'Timestamp peripherals for precise real-time programming,' 2023.

[69] S. A. Edwards and J. Hui, 'The sparse synchronous model,' in *2020 Forum for Specification and Design Languages (FDL)*, 2020, pp. 1–8.

[70] J. Hui and S. A. Edwards, 'The sparse synchronous model on real hardware,' *ACM Transactions on Embedded Computing Systems*,

[71] M. Lohstroh, C. Menard, S. Bateni and E. A. Lee, 'Toward a Lingua Franca for deterministic concurrent systems,' *ACM Transactions on Embedded Computing Systems (TECS), Special Issue on FDL'19*, vol. 20, no. 4, Article 36, May 2021. DOI: 10.1145/3448128.

[72] H. M. F. Lohstroh, *Reactors: A deterministic model of concurrent computation for reactive systems*. University of California, Berkeley, 2020.

[73] S. Natarajan and D. Broman, 'Timed c: An extension to the c programming language for real-time systems,' in *24th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS 2018)*, [ed] Pellizzoni, R, IEEE, 2018, pp. 227–239, ISBN: 978-1-5386-5295-4. DOI: 10.1109/RTAS.2018.00031. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-235153.

[74] N. P. Jouppi, C. Young, N. Patil and D. Patterson, 'A domain-specific architecture for deep neural networks,' *Commun. ACM*, vol. 61, no. 9, pp. 50–59, Aug. 2018, ISSN: 0001-0782. DOI: 10.1145/3154484. [Online]. Available: https://doi.org/10.1145/3154484.

[75] G. E. Moore, 'Cramming more components onto integrated circuits,' *Proceedings of the IEEE*, vol. 86, no. 1, pp. 82–85, 1998.

[76] R. Dennard, F. Gaensslen, H.-N. Yu, V. Rideout, E. Bassous and A. LeBlanc, 'Design of ion-implanted mosfet's with very small physical dimensions,' *IEEE Journal of Solid-State Circuits*, vol. 9, no. 5, pp. 256–268, 1974. DOI: 10.1109/JSSC.1974.1050511.

[77] E. A. Lee, 'The problem with threads,' *Computer*, vol. 39, no. 5, pp. 33–42, 2006. DOI: 10.1109/MC.2006.180.

[78] T. Mitra, 'Time-predictable computing by design: Looking back, looking forward,' in *Proceedings of the 56th Annual Design Automation Conference 2019*, ser. DAC '19, Las Vegas, NV, USA: Association for Computing Machinery, 2019, ISBN: 9781450367257. DOI: 10.1145/3316781.3323489. [Online]. Available: https://doi.org/10.1145/3316781.3323489.

[79] M. Schoeberl, 'A time predictable Java processor,' in *DATE*, 2006, pp. 800–805.

[80] M. Schoeberl, P. Schleuniger, W. Puffitsch, F. Brandner, C. W. Probst, S. Karlsson and T. Thorn, 'Towards a time-predictable dual-issue microprocessor: The patmos approach,' in *Workshop on Bringing Theory to Practice: Predictability and Performance in Embedded Systems*, OASICS, 2011.

[81] M. Schoeberl, S. Abbaspour, B. Akesson, N. Audsley, R. Capasso, J. Garside, K. Goossens, S. Goossens, S. Hansen, R. Heckmann, S. Hepp, B. Huber, A. Jordan, E. Kasapaki, J. Knoop, Y. Li, D. Prokesch, W. Puffitsch, P. Puschner, A. Rocha, C. Silva, J. Sparsø and A. Tocchi, 'T-crest: Time-predictable multi-core architecture for embedded systems,' *Journal of Systems Architecture*, vol. 61, no. 9, pp. 449–471, 2015, ISSN: 1383-7621. DOI: https://doi.org/10.1016/j.sysarc.2015.04.002. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762115000193.

[82] S. A. Edwards, S. Kim, E. A. Lee, I. Liu, H. Patel and M. Schoeberl, 'A disruptive computer design idea: Architectures with repeatable timing,' in *International Conference on Computer Design (ICCD)*, IEEE, 2009, pp. 54–59. DOI: 10.1109/ICCD.2009.5413177.

[83] I. Liu, J. Reineke and E. A. Lee, 'A pret architecture supporting concurrent programs with composable timing properties,' in *Asilomar Conference on Signals, Systems, and Computers*, 2010. [Online]. Available: http://chess.eecs.berkeley.edu/pubs/803.html.

[84] I. Liu, 'Precision timed machines,' EECS Department, UC Berkeley, Report UCB/EECS-2012-113, May 2012. [Online]. Available: http://www.eecs.berkeley.edu/Pubs/TechRpts/2012/EECS-2012-113.html.

[85] M. Zimmer, 'Predictable processors for mixed-criticality systems and precision-timed i/o,' PhD thesis, EECS Department, University of California, Berkeley, Aug. 2015.

[86] N. Hili, A. Girault and E. Jenn, 'Worst-case reaction time optimization on deterministic multi-core architectures with synchronous languages,' in *2019 IEEE 25th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, IEEE, 2019, pp. 1–11.

[87] M. Paolieri, J. Mische, S. Metzlaff, M. Gerdes, E. Quiñones, S. Uhrig, T. Ungerer and F. J. Cazorla, 'A hard real-time capable multi-core smt processor,' *ACM Trans. Embed. Comput. Syst.*, vol. 12, no. 3, 79:1–79:26, Apr. 2013, ISSN: 1539-9087. DOI: http://dx.doi.org/10.1145/2442116.2442129.

[88] D. C. Metz and E. R. Jellum, 'Evaluating fifo-based instruction scheduling techniques using fpgas,' Master's thesis, NTNU, 2020.

[89] S. Edwards, 'The challenges of synthesizing hardware from c-like languages,' *IEEE Design & Test of Computers*, vol. 23, no. 5, pp. 375–386, 2006. DOI: 10.1109/MDT.2006.134.

[90] I. Sander, 'System modeling and design refinement in ForSyDe,' Thesis, 2003.

[91] S. M. Albrektsen and T. A. Johansen, 'Syncboard - a high accuracy sensor timing board for uav payloads,' in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 1706–1715. DOI: 10.1109/ICUAS.2017.7991410.

[92] J. Mogul, P. D. L. Mills, U. Windl, J. Stone and J. Brittenson, *Pulse-Per-Second API for UNIX-like Operating Systems, Version 1.0*, RFC 2783, Mar. 2000. DOI: 10.17487/RFC2783. [Online]. Available: https://www.rfc-editor.org/info/rfc2783.

[93] D. L. Mills, 'A brief history of NTP time: Confessions of an internet timekeeper,' *ACM Computer Communications Review*, vol. 33, 2003.

[94] 'Ieee standard for a precision clock synchronization protocol for networked measurement and control systems,' *IEEE Std 1588-2019 (Revision ofIEEE Std 1588-2008)*, pp. 1–499, 2020. DOI: 10.1109/IEEESTD.2020.9120376.

[95] S. Williams, A. Waterman and D. Patterson, 'Roofline: An insightful visual performance model for floating-point programs and multicore architectures,' Feb. 2009.

[96] A. Sarkar, B. J. Svensson and M. Sheeran, *Synchron – an API and runtime for embedded systems*, 2022. DOI: 10.48550/ARXIV.2205.03262. [Online]. Available: https://arxiv.org/abs/2205.03262.

[97] T. W. Barr and S. Rixner, 'Medusa: Managing concurrency and communication in embedded systems,' in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIX ATC'14, Philadelphia, PA: USENIX Association, 2014, pp. 439–450, ISBN: 9781931971102.

[98] M. Zimmer, D. Broman, C. Shaver and E. A. Lee, 'FlexPRET: A processor platform for mixed-criticality systems,' in *Real-Time and Embedded Technology and Application Symposium (RTAS)*, 2014. [Online]. Available: http://chess.eecs.berkeley.edu/pubs/1048.html.

[99] M. Schoeberl, L. Pezzarossa and J. Sparsø, 'S4noc: A minimalistic network-on-chip for real-time multicores,' in *Proceedings of the 12th International Workshop on Network on Chip Architectures*, ACM, Oct. 2019, 8:1–8:6. DOI: 10.1145/3356045.3360714.

[100] S. Natarajan and D. Broman, 'Timed c: An extension to the c programming language for real-time systems,' in *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, Apr. 2018, pp. 227–239. DOI: 10.1109/RTAS.2018.00031.

[101] S. Natarajan and D. Broman, 'Temporal property-based testing of a timed c compiler using time-flow graph semantics,' in *Proceedings 2020 Forum on specification & Design Languages (FDL).*, [ed] Alain Girault, Institute of Electrical and Electronics Engineers (IEEE), 2020, pp. 1–8. DOI: 10.1109/FDL50818.2020.9232935. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-292191.

[102] 'Standard for local and metropolitan area network–bridges and bridged networks,' *IEEE Std 802.1Q-2018 (Revision of IEEE Std 802.1Q-2014)*, pp. 1–1993, 2018. DOI: 10.1109/IEEESTD.2018.8403927.

[103] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek and K. Asanovic, 'Chisel: Constructing hardware in a scala embedded language,' in *The 49th Annual Design Automation Conference (DAC 2012)*, San Francisco, CA, USA: ACM, Jun. 2012, pp. 1216–1225, ISBN: 978-1-4503-1199-1.

[104] D. P. Bertsekas, 'The auction algorithm: A distributed relaxation method for the assignment problem,' *Ann. Oper. Res.*, vol. 14, no. 1–4, pp. 105–123, Jun. 1988, ISSN: 0254-5330. DOI: 10.1007/BF02186476. [Online]. Available: https://doi.org/10.1007/BF02186476.

[105] A. Schulz-Rosengarten, R. von Hanxleden, M. Lohstroh, E. A. Lee and S. Bateni, 'Polyglot modal models through lingua franca,' in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, 2023, pp. 337–342.

ISBN 978-82-326-7700-9 (printed ver.)
ISBN 978-82-326-7699-6 (electronic ver.)
ISSN 1503-8181 (printed ver.)
ISSN 2703-8084 (online ver.)

NTNU

Norwegian University of
Science and Technology